Minimal Discrete Dynamical Systems

From Iterated Maps to Large Language Models

Contents

I Foundations of Discrete Dynamical Systems **1**

1 Iterated Maps on Finite Sets **3**
1.1 Introduction . . . 3
1.2 Basic Definitions . . . 3
1.3 The Functional Graph . . . 4
1.4 Worked Examples . . . 4
1.5 Counting Functions and Permutations . . . 6
1.6 Transient Length, Cycle Length, and Rho Length . . . 6
1.7 Product Systems . . . 7
1.8 Random Maps . . . 7
1.9 Collatz-like Maps on Finite Rings . . . 8
1.10 Connection to the $(f, x) \to (\varphi(f, x), f(x))$ Framework . . . 8
1.11 Summary . . . 9
1.12 References . . . 9
1.13 Recommended Reading . . . 10

2 Symbolic Dynamics **11**
2.1 Introduction and Motivation . . . 11
2.2 The Full Shift . . . 11
2.2.1 2.2.1 Definition . . . 11
2.2.2 2.2.2 The Product Topology . . . 11
2.2.3 2.2.3 The Shift Map . . . 12
2.3 Subshifts . . . 12
2.3.1 2.3.1 Definition via Topology . . . 12
2.3.2 2.3.2 Forbidden Words Characterization . . . 12
2.4 Subshifts of Finite Type (SFTs) . . . 13
2.4.1 2.4.1 Definition . . . 13
2.4.2 2.4.2 Edge Shifts and Adjacency Matrices . . . 13
2.4.3 2.4.3 Example: The Golden Mean Shift . . . 13
2.5 Sofic Shifts . . . 14
2.5.1 2.5.1 Sliding Block Codes and Factor Maps . . . 14
2.5.2 2.5.2 Definition of Sofic Shifts . . . 14
2.5.3 2.5.3 Example: The Even Shift . . . 14
2.6 The Hierarchy of Shift Spaces . . . 14
2.7 Topological Entropy . . . 15
2.7.1 2.7.1 Definition . . . 15
2.7.2 2.7.2 Entropy of SFTs via the Perron Eigenvalue . . . 15
2.7.3 2.7.3 Worked Examples . . . 15
2.8 Invariant Measures and the Variational Principle . . . 16
2.8.1 2.8.1 Shift-Invariant Measures . . . 16
2.8.2 2.8.2 Measure-Theoretic Entropy . . . 16

2.8.3 2.8.3 The Variational Principle . . . 17
2.8.4 2.8.4 Markov Measures on SFTs . . . 17
2.9 One-Sided vs. Two-Sided Shifts . . . 18
2.10 Application: Modeling LLM Output Iteration . . . 18
2.11 Summary . . . 19
2.12 References . . . 19
2.13 Recommended Reading . . . 20

3 Sliding Block Codes and the Curtis-Hedlund-Lyndon Theorem 21
3.1 Introduction . . . 21
3.2 Sliding Block Codes: Definition . . . 21
3.3 Examples . . . 22
3.3.1 Example 3.3.1: XOR map (2-block code) . . . 22
3.3.2 Example 3.3.2: Majority vote map (radius-1) . . . 22
3.3.3 Example 3.3.3: Higher-order block code and the golden mean shift . . . 23
3.4 Continuity and Shift-Commutation . . . 23
3.5 The Curtis-Hedlund-Lyndon Theorem . . . 23
3.6 Why the Theorem Matters . . . 24
3.7 Factor Maps and Conjugacies . . . 24
3.8 Endomorphisms and Automorphisms of the Full Shift . . . 25
3.9 One-Sided Sliding Block Codes (Causal Maps) . . . 26
3.10 Connection to Cellular Automata . . . 26
3.11 Connection to Autoregressive Language Models . . . 26
3.12 Summary of Key Results . . . 27
3.13 References . . . 27
3.14 Recommended Reading . . . 28

4 Post Tag Systems 29
4.1 Introduction . . . 29
4.2 Definition . . . 29
4.3 Worked Example 1: Growth and Cycling . . . 30
4.4 Worked Example 2: A Halting Tag System . . . 30
4.5 Worked Example 3: A Looping Tag System . . . 31
4.6 Post's Normal Form Theorem (1943) . . . 31
4.7 Turing-Completeness of Tag Systems . . . 32
4.7.1 Minsky's Theorem (1961) . . . 32
4.7.2 The Cocke–Minsky Theorem (1964) . . . 32
4.8 Cyclic Tag Systems . . . 32
4.9 The Collatz Connection . . . 33
4.10 The Decidability Boundary . . . 33
4.11 Connection to the (f, x) Framework . . . 34
4.12 Connection to Autoregressive Language Models . . . 34
4.13 Summary . . . 35
4.14 References . . . 35
4.15 Recommended Reading . . . 36

5 Substitution Systems and Automatic Sequences 37
5.1 Morphisms on Words . . . 37
5.2 Fixed Points of Morphisms . . . 37
5.3 The Thue-Morse Sequence . . . 38
5.3.1 5.3.1 Construction . . . 38
5.3.2 5.3.2 Properties . . . 38

5.3.3 5.3.3 The Thue-Morse Dynamical System . . . 39
5.4 The Fibonacci Word . . . 39
5.4.1 5.4.1 Connection to the Golden Ratio . . . 39
5.4.2 5.4.2 Sturmian Sequences . . . 40
5.5 The Rudin-Shapiro Sequence . . . 40
5.6 Morphic Sequences . . . 40
5.7 Automatic Sequences . . . 41
5.7.1 5.7.1 Definition via Finite Automata . . . 41
5.7.2 5.7.2 The k-kernel . . . 41
5.7.3 5.7.3 Cobham's Characterization . . . 41
5.8 Cobham's Theorem . . . 41
5.9 Automatic vs. Morphic: The Hierarchy . . . 42
5.10 The Substitution Dynamical System . . . 42
5.10.1 5.10.1 Construction . . . 42
5.10.2 5.10.2 Recognizability . . . 42
5.11 Spectral Theory . . . 43
5.12 Connection to Our Framework . . . 43
5.13 Summary . . . 44
5.14 References . . . 44
5.15 Recommended Reading . . . 45

6 Iterated Function Systems 47
6.1 Introduction . . . 47
6.2 Basic Definitions . . . 47
6.3 The Hutchinson Operator . . . 48
6.4 Hutchinson's Theorem . . . 48
6.5 Classical Examples . . . 48
6.6 The Chaos Game . . . 50
6.7 IFS with Probabilities and the Invariant Measure . . . 50
6.8 Hausdorff Dimension of IFS Attractors . . . 51
6.9 IFS on Finite and Discrete Sets . . . 51
6.10 Recurrent IFS . . . 52
6.11 The Collage Theorem . . . 53
6.12 Connection to the (f, x) Framework . . . 53
6.13 Summary . . . 54
6.14 References . . . 54
6.15 Recommended Reading . . . 54

7 Cellular Automata as Dynamical Systems 57
7.1 Definition . . . 57
7.2 Elementary Cellular Automata . . . 57
7.2.1 7.2.1 Wolfram's Numbering Scheme . . . 58
7.2.2 7.2.2 Rule 30 . . . 58
7.2.3 7.2.3 Rule 110 . . . 58
7.2.4 7.2.4 Rule 90 . . . 58
7.3 Worked Examples: Tracing ECA Evolution . . . 58
7.4 Spacetime Diagrams . . . 60
7.5 Wolfram's Four Classes (1984) . . . 60
7.6 Kurka's Topological Classification (1997) . . . 60
7.7 CA as Endomorphisms of the Shift: The Curtis-Hedlund-Lyndon Theorem . . . 61
7.8 Universality: Rule 110 is Turing-Complete . . . 62
7.9 The Garden of Eden Theorem . . . 62

7.10 Reversible Cellular Automata . . . 63
7.11 Topological Entropy of a CA . . . 63
7.12 Two-Dimensional CA: Conway's Game of Life . . . 64
7.13 Connection to Our Framework . . . 64
7.14 Summary . . . 65
7.15 References . . . 65
7.16 Recommended Reading . . . 66

II Complexity and Computation 67

8 Kolmogorov Complexity and Algorithmic Information Theory 69
8.1 Motivation: Information Content of Individual Objects . . . 69
8.2 Definition . . . 69
8.3 The Invariance Theorem . . . 70
8.4 Basic Properties . . . 70
8.5 Incompressible Strings . . . 70
8.6 Conditional and Joint Complexity . . . 71
8.7 Prefix-Free Complexity . . . 72
8.8 Algorithmic Probability and Solomonoff's Prior . . . 72
8.9 Martin-Löf Randomness . . . 73
8.10 Brudno's Theorem: Complexity Meets Ergodic Theory . . . 73
8.11 Worked Examples . . . 73
8.12 Practical Approximation . . . 75
8.13 Connection to Discrete Dynamical Systems . . . 75
8.14 Summary of Key Results . . . 76
8.15 References . . . 76
8.16 Recommended Reading . . . 77

9 Logical Depth and Sophistication 79
9.1 Motivation: The Inadequacy of Kolmogorov Complexity Alone . . . 79
9.2 Bennett's Logical Depth . . . 79
9.2.1 9.2.1 Informal Idea . . . 79
9.2.2 9.2.2 Formal Definition . . . 80
9.2.3 9.2.3 The Three Categories . . . 80
9.2.4 9.2.4 The Slow Growth Law . . . 80
9.3 Worked Examples . . . 81
9.4 Sophistication . . . 82
9.4.1 9.4.1 Motivation . . . 82
9.4.2 9.4.2 Formal Definition . . . 82
9.4.3 9.4.3 Properties . . . 82
9.5 Effective Complexity . . . 83
9.6 Relationships Between the Notions . . . 83
9.7 Connection to Creativity and "Plausible History of Origin" . . . 84
9.8 Connection to Discrete Dynamical Systems . . . 84
9.9 Practical Approximation . . . 84
9.10 Summary . . . 85
9.11 References . . . 85
9.12 Recommended Reading . . . 86

10 Computational Mechanics and Epsilon-Machines **87**
10.1 Introduction and Motivation . . . 87
10.2 Setup: Stationary Stochastic Processes . . . 87
10.3 Causal States . . . 88
10.4 The Epsilon-Machine . . . 89
10.5 Statistical Complexity and Entropy Rate . . . 89
10.6 Excess Entropy (Predictive Information) . . . 90
10.7 The Complexity-Entropy Plane . . . 91
10.8 Worked Example 1: The Fair Coin (IID Bernoulli-1/2) . . . 91
10.9 Worked Example 2: The Period-2 Process ("010101...") . . . 92
10.10 Worked Example 3: The Golden Mean Process . . . 92
10.11 Worked Example 4: The Even Process . . . 94
10.12 Summary of Examples . . . 95
10.13 The CSSR Algorithm . . . 95
10.14 Connection to Dynamical Systems . . . 96
10.15 Further Results and Connections . . . 97
10.16 Summary . . . 98
10.17 References . . . 98
10.18 Recommended Reading . . . 99

III Self-Reference and Reflection **101**

11 Kleene's Recursion Theorem and Fixed Points of Program Transformations **103**
11.1 Introduction . . . 103
11.2 Setup: Acceptable Numberings and the s-m-n Theorem . . . 103
11.2.1 11.2.1 Gödel Numbering . . . 103
11.2.2 11.2.2 The s-m-n Theorem (Parameter Theorem) . . . 104
11.3 Kleene's Second Recursion Theorem . . . 104
11.3.1 11.3.1 Statement . . . 104
11.3.2 11.3.2 Proof . . . 104
11.3.3 11.3.3 Dissecting the Proof . . . 105
11.4 Rogers's Fixed-Point Theorem . . . 105
11.5 What the Theorem Does NOT Say . . . 105
11.6 Worked Examples . . . 106
11.6.1 11.6.1 Example 1: A Concrete Fixed Point . . . 106
11.6.2 11.6.2 Example 2: Quine Construction . . . 106
11.6.3 11.6.3 Example 3: A Fixed Point of "Double the Program" . . . 107
11.6.4 11.6.4 Example 4: Self-Referential Programs . . . 107
11.7 Applications . . . 107
11.7.1 11.7.1 Application 1: Self-Replicating Programs (Viruses) . . . 107
11.7.2 11.7.2 Application 2: Rice's Theorem . . . 108
11.7.3 11.7.3 Application 3: Fixed Points in Learning Theory . . . 108
11.8 The Recursion Theorem for Dynamical Systems . . . 108
11.8.1 11.8.1 Meta-Rules and Fixed Points . . . 108
11.8.2 11.8.2 Consequences for the (f, x) Framework . . . 109
11.8.3 11.8.3 From Extensional to Intensional Fixed Points in Finite Systems . . . 109
11.9 Limitations and Caveats . . . 110
11.10 Summary . . . 110
11.11 References . . . 110
11.12 Recommended Reading . . . 111

12 Self-Modifying Computation and Reflective Towers **113**
12.1 Introduction . . . 113
12.2 Von Neumann's Self-Reproducing Automata . . . 113
12.3 Program-Data Duality . . . 114
12.4 Procedural Reflection: Smith's 3-Lisp . . . 114
12.4.1 12.4.1 The Reflective Tower . . . 115
12.4.2 12.4.2 Reification and Reflection . . . 115
12.4.3 12.4.3 Formal Sketch . . . 116
12.5 The Implementation Problem: Lazy Tower Construction . . . 116
12.6 Denotational Semantics: Wand and Friedman . . . 116
12.7 Collapsing Towers: Amin and Rompf . . . 117
12.8 State-Update versus Function-Update: Kampis . . . 118
12.9 Connection to the (f, x) Framework . . . 118
12.9.1 12.9.1 The Tower as a Stack of (f_i, x_i) Pairs . . . 118
12.9.2 12.9.2 Reflective Operations as Meta-Rule Applications . . . 119
12.9.3 12.9.3 The Tower Arises from Self-Applicable Meta-Rules . . . 119
12.9.4 12.9.4 Worked Example . . . 119
12.10 Self-Modifying Code in Practice . . . 120
12.10.1 12.10.1 JIT Compilation . . . 120
12.10.2 12.10.2 Genetic Programming . . . 120
12.10.3 12.10.3 Neural Architecture Search . . . 120
12.11 Summary . . . 121
12.12 References . . . 121
12.13 Recommended Reading . . . 122

13 AIXI and Gödel Machines **125**
13.1 Introduction . . . 125
13.2 Solomonoff Induction . . . 125
13.2.1 13.2.1 The Prediction Problem . . . 125
13.2.2 13.2.2 The Universal Prior . . . 125
13.2.3 13.2.3 The Dominance Property . . . 126
13.2.4 13.2.4 Convergence of Solomonoff Prediction . . . 126
13.2.5 13.2.5 Incomputability . . . 127
13.3 AIXI . . . 127
13.3.1 13.3.1 The Agent-Environment Framework . . . 127
13.3.2 13.3.2 The AIXI Agent . . . 127
13.3.3 13.3.3 A Concrete Scenario . . . 128
13.3.4 13.3.4 Optimality . . . 128
13.3.5 13.3.5 Limitations . . . 129
13.3.6 13.3.6 AIXItl: A Computable Approximation . . . 129
13.4 Gödel Machines . . . 129
13.4.1 13.4.1 Motivation . . . 129
13.4.2 13.4.2 Structure . . . 129
13.4.3 13.4.3 The Self-Referential Proof Search . . . 130
13.4.4 13.4.4 Key Properties . . . 130
13.4.5 13.4.5 Comparison with AIXI . . . 131
13.5 OOPS: Optimal Ordered Problem Solver . . . 131
13.5.1 13.5.1 Motivation . . . 131
13.5.2 13.5.2 Setup . . . 131
13.5.3 13.5.3 Bias-Optimality . . . 132
13.5.4 13.5.4 OOPS as a Dynamical System . . . 132

13.6 Synthesis: Self-Modifying Dynamical Systems . . . 132
13.6.1 13.6.1 Three Approaches to $(f, x) \mapsto (f', x')$ . . . 132
13.6.2 13.6.2 The Unconstrained Case . . . 133
13.6.3 13.6.3 What These Constructions Share . . . 133
13.7 Summary . . . 133
13.8 References . . . 134
13.9 Recommended Reading . . . 134

IV Language Models and Novelty 137

14 LLMs as Dynamical Systems 139
14.1 Introduction . . . 139
14.2 The LLM as a Map . . . 139
14.2.1 14.2.1 The Conditional Distribution . . . 139
14.2.2 14.2.2 Context Windows and Finite Memory . . . 139
14.2.3 14.2.3 Deterministic Decoding as a Sliding Block Code . . . 140
14.2.4 14.2.4 The Generated Subshift . . . 140
14.2.5 14.2.5 Stochastic Generation as a Measure . . . 141
14.3 Iterated LLM Generation . . . 141
14.3.1 14.3.1 The Iteration Scheme . . . 141
14.3.2 14.3.2 Convergence to Periodic Attractors . . . 142
14.3.3 14.3.3 Geometric Framework . . . 142
14.3.4 14.3.4 The Telephone Game . . . 142
14.4 The (f, x) Decomposition . . . 142
14.4.1 14.4.1 Prompts Encode Functions and Inputs . . . 142
14.4.2 14.4.2 The (f, x) Dynamical System . . . 143
14.4.3 14.4.3 Worked Example: Iterated Increment . . . 143
14.4.4 14.4.4 Worked Example: Instruction Drift . . . 144
14.5 Model Collapse . . . 144
14.5.1 14.5.1 The Phenomenon . . . 144
14.5.2 14.5.2 Information-Theoretic Explanation . . . 144
14.5.3 14.5.3 Self-Consuming Loops . . . 145
14.5.4 14.5.4 Model Collapse in the (f, x) Framework . . . 145
14.6 Self-Improvement and Its Limits . . . 145
14.6.1 14.6.1 Self-Refine . . . 145
14.6.2 14.6.2 Self-Rewarding . . . 146
14.6.3 14.6.3 Limits of Self-Correction . . . 146
14.6.4 14.6.4 Constitutional AI . . . 146
14.7 Computational Power of Iterated LLMs . . . 146
14.7.1 14.7.1 A Single Forward Pass: TC^0 . . . 146
14.7.2 14.7.2 Chain of Thought: Turing Completeness . . . 147
14.7.3 14.7.3 Prompt-Based Turing Completeness . . . 147
14.7.4 14.7.4 Implications for the (f, x) Framework . . . 147
14.8 Multi-Agent LLM Systems . . . 147
14.8.1 14.8.1 The Product Dynamical System . . . 147
14.8.2 14.8.2 Role-Playing Agents: CAMEL . . . 148
14.8.3 14.8.3 Emergent Collective Dynamics . . . 148
14.9 Open Questions . . . 148
14.10Summary . . . 149
14.11References . . . 149
14.12Recommended Reading . . . 150

15 Measuring Novelty Within a Sequence **153**
15.1 Introduction . . . 153
15.2 The Setting . . . 153
15.2.1 15.2.1 Finite Sequences . . . 153
15.2.2 15.2.2 Two Notions of Novelty . . . 153
15.3 Type-Token Measures . . . 154
15.3.1 15.3.1 Type-Token Ratio . . . 154
15.3.2 15.3.2 The Length Dependence Problem . . . 154
15.3.3 15.3.3 Length-Normalized Variants . . . 154
15.3.4 15.3.4 Hapax Legomena . . . 155
15.4 N-gram Diversity Measures . . . 155
15.4.1 15.4.1 Distinct-n . . . 155
15.4.2 15.4.2 Repetition Rate . . . 155
15.4.3 15.4.3 Self-BLEU . . . 155
15.5 Entropy-Based Measures . . . 156
15.5.1 15.5.1 Empirical Entropy . . . 156
15.5.2 15.5.2 Higher-Order Empirical Entropy . . . 156
15.5.3 15.5.3 Perplexity . . . 157
15.6 Compression-Based Measures . . . 157
15.6.1 15.6.1 Connection to Kolmogorov Complexity . . . 157
15.6.2 15.6.2 Compression Ratio . . . 157
15.6.3 15.6.3 Lempel-Ziv Complexity . . . 157
15.6.4 15.6.4 Normalized Compression Distance . . . 158
15.7 The Quality-Diversity Frontier . . . 158
15.7.1 15.7.1 The Trade-off . . . 158
15.7.2 15.7.2 Pareto Optimality . . . 158
15.7.3 15.7.3 Entropy Bounds in Human Text . . . 158
15.8 Connection to Logical Depth . . . 159
15.8.1 15.8.1 Review of Logical Depth . . . 159
15.8.2 15.8.2 Depth as Structured Novelty . . . 159
15.8.3 15.8.3 Implications for LLM Outputs . . . 159
15.9 Connection to Computational Mechanics . . . 159
15.9.1 15.9.1 Epsilon-Machines and Statistical Complexity . . . 159
15.9.2 15.9.2 Statistical Complexity of a Finite Sequence . . . 159
15.9.3 15.9.3 Excess Entropy . . . 160
15.9.4 15.9.4 Complexity vs. Entropy . . . 160
15.10 Practical Metrics for LLM Evaluation . . . 160
15.10.1 15.10.1 Standard Metrics . . . 160
15.10.2 15.10.2 Implementation . . . 160
15.10.3 15.10.3 Interpreting the Metrics . . . 162
15.11 Connection to Model Collapse . . . 162
15.11.1 15.11.1 Intra-Sequence Novelty During Collapse . . . 162
15.11.2 15.11.2 Diversity as Collapse Diagnostic . . . 162
15.12 Novelty-Increasing Dynamics in the (f, x) Framework . . . 162
15.12.1 15.12.1 The Default: Novelty Decay . . . 162
15.12.2 15.12.2 Novelty-Expanding Functions . . . 163
15.12.3 15.12.3 The Exploratory Regime Revisited . . . 163
15.12.4 15.12.4 Strategies for Sustained Novelty . . . 164
15.12.5 15.12.5 Patterns of Novelty-Increasing Systems . . . 164
15.13 Relative Novelty . . . 165
15.13.1 15.13.1 The Reference Problem . . . 165

15.13.2 15.13.2 Reference Sets . . . 165
15.13.3 15.13.3 Distance-Based Relative Novelty . . . 165
15.13.4 15.13.4 Information-Theoretic Relative Novelty . . . 166
15.13.5 15.13.5 Kernel-Based Novelty . . . 166
15.13.6 15.13.6 N-gram Overlap Measures . . . 166
15.13.7 15.13.7 Self-Relative Novelty: Novelty Over Time . . . 167
15.13.8 15.13.8 The Novelty-Coherence-Reference Triangle . . . 167
15.13.9 15.13.9 Formalizing "Interestingness" . . . 167
15.14 Additional Mathematical Frameworks . . . 168
15.14.1 15.14.1 Optimal Transport and Wasserstein Distance . . . 168
15.14.2 15.14.2 Rate-Distortion Theory . . . 168
15.14.3 15.14.3 Determinantal Point Processes . . . 169
15.14.4 15.14.4 Information Geometry . . . 169
15.14.5 15.14.5 Algorithmic Statistics and Typicality . . . 169
15.14.6 15.14.6 Intrinsic Motivation and Curiosity . . . 170
15.14.7 15.14.7 Topological Data Analysis . . . 170
15.14.8 15.14.8 Renormalization and Multi-Scale Analysis . . . 170
15.15 Cognitive Foundations of Novelty . . . 171
15.15.1 15.15.1 Predictive Processing and Surprisal . . . 171
15.15.2 15.15.2 Hippocampal Novelty Detection . . . 171
15.15.3 15.15.3 Habituation and Cumulative Novelty . . . 172
15.15.4 15.15.4 Curiosity and the Goldilocks Zone . . . 172
15.15.5 15.15.5 Schema Violation and Bisociation . . . 172
15.15.6 15.15.6 Conceptual Spaces . . . 173
15.15.7 15.15.7 Working Memory and Chunk Diversity . . . 173
15.15.8 15.15.8 Semantic Priming and Spreading Activation . . . 173
15.15.9 15.15.9 The Orienting Response . . . 174
15.15.10 15.15.10 Toward a Unified Cognitive-Computational Theory . . . 174
15.16 Open Questions . . . 175
15.17 Summary . . . 175
15.18 Recommended Reading . . . 176
15.19 References . . . 177
15.20 Exercises . . . 179

16 Evolution, Discovery, and Open-Ended Systems 181
16.1 Introduction . . . 181
16.2 Scientific Discovery as a Dynamical System . . . 181
16.2.1 16.2.1 The Scientific Method as Iteration . . . 181
16.2.2 16.2.2 The (f, x) Decomposition of Science . . . 182
16.2.3 16.2.3 Attractors in Hypothesis Space . . . 182
16.2.4 16.2.4 Novelty in Scientific Discovery . . . 183
16.2.5 16.2.5 Boden's Taxonomy of Creativity . . . 183
16.3 Evolutionary Computation as a Dynamical System . . . 184
16.3.1 16.3.1 Genetic Algorithms . . . 184
16.3.2 16.3.2 Fitness Landscapes . . . 184
16.3.3 16.3.3 Premature Convergence as Collapse . . . 184
16.3.4 16.3.4 Diversity Maintenance . . . 185
16.4 Novelty Search and Quality-Diversity . . . 185
16.4.1 16.4.1 Novelty Search . . . 185
16.4.2 16.4.2 The Paradox of Objectives . . . 185
16.4.3 16.4.3 Quality-Diversity Algorithms . . . 185

16.4.4 16.4.4 Connection to Chapter 15 Metrics . . . 186
16.5 LLM-Augmented Evolution . . . 186
16.5.1 16.5.1 Evolution Through Large Models (ELM) . . . 186
16.5.2 16.5.2 FunSearch: LLM + Evolution for Mathematical Discovery . . . 187
16.5.3 16.5.3 AlphaEvolve: General-Purpose Algorithm Discovery . . . 187
16.5.4 16.5.4 The (f, x) View of LLM-Evolution . . . 187
16.6 Autonomous AI Scientists . . . 188
16.6.1 16.6.1 The AI Scientist . . . 188
16.6.2 16.6.2 AI Scientist v2 and Agentic Tree Search . . . 188
16.6.3 16.6.3 Open-Ended Scientific Discovery . . . 189
16.7 The Adjacent Possible and Open-Endedness . . . 189
16.7.1 16.7.1 The Adjacent Possible . . . 189
16.7.2 16.7.2 Formalizing Open-Endedness . . . 189
16.7.3 16.7.3 Obstacles to Open-Endedness . . . 189
16.7.4 16.7.4 Conditions for Open-Endedness . . . 190
16.8 Synthesis: The DDS Framework for Discovery and Evolution . . . 190
16.8.1 16.8.1 Unifying Table . . . 190
16.8.2 16.8.2 The Quality-Diversity Lyapunov Exponent . . . 190
16.8.3 16.8.3 The Role of LLMs . . . 190
16.8.4 16.8.4 Implementation Examples . . . 191
16.9 Open Questions . . . 196
16.10Summary . . . 197
16.11Recommended Reading . . . 197
16.12References . . . 199
16.13Exercises . . . 199

Part I

Foundations of Discrete Dynamical Systems

Chapter 1

Iterated Maps on Finite Sets

1.1 Introduction

Many systems in mathematics and computer science evolve by the repeated application of a fixed rule. A cellular automaton updates its cells according to a local rule; a hash function chains outputs back as inputs; a pseudorandom number generator feeds its output into itself. The simplest abstraction that captures all of these is the **iterated map on a finite set**: given a finite set X and a function $f : X \to X$, we study the sequence $x, f(x), f(f(x)), \ldots$ obtained by repeatedly applying f.

Despite the simplicity of this setup, a surprisingly rich theory emerges. Every orbit must eventually cycle. The global structure of the map decomposes into a forest of trees hanging off disjoint cycles. The statistics of random maps connect to the birthday paradox and to the analysis of algorithms. This chapter develops the core theory from scratch.

1.2 Basic Definitions

Definition 1.1 (Discrete Dynamical System)**.** A *discrete dynamical system* (DDS) on a finite set is a pair (X, f) where X is a finite set and $f : X \to X$ is a function. The *state space* is X and f is the *transition map* (or simply the *map*).

Given an initial state $x_0 \in X$, the system evolves deterministically:

$$x_0, \quad x_1 = f(x_0), \quad x_2 = f(x_1) = f^2(x_0), \quad x_3 = f^3(x_0), \quad \ldots$$

where f^k denotes the k-fold composition of f with itself, and f^0 is the identity.

Definition 1.2 (Orbit)**.** The *orbit* (or *trajectory*) of a point $x_0 \in X$ under f is the sequence

$$\mathcal{O}(x_0) = (x_0, f(x_0), f^2(x_0), \ldots).$$

Since X is finite, this sequence must eventually revisit a state.

Definition 1.3 (Fixed Points and Periodic Points)**.**

- A point $x \in X$ is a *fixed point* of f if $f(x) = x$.
- A point x is a *periodic point* of period k if $f^k(x) = x$ and $k \geq 1$ is the smallest such integer. We also say x belongs to a *k-cycle*. A fixed point is periodic with period 1.
- A point x is *eventually periodic* if there exist integers $t \geq 0$ and $k \geq 1$ such that $f^{t+k}(x) = f^t(x)$. The smallest such t is the *transient length* (or *tail length*) and the smallest such k is the *cycle length*.

Proposition 1.4. *Every point in a finite DDS is eventually periodic.*

Proof. Let $|X| = n$. The sequence $x_0, f(x_0), \ldots, f^n(x_0)$ contains $n+1$ elements drawn from a set of size n. By the pigeonhole principle, there exist indices $0 \leq i < j \leq n$ with $f^i(x_0) = f^j(x_0)$. Setting $t = i$ and $k = j - i$, we have $f^{t+k}(x_0) = f^t(x_0)$.

□

1.3 The Functional Graph

Definition 1.5 (Functional Graph)**.** The *functional graph* of a map $f : X \to X$ is the directed graph $G_f = (X, E)$ where the edge set is $E = \{(x, f(x)) : x \in X\}$. Every vertex has out-degree exactly 1.

This is the central combinatorial object of this chapter. A directed graph in which every vertex has out-degree exactly 1 is sometimes called a *functional digraph* or *1-regular digraph.* The structure of such graphs is completely determined by the following theorem.

Theorem 1.6 (Structure Theorem for Functional Graphs)**.** *Let G_f be the functional graph of $f : X \to X$ where X is finite. Then every weakly connected component of G_f consists of a single directed cycle with directed trees (arborescences) rooted at the cycle nodes, with edges directed toward the roots. Equivalently, every connected component has the shape of the Greek letter ρ: a "tail" path leading into a cycle.*

Proof. Let C be a weakly connected component of G_f. We prove the structure in two steps.

Step 1: Every component contains exactly one cycle. Consider any vertex v in C. The sequence $v, f(v), f^2(v), \ldots$ must eventually repeat (Proposition 1.4), so it enters a cycle. Thus C contains at least one cycle. Suppose C contains two distinct cycles γ_1 and γ_2. Since C is weakly connected, there is an undirected path between some $u_1 \in \gamma_1$ and $u_2 \in \gamma_2$. But every vertex on a cycle has its successor on the same cycle (since cycles are invariant under f), and every vertex not on a cycle has a unique successor that is one step closer to a cycle. Following the forward orbit of any vertex in C must lead to a single cycle. If there were two cycles, some vertex would need to have out-degree 2 (its orbit would need to "choose" which cycle to join), which contradicts the fact that f is a function. Hence there is exactly one cycle per component.

Step 2: The non-cycle vertices form a forest of trees rooted at cycle nodes. Let γ be the unique cycle in C. Remove the edges of γ. The remaining graph restricted to non-cycle vertices has out-degree 1 everywhere except at cycle nodes (which are now roots), and no cycles. A directed graph with out-degree ≤ 1 and no cycles is a forest of rooted directed trees. The edge directions point toward the roots (cycle vertices), since iterating f from any non-cycle vertex eventually reaches the cycle. □

Remark 1.7. The "ρ shape" terminology comes from the visual resemblance: starting from a tail vertex, the orbit traces a path (the stroke of ρ) that enters a loop (the bowl of ρ). The transient length of a vertex x is the distance from x to the cycle, and the cycle length is the length of that cycle. The *rho length* of a vertex is the sum $\lambda(x) = t(x) + c(x)$, where $t(x)$ is the transient length and $c(x)$ the cycle length.

1.4 Worked Examples

Example: $f(x) = x^2 \bmod 10$

Let $X = \{0, 1, 2, \ldots, 9\}$ and $f(x) = x^2 \bmod 10$. We compute the function table:

x	0	1	2	3	4	5	6	7	8	9
$f(x)$	0	1	4	9	6	5	6	9	4	1

Now we trace orbits:

- $0 \to 0$. Fixed point. Transient 0, cycle length 1.
- $1 \to 1$. Fixed point.
- $2 \to 4 \to 6 \to 6$. Transient 2, cycle length 1 (6 is a fixed point).
- $3 \to 9 \to 1 \to 1$. Transient 2, cycle length 1.
- $4 \to 6 \to 6$. Transient 1, cycle length 1.
- $5 \to 5$. Fixed point.
- $6 \to 6$. Fixed point.
- $7 \to 9 \to 1 \to 1$. Transient 2, cycle length 1.
- $8 \to 4 \to 6 \to 6$. Transient 2, cycle length 1.
- $9 \to 1 \to 1$. Transient 1, cycle length 1.

The functional graph has four fixed points: $\{0, 1, 5, 6\}$. No non-trivial cycles exist. The maximum transient length is 2 (attained by $\{2, 3, 7, 8\}$). The graph has four connected components: $\{0\}$, $\{3, 7, 9, 1\}$, $\{2, 8, 4, 6\}$, and $\{5\}$. Each conforms to the ρ-structure theorem (with trivial cycles of length 1).

Example: $f(x) = 2x \bmod 11$

Let $X = \{0, 1, \ldots, 10\}$ and $f(x) = 2x \bmod 11$. Since $\gcd(2, 11) = 1$, f is a bijection (a permutation) on X. We compute:

x	0	1	2	3	4	5	6	7	8	9	10
$f(x)$	0	2	4	6	8	10	1	3	5	7	9

Tracing orbits:

- $0 \to 0$. Fixed point.
- $1 \to 2 \to 4 \to 8 \to 5 \to 10 \to 9 \to 7 \to 3 \to 6 \to 1$. This is a cycle of length 10.

Since f is a permutation, every point is periodic (transient length 0 for all vertices). The cycle structure is: one fixed point at 0, one 10-cycle containing $\{1, 2, \ldots, 10\}$. The order of 2 in the multiplicative group $(\mathbb{Z}/11\mathbb{Z})^*$ is 10, confirming that 2 is a primitive root modulo 11.

In general, when f is a permutation, the functional graph consists entirely of disjoint cycles with no tails. This is the special case addressed by classical permutation theory.

Example: $f(x) = 2x \bmod 12$

Let $X = \{0, 1, \ldots, 11\}$ and $f(x) = 2x \bmod 12$. Now $\gcd(2, 12) = 2 \neq 1$, so f is not a bijection.

x	0	1	2	3	4	5	6	7	8	9	10	11
$f(x)$	0	2	4	6	8	10	0	2	4	6	8	10

Tracing:

- $0 \to 0$. Fixed point.

- $1 \to 2 \to 4 \to 8 \to 4$. Transient 2, cycle $\{4, 8\}$ of length 2.
- $3 \to 6 \to 0 \to 0$. Transient 2, cycle length 1.
- $5 \to 10 \to 8 \to 4 \to 8$. Transient 2, cycle $\{4, 8\}$ of length 2.
- $7 \to 2 \to 4 \to 8 \to 4$. Transient 2, cycle $\{4, 8\}$ of length 2.
- $9 \to 6 \to 0 \to 0$. Transient 2, cycle length 1.
- $11 \to 10 \to 8 \to 4 \to 8$. Transient 2, cycle $\{4, 8\}$ of length 2.

This map has two connected components. One is rooted at the fixed point 0 (with $3, 6, 9$ as tail nodes). The other contains the 2-cycle $\{4, 8\}$ (with $1, 2, 5, 7, 10, 11$ as tail nodes). The ρ-shape is evident: every orbit has a tail of length 1 or 2 leading into a cycle of length 1 or 2.

1.5 Counting Functions and Permutations

We now turn to enumerative questions. Let $X = \{0, 1, \ldots, n-1\}$.

Proposition 1.8 (Total count)**.** *The number of functions $f : X \to X$ is n^n, since each of the n elements of X can be mapped to any of n elements independently.*

Proposition 1.9 (Permutations)**.** *The number of bijections $f : X \to X$ is $n!$. The ratio $n!/n^n$ (the probability that a uniformly random function is a bijection) tends to 0 rapidly: by Stirling's approximation, $n!/n^n \sim \sqrt{2\pi n}\, e^{-n}$.*

Proposition 1.10 (Functions with exactly k fixed points)**.** *The number of functions $f : X \to X$ with exactly k fixed points is*

$$\binom{n}{k}(n-1)^{n-k}.$$

Proof. Choose the k fixed points in $\binom{n}{k}$ ways. Each of the remaining $n-k$ elements must map to some element other than itself, giving $n-1$ choices per non-fixed element (it may map to any element of X except itself, including other non-fixed elements). Hence the total is $\binom{n}{k}(n-1)^{n-k}$.

Remark 1.11. Setting $k = 0$: the number of fixed-point-free functions is $(n-1)^n$. This is not the same as the number of *derangements* $D_n = n! \sum_{i=0}^{n} (-1)^i/i!$, which counts fixed-point-free *permutations*. The formula $\binom{n}{k}(n-1)^{n-k}$ counts all functions (not just bijections) with exactly k fixed points.

Example 1.12. For $n = 3$: there are $3^3 = 27$ total functions. Functions with exactly 0 fixed points: $\binom{3}{0} \cdot 2^3 = 8$. With exactly 1: $\binom{3}{1} \cdot 2^2 = 12$. With exactly 2: $\binom{3}{2} \cdot 2^1 = 6$. With exactly 3 (the identity): $\binom{3}{3} \cdot 2^0 = 1$. Total: $8 + 12 + 6 + 1 = 27$. Check.

□

1.6 Transient Length, Cycle Length, and Rho Length

For a point x in a DDS (X, f), we define:

- **Transient length** $\tau(x)$: the smallest $t \geq 0$ such that $f^t(x)$ is periodic.
- **Cycle length** $\lambda(x)$: the period of the periodic point $f^{\tau(x)}(x)$; equivalently, the smallest $k \geq 1$ such that $f^{\tau(x)+k}(x) = f^{\tau(x)}(x)$.

- **Rho length** $\rho(x) = \tau(x) + \lambda(x)$: the total number of distinct points in the orbit of x.

Note that $\rho(x) = |\mathcal{O}(x)|$, the number of distinct elements visited by the orbit.

Algorithm (Floyd's cycle detection). Given only the ability to evaluate f, one can determine $\tau(x)$ and $\lambda(x)$ using $O(1)$ memory and $O(\tau+\lambda)$ evaluations of f, via Floyd's "tortoise and hare" algorithm. This algorithm, discovered independently by Floyd and others in the 1960s, uses two pointers moving at different speeds; it is described in detail in Knuth's *The Art of Computer Programming* (Vol. 2, §3.1). Floyd's cycle detection is central to Pollard's rho factoring algorithm, which derives its name from the ρ-shaped orbits of iterated maps.

1.7 Product Systems

Given two DDS (X_1, f_1) and (X_2, f_2), the **product system** is $(X_1 \times X_2, f_1 \times f_2)$ where $(f_1 \times f_2)(x_1, x_2) = (f_1(x_1), f_2(x_2))$. The orbit of (x_1, x_2) in the product is determined componentwise: the cycle length of (x_1, x_2) is $\mathrm{lcm}(\lambda(x_1), \lambda(x_2))$ and the transient length is $\max(\tau(x_1), \tau(x_2))$.

More generally, one can consider *coupled* product systems where the components interact:

$$F(x_1, x_2) = (g_1(x_1, x_2), g_2(x_1, x_2)).$$

These are no longer decomposable and can exhibit qualitatively richer dynamics.

A particularly important coupled product system is the **(f, x) framework** studied in subsequent chapters: the state space is $F \times X$ where $F = X^X$ is the set of all maps on X, and the transition is

$$(f, x) \longmapsto (\varphi(f, x),\ f(x))$$

where $\varphi : F \times X \to F$ is a "meta-rule" that updates the function itself. When φ is the identity (i.e., $\varphi(f, x) = f$ for all f, x), this reduces to ordinary iteration of f. When φ is nontrivial, the system rewrites its own transition rule at each step–a form of self-modification. This product system is itself a DDS on the set $F \times X$ of size $n^n \cdot n = n^{n+1}$, and all the theory of this chapter applies to it. However, the structure of its functional graph can be far more complex than that of a single map f.

1.8 Random Maps

Perhaps the most remarkable aspect of iterated maps on finite sets is what happens when f is chosen uniformly at random from all n^n functions $X \to X$. The connection to the *birthday problem* is immediate: iterating a random map is analogous to drawing values from $\{0, \ldots, n-1\}$ where collisions (revisiting a state) are governed by the same probabilistic mechanism.

The foundational results are due to Harris (1960), Mutafchiev (1988), and especially the landmark paper of Flajolet and Odlyzko (1990), who used analytic combinatorics (singularity analysis of generating functions) to derive precise asymptotics for virtually all parameters of random functional graphs.

Theorem 1.13 (Flajolet & Odlyzko, 1990)**.** *Let $f : X \to X$ be a function chosen uniformly at random from all n^n functions, and let $x \in X$ be a uniformly random starting point. Then:*

- The expected tail length is $E[\tau] \sim \sqrt{\pi n/8} \approx 0.6267\sqrt{n}$. - The expected cycle length is $E[\lambda] \sim \sqrt{\pi n/8} \approx 0.6267\sqrt{n}$. - The expected rho length is $E[\rho] \sim \sqrt{\pi n/2} \approx 1.2533\sqrt{n}$.

More precisely, Flajolet and Odlyzko prove that if T_n is the tail length of a random point in a random map of size n, then

$$E[T_n] = \sqrt{\frac{\pi n}{8}} \left(1 + O\left(\frac{1}{\sqrt{n}}\right)\right).$$

The cycle length has the same asymptotic expectation, $E[\Lambda_n] \sim \sqrt{\pi n/8}$, but note that this is a different random variable. The rho length $\rho = \tau + \lambda$ has expectation $\sim \sqrt{\pi n/2}$.

Birthday paradox connection. *The expected rho length $\sqrt{\pi n/2}$ is exactly the birthday paradox threshold: the expected number of independent uniform draws from $\{1, \ldots, n\}$ before a collision is $\sqrt{\pi n/2}$. This is not a coincidence. The orbit of a random map behaves, until it first revisits a state, as a sequence of independent uniform draws (since f was chosen randomly and the orbit has not yet "explored" any previously-seen value of f). The first collision ends the tail and initiates the cycle.*

Further statistics. *Flajolet and Odlyzko also establish: - The expected number of connected components of a random functional graph is $\sim \frac{1}{2} \ln n$. - The expected number of cyclic points (points lying on a cycle) is $\sim \sqrt{\pi n/2}$. - The expected size of the largest tree is $\sim c \cdot n$ for an explicit constant c.*

These results are obtained by encoding functional graphs as labeled combinatorial structures, writing down their exponential generating functions, and extracting asymptotics via transfer theorems from analytic combinatorics.

1.9 Collatz-like Maps on Finite Rings

The Collatz map on $\mathbb{N}$ (send x to $x/2$ if even, $3x + 1$ if odd) can be studied modulo n to yield a finite DDS. Define $f : \mathbb{Z}/n\mathbb{Z} \to \mathbb{Z}/n\mathbb{Z}$ by:

$$f(x) = \begin{cases} x/2 \pmod{n} \& \text{if } x \text{ is even}, \\ 3x + 1 \pmod{n} \& \text{if } x \text{ is odd}. \end{cases}$$

Here "even" and "odd" refer to the residue class of x, and $x/2$ mod n requires that n be odd (so that 2 is invertible mod n), or a suitable convention when n is even. These finite Collatz-like maps provide test beds for studying the interaction of arithmetic structure with dynamical structure.

Example 1.14. Consider the Collatz map modulo $n = 7$. Since 7 is odd, $2^{-1} \equiv 4 \pmod 7$, so $x/2 \equiv 4x \pmod 7$ when x is even.

x	0	1	2	3	4	5	6
parity	E	O	E	O	E	O	E
$f(x)$	0	4	1	3	2	2	3

Orbits: - $0 \to 0$. Fixed point. - $1 \to 4 \to 2 \to 1$. Cycle of length 3. - $3 \to 3$. Fixed point. - $5 \to 2 \to 1 \to 4 \to 2$. Transient 1, cycle $\{1, 2, 4\}$ of length 3. - $6 \to 3$. Transient 1, cycle length 1.

The functional graph has three components: $\{0\}$, $\{3, 6\}$ (with 6 a tail node), and $\{1, 2, 4, 5\}$ (cycle $\{1, 2, 4\}$ with 5 as a tail node).

1.10 Connection to the $(f, x) \to (\varphi(f, x), f(x))$ Framework

The theory of this chapter–orbits, cycles, transients, functional graphs, the ρ-structure theorem, and random map statistics–applies directly to the product system $(f, x) \mapsto (\varphi(f, x), f(x))$ on $F \times X$. This system is simply a DDS on the larger state space $F \times X$, with $|F \times X| = n^{n+1}$.

However, the structure of the functional graph depends sensitively on the meta-rule φ:

- **Identity meta-rule** ($\varphi(f, x) = f$): The F-component is constant along each orbit; the system decomposes into $|F| = n^n$ independent copies of the DDS (X, f), one for each f. The functional graph is a disjoint union of functional graphs of individual maps.

- **Non-trivial meta-rules**: The function f mutates along the orbit. The orbit in the full space $F \times X$ can visit many different functions before cycling. This breaks the product structure and can produce dramatically longer transients, larger cycles, and richer graph topology than any single map on X.

The random map statistics of Section 1.8 provide a baseline: if φ and the initial state are "generic" in some sense, the expected transient and cycle lengths on $F \times X$ should scale as $\sqrt{\pi \cdot n^{n+1}/2}$, which grows super-exponentially in n. In practice, structured meta-rules produce dynamics far from this generic case, and understanding which meta-rules produce which dynamical signatures is a central question.

1.11 Summary

The key takeaways of this chapter are:

1. A DDS on a finite set is determined by a single function $f : X \to X$.
2. Every orbit is eventually periodic (Proposition 1.4).
3. The functional graph has a canonical decomposition: each connected component is a cycle with trees hanging off it (Theorem 1.6).
4. There are n^n functions on an n-element set, $n!$ permutations, and $\binom{n}{k}(n-1)^{n-k}$ functions with exactly k fixed points.
5. For random maps, the expected tail and cycle lengths both scale as $\sqrt{\pi n/8}$, and the expected rho length scales as $\sqrt{\pi n/2}$, intimately connected to the birthday paradox.
6. All of this applies to the product system $(f, x) \mapsto (\varphi(f, x), f(x))$ on $F \times X$, which is itself a DDS on a larger state space.

1.12 References

- Flajolet, P. and Odlyzko, A. M. (1990). "Random Mapping Statistics." *Advances in Cryptology – EUROCRYPT '89*, Lecture Notes in Computer Science, vol. 434, pp. 329–354. Springer. The foundational paper on the asymptotic analysis of random functional graphs using analytic combinatorics.

- Harris, B. (1960). "Probability Distributions Related to Random Mappings." *Annals of Mathematical Statistics*, 31(4), pp. 1045–1062. Early probabilistic results on random mappings, including component counts and tree sizes.

- Mutafchiev, L. (1988). "Large Trees in Random Mappings." *European Journal of Combinatorics*, 9(5), pp. 493–504. Asymptotic results on the largest tree component in random functional graphs.

- Flajolet, P. and Sedgewick, R. (2009). *Analytic Combinatorics*. Cambridge University Press. Chapters II and VIII provide the general framework (symbolic method, singularity analysis) underlying the random mapping results.

- Stanley, R. P. (2012). *Enumerative Combinatorics*, Volume 1, 2nd edition. Cambridge University Press. Standard reference for counting functions, permutations, and related combinatorial objects.

- Knuth, D. E. (1998). *The Art of Computer Programming*, Volume 2: *Seminumerical Algorithms*, 3rd edition. Addison-Wesley. Section 3.1 discusses random number generation via iterated maps; Exercise 3.1-12 treats the birthday paradox connection.

- Pollard, J. M. (1975). "A Monte Carlo Method for Factorization." *BIT Numerical Mathematics*, 15(3), pp. 331–334. Introduces the rho factorization algorithm, a celebrated application of cycle detection in iterated maps.

1.13 Recommended Reading

For readers new to discrete dynamical systems on finite sets:

- **Start here**: Chapter II of Flajolet & Sedgewick's *Analytic Combinatorics* (2009) provides a modern, self-contained treatment of labeled structures including functional graphs. Freely available at https://algo.inria.fr/flajolet/Publications/book.pdf.

- **Classical reference**: Knuth's *TAOCP* Volume 2, Section 3.1, offers a programmer's perspective on random number generators as iterated maps, with exercises on cycle detection and the birthday paradox.

- **For probabilists**: The original Flajolet & Odlyzko (1990) paper is highly readable and derives the $\sqrt{n}$ scaling laws with full proofs via generating functions.

- **Deeper exploration**: Readers interested in the combinatorics of permutations specifically should consult Stanley's *Enumerative Combinatorics* Volume 1, Chapter 1.

For connections to cryptography and hash functions:

- Menezes, van Oorschot, and Vanstone, *Handbook of Applied Cryptography* (1996), Chapter 9, discusses cycle detection in the context of discrete logarithm and factoring algorithms.

Chapter 2

Symbolic Dynamics

2.1 Introduction and Motivation

Symbolic dynamics studies sequences of symbols drawn from a finite alphabet, together with the shift operation that slides the sequence one position to the left. Despite the apparent simplicity of this setup, the resulting theory is remarkably rich: it provides a combinatorial framework for studying general dynamical systems, connects to automata theory and coding, and supplies computable invariants—most notably topological entropy—that distinguish qualitatively different types of dynamical behavior.

The subject originates with Hadamard (1898), who used symbolic sequences to study geodesics on surfaces of negative curvature, and was placed on firm foundations by Morse and Hedlund (1938). The modern treatment, emphasizing the interplay between combinatorics, topology, and linear algebra, is developed comprehensively in Lind and Marcus (2021).

This chapter introduces the core objects of symbolic dynamics—shift spaces, subshifts of finite type, sofic shifts—and their principal invariants. We close with a discussion of why these ideas are relevant to modeling the iteration of language model outputs.

2.2 The Full Shift

2.2.1 2.2.1 Definition

Let Σ be a finite set with $|\Sigma| \geq 2$, called the **alphabet**. Its elements are called **symbols** or **letters**.

Definition 2.1 (Full shift)**.** The **full shift** on Σ is the set of all bi-infinite sequences of symbols:

$$\Sigma^{\mathbb{Z}} = \{x = (\ldots, x_{-1}, x_0, x_1, \ldots) : x_i \in \Sigma \text{ for all } i \in \mathbb{Z}\}.$$

When we restrict to one-sided (forward) sequences, we write:

$$\Sigma^{\mathbb{N}} = \{x = (x_0, x_1, x_2, \ldots) : x_i \in \Sigma \text{ for all } i \geq 0\}.$$

We will work primarily with the two-sided case $\Sigma^{\mathbb{Z}}$, noting differences with $\Sigma^{\mathbb{N}}$ where they arise (see Section 2.9).

2.2.2 2.2.2 The Product Topology

We equip Σ with the discrete topology and $\Sigma^{\mathbb{Z}}$ with the product topology. This is the coarsest topology making every coordinate projection $\pi_i : \Sigma^{\mathbb{Z}} \to \Sigma$ continuous.

A basis of open sets is given by **cylinder sets**: for indices $i_1 < i_2 < \cdots < i_k$ and symbols $a_1, \ldots, a_k \in \Sigma$,

$$C_{i_1,\ldots,i_k}(a_1,\ldots,a_k) = \{x \in \Sigma^{\mathbb{Z}} : x_{i_j} = a_j \text{ for } j = 1,\ldots,k\}.$$

This topology is metrizable. A standard compatible metric is:

$$d(x,y) = \sum_{i=-\infty}^{\infty} \frac{\delta(x_i, y_i)}{2^{|i|}},$$

where $\delta(a,b) = 0$ if $a = b$ and $\delta(a,b) = 1$ if $a \neq b$. Intuitively, two sequences are close if they agree on a long central block.

Proposition 2.2. *$\Sigma^{\mathbb{Z}}$ with the product topology is compact, metrizable, and totally disconnected (i.e., it is a Cantor set when $|\Sigma| \geq 2$).*

Proof sketch. Compactness follows from Tychonoff's theorem, since Σ is finite (hence compact). Total disconnectedness follows because cylinder sets are clopen.

2.2.3 2.2.3 The Shift Map

Definition 2.3 (Shift map)**.** The **shift map** $\sigma : \Sigma^{\mathbb{Z}} \to \Sigma^{\mathbb{Z}}$ is defined by

$$(\sigma(x))_i = x_{i+1}.$$

That is, σ shifts every entry one position to the left. The shift map is a homeomorphism of $\Sigma^{\mathbb{Z}}$ onto itself (its inverse shifts to the right).

The pair $(\Sigma^{\mathbb{Z}}, \sigma)$ is a topological dynamical system: a continuous map acting on a compact metrizable space. This is the **full shift dynamical system**.

2.3 Subshifts

2.3.1 2.3.1 Definition via Topology

Definition 2.4 (Subshift)**.** A **subshift** (or **shift space**) is a subset $X \subseteq \Sigma^{\mathbb{Z}}$ that is:

1. **Closed** in the product topology, and 2. **Shift-invariant**: $\sigma(X) = X$.

A subshift inherits the structure of a topological dynamical system $(X, \sigma|_X)$.

2.3.2 2.3.2 Forbidden Words Characterization

A **word** (or **block**) of length n over Σ is an element $w = w_1 w_2 \cdots w_n \in \Sigma^n$. We say that a word w **appears** in a sequence x if there exists i such that $x_i x_{i+1} \cdots x_{i+n-1} = w$.

Definition 2.5 (Language)**.** The **language** of a subshift X, denoted $\mathcal{B}(X)$, is the set of all finite words appearing in some sequence in X. The set of words of length n in the language is denoted $\mathcal{B}_n(X)$.

Given a collection $\mathcal{F} \subseteq \bigcup_{n\geq 1} \Sigma^n$ of **forbidden words**, define:

$$X_{\mathcal{F}} = \{x \in \Sigma^{\mathbb{Z}} : \text{no word from } \mathcal{F} \text{ appears in } x\}.$$

Theorem 2.6 (Characterization of subshifts)**.** *A subset $X \subseteq \Sigma^{\mathbb{Z}}$ is a subshift if and only if $X = X_{\mathcal{F}}$ for some set $\mathcal{F}$ of forbidden words.*

Proof. ($\Leftarrow$) For each forbidden word w, the set of sequences containing w is open (it is a union of cylinder sets), so its complement is closed. Then $X_{\mathcal{F}}$ is an intersection of closed sets, hence closed. Shift-invariance is immediate since a shifted sequence contains the same set of words.

($\Rightarrow$) Given a subshift X, take $\mathcal{F} = \{w : w \text{ does not appear in any } x \in X\}$. Then $X_{\mathcal{F}} = X$, which can be verified by showing that any sequence all of whose finite subwords appear in X must itself lie in X (by compactness). $\square$

2.4 Subshifts of Finite Type (SFTs)

2.4.1 2.4.1 Definition

Definition 2.7 (SFT)**.** A subshift X is a **subshift of finite type** (SFT) if $X = X_{\mathcal{F}}$ for some *finite* set $\mathcal{F}$ of forbidden words.

If all forbidden words in $\mathcal{F}$ have length at most $N + 1$, we say X is an N**-step SFT**. A 1-step SFT is determined by which pairs of consecutive symbols are allowed.

Proposition 2.8 (Reduction to 1-step)**.** *Every N-step SFT is conjugate (topologically isomorphic as a dynamical system) to a 1-step SFT, via a standard recoding that uses words of length N as symbols in a new alphabet.*

*This recoding is called the **higher block presentation** and is a key technical tool.*

2.4.2 2.4.2 Edge Shifts and Adjacency Matrices

A 1-step SFT is naturally described by a directed graph (digraph).

Definition 2.9 (Edge shift)**.** Let $G = (V, E)$ be a directed graph with vertex set V and edge set E, where each edge $e \in E$ has an initial vertex $\mathbf{i}(e)$ and terminal vertex $\mathbf{t}(e)$. The **edge shift** X_G is the subshift:

$$X_G = \{(\ldots, e_{-1}, e_0, e_1, \ldots) \in E^{\mathbb{Z}} : \mathbf{t}(e_i) = \mathbf{i}(e_{i+1}) \text{ for all } i\}.$$

That is, the sequences of edges forming bi-infinite walks on G.

Theorem 2.10. *A subshift is an SFT if and only if it is conjugate to an edge shift.*

Definition 2.11 (Adjacency matrix)**.** The **adjacency matrix** $A = A_G$ of a directed graph G with vertices $\{v_1, \ldots, v_k\}$ is the $k \times k$ matrix where A_{ij} is the number of edges from v_i to v_j.

The adjacency matrix encodes the combinatorics of the SFT. In particular:

Proposition 2.12. *The number of allowed words of length $n+1$ in the edge shift X_G equals the sum of all entries of A^n, i.e., $|\mathcal{B}_{n+1}(X_G)| = \mathbf{1}^T A^n \mathbf{1}$, where $\mathbf{1}$ is the all-ones vector of appropriate dimension.*

More precisely, $(A^n)_{ij}$ counts the number of walks of length n from vertex v_i to vertex v_j.

2.4.3 2.4.3 Example: The Golden Mean Shift

Example 2.13 (Golden mean shift)**.** Let $\Sigma = \{0, 1\}$ and forbid the word 11. The resulting SFT is called the **golden mean shift**, denoted X_{gm}. It consists of all binary sequences with no two consecutive 1s.

The graph G has two vertices (corresponding to states "last symbol was 0" and "last symbol was 1") and three edges:

- $0 \to 0$ (emit 0, stay in state 0), - $0 \to 1$ (emit 1, go to state 1), - $1 \to 0$ (emit 0, go to state 0).

The adjacency matrix is:

$$A = \begin{pmatrix} 1\&1 \\ 1\&0 \end{pmatrix}.$$

The eigenvalues of A are $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$ (the golden ratio) and $\frac{1-\sqrt{5}}{2} \approx -0.618$. As we will see, the topological entropy is $h(X_{\text{gm}}) = \log \varphi$.

2.5 Sofic Shifts

2.5.1 2.5.1 Sliding Block Codes and Factor Maps

Definition 2.14 (Sliding block code)**.** Let $X \subseteq \Sigma^{\mathbb{Z}}$ and $Y \subseteq \Delta^{\mathbb{Z}}$ be subshifts. A **sliding block code** (or simply a **code**) from X to Y is a continuous, shift-commuting map $\phi : X \to Y$. The Curtis–Hedlund–Lyndon theorem states that every such map is determined by a local rule: there exist integers $m \leq a$ (the **memory** and **anticipation**) and a block map $\Phi : \Sigma^{a-m+1} \to \Delta$ such that

$$(\phi(x))_i = \Phi(x_{i+m}, x_{i+m+1}, \ldots, x_{i+a}).$$

If ϕ is surjective, we call Y a **factor** of X, and ϕ a **factor map**. If ϕ is a bijection (hence a homeomorphism), we call it a **conjugacy**, and X and Y are **conjugate**.

2.5.2 2.5.2 Definition of Sofic Shifts

Definition 2.15 (Sofic shift)**.** A subshift Y is **sofic** if it is a factor of some SFT. Equivalently (Fischer, 1975), Y is sofic if and only if it can be presented as the set of label sequences of a **labeled graph** $(G, \mathcal{L})$, where G is a finite directed graph and $\mathcal{L} : E \to \Delta$ is a labeling of the edges.

This equivalence connects sofic shifts to the theory of finite automata: a sofic shift is precisely the set of bi-infinite sequences accepted by a finite-state automaton.

2.5.3 2.5.3 Example: The Even Shift

Example 2.16 (Even shift)**.** The **even shift** X_{even} over $\Sigma = \{0, 1\}$ is defined by the rule: between any two consecutive 0s, the number of 1s must be even (including zero). That is, the forbidden patterns are $0\,1^{2k+1}\,0$ for all $k \geq 0$.

Claim: The even shift is sofic but not an SFT.

Sofic: Consider the labeled graph with two vertices A, B and edges:

- $A \xrightarrow{0} A$ - $A \xrightarrow{1} B$ - $B \xrightarrow{1} A$

The label sequences of bi-infinite walks on this graph are exactly X_{even}. After seeing a 0, we are in state A. Each 1 toggles between A and B, and a 0 is only allowed from state A (i.e., after an even number of 1s). Hence X_{even} is sofic.

Not SFT: Suppose for contradiction that $X_{\text{even}} = X_{\mathcal{F}}$ for a finite set $\mathcal{F}$ of forbidden words, with all words in $\mathcal{F}$ having length at most N. Consider the word $w = 0\,1^{2N+1}\,0$. This word is forbidden in X_{even} (an odd number of 1s between two 0s). However, every subword of w of length $\leq N$ is either a block of 1s (which is allowed) or contains at most one 0, and any block of the form $0\,1^j$ or $1^j\,0$ with $j < N$ appears in allowed sequences (e.g., $\ldots 0\,1^{2j}\,0 \ldots$ contains $0\,1^j$ as a subword). Therefore no subword of length $\leq N$ of w belongs to $\mathcal{F}$, contradicting the assumption that $\mathcal{F}$ forbids w. $\square$

2.6 The Hierarchy of Shift Spaces

The classes of shift spaces form a strict hierarchy:

$$\text{SFT} \subsetneq \text{Sofic} \subsetneq \text{All subshifts}.$$

We have already shown the first strict inclusion (the even shift is sofic but not SFT). For the second:

Example 2.17 (A non-sofic subshift)**.** The **context-free shift** over $\Sigma = \{0, 1\}$ defined by forbidding $0^n 1^n$ for all $n \geq 1$ is not sofic. More directly, consider the subshift X over $\{0, 1\}$ where the allowed sequences are those in which the word $10^n 1$ appears only when n is a prime number. This subshift is not sofic because the set of primes cannot be recognized by a finite automaton.

A cleaner and classical example: the **Dyck shift** (parenthesis matching sequences) is an example of a coded shift that is not sofic.

The following table summarizes the hierarchy with canonical examples:

Class	Example	Characterization
Full shift	$\{0, 1\}^{\mathbb{Z}}$	No forbidden words
SFT	Golden mean shift	Finitely many forbidden words
Sofic	Even shift	Factor of an SFT; finite automaton
General subshift	(various)	Possibly infinitely many forbidden words

2.7 Topological Entropy

2.7.1 2.7.1 Definition

Definition 2.18 (Topological entropy)**.** Let X be a subshift and $\mathcal{B}_n(X)$ the set of allowed words of length n in X. The **topological entropy** of X is:

$$h(X) = \lim_{n\to\infty} \frac{1}{n} \log |\mathcal{B}_n(X)|.$$

The limit exists because $\log |\mathcal{B}_n(X)|$ is subadditive (since $|\mathcal{B}_{m+n}(X)| \leq |\mathcal{B}_m(X)| \cdot |\mathcal{B}_n(X)|$ by the concatenation bound), and one applies the Fekete subadditivity lemma.

The entropy measures the exponential growth rate of the number of allowed words. It is a topological conjugacy invariant: conjugate subshifts have the same entropy.

Convention: Unless otherwise stated, logarithms are natural (base e). Some authors use $\log_2$, which differs by a constant factor.

2.7.2 2.7.2 Entropy of SFTs via the Perron Eigenvalue

Theorem 2.19 (Entropy of an SFT)**.** *Let X_G be the edge shift of a directed graph G with adjacency matrix A. If A is irreducible (equivalently, G is strongly connected), then:*

$$h(X_G) = \log \lambda_A,$$

*where λ_A is the **Perron eigenvalue** (the largest real eigenvalue) of A.*

This is a consequence of the Perron–Frobenius theorem, which guarantees that an irreducible non-negative matrix has a unique largest real eigenvalue $\lambda_A > 0$, and that the entries of A^n grow asymptotically as λ_A^n.

For reducible A, the entropy is log *of the spectral radius $\rho(A)$, which equals the maximum of the Perron eigenvalues of the irreducible components.*

2.7.3 2.7.3 Worked Examples

Example 2.20 (Full shift)**.** The full shift $\{0, 1, \ldots, k-1\}^{\mathbb{Z}}$ has $|\mathcal{B}_n| = k^n$, so:

$$h = \lim_{n\to\infty} \frac{1}{n} \log k^n = \log k.$$

The adjacency matrix is the $k \times k$ all-ones matrix (or, for the vertex shift, the $k \times k$ matrix of all 1s), which has Perron eigenvalue k, confirming $h = \log k$.

Example 2.21 (Golden mean shift)**.** Recall from Example 2.13 that the adjacency matrix is $A = \begin{pmatrix} 1\&1 \\ 1\&0 \end{pmatrix}$ with Perron eigenvalue $\varphi = \frac{1+\sqrt{5}}{2}$. Therefore:

$$h(X_{\text{gm}}) = \log \varphi \approx 0.4812.$$

We can verify combinatorially. Let $f(n) = |\mathcal{B}_n(X_{\text{gm}})|$. A word of length n with no two consecutive 1s can end in 0 or 1. If it ends in 0, the first $n-1$ symbols form any allowed word of length $n-1$. If it ends in 1, the $(n-1)$-th symbol must be 0, and the first $n-2$ symbols form any allowed word of length $n-2$. Thus:

$$f(n) = f(n-1) + f(n-2),$$

with $f(1) = 2$ (words: $0, 1$) and $f(2) = 3$ (words: $00, 01, 10$). This is a shifted Fibonacci recurrence, and $f(n) \sim C \cdot \varphi^n$, confirming $h = \log \varphi$.

Example 2.22 (Even shift)**.** The even shift is sofic, presented by the labeled graph in Example 2.16 with vertices A, B and edges: $A \to A$ (label 0), $A \to B$ (label 1), $B \to A$ (label 1). The adjacency matrix of this underlying graph is:

$$A = \begin{pmatrix} 1\&1 \\ 1\&0 \end{pmatrix}.$$

This is the same matrix as the golden mean shift. The Perron eigenvalue is $\varphi = \frac{1+\sqrt{5}}{2}$, so:

$$h(X_{\text{even}}) = \log \varphi.$$

The even shift and the golden mean shift have the same entropy despite being different subshifts. The key insight is that entropy depends only on the underlying graph structure of the Fischer cover (the minimal right-resolving presentation), not on the edge labels. Both shifts happen to have the same underlying graph, so they have the same entropy. This illustrates a general principle: different label assignments on the same graph yield sofic shifts with the same topological entropy but potentially different languages.

2.8 Invariant Measures and the Variational Principle

2.8.1 2.8.1 Shift-Invariant Measures

A **Borel probability measure** μ on a subshift X is **shift-invariant** if $\mu(\sigma^{-1}(B)) = \mu(B)$ for every Borel set B. The set of all shift-invariant probability measures on X is denoted $\mathcal{M}_\sigma(X)$. By the Krylov–Bogolyubov theorem, $\mathcal{M}_\sigma(X) \neq \emptyset$ for any subshift X, and this set is convex and compact in the weak-$*$ topology.

2.8.2 2.8.2 Measure-Theoretic Entropy

Definition 2.23 (Measure-theoretic entropy)**.** Given $\mu \in \mathcal{M}_\sigma(X)$, the **measure-theoretic (or Kolmogorov–Sinai) entropy** of μ is:

$$h_\mu(\sigma) = \lim_{n\to\infty} \frac{1}{n} H_\mu(\mathcal{P}^{(n)}),$$

where $\mathcal{P}^{(n)}$ is the partition of X into cylinder sets of length n (determined by coordinates $0, 1, \ldots, n-1$), and:

$$H_\mu(\mathcal{P}^{(n)}) = -\sum_{w \in \mathcal{B}_n(X)} \mu([w]) \log \mu([w]),$$

with $[w]$ denoting the cylinder set $\{x \in X : x_0 x_1 \cdots x_{n-1} = w\}$.

The measure-theoretic entropy quantifies the average rate of information production per step, as seen by the measure μ.

2.8.3 2.8.3 The Variational Principle

Theorem 2.24 (Variational Principle; Goodwyn 1969, Goodman 1971, Walters 1975)**.** *For any subshift X:*

$$h_{\text{top}}(X) = \sup_{\mu \in \mathcal{M}_\sigma(X)} h_\mu(\sigma).$$

Moreover, the supremum is attained: there exists a measure $\mu^* \in \mathcal{M}_\sigma(X)$ with $h_{\mu^*}(\sigma) = h_{\text{top}}(X)$. Such a measure is called a ***measure of maximal entropy*** *(MME).*

The variational principle is one of the deepest results connecting topological and measure-theoretic dynamics. It says that topological entropy—a purely combinatorial quantity defined by word counting—equals the supremum of a probabilistic quantity over all invariant measures.

Remark. The variational principle holds for any continuous map on a compact metrizable space, not just subshifts (see Walters, 1982, Chapter 8).

2.8.4 2.8.4 Markov Measures on SFTs

For SFTs, the natural class of invariant measures consists of **Markov measures**.

Definition 2.25 (Markov measure)**.** Let X_G be a 1-step SFT with adjacency matrix A, and let P be a stochastic matrix compatible with A (i.e., $P_{ij} > 0$ only if $A_{ij} > 0$). Let π be a stationary distribution for P (i.e., $\pi P = \pi$). The **Markov measure** $\mu_{\pi,P}$ is the shift-invariant measure defined on cylinder sets by:

$$\mu_{\pi,P}([i_0\, i_1 \cdots i_{n-1}]) = \pi_{i_0} P_{i_0 i_1} P_{i_1 i_2} \cdots P_{i_{n-2} i_{n-1}}.$$

The measure-theoretic entropy of a Markov measure is:

$$h_{\mu_{\pi,P}}(\sigma) = -\sum_{i,j} \pi_i P_{ij} \log P_{ij}.$$

Theorem 2.26 (Parry, 1964)**.** *For an irreducible SFT with adjacency matrix A and Perron eigenvalue λ, the unique measure of maximal entropy is the* ***Parry measure****, the Markov measure with transition probabilities:*

$$P_{ij} = \frac{A_{ij}\, r_j}{\lambda\, r_i},$$

where $r = (r_1, \ldots, r_k)$ is the right Perron eigenvector of A (i.e., $Ar = \lambda r$, $r_i > 0$). The stationary distribution is $\pi_i = l_i r_i$, where l is the left Perron eigenvector normalized so that $\sum_i l_i r_i = 1$.

Example 2.27 (Parry measure for the golden mean shift)**.** The adjacency matrix is $A = \begin{pmatrix} 1\&1 \\ 1\&0 \end{pmatrix}$ with Perron eigenvalue $\lambda = \varphi = \frac{1+\sqrt{5}}{2}$.

The right eigenvector satisfies $Ar = \varphi r$:

$$r_1 + r_2 = \varphi r_1, \quad r_1 = \varphi r_2.$$

So $r_2 = r_1/\varphi$. Choosing $r_1 = \varphi$, $r_2 = 1$, the transition matrix is:

$$P = \begin{pmatrix} 1/\varphi \& 1/\varphi^2 \\ 1\&0 \end{pmatrix} = \begin{pmatrix} \varphi - 1 \& 2 - \varphi \\ 1\&0 \end{pmatrix} \approx \begin{pmatrix} 0.618\&0.382 \\ 1\&0 \end{pmatrix}.$$

The entropy of this Parry measure equals $\log \varphi$, confirming the variational principle.

2.9 One-Sided vs. Two-Sided Shifts

The **one-sided full shift** $\Sigma^{\mathbb{N}}$ consists of sequences indexed by $\mathbb{N} = \{0, 1, 2, \ldots\}$. The shift map $\sigma : \Sigma^{\mathbb{N}} \to \Sigma^{\mathbb{N}}$ defined by $(\sigma(x))_i = x_{i+1}$ is surjective but **not injective** (information at position 0 is lost). Key differences from the two-sided case:

Property	Two-sided $\Sigma^{\mathbb{Z}}$	One-sided $\Sigma^{\mathbb{N}}$
Shift map	Homeomorphism (bijective)	Surjective, not injective
Invertibility	σ^{-1} exists	σ^{-1} does not exist
Topological entropy	Same formula	Same formula
Natural measures	Parry measure (same)	Parry measure (same)
Conjugacy theory	Richer (Williams' theorem)	Factor maps are easier

For most entropy and language questions, the one-sided and two-sided versions give the same answers: $\mathcal{B}_n(X)$ is the same in both cases, so h_{top} is identical.

The one-sided shift is often more natural for modeling sequential processes (such as language generation), where the future is being produced one token at a time and the past is fixed.

2.10 Application: Modeling LLM Output Iteration

We close with a remark on why symbolic dynamics provides a natural framework for analyzing the iterative behavior of large language models (LLMs).

Consider an LLM with a finite token vocabulary Σ (a typical vocabulary has $|\Sigma| \approx 32{,}000$ to 100,000 tokens). When the LLM generates text autoregressively, it produces a sequence $x_0, x_1, x_2, \ldots$ of tokens, where each x_{t+1} depends on the preceding context $(x_0, \ldots, x_t)$.

Now consider the following iterative scheme: take the LLM's output, feed it back as input, and generate again. The set of sequences that can arise as outputs of this iterated process is, in principle, a subset of $\Sigma^{\mathbb{N}}$ that is:

- **Closed** (limits of realizable sequences are realizable, under reasonable continuity assumptions on the model), and
- **Shift-invariant** (if a sequence can be generated starting from some context, then its shift can be generated starting from a shifted context).

This makes the set of output sequences a **one-sided subshift** (or an approximation thereof). The structure of this subshift—whether it is of finite type, sofic, or more complex—depends on the architecture and parameters of the model:

- A model with a finite context window of length N produces sequences where the constraint on the next token depends only on the preceding N tokens. This is exactly a **subshift of finite type** (an N-step SFT).
- A model with unbounded effective memory (e.g., via attention over the entire context) can, in principle, produce sofic or even non-sofic subshifts.

The **topological entropy** $h(X)$ of this subshift measures the exponential growth rate of the number of distinct output sequences of length n, and thus quantifies the "richness" or "diversity" of the model's dynamical behavior under iteration. A higher entropy means a greater variety of possible output trajectories; entropy zero means the model's iterated outputs are highly constrained (e.g., converging to periodic orbits or a small number of fixed points).

The **variational principle** then tells us that there exists an invariant measure that realizes this maximal complexity, providing a probabilistic characterization of the "most complex" mode of the model's behavior.

This perspective transforms questions about LLM behavior under iteration—stability, periodicity, diversity of outputs—into the well-developed language of symbolic dynamics.

2.11 Summary

Concept	Definition	Key Property
Full shift $\Sigma^{\mathbb{Z}}$	All bi-infinite sequences over Σ	Compact, metrizable, Cantor set
Shift map σ	$(\sigma(x))_i = x_{i+1}$	Homeomorphism (two-sided)
Subshift	Closed, shift-invariant $X \subseteq \Sigma^{\mathbb{Z}}$	$= X_{\mathcal{F}}$ for forbidden words $\mathcal{F}$
SFT	$\mathcal{F}$ is finite	Edge shifts; adjacency matrix
Sofic shift	Factor of an SFT	Labeled graphs; finite automata
Entropy $h(X)$	$\lim \frac{1}{n} \log \lvert\mathcal{B}_n(X)\rvert$	Conjugacy invariant
Variational principle	$h_{\text{top}} = \sup_\mu h_\mu$	Bridges topology and measure theory

2.12 References

- **Lind, D. and Marcus, B.** (2021). *An Introduction to Symbolic Dynamics and Coding*, 2nd ed. Cambridge University Press. The standard modern reference; covers SFTs, sofic shifts, entropy, and coding theory comprehensively.

- **Kitchens, B.** (1998). *Symbolic Dynamics: One-sided, Two-sided, and Countable State Markov Shifts*. Springer. Emphasizes the Markov chain perspective and connections to ergodic theory.

- **Morse, M. and Hedlund, G. A.** (1938). Symbolic dynamics. *American Journal of Mathematics*, 60(4), 815–866. The foundational paper that established symbolic dynamics as a field.

- **Walters, P.** (1982). *An Introduction to Ergodic Theory*. Springer. Excellent treatment of measure-theoretic entropy and the variational principle in the general (non-symbolic) setting.

- **Parry, W.** (1964). Intrinsic Markov chains. *Transactions of the American Mathematical Society*, 112(1), 55–66. Introduces the measure of maximal entropy for topological Markov chains (SFTs).

- **Fischer, R.** (1975). Sofic systems and graphs. *Monatshefte fur Mathematik*, 80, 179–186. Establishes the connection between sofic shifts and finite automata.

- **Weiss, B.** (1973). Subshifts of finite type and sofic systems. *Monatshefte fur Mathematik*, 77, 462–474. Introduces the term "sofic" and establishes basic properties.

2.13 Recommended Reading

For newcomers to symbolic dynamics:

- **Start here**: Lind & Marcus (2021), Chapters 1–4. The book is exceptionally clear and includes numerous exercises. An early draft is available online.

- **Prerequisite-light introduction**: Kitchens (1998) emphasizes the probabilistic (Markov chain) viewpoint and may be easier for readers with a probability background.

For deeper study of entropy:

- **Walters (1982)**, Chapters 4 and 8, covers the general theory of topological and measure-theoretic entropy beyond the symbolic setting. Essential for understanding the variational principle in full generality.

- **Downarowicz, T.** (2011). *Entropy in Dynamical Systems.* Cambridge University Press. A comprehensive treatment of all flavors of dynamical entropy.

For connections to automata theory and formal languages:

- **Hopcroft, Motwani, and Ullman** (2006). *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. The connection between sofic shifts and regular languages is immediate once you know both theories.

For applications to coding and information theory:

- **Cover, T. and Thomas, J.** (2006). *Elements of Information Theory*, 2nd ed. Chapters 4–5 cover entropy rate of stochastic processes, closely related to measure-theoretic entropy of Markov measures.

- **Marcus, B., Roth, R., and Siegel, P.** (1998). "Constrained systems and coding for recording channels." In *Handbook of Coding Theory*, Vol. II. Direct applications of symbolic dynamics to data storage.

Chapter 3

Sliding Block Codes and the Curtis-Hedlund-Lyndon Theorem

3.1 Introduction

The central objects of symbolic dynamics are shift spaces (subshifts) and the maps between them. In the previous chapters we defined shift spaces as closed, shift-invariant subsets of $\Sigma^{\mathbb{Z}}$ (or $\Sigma^{\mathbb{N}}$). The natural question is: **what are the "right" morphisms between these objects?**

The answer turns out to be remarkably clean. The Curtis-Hedlund-Lyndon theorem (1969) says that the structure-preserving maps between shift spaces are precisely the **sliding block codes**: maps defined by applying a fixed local rule uniformly across every position. This is one of the foundational results of symbolic dynamics, and it connects the topological viewpoint (continuity + shift-commutation) to the combinatorial viewpoint (local rules on finite windows).

This chapter develops the theory of sliding block codes, states and sketches the proof of the Curtis-Hedlund-Lyndon theorem, and explores consequences including factor maps, conjugacies, automorphisms, and the connection to cellular automata and autoregressive language models.

3.2 Sliding Block Codes: Definition

Let Σ and Γ be finite alphabets. Let $X \subseteq \Sigma^{\mathbb{Z}}$ and $Y \subseteq \Gamma^{\mathbb{Z}}$ be subshifts.

Definition 3.2.1 (Block map). A function $f : \Sigma^{m+a+1} \to \Gamma$, where $m, a \geq 0$, is called a **local rule** (or **block map**) with **memory** m and **anticipation** a. The quantity $r = \max(m, a)$ is called the **radius** of the block map. When $m = a = r$, the domain is Σ^{2r+1} and we say the block map has **symmetric radius** r.

Definition 3.2.2 (Sliding block code). Given a local rule $f : \Sigma^{m+a+1} \to \Gamma$ with memory m and anticipation a, the **sliding block code** (or **∞-block map**) induced by f is the map $\Phi : \Sigma^{\mathbb{Z}} \to \Gamma^{\mathbb{Z}}$ defined by

$$[\Phi(x)]_i = f(x_{i-m}, x_{i-m+1}, \ldots, x_i, \ldots, x_{i+a})$$

for all $i \in \mathbb{Z}$ and all $x \in \Sigma^{\mathbb{Z}}$.

In words: to compute the i-th coordinate of the output, we look at a window of $m + a + 1$ symbols centered (with possible asymmetry) at position i in the input, and apply the fixed local rule f.

If $\Phi(X) \subseteq Y$, we say Φ is a sliding block code **from** X **to** Y and write $\Phi : X \to Y$.

Remark 3.1. Every sliding block code with memory m and anticipation a can be **recoded** as one with symmetric radius $r = \max(m, a)$ by simply ignoring extra coordinates in the local rule. A more useful recoding trick (the **higher block presentation**, see Section 3.7) shows

that every sliding block code can be recoded as a **1-block map** (memory 0, anticipation 0) after passing to a higher-block shift.

3.3 Examples

3.3.1 Example 3.3.1: XOR map (2-block code)

Let $\Sigma = \Gamma = \{0, 1\}$. Define the local rule $f : \{0, 1\}^2 \to \{0, 1\}$ by

$$f(a, b) = a \oplus b$$

where $\oplus$ denotes addition mod 2 (XOR). This has memory $m = 0$ and anticipation $a = 1$ (equivalently, memory 1 and anticipation 0, depending on convention). The induced sliding block code $\Phi : \{0, 1\}^{\mathbb{Z}} \to \{0, 1\}^{\mathbb{Z}}$ is

$$[\Phi(x)]_i = x_i \oplus x_{i+1}.$$

Worked computation. Let $x = \ldots 0\,0\,1\,1\,0\,1\,0\,0 \ldots$ (with positions labeled $\ldots, -1, 0, 1, 2, 3, 4, 5, 6, \ldots$). Then:

i	0	1	2	3	4	5
x_i	0	1	1	0	1	0
x_{i+1}	1	1	0	1	0	0
$[\Phi(x)]_i$	1	0	1	1	1	0

So $\Phi(x) = \ldots 1\,0\,1\,1\,1\,0 \ldots$ in positions $0, \ldots, 5$.

Note that Φ is a 2-to-1 map on $\{0, 1\}^{\mathbb{Z}}$: for any output y, choosing $x_0 \in \{0, 1\}$ determines the rest of x uniquely. This map is also known as **elementary cellular automaton Rule 102** (see Section 3.10).

3.3.2 Example 3.3.2: Majority vote map (radius-1)

Let $\Sigma = \Gamma = \{0, 1\}$. Define $f : \{0, 1\}^3 \to \{0, 1\}$ by

$$f(a, b, c) = \text{majority}(a, b, c) = \begin{cases} 1\&\text{if } a + b + c \geq 2, \\ 0\&\text{otherwise.} \end{cases}$$

This has symmetric radius $r = 1$. The induced map is

$$[\Phi(x)]_i = \text{majority}(x_{i-1}, x_i, x_{i+1}).$$

Worked computation. Let $x = \ldots 0\,1\,1\,0\,0\,1\,1\,1 \ldots$ at positions $0, \ldots, 7$.

i	1	2	3	4	5	6
(x_{i-1}, x_i, x_{i+1})	$(0, 1, 1)$	$(1, 1, 0)$	$(1, 0, 0)$	$(0, 0, 1)$	$(0, 1, 1)$	$(1, 1, 1)$
$[\Phi(x)]_i$	1	1	0	0	1	1

This map acts as a "smoothing" or "noise reduction" operator: isolated 0s surrounded by 1s become 1, and vice versa. It is **not** invertible. As a cellular automaton, this is **Rule 232**.

3.3.3 Example 3.3.3: Higher-order block code and the golden mean shift

Let $X \subseteq \{0,1\}^{\mathbb{Z}}$ be the **golden mean shift**: the subshift defined by the forbidden word $\mathcal{F} = \{11\}$, so no two consecutive 1s appear. Let $\Gamma = \{a, b, c\}$ and define $f : \{0,1\}^2 \to \{a, b, c\}$ by

$$f(0,0) = a, \quad f(0,1) = b, \quad f(1,0) = c, \quad f(1,1) = (\text{never occurs in } X).$$

The induced map $\Phi : X \to \Gamma^{\mathbb{Z}}$ sends the golden mean shift to the subshift $Y \subseteq \{a, b, c\}^{\mathbb{Z}}$ whose allowed transitions are $a \to a$, $a \to b$, $b \to c$, $c \to a$, $c \to b$ (one can verify: after a b, the underlying pair was $(0,1)$, so the next symbol in X must be 0, giving either $(1,0) \mapsto c$). This is the **higher block presentation** of the golden mean shift; it converts a subshift defined by forbidden words into an **edge shift** (a 1-step shift of finite type).

3.4 Continuity and Shift-Commutation

We equip $\Sigma^{\mathbb{Z}}$ with the **product topology** (equivalently, the topology induced by the metric $d(x, y) = 2^{-\min\{|i| : x_i \neq y_i\}}$). In this topology, two sequences are close if they agree on a large window around the origin.

Proposition 3.4.1. Every sliding block code $\Phi : \Sigma^{\mathbb{Z}} \to \Gamma^{\mathbb{Z}}$ is:

1. **Continuous** in the product topology.
2. **Shift-commuting**: $\Phi \circ \sigma = \sigma \circ \Phi$, where σ is the left shift $[\sigma(x)]_i = x_{i+1}$.

Proof sketch. (1) Fix Φ with memory m and anticipation a. If x and y agree on positions $\{-N, \ldots, N\}$, then $\Phi(x)$ and $\Phi(y)$ agree on positions $\{-N+m, \ldots, N-a\}$. So agreement on a large window in the input implies agreement on a large window in the output, which is exactly continuity in the product topology.

(2) We compute directly:

$$[\Phi(\sigma(x))]_i = f((\sigma x)_{i-m}, \ldots, (\sigma x)_{i+a}) = f(x_{i-m+1}, \ldots, x_{i+a+1})$$

$$[\sigma(\Phi(x))]_i = [\Phi(x)]_{i+1} = f(x_{(i+1)-m}, \ldots, x_{(i+1)+a}) = f(x_{i-m+1}, \ldots, x_{i+a+1}).$$

These are equal for all i, so $\Phi \circ \sigma = \sigma \circ \Phi$.

The remarkable content of the Curtis-Hedlund-Lyndon theorem is that the converse holds.

3.5 The Curtis-Hedlund-Lyndon Theorem

Theorem 3.5.1 (Curtis-Hedlund-Lyndon, 1969). Let $X \subseteq \Sigma^{\mathbb{Z}}$ and $Y \subseteq \Gamma^{\mathbb{Z}}$ be subshifts, and let $\Phi : X \to Y$ be a map. Then Φ is a sliding block code if and only if:

1. Φ is **continuous** (with respect to the product topology), and
2. Φ **commutes with the shift**: $\Phi \circ \sigma = \sigma \circ \Phi$.

The forward direction is Proposition 3.4.1. The content is the reverse: every continuous, shift-commuting map is induced by a local rule.

Proof sketch of the reverse direction.

Assume $\Phi : X \to Y$ is continuous and shift-commuting. We must show that there exist $m, a \geq 0$ and a local rule $f : \Sigma^{m+a+1} \to \Gamma$ such that $[\Phi(x)]_i = f(x_{i-m}, \ldots, x_{i+a})$ for all $x \in X$ and all i.

Step 1: Reduce to the zeroth coordinate. By shift-commutation, it suffices to show that $[\Phi(x)]_0$ depends only on finitely many coordinates of x. Indeed, once we know that $[\Phi(x)]_0 = f(x_{-m}, \ldots, x_a)$ for some function f, then

$$[\Phi(x)]_i = [\sigma^{-i}(\Phi(x))]_0 = [\Phi(\sigma^{-i}(x))]_0 = f((\sigma^{-i}x)_{-m}, \ldots, (\sigma^{-i}x)_a) = f(x_{i-m}, \ldots, x_{i+a})$$

where we used shift-commutation in the second equality.

Step 2: Compactness argument. Consider the map $\pi_0 \circ \Phi : X \to \Gamma$, where π_0 projects onto the zeroth coordinate. This is a continuous function from X to the discrete space Γ. For each $\gamma \in \Gamma$, the preimage $(\pi_0 \circ \Phi)^{-1}(\gamma)$ is clopen (both open and closed) in X.

The sets of the form $C(i_1, \ldots, i_k; a_1, \ldots, a_k) = \{x \in X : x_{i_j} = a_j \text{ for all } j\}$ (cylinder sets) form a basis for the topology on X. Since $(\pi_0 \circ \Phi)^{-1}(\gamma)$ is open, it is a union of cylinder sets. Since $X \subseteq \Sigma^{\mathbb{Z}}$ is compact (as a closed subset of the compact space $\Sigma^{\mathbb{Z}}$, by Tychonoff's theorem), and $(\pi_0 \circ \Phi)^{-1}(\gamma)$ is closed (hence compact), it can be covered by **finitely many** cylinder sets.

Each cylinder set depends on finitely many coordinates. A finite union of such sets depends on finitely many coordinates. Since Γ is finite, taking the union over all γ, we find a finite set of coordinates $\{-m, -m+1, \ldots, a\}$ (after relabeling) such that $[\Phi(x)]_0$ depends only on $x_{-m}, \ldots, x_a$.

Step 3: Define the local rule. For each word $w = w_{-m} \cdots w_a \in \mathcal{B}_{m+a+1}(X)$ (the set of $(m+a+1)$-blocks appearing in X), pick any $x \in X$ with $x_{-m} \cdots x_a = w$ and define $f(w) = [\Phi(x)]_0$. By Step 2, this is well-defined (independent of the choice of x). Then Φ is the sliding block code induced by f.

Remark on the domain. The theorem requires the domain X to be a subshift (in particular, compact). This is essential: on non-compact subsets of $\Sigma^{\mathbb{Z}}$, there exist continuous shift-commuting maps that are not sliding block codes.

3.6 Why the Theorem Matters

The Curtis-Hedlund-Lyndon theorem says that the category of subshifts with continuous shift-commuting maps is the same as the category of subshifts with sliding block codes. This has several important consequences:

1. **Complete characterization of morphisms.** We do not need to search for exotic continuous shift-commuting maps; they are all given by local rules on finite windows. The theory is entirely combinatorial.

1. **Computability.** To specify a map between subshifts, it suffices to give a finite lookup table (the local rule). This makes the theory computationally tractable.

1. **Composition.** The composition of two sliding block codes is again a sliding block code (with radius at most the sum of the radii). This follows immediately from the topological characterization: a composition of continuous maps is continuous, and a composition of shift-commuting maps is shift-commuting.

1. **Foundation for classification.** It provides the correct notion of isomorphism (conjugacy) between subshifts, which underlies the entire classification theory of symbolic dynamics.

3.7 Factor Maps and Conjugacies

Definition 3.7.1. Let X and Y be subshifts.

- A **factor map** is a surjective sliding block code $\Phi : X \to Y$. We say Y is a **factor** of X, and X is an **extension** of Y.
- A **conjugacy** is a bijective sliding block code $\Phi : X \to Y$ whose inverse $\Phi^{-1} : Y \to X$ is also a sliding block code. Two subshifts are **conjugate** if there exists a conjugacy between them.

Remark 3.2. If Φ is a bijective sliding block code, then Φ^{-1} is automatically a sliding block code. This follows because Φ is a continuous bijection from the compact space X to the Hausdorff space Y, hence a homeomorphism, and Φ^{-1} commutes with the shift (since Φ does). By the Curtis-Hedlund-Lyndon theorem, Φ^{-1} is a sliding block code. So in the definition of conjugacy, it suffices to require bijectivity.

Conjugacy is the correct notion of **isomorphism** for subshifts. Two conjugate subshifts are "the same" dynamical system up to a re-encoding of symbols. Any dynamical invariant (topological entropy, zeta function, etc.) is preserved by conjugacy.

Example 3.7.2. Every subshift of finite type is conjugate to an edge shift (a 1-step SFT) via a higher block code. This is the standard recoding trick: if X is an M-step SFT over Σ, define the new alphabet $\tilde{\Sigma} = \mathcal{B}_M(X)$ (the set of allowed M-blocks), and the 1-block conjugacy sends each M-block to its corresponding symbol in $\tilde{\Sigma}$.

3.8 Endomorphisms and Automorphisms of the Full Shift

The set of all sliding block codes from a full shift $\Sigma^{\mathbb{Z}}$ to itself has rich algebraic structure.

Definition 3.8.1. Let Σ be a finite alphabet with $|\Sigma| \geq 2$.

- $\mathrm{End}(\Sigma^{\mathbb{Z}})$ denotes the set of all sliding block codes $\Phi : \Sigma^{\mathbb{Z}} \to \Sigma^{\mathbb{Z}}$. Under composition, this is a **monoid** (associative with identity, where the identity is the 0-block code $[\Phi(x)]_i = x_i$).
- $\mathrm{Aut}(\Sigma^{\mathbb{Z}})$ denotes the set of all **invertible** sliding block codes (conjugacies from $\Sigma^{\mathbb{Z}}$ to itself). Under composition, this is a **group**, called the **automorphism group** of the full shift.

Basic elements of Aut($\Sigma^{\mathbb{Z}}$):

- The **shift** σ itself (a 0-block code after re-indexing; it is the conjugacy induced by the identity local rule but with a coordinate offset).
- **Symbol permutations**: for any permutation $\pi : \Sigma \to \Sigma$, the 0-block code $[\Phi(x)]_i = \pi(x_i)$ is an automorphism.
- **Marker automorphisms** and more exotic constructions.

Theorem 3.8.2 (Hedlund, 1969). For $|\Sigma| \geq 2$, the group $\mathrm{Aut}(\Sigma^{\mathbb{Z}})$ is countably infinite. It contains:

- The shift σ (of infinite order),
- All finite symmetric groups S_n for $n \leq |\Sigma|$ (via symbol permutations),
- Free groups and free products,
- Every finite group as a subgroup (Ryan's theorem, 1972).

The structure of $\mathrm{Aut}(\Sigma^{\mathbb{Z}})$ remains deeply mysterious. For example, it is unknown whether $\mathrm{Aut}(\{0,1\}^{\mathbb{Z}})$ is generated by σ together with all involutions.

Example 3.8.3. For $\Sigma = \{0,1\}$, the map Φ defined by $[\Phi(x)]_i = x_i \oplus x_{i+1}$ from Example 3.3.1 is an endomorphism (it maps $\{0,1\}^{\mathbb{Z}}$ to itself) but **not** an automorphism, since it is 2-to-1.

The map Ψ defined by $[\Psi(x)]_i = 1 - x_i$ (symbol swap) is an automorphism of order 2.

3.9 One-Sided Sliding Block Codes (Causal Maps)

In many applications, we are interested in maps that depend only on the past and present, not the future.

Definition 3.9.1. A **one-sided sliding block code** (or **causal sliding block code**) is a sliding block code with anticipation $a = 0$. That is, $[\Phi(x)]_i = f(x_{i-m}, \ldots, x_i)$ for some local rule $f : \Sigma^{m+1} \to \Gamma$.

Equivalently, the output at position i depends only on the input at positions $\leq i$.

On the one-sided shift space $\Sigma^{\mathbb{N}}$ (sequences indexed by $\mathbb{N} = \{0, 1, 2, \ldots\}$), the Curtis-Hedlund-Lyndon theorem holds in the same form, and all sliding block codes $\Phi : X \to Y$ between one-sided subshifts $X \subseteq \Sigma^{\mathbb{N}}$, $Y \subseteq \Gamma^{\mathbb{N}}$ are necessarily causal: $[\Phi(x)]_i$ can depend on $x_0, \ldots, x_{i+a}$ for some fixed a, and by re-indexing this is equivalent to a causal map on a higher block presentation.

Remark 3.3. One-sided sliding block codes are strictly less expressive than two-sided ones. For example, the map $[\Phi(x)]_i = x_{i+1}$ (a "look-ahead" by one step) is a perfectly valid two-sided sliding block code but is not causal.

3.10 Connection to Cellular Automata

Definition 3.10.1. A **(one-dimensional) cellular automaton** (CA) is a sliding block code $\Phi : \Sigma^{\mathbb{Z}} \to \Sigma^{\mathbb{Z}}$ from the full shift to itself.

By the Curtis-Hedlund-Lyndon theorem, this is equivalent to: a CA is a continuous, shift-commuting map from $\Sigma^{\mathbb{Z}}$ to itself. Thus:

$$\{\text{cellular automata on } \Sigma\} = \mathrm{End}(\Sigma^{\mathbb{Z}}).$$

This is the endomorphism monoid of the full shift, as defined in Section 3.8.

Wolfram's elementary cellular automata are exactly the sliding block codes $\Phi : \{0,1\}^{\mathbb{Z}} \to \{0,1\}^{\mathbb{Z}}$ with symmetric radius $r = 1$. There are $2^{2^3} = 256$ such local rules $f : \{0,1\}^3 \to \{0,1\}$, conventionally numbered 0 through 255. For instance:

- **Rule 102**: $f(a, b, c) = b \oplus c$. This is the XOR map from Example 3.3.1 (after a shift in indexing).
- **Rule 232**: $f(a, b, c) = \mathrm{majority}(a, b, c)$. This is the majority vote from Example 3.3.2.
- **Rule 110**: Known to be Turing-complete (Cook, 2004).

The perspective of cellular automata as endomorphisms of the full shift provides a rich algebraic and topological framework for their study, going beyond the purely computational viewpoint.

3.11 Connection to Autoregressive Language Models

An **autoregressive language model** (such as a transformer-based LLM) generates a sequence of tokens $x_0, x_1, x_2, \ldots$ from a finite vocabulary Σ (the "token alphabet") by the rule

$$x_{t+1} \sim P(\cdot \mid x_{\max(0,t-k+1)}, \ldots, x_t)$$

where k is the **context window** length. If we ignore the stochastic sampling and consider the deterministic **greedy decoding** map (always pick the most likely next token), this becomes a deterministic function

$$x_{t+1} = f(x_{\max(0,t-k+1)}, \ldots, x_t)$$

for some fixed $f : \Sigma^{\leq k} \to \Sigma$. Once the sequence is long enough that $t \geq k-1$, this is exactly a **one-sided sliding block code** with memory $m = k-1$ and anticipation $a = 0$:

$$[\Phi(x)]_i = f(x_{i-k+1}, \ldots, x_i).$$

More precisely:

Symbolic dynamics concept	LLM concept
Alphabet Σ	Token vocabulary
One-sided shift space $\Sigma^{\mathbb{N}}$	Space of all token sequences
Memory $m = k-1$	Context window minus 1
Anticipation $a = 0$	Causal (autoregressive) generation
Local rule f	Deterministic decoding of the model
Sliding block code Φ	One step of autoregressive generation

This correspondence is exact in the deterministic case and approximate in the stochastic case (where the model samples rather than taking the argmax). The subshift generated by iterating Φ from a prompt is the **orbit** of the prompt under the dynamical system defined by Φ.

Key differences from the classical theory:

1. LLMs operate on **one-sided** sequences (causal generation), not two-sided.
2. The local rule f is astronomically complex (billions of parameters), whereas classical symbolic dynamics studies simple local rules.
3. The stochastic case (nonzero temperature) leads to random dynamical systems and stationary measures on shift spaces, not deterministic orbits.

Nevertheless, the sliding block code framework provides the correct mathematical scaffolding for understanding the structure of autoregressive generation.

3.12 Summary of Key Results

Concept	Definition
Sliding block code	$[\Phi(x)]_i = f(x_{i-m}, \ldots, x_{i+a})$ for a fixed local rule f
Curtis-Hedlund-Lyndon	Continuous + shift-commuting $\iff$ sliding block code
Factor map	Surjective sliding block code
Conjugacy	Bijective sliding block code (inverse is automatic)
$\mathrm{End}(\Sigma^{\mathbb{Z}})$	Monoid of CAs = endomorphisms of the full shift
$\mathrm{Aut}(\Sigma^{\mathbb{Z}})$	Group of invertible CAs = automorphisms of the full shift
Causal SBC	Anticipation $a = 0$; relevant to autoregressive models

3.13 References

- **Curtis, A., Hedlund, G. A., and Lyndon, R. C.** The result is attributed to all three; the published proof appears in Hedlund (1969).
- **Hedlund, G. A.** (1969). Endomorphisms and automorphisms of the shift dynamical system. *Mathematical Systems Theory*, 3(4), 320–375.

- **Lind, D. and Marcus, B.** (2021). *An Introduction to Symbolic Dynamics and Coding*, 2nd edition. Cambridge University Press. (Chapter 6 covers sliding block codes; Chapter 1 covers shift spaces.)

- **Kurka, P.** (2003). *Topological and Symbolic Dynamics.* Cours Specialises, Societe Mathematique de France. (Chapter 4 treats cellular automata as endomorphisms.)

- **Ryan, J. P.** (1972). The shift and commutativity. *Mathematical Systems Theory*, 6(1–2), 82–85.

- **Cook, M.** (2004). Universality in elementary cellular automata. *Complex Systems*, 15(1), 1–40.

3.14 Recommended Reading

For the core theory:

- **Lind & Marcus (2021)**, Chapter 6, gives the definitive modern treatment of sliding block codes, including full proofs and many exercises. This is the standard reference.

- **Hedlund (1969)** is the original source and remains highly readable. It establishes the foundational results on endomorphisms and automorphisms of shift spaces.

For cellular automata:

- **Wolfram, S.** (2002). *A New Kind of Science.* Wolfram Media. Controversial but influential; contains extensive empirical study of elementary cellular automata.

- **Ilachinski, A.** (2001). *Cellular Automata: A Discrete Universe.* World Scientific. Comprehensive treatment of cellular automata from multiple perspectives.

For deeper algebraic structure:

- **Ceccherini-Silberstein, T. and Coornaert, M.** (2010). *Cellular Automata and Groups.* Springer. Explores the connection between CAs and group theory, generalizing beyond $\mathbb{Z}$ actions.

- **Boyle, M., Lind, D., and Rudolph, D.** (1988). "The automorphism group of a shift of finite type." *Transactions of the AMS*, 306(1), 71–114. Deep results on $\mathrm{Aut}(X)$ for SFTs.

For connections to computation and language:

- **Kari, J.** (2005). "Theory of cellular automata: A survey." *Theoretical Computer Science*, 334(1-3), 3–33. Emphasizes decidability questions and the computational perspective.

Chapter 4

Post Tag Systems

4.1 Introduction

Tag systems are among the simplest string-rewriting models that achieve computational universality. Introduced by Emil Post in the 1940s, they operate by a monotonously simple rule: read the first symbol, delete a fixed number of symbols from the front, and append a string determined by that first symbol. Despite this extreme simplicity, tag systems can simulate arbitrary Turing machines — a fact that has made them a key tool in the study of the boundary between decidable and undecidable computation.

This chapter defines tag systems precisely, traces several worked examples, surveys the main universality and decidability results, and connects tag systems to the (f, x) dynamical systems framework developed in earlier chapters.

4.2 Definition

Definition 4.1 (Post Tag System)**.** A *tag system* is a triple $T = (\Sigma, m, P)$ where:

- Σ is a finite alphabet,
- $m \geq 1$ is the *deletion number*,
- $P : \Sigma \to \Sigma^*$ is the *production function*, mapping each symbol to a (possibly empty) string over Σ.

Computation step. Given a string $w = a_1 a_2 \cdots a_n$ with $n \geq m$:

1. Read the first symbol a_1. 2. Delete the first m symbols, yielding $a_{m+1} a_{m+2} \cdots a_n$. 3. Append $P(a_1)$, yielding $a_{m+1} a_{m+2} \cdots a_n \cdot P(a_1)$.

Halting condition. The system halts when the current string has length less than m. In particular, if a production maps some symbol to the empty string ε, the string may shrink and eventually halt.

We write $w \vdash_T w'$ for a single step and $w \vdash_T^* w'$ for the reflexive-transitive closure. The *orbit* of a string w_0 is the sequence $w_0, w_1, w_2, \ldots$ where w_{i+1} is obtained from w_i by one step (or the sequence terminates if w_i halts).

Remark 4.2. A tag system with $m = 1$ is trivial: each step replaces the first symbol with a string but the "queue" never advances faster than it grows. The interesting case is $m \geq 2$, where there is genuine competition between deletion and production.

4.3 Worked Example 1: Growth and Cycling

Consider the tag system $T_1 = (\{0, 1\}, 2, P)$ with:

$$P(0) = \texttt{00}, \quad P(1) = \texttt{1101}$$

Starting from $w_0 = \texttt{10}$:

Step	String	Length	First symbol	Deleted prefix	Appended
0	‘10‘	2	‘1‘	‘10‘	‘1101‘
1	‘1101‘	4	‘1‘	‘11‘	‘1101‘
2	‘011101‘	6	‘0‘	‘01‘	‘00‘
3	‘110100‘	6	‘1‘	‘11‘	‘1101‘
4	‘01001101‘	8	‘0‘	‘01‘	‘00‘
5	‘00110100‘	8	‘0‘	‘00‘	‘00‘
6	‘11010000‘	8	‘1‘	‘11‘	‘1101‘
7	‘0100001101‘	10	‘0‘	‘01‘	‘00‘
8	‘0000110100‘	10	‘0‘	‘00‘	‘00‘
9	‘0011010000‘	10	‘0‘	‘00‘	‘00‘
10	‘1101000000‘	10	‘1‘	‘11‘	‘1101‘

Several features are visible:

P(a)	$> \mathrm{m} forthesymbolabeingread.Here$	P(1)

- **Complex dynamics.** Despite the simplicity of the rules, the string's evolution is non-trivial: ‘0‘s and ‘1‘s interleave in patterns that are not immediately periodic.

Continuing the trace, one can verify computationally that this system does not halt for the initial string ‘10‘. The orbit grows and exhibits quasi-periodic behavior.

4.4 Worked Example 2: A Halting Tag System

Consider $T_2 = (\{a, b, c\}, 3, P)$ with:

$$P(a) = \texttt{bc}, \quad P(b) = \texttt{a}, \quad P(c) = \texttt{aaa}$$

Starting from $w_0 = \texttt{aaa}$:

Step	String	Length	First symbol	Action
0	‘aaa‘	3	‘a‘	delete ‘aaa‘, append ‘bc‘
1	‘bc‘	2	—	**HALT** (length $2 < m = 3$)

The system halts in one step. The production $P(a) = \texttt{bc}$ has length 2, which is less than $m = 3$, so a net loss of 1 symbol occurs. Starting from a string of length exactly m, this guarantees halting.

A more interesting halting example: same alphabet and m, but start from $w_0 = \texttt{abcabc}$:

Step	String	Length	First symbol	Appended
0	‘abcabc‘	6	‘a‘	‘bc‘
1	‘abcbc‘	5	‘a‘	‘bc‘
2	‘bcbc‘	4	‘b‘	‘a‘
3	‘ca‘	2	—	**HALT**

This halts in 3 steps. The string length decreases from 6 to 5 to 4 to 2.

4.5 Worked Example 3: A Looping Tag System

Consider $T_3 = (\{0, 1\}, 2, P)$ with:

$$P(0) = \texttt{01}, \quad P(1) = \texttt{10}$$

Both productions have length 2, equal to m. Every step preserves the string length. Starting from $w_0 = \texttt{01}$:

Step	String	First symbol	Appended
0	‘01‘	‘0‘	‘01‘
1	‘01‘	‘0‘	‘01‘

The system enters a fixed point immediately: ‘01‘ maps to ‘01‘. Now start from $w_0 = \texttt{10}$:

Step	String	First symbol	Appended
0	‘10‘	‘1‘	‘10‘
1	‘10‘	‘1‘	‘10‘

Again a fixed point. Now try $w_0 = \texttt{0110}$:

Step	String	First symbol	Appended
0	‘0110‘	‘0‘	‘01‘
1	‘1001‘	‘1‘	‘10‘
2	‘0110‘	‘0‘	‘01‘

This is a cycle of period 2: $\texttt{0110} \to \texttt{1001} \to \texttt{0110}$. Since all productions have length m, no string can halt or grow; every orbit is eventually periodic with bounded period. This illustrates that length-preserving tag systems have relatively tame dynamics.

4.6 Post’s Normal Form Theorem (1943)

The original motivation for tag systems comes from Post’s study of *canonical systems* — a general class of string-rewriting systems that Post developed in parallel with Turing’s work on computability.

Theorem 4.3 (Post’s Normal Form Theorem, 1943)**.** *Every Post canonical system can be reduced to a tag system (possibly over a larger alphabet).*

More precisely, any computation that can be expressed as a sequence of string productions in a canonical system can be simulated by a tag system, at the cost of increasing the alphabet size. Post’s original formulation used "normal systems," which are a special case of canonical systems where productions are constrained to delete a prefix and append a suffix — exactly the structure of a tag system.

Post arrived at tag systems in the 1920s and struggled with the question of their decidability for over two decades before publishing in 1943. His frustration with the problem — he called it his "principal unsolved problem" — is documented in his correspondence and was only resolved after his death (Post, 1943).

4.7 Turing-Completeness of Tag Systems

4.7.1 Minsky's Theorem (1961)

Theorem 4.4 (Minsky, 1961)**.** *For any $m \geq 2$, tag systems with deletion number m are Turing-complete.*

Sketch of proof. Minsky showed how to encode the configuration of a Turing machine as a tag string and simulate each transition by a sequence of tag steps.

The key idea is as follows. A Turing machine configuration consists of a state q, a tape content, and a head position. This is encoded as a string over an enlarged alphabet where:

1. The tape content to the right of the head is encoded as a sequence of "data symbols" using a block code (each tape symbol is represented by a fixed-length block of tag symbols). 2. The state and the symbol currently under the head are encoded jointly as a special "state-symbol" marker at the front of the string. 3. The tape content to the left of the head is encoded at the end of the string.

A single Turing machine step is simulated by multiple tag steps that: - Read the state-symbol marker at the front of the string. - Use the production rules to output the appropriate new marker (encoding the new state and the symbol to write) plus routing symbols. - The deletion of m symbols from the front "advances" through the encoded tape, effectively moving the head.

The encoding requires an alphabet whose size depends on the number of Turing machine states and tape symbols, but m can remain fixed. The overhead is polynomial in the description of the Turing machine.

4.7.2 The Cocke–Minsky Theorem (1964)

Theorem 4.5 (Cocke–Minsky, 1964)**.** *2-tag systems (tag systems with deletion number $m = 2$) are universal.*

This is a strengthening of Theorem 4.3: the deletion number need not be arbitrary but can be fixed at 2, the smallest non-trivial value. The proof proceeds by showing that any tag system with $m > 2$ can be simulated by a 2-tag system over a larger alphabet, combined with Minsky's theorem.

The universality of 2-tag systems makes them one of the simplest known Turing-complete formalisms, alongside two-state three-symbol Turing machines and Rule 110 cellular automata.

4.8 Cyclic Tag Systems

Definition 4.6 (Cyclic Tag System)**.** A *cyclic tag system* is a tag system variant defined by a finite sequence of productions $P_0, P_1, \ldots, P_{k-1}$ over the alphabet $\{0, 1\}$. The system operates on binary strings as follows:

At step t: 1. Read and delete the first symbol of the string. 2. If the first symbol was '1', append $P_{t \bmod k}$. 3. If the first symbol was '0', append nothing.

The production applied at each step cycles through the list $P_0, P_1, \ldots, P_{k-1}, P_0, P_1, \ldots$ regardless of the symbol read.

Example. Let $k = 3$ with productions $P_0 = \texttt{11}$, $P_1 = \texttt{10}$, $P_2 = \varepsilon$ (empty). Starting from $w_0 = \texttt{11}$:

Step t	String	First symbol	Production $P_{t \bmod 3}$	Append?	Result
0	‘11‘	‘1‘	$P_0 = \texttt{11}$	yes	‘111‘
1	‘111‘	‘1‘	$P_1 = \texttt{10}$	yes	‘1110‘
2	‘1110‘	‘1‘	$P_2 = \varepsilon$	yes (nothing)	‘110‘
3	‘110‘	‘1‘	$P_0 = \texttt{11}$	yes	‘1011‘
4	‘1011‘	‘1‘	$P_1 = \texttt{10}$	yes	‘01110‘
5	‘01110‘	‘0‘	$P_2 = \varepsilon$	no	‘1110‘

Note step 5: the first symbol is ‘0‘, so nothing is appended regardless of the production.

Cyclic tag systems were introduced by Matthew Cook in connection with his proof that the elementary cellular automaton Rule 110 is Turing-complete (Cook, 2004). The proof proceeds in two stages:

1. Show that cyclic tag systems are Turing-complete (by simulating arbitrary 2-tag systems).
2. Show that Rule 110 can simulate cyclic tag systems.

Cyclic tag systems serve as a bridge because their purely periodic control structure is well-suited for embedding in the spatially periodic patterns of cellular automata.

4.9 The Collatz Connection

De Mol (2008) observed a striking connection between certain tag systems and Collatz-like iterations in number theory.

The Collatz function $C : \mathbb{N} \to \mathbb{N}$ is defined by:

$$C(n) = \begin{cases} n/2 \& \text{if } n \text{ is even} \\ 3n + 1 \& \text{if } n \text{ is odd} \end{cases}$$

Consider a 2-tag system over $\{a, b\}$ with $P(a) = \texttt{b}$ (length 1, causing the string to shrink) and $P(b) = \texttt{aab}$ (length 3, causing the string to grow). If we track the length ℓ of the string:

- When the first symbol is a: new length $= \ell - 2 + 1 = \ell - 1$.
- When the first symbol is b: new length $= \ell - 2 + 3 = \ell + 1$.

The *pattern* of as and bs at the head of the string determines a sequence of increments and decrements that, when analyzed modulo arithmetic, produces dynamics analogous to iterated Collatz-type maps. De Mol showed that proving termination (or non-termination) for certain tag systems is equivalent to resolving Collatz-type conjectures, thereby connecting string rewriting to deep open problems in number theory.

This provides another lens on why tag system behavior is hard to predict: even the question "does the string length remain bounded?" can encode unsolved problems.

4.10 The Decidability Boundary

The halting problem for tag systems asks: given a tag system T and an initial string w, does the computation starting from w eventually halt?

Theorem 4.7 (Undecidability, from Minsky 1961 / Cocke–Minsky 1964)**.** *The halting problem for 2-tag systems is undecidable.*

This follows immediately from universality: if we could decide halting for 2-tag systems, we could decide the halting problem for Turing machines.

The interesting question is: how small can a tag system be while remaining undecidable?

De Mol (2007) investigated this boundary systematically. For 2-tag systems over a binary alphabet $\{0, 1\}$*, there are only finitely many possible production tables (up to production length bounds). De Mol showed:*

- Many small 2-tag systems have decidable behavior (they always halt, always loop, or can be analyzed by tracking string length). - The smallest known undecidable 2-tag systems require relatively few production rules but with productions of moderate length. - There exist 2-tag systems whose behavior is unknown for specific inputs — systems that may halt, loop, or grow without bound, and current techniques cannot determine which.

The precise boundary remains open, but the general picture is clear: the threshold of undecidability occurs at remarkably small system sizes, consistent with the theme that computational universality requires very little structure.

4.11 Connection to the (f, x) Framework

A tag system fits naturally into the dynamical systems framework of Chapter 1.

In a standard tag system, the state is the current string $x \in \Sigma^*$, and the transition function f is determined by the fixed triple (Σ, m, P). Thus a tag system is a dynamical system of the form:

$$x_{n+1} = f(x_n)$$

where f is the tag-step function. The "program" f (encoded by the production table P) is fixed; only the "data" x evolves. In our (f, x) notation, the system is:

$$(f, x) \to (f, f(x))$$

with f constant — the simplest case of the general framework.

A *self-modifying tag system* would be one in which the production table is encoded within the string itself, so that applying the current rules transforms both the "data" portion and the "rule" portion of the string. In (f, x) notation:

$$(f, x) \to (f', x')$$

where f' is extracted from the string x'. This is not merely a theoretical curiosity: it corresponds to a system whose rewriting rules are part of its own substrate, analogous to self-modifying code or — more relevantly — to an LLM whose prompt contains instructions that are themselves rewritten by the generation process.

Such self-modifying tag systems inherit the universality of ordinary tag systems but add a layer of meta-level dynamics: the function itself undergoes evolution, not just the data. The dynamical questions from Chapter 1 (transient length, cycle structure, attractor complexity) become substantially harder, because the "program" is no longer a fixed point of the meta-rule φ; it co-evolves with the data.

4.12 Connection to Autoregressive Language Models

There is a suggestive structural parallel between tag systems and autoregressive language generation with a finite context window.

Tag System	Autoregressive LLM
Finite alphabet Σ	Token vocabulary V
Current string w	Context window contents
Read first symbol a	Attend to context (especially early tokens)
Delete first m symbols	Sliding window drops oldest m tokens
Append $P(a)$	Generate and append new tokens
Production table P	Model weights (fixed during inference)

In both systems, the core loop is: *read context, discard old material, produce new material.* The production function P in a tag system is a lookup table; in an LLM, it is a neural network — vastly more expressive, but playing the same structural role.

The analogy has limits. An LLM attends to its entire context window (not just the first token), and its "production" depends on the full context via attention, not on a single symbol lookup. Nevertheless, the tag system abstraction captures the essential tension: the system's future behavior is determined by a finite window of past output, with old information irreversibly discarded. This is precisely the regime where complex, hard-to-predict dynamics arise — as the universality results of this chapter confirm.

4.13 Summary

Tag systems illustrate a recurring theme in the theory of computation: extremely simple rewriting rules can generate behavior that is computationally universal and therefore undecidable. The key results form a clear logical chain:

1. **Post (1943):** Canonical systems reduce to tag systems.
2. **Minsky (1961):** Tag systems with $m \geq 2$ are Turing-complete.
3. **Cocke–Minsky (1964):** Even 2-tag systems suffice for universality.
4. **Cook (2004):** Cyclic tag systems mediate between 2-tag systems and cellular automata.
5. **De Mol (2007, 2008):** The decidability boundary is surprisingly low, and tag system dynamics connect to Collatz-type problems in number theory.

For our purposes, tag systems serve as a bridge between the abstract (f, x) dynamics of Chapter 1 and the concrete mechanics of string rewriting. They are the simplest non-trivial model in which "read, delete, append" produces universal computation — and they suggest that the same dynamical pattern, scaled up with neural networks, may account for some of the surprising capabilities of autoregressive language models.

4.14 References

- Cocke, J. and Minsky, M. (1964). "Universality of Tag Systems with P = 2." *Journal of the ACM*, 11(1), 15–20.
- Cook, M. (2004). "Universality in Elementary Cellular Automata." *Complex Systems*, 15(1), 1–40.
- De Mol, L. (2007). "Tag Systems and Collatz-like Functions." *Theoretical Computer Science*, 390(1), 92–101.
- De Mol, L. (2008). "Closing the Circle: An Analysis of Emil Post's Early Work." *Bulletin of Symbolic Logic*, 12(2), 267–289.

- Minsky, M. (1961). "Recursive Unsolvability of Post's Problem of 'Tag' and Other Topics in Theory of Turing Machines." *Annals of Mathematics*, 74(3), 437–455.
- Post, E. L. (1943). "Formal Reductions of the General Combinatorial Decision Problem." *American Journal of Mathematics*, 65(2), 197–215.
- Wolfram, S. (2002). *A New Kind of Science*. Wolfram Media. Chapter 3: "The World of Simple Programs."

4.15 Recommended Reading

For historical context and Post's contributions:

- **Davis, M.** (1965). *The Undecidable*. Raven Press. Contains Post's original 1943 paper along with other foundational papers in computability. Essential primary source.
- **De Mol, L.** (2008). "Closing the Circle: An Analysis of Emil Post's Early Work." Provides excellent historical analysis of how Post arrived at tag systems and their place in the development of computability theory.

For universality proofs:

- **Minsky, M.** (1967). *Computation: Finite and Infinite Machines*. Prentice-Hall. Chapter 14 gives a readable account of tag system universality with full details.
- **Rogozhin, Y.** (1996). "Small Universal Turing Machines." *Theoretical Computer Science*, 168(2), 215–240. Context on the quest for minimal universal systems, of which tag systems are a key example.

For connections to cellular automata:

- **Cook, M.** (2004). The original proof that Rule 110 is universal, via cyclic tag systems. Technical but foundational.

For the decidability boundary and open problems:

- **De Mol, L.** (2007). Explores the connection between tag systems and Collatz-type problems, showing that seemingly simple systems encode deep number-theoretic questions.

For broader context on string rewriting:

- **Book, R. and Otto, F.** (1993). *String-Rewriting Systems*. Springer. Comprehensive treatment of rewriting systems, with tag systems as a special case.

Chapter 5

Substitution Systems and Automatic Sequences

5.1 Morphisms on Words

Let Σ be a finite alphabet. We write Σ^* for the free monoid over Σ—the set of all finite words (including the empty word ε) with concatenation as the monoid operation.

Definition 5.1 (Morphism)**.** A *morphism* is a map $\sigma : \Sigma \to \Sigma^*$, extended to $\sigma : \Sigma^* \to \Sigma^*$ by the concatenation rule:

$$\sigma(a_1 a_2 \cdots a_n) = \sigma(a_1)\sigma(a_2)\cdots\sigma(a_n).$$

In other words, σ is a monoid homomorphism from $(\Sigma^*, \cdot)$ to itself. A morphism is completely determined by specifying $\sigma(a)$ for each letter $a \in \Sigma$.

This is the fundamental observation: to define a morphism, you only need to say what each letter maps to. The extension to arbitrary words is forced by the homomorphism property.

Definition 5.2 (Uniform and non-uniform morphisms)**.** A morphism σ is *k-uniform* if $|\sigma(a)| = k$ for every $a \in \Sigma$, where $|\cdot|$ denotes word length. A morphism is *uniform* if it is k-uniform for some $k \geq 1$. Otherwise it is *non-uniform*.

A 1-uniform morphism is simply a letter-to-letter map (a permutation or coding). A 2-uniform morphism doubles the length of every word it is applied to.

Definition 5.3 (Coding)**.** A *coding* (or *letter-to-letter morphism*) is a 1-uniform morphism $\tau : \Sigma \to \Delta$, possibly between different alphabets. A coding simply renames letters.

Definition 5.4 (Primitive morphism)**.** A morphism $\sigma : \Sigma \to \Sigma^*$ is *primitive* if there exists an integer $n \geq 1$ such that for every pair of letters $a, b \in \Sigma$, the letter b appears in $\sigma^n(a)$. Equivalently, the incidence matrix M_σ (where $(M_\sigma)_{ij}$ counts occurrences of letter j in $\sigma(i)$) is a primitive matrix (some power has all positive entries).

Primitivity is the substitution analogue of irreducibility plus aperiodicity in Markov chains. It ensures that every part of the system communicates with every other part.

5.2 Fixed Points of Morphisms

The central construction in substitution theory is iteration.

Definition 5.5 (Prolongable morphism)**.** A morphism σ is *prolongable on a letter a* if $\sigma(a) = aw$ for some non-empty word $w \in \Sigma^+$. In this case:

$$\sigma^n(a) = a \cdot w \cdot \sigma(w) \cdot \sigma^2(w) \cdots \sigma^{n-1}(w),$$

so $\sigma^n(a)$ is a proper prefix of $\sigma^{n+1}(a)$ for all $n \geq 0$, and the sequence of words $\sigma^n(a)$ converges (in the product topology on Σ^ω) to a unique infinite word

$$\mathbf{x} = \lim_{n\to\infty} \sigma^n(a).$$

Theorem 5.6. *If σ is prolongable on a, then the infinite word $\mathbf{x} = \lim_{n\to\infty} \sigma^n(a)$ is a fixed point of σ, meaning $\sigma(\mathbf{x}) = \mathbf{x}$, where we extend σ to infinite words in the obvious way.*

Proof sketch. Since $\sigma(\sigma^n(a)) = \sigma^{n+1}(a)$, and both $\sigma^n(a)$ and $\sigma^{n+1}(a)$ converge to $\mathbf{x}$, continuity of σ (in the product topology) gives $\sigma(\mathbf{x}) = \mathbf{x}$.

5.3 The Thue-Morse Sequence

5.3.1 5.3.1 Construction

Definition 5.7. The *Thue-Morse morphism* is the 2-uniform morphism $\mu : \{0,1\} \to \{0,1\}^*$ defined by

$$\mu(0) = 01, \qquad \mu(1) = 10.$$

Since $\mu(0)$ starts with 0, the morphism is prolongable on 0.

Example 5.8 (Iterated construction of the Thue-Morse sequence)**.**

Iteration	Word
$\mu^0(0)$	0
$\mu^1(0)$	01
$\mu^2(0)$	0110
$\mu^3(0)$	01101001
$\mu^4(0)$	0110100110010110

At each step, we replace every 0 by 01 and every 1 by 10. Let us verify $\mu^3(0)$ in detail:

$$\mu^3(0) = \mu^2(01) = \mu^2(0)\mu^2(1) = 0110 \cdot 1001 = 01101001. \quad \checkmark$$

The fixed point is the infinite *Thue-Morse sequence*:

$$\mathbf{t} = 0\,1\,1\,0\,1\,0\,0\,1\,1\,0\,0\,1\,0\,1\,1\,0\,\ldots$$

Alternative characterization. The n-th term is $t_n = s_2(n) \bmod 2$, where $s_2(n)$ is the sum of the binary digits of n. This is the simplest example of a 2-automatic sequence (Section 5.7).

5.3.2 5.3.2 Properties

The Thue-Morse sequence has a remarkable collection of properties, which is why it appears throughout combinatorics, number theory, and dynamical systems.

Theorem 5.9 (Overlap-freeness, Thue 1906, 1912)**.** *The Thue-Morse sequence is overlap-free: it contains no factor of the form $axaxa$ where a is a letter and x is a (possibly empty) word. Equivalently, it avoids the pattern $\alpha^2\alpha'$ where α' is a prefix of α.*

This was Thue's original motivation: to construct an infinite binary word with no overlapping squares. Note that a binary word cannot avoid all squares (the factors 00 and 11 are unavoidable), but it can avoid overlaps.

Theorem 5.10 (Not eventually periodic)**.** *The Thue-Morse sequence is not eventually periodic. That is, there are no integers n_0 and $p \geq 1$ such that $t_n = t_{n+p}$ for all $n \geq n_0$.*

Proof sketch. If $\mathbf{t}$ were eventually periodic with period p, it would contain arbitrarily long squares ww. But $\mathbf{t}$ is overlap-free, and any word containing arbitrarily long squares must contain an overlap (by a pigeonhole argument on the positions). Contradiction.

Theorem 5.11 (Balanced)**.** *Define $|w|_a$ as the number of occurrences of letter a in the word w. For any two factors u, v of $\mathbf{t}$ with $|u| = |v|$, we have*

$$\big||u|_0 - |v|_0\big| \leq C$$

for a universal constant C. That is, the Thue-Morse sequence is C-balanced. (In fact, one can take $C = 2$; note it is not 1-balanced, unlike Sturmian sequences.)

5.3.3 5.3.3 The Thue-Morse Dynamical System

Define the *shift map* $T : \Sigma^\omega \to \Sigma^\omega$ by $T(x_0x_1x_2\cdots) = x_1x_2x_3\cdots$. The *orbit closure* of $\mathbf{t}$ under T is

$$X_\mu = \overline{\{T^n(\mathbf{t}) : n \geq 0\}},$$

where the closure is in the product topology. The pair (X_μ, T) is a topological dynamical system called the *Thue-Morse subshift*.

Since μ is primitive (the matrix $M_\mu = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right)$ is already positive), the subshift (X_μ, T) is *minimal*: every orbit is dense, or equivalently, every factor of every element of X_μ is a factor of $\mathbf{t}$. It is also *uniquely ergodic*: there is a unique shift-invariant Borel probability measure on X_μ.

5.4 The Fibonacci Word

Definition 5.12. The *Fibonacci morphism* is $\varphi : \{0, 1\} \to \{0, 1\}^*$ defined by

$$\varphi(0) = 01, \qquad \varphi(1) = 0.$$

This morphism is non-uniform: $|\varphi(0)| = 2$ and $|\varphi(1)| = 1$. It is prolongable on 0.

Example 5.13 (Iterated construction of the Fibonacci word)**.**

Iteration	Word	Length
$\varphi^0(0)$	0	1
$\varphi^1(0)$	01	2
$\varphi^2(0)$	010	3
$\varphi^3(0)$	01001	5
$\varphi^4(0)$	01001010	8
$\varphi^5(0)$	0100101001001	13

Let us verify $\varphi^4(0)$:

$$\varphi^4(0) = \varphi^3(01) = \varphi^3(0)\varphi^3(1) = 01001 \cdot 010 = 01001010. \quad \checkmark$$

The lengths are Fibonacci numbers $1, 2, 3, 5, 8, 13, 21, \ldots$ — hence the name. This follows from the recurrence: $|\varphi^{n+1}(0)| = |\varphi^n(0)| + |\varphi^n(1)| = |\varphi^n(0)| + |\varphi^{n-1}(0)|$, which is exactly the Fibonacci recurrence.

5.4.1 5.4.1 Connection to the Golden Ratio

The incidence matrix of φ is

$$M_\varphi = \begin{pmatrix} 1\&1 \\ 1\&0 \end{pmatrix},$$

whose eigenvalues are $\phi = (1 + \sqrt{5})/2$ (the golden ratio) and $\hat{\phi} = (1 - \sqrt{5})/2$. The Perron-Frobenius eigenvalue ϕ controls the growth rate of the word lengths: $|\varphi^n(0)| \sim C\phi^n$ as $n \to \infty$.

5.4.2 5.4.2 Sturmian Sequences

The Fibonacci word is the prototypical example of a *Sturmian sequence.*

Definition 5.14. A sequence $\mathbf{x} \in \{0,1\}^\omega$ is *Sturmian* if its *factor complexity* satisfies $p_{\mathbf{x}}(n) = n+1$ for all $n \geq 0$, where $p_{\mathbf{x}}(n)$ is the number of distinct factors of length n in $\mathbf{x}$.

The Morse-Hedlund theorem states that $p_{\mathbf{x}}(n) \geq n+1$ for all n if and only if $\mathbf{x}$ is not eventually periodic. So Sturmian sequences have the minimal complexity among non-eventually-periodic sequences. They are, in a precise sense, the "simplest" aperiodic sequences.

Sturmian sequences can be constructed by *cutting sequences*: take a line of irrational slope α in the plane and record the order in which it crosses horizontal and vertical grid lines. The Fibonacci word corresponds to slope $\alpha = 1/\phi = \phi - 1 = (\sqrt{5}-1)/2$.

5.5 The Rudin-Shapiro Sequence

Definition 5.15. The *Rudin-Shapiro morphism* operates on the four-letter alphabet $\{0,1,2,3\}$:

$$\sigma(0) = 02, \quad \sigma(1) = 32, \quad \sigma(2) = 01, \quad \sigma(3) = 31.$$

The *Rudin-Shapiro sequence* $\mathbf{r}$ is obtained by applying the coding $\tau : \{0,1,2,3\} \to \{+1,-1\}$ defined by $\tau(0) = \tau(2) = +1$, $\tau(1) = \tau(3) = -1$ to the fixed point of σ starting from 0.

Example 5.16 (First iterations of the Rudin-Shapiro morphism)**.**

Iteration	Word
$\sigma^0(0)$	0
$\sigma^1(0)$	02
$\sigma^2(0)$	0201
$\sigma^3(0)$	02010232

Let us verify $\sigma^3(0)$:

$$\sigma^3(0) = \sigma^2(02) = \sigma^2(0)\sigma^2(2) = 0201 \cdot 0232.$$

Check $\sigma^2(2)$: $\sigma(2) = 01$, so $\sigma^2(2) = \sigma(0)\sigma(1) = 02 \cdot 32 = 0232$. ✓

Applying the coding: $\sigma^3(0) = 02010232 \mapsto (+1,+1,+1,-1,+1,+1,-1,+1)$.

Alternative characterization. Let $r_n = (-1)^{f(n)}$ where $f(n)$ counts the number of (possibly overlapping) occurrences of the block 11 in the binary expansion of n.

The Rudin-Shapiro sequence was introduced independently by Rudin (1959) and Shapiro (1951) in the study of flat polynomials. Its key analytic property is that the partial sums of the associated trigonometric polynomial $\sum_{n=0}^{N-1} r_n z^n$ satisfy $|\sum r_n z^n| = O(\sqrt{N})$ on the unit circle, which is optimal up to a constant.

5.6 Morphic Sequences

Definition 5.17 (Morphic sequence)**.** A sequence $\mathbf{x} \in \Delta^\omega$ is *morphic* if there exist: 1. a morphism $\sigma : \Sigma \to \Sigma^*$ prolongable on some letter $a \in \Sigma$, and 2. a coding $\tau : \Sigma \to \Delta$,

such that $\mathbf{x} = \tau(\sigma^\omega(a))$, where $\sigma^\omega(a) = \lim_{n\to\infty} \sigma^n(a)$ is the fixed point.

If σ is k-uniform, then $\mathbf{x}$ is called *k-automatic* (see Section 5.7). In general, morphic sequences are strictly more expressive than automatic sequences.

A sequence that is a fixed point of a morphism (no coding needed) is called *purely morphic* or a *pure morphism fixed point.*

Example 5.18. The Thue-Morse sequence is purely morphic (it is the fixed point of μ itself, no coding needed) and 2-automatic. The Fibonacci word is purely morphic but not k-automatic for any k, since φ is non-uniform. The Rudin-Shapiro sequence is morphic (it requires a coding τ) and 2-automatic (since σ is 2-uniform).

5.7 Automatic Sequences

5.7.1 5.7.1 Definition via Finite Automata

Definition 5.19 (k-automatic sequence)**.** Let $k \geq 2$. A sequence $(a_n)_{n\geq 0}$ over a finite alphabet Δ is *k-automatic* if there exists a deterministic finite automaton with output (DFAO) $\mathcal{A} = (Q, \Sigma_k, \delta, q_0, \tau)$ such that

$$a_n = \tau(\delta(q_0, (n)_k))$$

for all $n \geq 0$, where $(n)_k$ is the base-k representation of n (with the most significant digit first), δ is the extended transition function, and $\tau : Q \to \Delta$ is the output function.

In words: to compute a_n, write n in base k, feed those digits into a finite automaton, and read off the output from the state you reach.

Example 5.20 (The Thue-Morse sequence is 2-automatic)**.** The DFAO has two states $\{q_0, q_1\}$, input alphabet $\{0, 1\}$, transitions $\delta(q_0, 0) = q_0$, $\delta(q_0, 1) = q_1$, $\delta(q_1, 0) = q_1$, $\delta(q_1, 1) = q_0$, and output $\tau(q_0) = 0$, $\tau(q_1) = 1$. The automaton simply tracks the parity of the number of 1's in the binary representation, and indeed $t_n = s_2(n) \bmod 2$.

5.7.2 5.7.2 The k-kernel

An equivalent algebraic characterization avoids automata entirely.

Definition 5.21. The *k-kernel* of a sequence $(a_n)_{n\geq 0}$ is the set

$$\mathcal{K}_k(\mathbf{a}) = \{(a_{k^e n+r})_{n\geq 0} : e \geq 0,\ 0 \leq r < k^e\}.$$

Theorem 5.22 (Eilenberg 1974)**.** *A sequence is k-automatic if and only if its k-kernel is finite.*

The k-kernel captures all subsequences obtained by "zooming in" at different scales and offsets in base k. Finiteness of this set is equivalent to the finite-state condition.

5.7.3 5.7.3 Cobham's Characterization

The following theorem connects the automata-theoretic and morphism-based viewpoints.

Theorem 5.23 (Cobham 1972)**.** *A sequence $\mathbf{x} \in \Delta^\omega$ is k-automatic if and only if it is the image under a coding of a fixed point of a k-uniform morphism. That is, $\mathbf{x}$ is k-automatic iff $\mathbf{x} = \tau(\sigma^\omega(a))$ for some k-uniform morphism σ prolongable on a and some coding τ.*

This is why the terms "k-automatic" and "image of a k-uniform morphism fixed point under a coding" are used interchangeably. The proof constructs the automaton from the morphism and vice versa; the states of the automaton correspond to the columns of the morphism iteration.

5.8 Cobham's Theorem

The following is one of the deepest results in the theory of automatic sequences.

Definition 5.24. Two integers $k, \ell \geq 2$ are *multiplicatively independent* if $\log k / \log \ell \notin \mathbb{Q}$, i.e., there are no positive integers m, n with $k^m = \ell^n$. For example, 2 and 3 are multiplicatively independent; 2 and 4 are not (since $4 = 2^2$).

Theorem 5.25 (Cobham 1969)**.** *Let k and ℓ be multiplicatively independent integers ≥ 2. If a sequence $\mathbf{x}$ is both k-automatic and ℓ-automatic, then $\mathbf{x}$ is eventually periodic.*

This is a rigidity theorem: it says that the only sequences recognizable by finite automata in two multiplicatively independent bases are the trivial ones. The Thue-Morse sequence is 2-automatic but not 3-automatic (and it cannot be, since it is not eventually periodic).

Cobham's original proof (1972) was notoriously difficult. Simpler proofs were given by Durand (2011) using dynamical methods and by Adamczewski and Bell (2011). The result has deep connections to the model theory of $\langle \mathbb{N}, +, V_k \rangle$ where $V_k(n)$ is the largest power of k dividing n.

Remark 5.26. For multiplicatively *dependent* k, ℓ (so $k = p^a$, $\ell = p^b$ for some p, a, b), the classes of k-automatic and ℓ-automatic sequences coincide. This is because base p^a and base p^b representations are trivially inter-convertible by grouping digits.

5.9 Automatic vs. Morphic: The Hierarchy

The relationships among these classes are:

$$\text{eventually periodic} \subsetneq k\text{-automatic} \subsetneq \text{morphic} \subsetneq \text{all sequences.}$$

Key facts:

1. Every eventually periodic sequence is k-automatic for all $k \geq 2$.
2. The Thue-Morse sequence is 2-automatic but not eventually periodic.
3. The Fibonacci word is morphic but not k-automatic for any k (its factor complexity is $n + 1$, but automatic sequences have factor complexity $\Theta(n)$ only in special cases; more precisely, a non-eventually-periodic automatic sequence has factor complexity $\Theta(n)$, while the Fibonacci word is Sturmian, so it is on the boundary—but the Fibonacci word fails to be automatic because its letter frequencies are irrational; see below).
4. A morphic sequence on a two-letter alphabet is k-automatic iff the letter frequencies are rational (Cobham). The Fibonacci word has frequency of 0 equal to $1/\phi$, which is irrational.

5.10 The Substitution Dynamical System

5.10.1 5.10.1 Construction

Given a morphism $\sigma : \Sigma \to \Sigma^*$ with fixed point $\mathbf{x} = \sigma^\omega(a)$, define the *substitution subshift*

$$X_\sigma = \overline{\{T^n(\mathbf{x}) : n \geq 0\}} \subseteq \Sigma^\omega,$$

where T is the shift and the closure is in the product topology. This is a compact, shift-invariant subset of Σ^ω.

Theorem 5.27. *If σ is primitive, then (X_σ, T) is: 1. Minimal: every orbit is dense (equivalently, every element of X_σ has the same set of factors as $\mathbf{x}$). 2. Uniquely ergodic: there exists a unique shift-invariant probability measure μ on X_σ.*

These are consequences of the Perron-Frobenius theory applied to the incidence matrix M_σ and the resulting uniform frequency estimates for factors.

5.10.2 5.10.2 Recognizability

Theorem 5.28 (Mossé 1992)**.** *If σ is primitive and aperiodic (the subshift is infinite), then σ is recognizable: every bi-infinite sequence in the two-sided version of X_σ has a unique decomposition into images of letters under σ. Formally, the "desubstitution map" is well-defined and continuous.*

Recognizability is the substitution analogue of the Markov partition property and is fundamental for computing invariants of the dynamical system.

5.11 Spectral Theory

Given a primitive substitution σ with substitution matrix M_σ, the associated dynamical system (X_σ, T, μ) is a measure-preserving system (with μ the unique invariant measure). A central question is: what is the spectral type of the unitary operator $U_T : L^2(X_\sigma, \mu) \to L^2(X_\sigma, \mu)$ defined by $U_T f = f \circ T$?

Definition 5.29. A measure-preserving system has *pure point (discrete) spectrum* if L^2 has an orthonormal basis of eigenfunctions of U_T.

The spectral type determines many dynamical properties: pure point spectrum implies the system is measurably conjugate to a rotation on a compact abelian group (the Halmos-von Neumann theorem).

Theorem 5.30 (Pisot conjecture, partially resolved)**.** *Let σ be a primitive aperiodic substitution with Perron-Frobenius eigenvalue θ. If θ is a Pisot number (an algebraic integer > 1 all of whose Galois conjugates have absolute value < 1), then the substitution dynamical system (X_σ, T, μ) has pure point spectrum.*

The converse—that pure point spectrum implies the Pisot condition—has been established for two-letter alphabets (Barge and Diamond, 2002; Barge, 2016 for the full result on irreducible Pisot substitutions). The general Pisot conjecture remains open but is one of the most important problems in the field.

Example 5.31. The Fibonacci morphism has Perron-Frobenius eigenvalue $\phi = (1 + \sqrt{5})/2$, which is a Pisot number (its conjugate is $(1 - \sqrt{5})/2 \approx -0.618$, with absolute value < 1). The Fibonacci substitution dynamical system therefore has pure point spectrum and is measurably conjugate to a rotation on the torus.

Example 5.32. The Thue-Morse morphism has Perron-Frobenius eigenvalue 2, which is a Pisot number (it has no other conjugates). However, the substitution matrix $M_\mu = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right)$ has second eigenvalue 0, which is a degenerate case. In fact, the Thue-Morse system does *not* have pure point spectrum; it has a partly continuous spectral component. This is consistent with the Pisot conjecture because the matrix M_μ is not irreducible in the required sense (the substitution is on a two-letter alphabet with both rows identical, so the second eigenvalue is 0 and the algebraic condition degenerates).

5.12 Connection to Our Framework

In the framework of discrete dynamical systems (f, x) studied in earlier chapters, a substitution system fits as follows:

- **State space** X: the set Σ^* of all finite words (or, in the limit, Σ^ω).
- **The map** f: the morphism σ itself. Each application of σ takes a word and produces a longer word.
- **Iteration**: starting from a single letter a, we compute $a, \sigma(a), \sigma^2(a), \ldots$
- **The fixed point as attractor**: the infinite word $\sigma^\omega(a)$ is the "attractor" of this iteration—every finite prefix eventually stabilizes.

There is an important difference from the finite DDS of earlier chapters: in a substitution system, the state space grows with each step (the words get longer). The dynamics is *non-contracting.* Nevertheless, when we pass to the orbit closure under the shift, we obtain a compact dynamical system (X_σ, T) on which the usual tools of topological and ergodic dynamics apply.

The meta-rule perspective also applies: a morphism σ can be viewed as a rule that operates on the structure of the word at every position simultaneously. In the language of Section 2, if we think of the current word as encoding both the "state" and the "rule" (since the positions determine which substitution rules fire), then the substitution system is a DDS where the function and the data are intertwined.

5.13 Summary

Sequence	Morphism	Uniform?	Automatic?	Morphic?	Complexity $p(n)$	Spect
Thue-Morse	$0 \mapsto 01, 1 \mapsto 10$	2-uniform	2-automatic	Yes	$\Theta(n)$	Mix
Fibonacci	$0 \mapsto 01, 1 \mapsto 0$	Non-uniform	No	Yes	$n+1$ (Sturmian)	Pure
Rudin-Shapiro	$0 \mapsto 02, \ldots +$ coding	2-uniform	2-automatic	Yes	$\Theta(n)$	Mix
Period-doubling	$0 \mapsto 01, 1 \mapsto 00$	2-uniform	2-automatic	Yes	$\Theta(n)$	Pure

The theory of substitution systems and automatic sequences lies at a crossroads of combinatorics on words, number theory, formal language theory, and dynamical systems. The interplay between the combinatorial properties of the morphism and the dynamical and spectral properties of the associated subshift is one of the richest areas in modern mathematics.

5.14 References

- Allouche, J.-P. and Shallit, J. (2003). *Automatic Sequences: Theory, Applications, Generalizations*. Cambridge University Press. The standard reference for automatic sequences and morphic words.
- Cobham, A. (1969). On the base-dependence of sets of numbers recognizable by finite automata. *Mathematical Systems Theory*, 3:186–192. The original statement of Cobham's theorem.
- Cobham, A. (1972). Uniform tag sequences. *Mathematical Systems Theory*, 6:164–192. Characterization of k-automatic sequences as images of k-uniform morphism fixed points.
- Durand, F. (2011). Cobham's theorem for substitutions. *Journal of the European Mathematical Society*, 13(6):1797–1812. A dynamical proof of Cobham's theorem.
- Eilenberg, S. (1974). *Automata, Languages, and Machines*, Vol. A. Academic Press. The kernel characterization of automatic sequences.
- Fogg, N. P. (2002). *Substitutions in Dynamics, Arithmetics, and Combinatorics*. Lecture Notes in Mathematics, Vol. 1794, Springer. Comprehensive treatment of substitution dynamical systems.
- Mossé, B. (1992). Puissances de mots et reconnaissabilité des points fixes d'une substitution. *Theoretical Computer Science*, 99(2):327–334. Recognizability of primitive aperiodic substitutions.
- Queffélec, M. (2010). *Substitution Dynamical Systems—Spectral Analysis*. Lecture Notes in Mathematics, Vol. 1294, 2nd edition, Springer. Spectral theory of substitution systems, including the Pisot conjecture.
- Rudin, W. (1959). Some theorems on Fourier coefficients. *Proceedings of the AMS*, 10(6):855–859.
- Thue, A. (1906). Über unendliche Zeichenreihen. *Norske Vid. Selsk. Skr. I Mat.-Nat. Kl.*, 7:1–22. The original construction of the overlap-free sequence.

5.15 Recommended Reading

For an accessible introduction:

- **Allouche & Shallit (2003)** is the definitive reference and surprisingly readable. Chapters 1–5 cover the basics of morphisms and automatic sequences; Chapters 6–7 cover Cobham's theorems.

For the dynamical systems perspective:

- **Queffélec (2010)** is essential for spectral theory. The first few chapters are accessible; later chapters require measure theory and functional analysis.

- **Fogg (2002)** is a collaborative volume (written by multiple authors under a pseudonym) covering substitutions from combinatorial, dynamical, and arithmetic viewpoints.

For combinatorics on words:

- **Lothaire, M.** (2002). *Algebraic Combinatorics on Words.* Cambridge University Press. Chapters 1–2 cover morphisms and Sturmian words; more advanced than Allouche-Shallit in some directions.

For number-theoretic connections:

- **Adamczewski, B. and Bugeaud, Y.** (2010). "Transcendence and Diophantine approximation." In *Bentlnagar et al. eds., Combinatorics, Automata and Number Theory.* Cambridge University Press. Deep connections between automatic sequences and transcendental number theory.

For the Pisot conjecture and related topics:

- **Barge, M.** (2016). "The Pisot conjecture for β-substitutions." *Ergodic Theory and Dynamical Systems*, 36(5):1501–1534. Recent progress on one of the main open problems in the field.

Chapter 6

Iterated Function Systems

6.1 Introduction

An **iterated function system** (IFS) is one of the most elegant constructions in dynamical systems and fractal geometry. The idea is simple: take a finite collection of maps on a space, apply them repeatedly, and ask what sets are invariant under the collective action of all the maps. The resulting theory, initiated by Hutchinson (1981) and popularized by Barnsley (1988), provides a rigorous framework for constructing and analyzing fractals, and connects deeply to probability theory, dimension theory, and—as we shall see—the discrete dynamical systems framework of this text.

6.2 Basic Definitions

Definition 6.1 (Iterated Function System)**.** Let (X, d) be a complete metric space. An *iterated function system* (IFS) is a finite collection of continuous maps

$$\mathcal{F} = \{f_1, f_2, \ldots, f_k\}$$

where each $f_i : X \to X$.

The system operates by repeatedly choosing one of the maps f_i and applying it. The central question is: what geometric or measure-theoretic structures are invariant under this process?

Before we can answer this, we need the key analytic ingredient.

Definition 6.2 (Contraction Mapping)**.** A map $f : X \to X$ on a metric space (X, d) is a *contraction* (or *contraction mapping*) if there exists a constant $0 \le c < 1$ such that

$$d(f(x), f(y)) \le c \cdot d(x, y) \quad \text{for all } x, y \in X.$$

The constant c is called the *contraction ratio* (or *Lipschitz constant*) of f. Intuitively, a contraction brings points closer together by at least a factor of c at every step.

The foundational result about contractions is:

Theorem 6.3 (Banach Fixed-Point Theorem, 1922)**.** *Let (X, d) be a complete metric space and $f : X \to X$ a contraction with ratio $c < 1$. Then:*

1. f has a unique fixed point $x^* \in X$, i.e., $f(x^*) = x^*$. 2. For any starting point $x_0 \in X$, the iterates $f^n(x_0) \to x^*$ as $n \to \infty$. 3. The convergence rate is geometric: $d(f^n(x_0), x^*) \le \frac{c^n}{1-c}\, d(x_0, f(x_0))$.

Proof sketch. The sequence $x_n = f^n(x_0)$ is Cauchy because $d(x_{n+1}, x_n) \le c^n\, d(x_1, x_0)$. Completeness gives a limit x^*. Continuity of f gives $f(x^*) = x^*$. Uniqueness: if $f(y) = y$ too, then $d(x^*, y) = d(f(x^*), f(y)) \le c\, d(x^*, y)$, which forces $d(x^*, y) = 0$.

The genius of the IFS construction is to apply the Banach theorem not to individual points, but to sets.

6.3 The Hutchinson Operator

To study how an IFS acts on sets rather than points, we need a metric on sets.

Definition 6.4 (Hausdorff Metric)**.** Let $\mathcal{H}(X)$ denote the collection of all non-empty compact subsets of (X, d). For $A, B \in \mathcal{H}(X)$, define

$$d_H(A, B) = \max\left\{\sup_{a\in A}\inf_{b\in B} d(a, b),\ \sup_{b\in B}\inf_{a\in A} d(a, b)\right\}.$$

This is the *Hausdorff metric*. Informally, $d_H(A, B) \leq \epsilon$ means every point of A is within ϵ of some point of B, and vice versa.

Fact. If (X, d) is complete, then $(\mathcal{H}(X), d_H)$ is also complete (see Falconer 2003, Ch. 2).

Definition 6.5 (Hutchinson Operator)**.** Given an IFS $\mathcal{F} = \{f_1, \ldots, f_k\}$, the *Hutchinson operator* $H : \mathcal{H}(X) \to \mathcal{H}(X)$ is defined by

$$H(A) = \bigcup_{i=1}^{k} f_i(A) = \bigcup_{i=1}^{k} \{f_i(a) : a \in A\}.$$

That is, H applies every map in the IFS to A and takes the union. A set A is *invariant* under the IFS if $H(A) = A$: it can be decomposed into k pieces, each of which is the image of the whole under one of the maps.

6.4 Hutchinson's Theorem

The following is the central theorem of IFS theory.

Theorem 6.6 (Hutchinson, 1981)**.** *Let (X, d) be a complete metric space, and let $\mathcal{F} = \{f_1, \ldots, f_k\}$ be an IFS where each f_i is a contraction with ratio $c_i < 1$. Then:*

1. The Hutchinson operator H is a contraction on $(\mathcal{H}(X), d_H)$ with ratio $c = \max_i c_i$. 2. There exists a unique non-empty compact set $A \in \mathcal{H}(X)$ such that $H(A) = A$. 3. For any non-empty compact set $B \in \mathcal{H}(X)$, the iterates $H^n(B) \to A$ in the Hausdorff metric.

The set A is called the ***attractor*** *(or invariant set) of the IFS.*

Proof of (1). Let $A, B \in \mathcal{H}(X)$. For any $a \in A$ and any i,

$$\inf_{b\in B} d(f_i(a), f_i(b)) \leq c_i \inf_{b\in B} d(a, b).$$

Taking the supremum over a and all i, and applying the symmetric argument, yields

$$d_H(H(A), H(B)) \leq c \cdot d_H(A, B).$$

Since $c < 1$, H is a contraction. Parts (2) and (3) then follow from the Banach fixed-point theorem applied to H on the complete metric space $(\mathcal{H}(X), d_H)$.

6.5 Classical Examples

Example: The Cantor Set

Consider $X = [0, 1]$ with the usual metric, and the IFS

$$f_1(x) = \frac{x}{3}, \qquad f_2(x) = \frac{x}{3} + \frac{2}{3}.$$

Both maps are contractions with ratio $c = 1/3$. The attractor A satisfies

$$A = f_1(A) \cup f_2(A) = \frac{A}{3} \cup \left(\frac{A}{3} + \frac{2}{3}\right).$$

Worked computation. Start with $B_0 = [0, 1]$:

- $H(B_0) = [0, 1/3] \cup [2/3, 1] = B_1$ (the first step of the Cantor construction).
- $H(B_1) = [0, 1/9] \cup [2/9, 1/3] \cup [2/3, 7/9] \cup [8/9, 1] = B_2$.
- Continuing, B_n consists of 2^n intervals each of length 3^{-n}.
- The attractor $A = \bigcap_{n=0}^{\infty} B_n$ is the classical **middle-thirds Cantor set**.

The Cantor set is uncountable, has Lebesgue measure zero, and is totally disconnected. Its Hausdorff dimension is $\log 2/\log 3 \approx 0.631$ (see Section 6.8).

Example: The Sierpinski Triangle

Let $X = \mathbb{R}^2$ and fix three vertices of an equilateral triangle, say $p_1 = (0, 0)$, $p_2 = (1, 0)$, $p_3 = (1/2, \sqrt{3}/2)$. Define

$$f_i(x) = \frac{x + p_i}{2}, \qquad i = 1, 2, 3.$$

Each f_i is a contraction with ratio $c = 1/2$. The attractor is the **Sierpinski triangle** (or Sierpinski gasket): the self-similar fractal obtained by repeatedly removing the central triangle from each remaining triangle.

Worked computation. Start with $B_0 =$ the filled triangle with vertices p_1, p_2, p_3:

- $H(B_0)$ consists of three half-scale copies of B_0, one anchored at each vertex. The central triangle (with vertices at the midpoints of B_0's edges) is missing.
- $H^2(B_0)$: each of the three sub-triangles has its own central triangle removed, giving $3^2 = 9$ triangles at scale $1/4$.
- At step n: 3^n triangles at scale 2^{-n}.
- The attractor $A = \lim_{n\to\infty} H^n(B_0)$ has Hausdorff dimension $\log 3/\log 2 \approx 1.585$.

Example: The Koch Curve

The Koch curve can be described as the attractor of an IFS with four maps on $\mathbb{R}^2$, each a similarity with ratio $c = 1/3$:

$$f_1(x) = \frac{1}{3}x, \qquad f_2(x) = \frac{1}{3}R_{60}(x) + \left(\frac{1}{3}, 0\right),$$

$$f_3(x) = \frac{1}{3}R_{-60}(x) + \left(\frac{1}{2}, \frac{\sqrt{3}}{6}\right), \qquad f_4(x) = \frac{1}{3}x + \left(\frac{2}{3}, 0\right),$$

where R_θ denotes counterclockwise rotation by θ degrees. Each map takes the unit interval $[0, 1]$ (embedded in $\mathbb{R}^2$) and maps it to one of the four segments of the Koch generator. The attractor has Hausdorff dimension $\log 4/\log 3 \approx 1.262$.

6.6 The Chaos Game

A remarkable fact about IFS attractors is that they can be generated by a simple stochastic algorithm.

Algorithm (The Chaos Game / Random Iteration Algorithm). Given an IFS $\{f_1, \ldots, f_k\}$:

1. Choose an arbitrary starting point $x_0 \in X$.
2. At each step n, choose an index $i_n \in \{1, \ldots, k\}$ uniformly at random (or with specified probabilities).
3. Set $x_{n+1} = f_{i_n}(x_n)$.
4. After discarding a transient (say the first 100 points), the orbit $\{x_n\}$ fills in the attractor A.

Theorem 6.7 (Barnsley & Demko, 1985)**.** *Let $\{f_1, \ldots, f_k\}$ be a contractive IFS on a complete metric space, with probabilities $p_1, \ldots, p_k > 0$ (summing to 1) assigned to the maps. Then for any starting point x_0, the empirical distribution of the orbit $\{x_0, x_1, x_2, \ldots\}$ converges almost surely (in the weak topology) to a unique invariant probability measure μ supported on the attractor A.*

Why does this work? *The orbit is a Markov chain on X. Because the maps are contractions, the chain "forgets" its initial condition exponentially fast. More precisely, two orbits starting from different points x_0 and y_0 but driven by the same random sequence of maps satisfy*

$$d(x_n, y_n) \leq c^n \, d(x_0, y_0) \to 0.$$

This coupling argument shows the stationary measure is unique. The orbit is dense in the support of μ, which is precisely A.

Worked example (Chaos game for the Sierpinski triangle). *Take the three maps from Example 6.8. Start at any point, say $x_0 = (0.5, 0.2)$. At each step, pick one of f_1, f_2, f_3 uniformly at random and apply it:*

- Step 1: Choose f_2. $x_1 = (x_0 + p_2)/2 = (0.75, 0.1)$. - Step 2: Choose f_1. $x_2 = (x_1 + p_1)/2 = (0.375, 0.05)$. - Step 3: Choose f_3. $x_3 = (x_2 + p_3)/2 = (0.4375, 0.4580\ldots)$.

After a few hundred iterations, plotting the points $\{x_n\}$ reveals the Sierpinski triangle. This works regardless of the starting point x_0—the first few points may lie outside the attractor, but they rapidly converge to it.

6.7 IFS with Probabilities and the Invariant Measure

Assigning probabilities to the maps gives a richer structure than the attractor alone.

Definition 6.8 (IFS with Probabilities)**.** An IFS with probabilities is a system $\{(f_1, p_1), \ldots, (f_k, p_k)\}$ where $p_i > 0$ and $\sum_{i=1}^{k} p_i = 1$. The *transfer operator* (or Markov operator) M acts on probability measures by

$$M(\mu) = \sum_{i=1}^{k} p_i \cdot (f_i)_*(\mu),$$

where $(f_i)_*(\mu)$ is the pushforward measure: $(f_i)_*(\mu)(B) = \mu(f_i^{-1}(B))$.

Theorem 6.9. *Under the hypotheses of Theorem 6.6, with $p_i > 0$, the operator M has a unique fixed point μ^*—a probability measure satisfying*

$$\mu^* = \sum_{i=1}^{k} p_i \cdot (f_i)_*(\mu^*).$$

*This is the **invariant measure** (or **self-similar measure**) of the IFS. The chaos game orbit converges in distribution to μ^*.*

The self-similarity equation says that μ^* can be decomposed: a fraction p_i of its mass is a scaled copy under f_i. Different probability vectors $(p_1, \ldots, p_k)$ produce different measures on the same attractor, concentrating mass on different parts. If all $p_i = 1/k$, the measure is the "most uniform" one on A (though it need not coincide with the Hausdorff measure in the natural dimension).

6.8 Hausdorff Dimension of IFS Attractors

For self-similar IFS, there is a clean formula for the fractal dimension.

Definition 6.10. An IFS $\{f_1, \ldots, f_k\}$ on $\mathbb{R}^n$ is **self-similar** if each f_i is a similarity: $\|f_i(x) - f_i(y)\| = c_i\|x - y\|$ for some contraction ratio $0 < c_i < 1$.

Definition 6.11 (Open Set Condition)**.** The IFS satisfies the *open set condition* (OSC) if there exists a non-empty bounded open set $U \subset \mathbb{R}^n$ such that

$$\bigcup_{i=1}^{k} f_i(U) \subseteq U \quad \text{and} \quad f_i(U) \cap f_j(U) = \emptyset \text{ for } i \neq j.$$

That is, the images of U under the maps are disjoint and fit inside U. This is a controlled-overlap condition ensuring the fractal pieces do not overlap "too much."

Theorem 6.12 (Moran–Hutchinson)**.** *Let $\{f_1, \ldots, f_k\}$ be a self-similar IFS with contraction ratios $c_1, \ldots, c_k$ satisfying the open set condition. Then the Hausdorff dimension s of the attractor A is the unique solution to the **Moran equation**:*

$$\sum_{i=1}^{k} c_i^s = 1.$$

Worked example. *For the Cantor set (Example 6.7): $k = 2$, $c_1 = c_2 = 1/3$. The Moran equation is*

$$2 \cdot (1/3)^s = 1 \implies (1/3)^s = 1/2 \implies s = \frac{\log 2}{\log 3} \approx 0.6309.$$

For the Sierpinski triangle (Example 6.8): $k = 3$, $c_i = 1/2$. Then $3 \cdot (1/2)^s = 1$, giving $s = \log 3/\log 2 \approx 1.5850$.

For the Koch curve (Example 6.9): $k = 4$, $c_i = 1/3$. Then $4 \cdot (1/3)^s = 1$, giving $s = \log 4/\log 3 \approx 1.2619$.

When the open set condition fails (excessive overlap), the Hausdorff dimension may be strictly less than the similarity dimension given by the Moran equation. Determining the dimension in such cases is generally difficult.

6.9 IFS on Finite and Discrete Sets

On a finite set X with $|X| = n$, every map $f : X \to X$ satisfies $d(f(x), f(y)) \leq d(x, y)$ trivially for any discrete metric $d(x, y) = \mathbf{1}[x \neq y]$, but strict contraction ($c < 1$) would require f

to be constant. Thus the metric theory of IFS does not directly apply to finite sets—every non-constant map has contraction ratio 1 in the discrete metric.

Instead, the natural objects of study are:

1. **The semigroup** $\langle f_1, \ldots, f_k \rangle$ generated by the maps under composition. For $|X| = n$, this is a sub-semigroup of the full transformation semigroup T_n (which has n^n elements). The size and structure of this semigroup characterize the reachability and complexity of the IFS.

1. **The functional graph** of random iteration. Fix a probability vector $(p_1, \ldots, p_k)$. The state space of the random process is X. The transition matrix is

$$P(x, y) = \sum_{i: f_i(x) = y} p_i.$$

This is a Markov chain on X. Its stationary distributions, mixing times, and absorbing states are the finite analogues of the invariant measure and attractor.

1. **Eventual periodicity.** Every orbit of a single map on a finite set is eventually periodic (it must enter a cycle). For an IFS with random map selection, the system is a Markov chain and (if irreducible) has a unique stationary distribution. If not irreducible, it decomposes into communicating classes—the analogue of the attractor's connected components.

Example 6.13. Let $X = \{0, 1, 2\}$ and consider the IFS $\{f_1, f_2\}$ where $f_1 = (0, 0, 1)$ and $f_2 = (1, 2, 2)$ (function tables). The semigroup generated by f_1, f_2 under composition includes:

- $f_1 \circ f_1 = (0, 0, 0)$ (constant map to 0) - $f_2 \circ f_2 = (2, 2, 2)$ (constant map to 2) - $f_1 \circ f_2 = (0, 1, 1)$ - $f_2 \circ f_1 = (1, 1, 2)$

The semigroup eventually includes all compositions; its structure reveals which states are reachable from which, under which sequences of map applications.

6.10 Recurrent IFS

Barnsley, Elton, Hardin, and Massopust (1989) introduced a generalization where the choice of map at each step depends on which map was applied in the previous step.

Definition 6.14 (Recurrent IFS)**.** A *recurrent IFS* consists of:

- Maps $\{f_1, \ldots, f_k\}$ on (X, d).
- A $k \times k$ transition matrix $T = (t_{ij})$ where $t_{ij} \geq 0$ and $\sum_j t_{ij} = 1$.

The joint state at time n is (i_n, x_n) where i_n is the index of the map just applied. The dynamics are:

1. Given (i_n, x_n), choose i_{n+1} according to the distribution $(t_{i_n,1}, \ldots, t_{i_n,k})$. 2. Set $x_{n+1} = f_{i_{n+1}}(x_n)$.

This is a Markov chain on $\{1, \ldots, k\} \times X$. By restricting which maps can follow which (setting some $t_{ij} = 0$), one obtains attractors that are strict subsets of the standard IFS attractor—only "allowed" compositions contribute.

Recurrent IFS can produce attractors with different connectivity or topology than the standard IFS, and they model situations where the rule for updating depends on context.

6.11 The Collage Theorem

A natural inverse problem: given a target shape T, find an IFS whose attractor approximates T.

Theorem 6.15 (Collage Theorem; Barnsley 1988)**.** *Let $\{f_1, \ldots, f_k\}$ be a contractive IFS with ratio $c = \max_i c_i < 1$ and attractor A. For any non-empty compact set $T \in \mathcal{H}(X)$,*

$$d_H(T, A) \leq \frac{1}{1-c}\, d_H(T, H(T)).$$

Interpretation. *To find an IFS whose attractor is close to T, it suffices to find maps such that $H(T) = \bigcup f_i(T)$ is close to T (the "collage" of the pieces approximates the original). The theorem bounds the attractor's distance to T in terms of the collage distance, amplified by $1/(1-c)$.*

This is the theoretical basis for ***fractal image compression*** *(Barnsley 1988; Jacquin 1992): encode an image by an IFS whose attractor approximates it. The IFS description (a few maps with their parameters) can be far more compact than the pixel data.*

6.12 Connection to the (f, x) Framework

The framework developed in earlier chapters studies systems with state (f, x) where $f : X \to X$ and $x \in X$, evolving under a meta-rule ϕ that updates the function at each step:

$$(f, x) \mapsto (\phi(f, x),\ f(x)).$$

An IFS is the special case where the "function" component is not updated by a deterministic meta-rule but rather *selected* from a fixed repertoire $\{f_1, \ldots, f_k\}$ at each step, either randomly or according to some schedule. That is, we can view the IFS dynamics as a sequence of states

$$(f_{i_0}, x_0) \to (f_{i_1}, x_1) \to (f_{i_2}, x_2) \to \cdots$$

where $x_{n+1} = f_{i_n}(x_n)$ and the sequence $i_0, i_1, i_2, \ldots$ is determined by:

- **Random IFS**: i_n drawn i.i.d. or as a Markov chain (recurrent IFS).
- **Deterministic schedule**: $i_n = \sigma(n)$ for some fixed sequence σ, e.g., cyclic application $f_1, f_2, f_1, f_2, \ldots$.
- **State-dependent selection**: $i_n = g(x_n)$ for some selector function $g : X \to \{1, \ldots, k\}$, making the choice of map depend on the current point. This is a meta-rule: $\phi(f, x) = f_{g(x)}$.

In the discrete (finite X) setting, the multi-map system $\{f_1, \ldots, f_k\}$ acting on X generates a semigroup of transformations. The functional graph of the random iteration is a Markov chain whose structure (communicating classes, absorbing sets, stationary distributions) parallels the attractor and invariant measure of the classical theory. The meta-rule ϕ that selects which map to apply is the mechanism that ties IFS to the broader framework: **multiple agents (maps) acting on a shared state, with a protocol (meta-rule) governing which agent acts when**.

6.13 Summary

Concept	Continuous setting	Finite/discrete setting
Maps	Contractions on (X, d)	Arbitrary maps $X \to X$
Invariant object	Attractor $A = H(A)$	Absorbing sets, recurrent classes
Convergence	$H^n(B) \to A$ (Hausdorff)	Markov chain mixing
Measure	Invariant measure μ^*	Stationary distribution π
Dimension	Hausdorff dim via Moran eq.	Not applicable (dim = 0)
Meta-rule	Map selection protocol	$\phi(f, x) = f_{g(x)}$

The IFS framework transforms the study of single-map iteration into the far richer study of multi-map interaction. In the continuous setting this yields fractal geometry; in the discrete setting it yields semigroup theory and Markov chains. Both are instances of the general (f, x) dynamical system with a non-trivial meta-rule.

6.14 References

- Banach, S. (1922). Sur les opérations dans les ensembles abstraits et leur application aux équations intégrales. *Fundamenta Mathematicae*, 3, 133–181.
- Barnsley, M. F. (1988). *Fractals Everywhere.* Academic Press. (2nd edition 1993, 3rd edition 2012.)
- Barnsley, M. F. & Demko, S. (1985). Iterated function systems and the global construction of fractals. *Proceedings of the Royal Society of London A*, 399, 243–275.
- Barnsley, M. F., Elton, J. H., Hardin, D. P., & Massopust, P. R. (1989). Hidden variable fractal interpolation functions. *SIAM Journal on Mathematical Analysis*, 20(5), 1218–1242.
- Falconer, K. (2003). *Fractal Geometry: Mathematical Foundations and Applications* (2nd ed.). Wiley.
- Hutchinson, J. E. (1981). Fractals and self-similarity. *Indiana University Mathematics Journal*, 30(5), 713–747.
- Jacquin, A. E. (1992). Image coding based on a fractal theory of iterated contractive image transformations. *IEEE Transactions on Image Processing*, 1(1), 18–30.
- Moran, P. A. P. (1946). Additive functions of intervals and Hausdorff measure. *Mathematical Proceedings of the Cambridge Philosophical Society*, 42(1), 15–23.

6.15 Recommended Reading

For an accessible introduction to IFS and fractals:

- **Barnsley (1988/2012)**, *Fractals Everywhere*, remains the most accessible introduction. The exposition is visual and builds intuition through many examples and exercises.
- **Peitgen, Jürgens, and Saupe** (2004). *Chaos and Fractals: New Frontiers of Science*, 2nd ed. Springer. A comprehensive and beautifully illustrated treatment for a general scientific audience.

For rigorous mathematical foundations:

- **Falconer (2003)** is the standard mathematical reference on fractal geometry. Chapters 2–3 cover IFS and dimension theory with full proofs.

- **Hutchinson (1981)** is the original paper establishing the mathematical framework. Highly readable and still the cleanest source for the core theorems.

For connections to probability and ergodic theory:

- **Diaconis, P. and Freedman, D.** (1999). "Iterated random functions." *SIAM Review*, 41(1), 45–76. Beautiful paper connecting IFS to the theory of Markov chains and coupling.

For applications in image compression:

- **Fisher, Y.** (1995). *Fractal Image Compression: Theory and Application.* Springer. Comprehensive treatment of the algorithmic applications of the Collage Theorem.

For dimension theory beyond IFS:

- **Mattila, P.** (1995). *Geometry of Sets and Measures in Euclidean Spaces.* Cambridge University Press. Advanced treatment of Hausdorff measure and dimension, essential for understanding when the Moran equation gives the exact dimension.

Chapter 7

Cellular Automata as Dynamical Systems

7.1 Definition

A **cellular automaton** (CA) consists of the following data:

1. A **lattice** Λ, typically $\mathbb{Z}$ (one-dimensional) or $\mathbb{Z}^d$ (d-dimensional).
2. A **finite state set** S with $|S| = k \geq 2$. Each cell $i \in \Lambda$ holds a state $s_i \in S$.
3. A **neighborhood** $\mathcal{N} = \{n_1, n_2, \ldots, n_m\} \subset \mathbb{Z}^d$, a fixed finite set of offset vectors.
4. A **local update rule** $f : S^{|\mathcal{N}|} \to S$ that maps the states of a cell's neighbors to the cell's new state.

A **configuration** is a function $c : \Lambda \to S$, i.e., an element of S^Λ. The **global map** $F : S^\Lambda \to S^\Lambda$ is defined by applying f synchronously to every cell:

$$F(c)_i = f(c_{i+n_1}, c_{i+n_2}, \ldots, c_{i+n_m})$$

The space S^Λ is given the product topology (each copy of S is discrete), making it a Cantor set when Λ is infinite. The global map F is continuous in this topology and commutes with all lattice translations σ_v for $v \in \Lambda$:

$$F \circ \sigma_v = \sigma_v \circ F$$

This translation-invariance is a defining property of CA.

7.2 Elementary Cellular Automata

The simplest nontrivial case is the **elementary cellular automaton** (ECA):

- Lattice: $\mathbb{Z}$ (one-dimensional)
- State set: $S = \{0, 1\}$ (two states)
- Neighborhood: radius $r = 1$, so $\mathcal{N} = \{-1, 0, 1\}$ (three cells: left, center, right)

The local rule $f : \{0, 1\}^3 \to \{0, 1\}$ is determined by its values on the $2^3 = 8$ possible input triples. Since each output is in $\{0, 1\}$, there are $2^8 = 256$ possible ECA rules.

7.2.1 7.2.1 Wolfram's Numbering Scheme

Wolfram (1984) introduced a canonical numbering of these 256 rules. The 8 input triples are listed in descending binary order:

Input (left, center, right)	111	110	101	100	011	010	001	000
Bit position	7	6	5	4	3	2	1	0

The outputs for these 8 inputs form an 8-bit binary number, read from bit 7 (MSB) down to bit 0 (LSB). The decimal value of this binary number is the **rule number**.

Example. If the outputs are $0, 1, 1, 0, 1, 1, 1, 0$ (for inputs 111 through 000 respectively), the binary string is $01101110_2 = 110_{10}$, giving **Rule 110**.

7.2.2 7.2.2 Rule 30

Rule 30 has rule number $30 = 00011110_2$. The lookup table is:

Input	111	110	101	100	011	010	001	000
Output	0	0	0	1	1	1	1	0

An equivalent algebraic expression: $f(l, c, r) = l \oplus (c \vee r)$, where $\oplus$ is XOR and $\vee$ is OR.

7.2.3 7.2.3 Rule 110

Rule 110 has rule number $110 = 01101110_2$:

Input	111	110	101	100	011	010	001	000
Output	0	1	1	0	1	1	1	0

7.2.4 7.2.4 Rule 90

Rule 90 has rule number $90 = 01011010_2$:

Input	111	110	101	100	011	010	001	000
Output	0	1	0	1	1	0	1	0

Algebraically: $f(l, c, r) = l \oplus r$ (the center cell is ignored; only the XOR of the two neighbors matters).

7.3 Worked Examples: Tracing ECA Evolution

Example: Rule 90

We trace Rule 90 ($f(l, c, r) = l \oplus r$) from a single live cell on a background of zeros. We use a finite window, padding with 0 outside the shown region.

```
t=0:  . . . . 1 . . . .
t=1:  . . . 1 . 1 . . .
t=2:  . . 1 . 0 . 1 . .
t=3:  . 1 . 1 1 1 . 1 .
t=4:  1 . 0 . 0 . 0 . 1
```

Here "." denotes 0, and we trace a few cells explicitly for $t = 0 \to 1$:

- Cell 4 (center at $t = 0$): neighbors are $(0, 1, 0)$. $f(0, 1, 0) = 0 \oplus 0 = 0$.
- Cell 3: neighbors are $(0, 0, 1)$. $f(0, 0, 1) = 0 \oplus 1 = 1$.
- Cell 5: neighbors are $(1, 0, 0)$. $f(1, 0, 0) = 1 \oplus 0 = 1$.

Continuing this process generates the **Sierpinski triangle** modulo 2 – a self-similar fractal pattern. Each row t corresponds to the binomial coefficients $\binom{t}{j} \mod 2$.

Example: Rule 30

We trace Rule 30 ($f(l, c, r) = l \oplus (c \vee r)$) from a single live cell:

```
t=0:  . . . . 1 . . . .
t=1:  . . . 1 1 1 . . .
t=2:  . . 1 1 0 0 1 . .
t=3:  . 1 1 0 1 1 1 1 .
t=4:  1 1 0 0 1 0 0 0 1
```

Verification of $t = 1 \to 2$ for a few cells:

- Cell 3: neighborhood from $t = 1$ is $(0, 1, 1)$. $f = 0 \oplus (1 \vee 1) = 0 \oplus 1 = 1$.
- Cell 4: neighborhood is $(1, 1, 1)$. $f = 1 \oplus (1 \vee 1) = 1 \oplus 1 = 0$.
- Cell 5: neighborhood is $(1, 1, 0)$. $f = 1 \oplus (1 \vee 0) = 1 \oplus 1 = 0$.
- Cell 6: neighborhood is $(1, 0, 0)$. $f = 1 \oplus (0 \vee 0) = 1 \oplus 0 = 1$.

The resulting spacetime diagram appears **chaotic**: the pattern shows no discernible regularity, and the center column passes statistical randomness tests. Wolfram used this as the basis of a pseudorandom number generator.

Example: Rule 110

We trace Rule 110 from a single live cell, using the table from Section 7.2.3:

```
t=0:  . . . . 1 . . . .
t=1:  . . . 1 1 . . . .
t=2:  . . 1 1 1 . . . .
t=3:  . 1 1 0 1 . . . .
t=4:  1 1 0 0 1 . . . .
```

Verification of $t = 2 \to 3$ for a few cells:

- Cell 2: neighborhood from $t = 2$ is $(0, 1, 1)$. Rule 110: $f(0, 1, 1) = 1$.
- Cell 3: neighborhood is $(1, 1, 1)$. $f(1, 1, 1) = 0$.
- Cell 4: neighborhood is $(1, 1, 0)$. $f(1, 1, 0) = 1$.
- Cell 5: neighborhood is $(1, 0, 0)$. $f(1, 0, 0) = 0$ (but there is right-boundary interaction; on the infinite lattice with zeros, this holds).

From a single seed, Rule 110 produces a leftward-growing triangular region. From random initial conditions on a large lattice, however, it produces **complex** behavior: persistent localized structures ("particles" or "gliders") that move and interact against a periodic background. This complex behavior is the key to its computational universality (see Section 7.8).

7.4 Spacetime Diagrams

A **spacetime diagram** displays the evolution of a 1D CA. Space (cell index) runs along the horizontal axis, and time (generation number) runs downward. Each cell is rendered as a pixel colored by its state – typically black for 1 and white for 0.

The qualitative character of the spacetime diagram is the basis for classification:

- **Rule 30**: The diagram from a single seed produces an irregular, apparently random pattern. No spatial or temporal periodicity is evident, even after millions of steps. The left side develops a nested structure, but the right side and center column appear fully disordered.

- **Rule 110**: The diagram from random initial conditions shows a periodic striped background punctuated by localized structures (gliders) that propagate at various speeds. Collisions between gliders produce new gliders or annihilate. This is the hallmark of Class 4 behavior.

- **Rule 90**: The diagram from a single seed produces the Sierpinski triangle – a self-similar fractal. At scale 2^n, the pattern repeats exactly. The Hausdorff dimension of the limit set is $\log_2 3 \approx 1.585$.

7.5 Wolfram's Four Classes (1984)

Based on extensive computational experiments, Wolfram (1984) proposed a classification of CA into four behavioral classes:

Class 1: Homogeneous. Almost all initial conditions rapidly converge to a single uniform fixed point. Example: **Rule 0** (all cells map to 0) and **Rule 32**.

Class 2: Periodic. Almost all initial conditions converge to periodic structures (fixed points or limit cycles), possibly with a short transient. The final pattern may be spatially inhomogeneous but is temporally periodic. Examples: **Rule 4**, **Rule 108**, **Rule 184**.

Class 3: Chaotic. Almost all initial conditions produce aperiodic, apparently random behavior. The spacetime diagram looks disordered. Statistically, the patterns are indistinguishable from noise by standard tests. Examples: **Rule 30**, **Rule 45**, **Rule 73**.

Class 4: Complex. The evolution produces long-lived, localized propagating structures (particles, gliders) on a periodic or quiescent background. These structures interact in complicated ways. Class 4 sits "at the edge of chaos" between Classes 2 and 3. Examples: **Rule 110**, **Rule 54**.

Wolfram conjectured that Class 4 rules are capable of universal computation, and that Class 4 behavior arises at a phase transition between order (Classes 1–2) and chaos (Class 3). While suggestive, this classification is informal: there is no precise mathematical definition of the four classes, and some rules are difficult to assign unambiguously. Wolfram further developed these ideas in *A New Kind of Science* (2002).

7.6 Kurka's Topological Classification (1997)

Kurka (1997) proposed a rigorous topological classification of CA based on the theory of equicontinuity in topological dynamics. This gives a mathematically precise refinement of Wolfram's informal scheme.

Let (X, F) be the dynamical system where $X = S^{\mathbb{Z}}$ with the product topology (metrized by the Cantor metric d) and F is the CA global map. Define:

- A point $x \in X$ is an **equicontinuity point** if for every $\varepsilon > 0$, there exists $\delta > 0$ such that $d(x, y) < \delta$ implies $d(F^n(x), F^n(y)) < \varepsilon$ for all $n \geq 0$.

- Let $E_F \subseteq X$ denote the set of equicontinuity points.

Kurka defines four classes:

E1: Equicontinuous. Every point is an equicontinuity point: $E_F = X$. Equivalently, all orbits are eventually periodic. This corresponds to Wolfram's Class 1 and the trivial end of Class 2.

E2: Almost equicontinuous (but not equicontinuous). The set E_F is residual (contains a dense G_δ set) but $E_F \neq X$. Most points behave regularly, but sensitivity can occur on a meager set. This captures the nontrivial part of Wolfram's Class 2.

E3: Sensitive (but not positively expansive). The set E_F is empty – the system is sensitive to initial conditions – but is not positively expansive. This corresponds roughly to Wolfram's Classes 3 and 4.

E4: Positively expansive. There exists $\varepsilon > 0$ such that for any $x \neq y$, there exists $n \geq 0$ with $d(F^n(x), F^n(y)) > \varepsilon$. This is the strongest form of chaos. Among ECA, positively expansive rules include certain "spreading" rules. Note: positive expansivity is impossible for CA on $\mathbb{Z}^d$ with $d \geq 2$ (Shereshevsky 1993).

Theorem 7.1 (Kurka 1997). *Every CA on $S^{\mathbb{Z}}$ falls into exactly one of the classes E1–E4. These classes are mutually exclusive and exhaustive.*

The advantage of Kurka's classification is that it is based on well-defined topological properties. Its limitation is that it does not distinguish between "chaotic" (Class 3) and "complex" (Class 4) within class E3 – both types of behavior yield an empty equicontinuity set without positive expansivity.

7.7 CA as Endomorphisms of the Shift: The Curtis-Hedlund-Lyndon Theorem

The full shift on the alphabet S is the dynamical system (Σ, σ) where $\Sigma = S^{\mathbb{Z}}$ and σ is the left shift map $\sigma(c)_i = c_{i+1}$.

A **sliding block code** (or block map) is a map $F : \Sigma \to \Sigma$ defined by a local rule applied uniformly:

$$F(c)_i = f(c_{i+n_1}, \ldots, c_{i+n_m})$$

for some finite neighborhood $\{n_1, \ldots, n_m\}$ and some function $f : S^m \to S$.

Observe that this is exactly the definition of a CA global map. The following fundamental theorem shows that the converse also holds:

Theorem 7.2 (Curtis-Hedlund-Lyndon, 1969). *Let $\Sigma = S^{\mathbb{Z}}$ with the product topology, and let σ be the shift. A map $F : \Sigma \to \Sigma$ is a sliding block code if and only if F is continuous and commutes with σ (i.e., $F \circ \sigma = \sigma \circ F$).*

In the language of symbolic dynamics, the endomorphisms of the full shift (Σ, σ) are exactly the cellular automata. This places CA theory squarely within the framework of symbolic dynamics. Hedlund (1969) developed many consequences of this perspective.

Corollary. The set of all CA on alphabet S forms a monoid under composition: the composition of two sliding block codes is again a sliding block code (with a larger neighborhood).

The Curtis-Hedlund-Lyndon theorem generalizes to subshifts: a continuous shift-commuting map between subshifts of finite type is always a sliding block code. See Lind & Marcus (2021), Chapter 1, for a thorough treatment.

7.8 Universality: Rule 110 is Turing-Complete

Theorem 7.3 (Cook, 2004). *Rule 110 is Turing-complete: it can simulate any Turing machine.*

This was conjectured by Wolfram and proved by Matthew Cook. The proof proceeds in several stages:

1. **Identify particles.** In Rule 110 from typical initial conditions, the background is periodic (with period 14 in space and 7 in time). Against this background, localized structures – called **gliders** or **particles** – propagate at rational speeds. Cook catalogued these particles and their collision rules.

1. **Simulate cyclic tag systems.** A **cyclic tag system** is a simple model of computation known to be Turing-universal (due to Cook, building on work of Post). It operates on a binary string by repeatedly applying productions from a fixed cyclic list.

1. **Encode the tag system.** Cook showed how to encode the state and transition rules of an arbitrary cyclic tag system into an initial configuration of Rule 110, using specific arrangements of gliders. The glider collisions implement the tag system's productions.

1. **Verify universality.** Since cyclic tag systems can simulate any Turing machine, Rule 110 inherits Turing-completeness.

The construction requires an infinite (but eventually periodic) initial configuration. Whether Rule 110 is universal from *finite* initial configurations remains open.

This result is remarkable: the simplest class of nontrivial CA (two states, radius 1) already contains a rule capable of universal computation. It supports the view that computational universality is not a rare or delicate property but is widespread among sufficiently complex dynamical rules.

7.9 The Garden of Eden Theorem

A configuration $c \in S^\Lambda$ is a **Garden of Eden** (GoE) if it has no preimage under the global map F: there is no configuration c' with $F(c') = c$. In other words, c can only appear as an initial condition – it can never be reached by the dynamics.

Two configurations c, c' are **asymptotic** if they differ on only finitely many cells: $\{i \in \Lambda : c_i \neq c'_i\}$ is finite. A CA is **pre-injective** if no two distinct asymptotic configurations have the same image.

Theorem 7.4 (Moore 1962, Myhill 1963 – Garden of Eden Theorem). *For a CA on $\mathbb{Z}^d$ with finite state set:*

(Moore) If the CA is surjective, then it is pre-injective.

(Myhill) If the CA is pre-injective, then it is surjective.

Equivalently: the CA is surjective if and only if it is pre-injective. Gardens of Eden exist if and only if there exist distinct asymptotic configurations with the same image ("mutually erasable patterns").

The proof uses a counting argument: both the number of patterns that can appear in a finite region in the image and the number of "orphan" patterns (that cannot appear in any image) are related to the injectivity properties of the local rule on finite blocks. The Myhill direction is typically proved using a compactness argument or a direct combinatorial argument involving the pigeonhole principle on finite approximations.

Example 7.1. Consider the ECA "majority rule" on $\mathbb{Z}$ with $S = \{0, 1\}$ and neighborhood $\{-1, 0, 1\}$:

$$f(l,c,r) = \begin{cases} 1\&\text{if } l+c+r \geq 2 \\ 0\&\text{otherwise} \end{cases}$$

This is Rule 232. Consider the two asymptotic configurations $c = \ldots 000 \ldots$ and $c' = \ldots 0010 \ldots$ (differing only at one cell). We have $F(c) = \ldots 000 \ldots$. For c', the cell with state 1 has neighborhood $(0,1,0)$, giving $f(0,1,0) = 0$, and its neighbors have neighborhoods $(0,0,1)$ and $(1,0,0)$, both giving 0. So $F(c') = \ldots 000 \ldots = F(c)$.

Since we found distinct asymptotic configurations with the same image, Rule 232 is **not** pre-injective, hence by the Garden of Eden theorem, it is **not** surjective. Gardens of Eden exist for this rule.

7.10 Reversible Cellular Automata

A CA is **reversible** if its global map $F : S^{\mathbb{Z}^d} \to S^{\mathbb{Z}^d}$ is bijective (both injective and surjective). By the Curtis-Hedlund-Lyndon theorem, if F is bijective and continuous and shift-commuting, then its inverse F^{-1} is also continuous and shift-commuting, hence also a CA. Thus:

Theorem 7.2. *A bijective CA has an inverse that is also a CA.*

The Garden of Eden theorem gives a powerful consequence for CA on $\mathbb{Z}^d$:

Corollary. *For a CA on $\mathbb{Z}^d$, injectivity implies surjectivity. Hence an injective CA on $\mathbb{Z}^d$ is automatically bijective, and therefore reversible.*

*The converse (surjectivity implies injectivity) is **false** in general for $d \geq 1$. There exist surjective CA that are not injective.*

*Reversible CA are important in physics because they conserve information (no Garden of Eden states) and can model time-reversible microscopic dynamics. The **Margolus neighborhood** provides a practical construction method: partition the lattice into blocks and apply a bijection to each block, alternating the partition at each time step.*

Remark 7.3. The statement "injectivity implies surjectivity" fails for CA on lattices other than $\mathbb{Z}^d$ – for instance, on free groups or on trees, counterexamples exist. This is related to the amenability of the group; the Garden of Eden theorem in its full form holds exactly for amenable groups (Ceccherini-Silberstein, Machi, and Scarabotti 1999; Bartholdi 2010).

7.11 Topological Entropy of a CA

Since the CA global map F is a continuous self-map of the compact metrizable space $S^{\mathbb{Z}}$, one can define its **topological entropy** $h(F)$ in the standard sense of dynamical systems theory.

For a CA with state set S, the maximum possible topological entropy is $\log |S|$ (the entropy of the full shift). Heuristically, maximal entropy means the CA "preserves all the randomness" in the configuration space.

Theorem 7.6 (Hedlund 1969). *A surjective CA on $S^{\mathbb{Z}}$ has topological entropy $h(F) = \log |S|$.*

Non-surjective CA can have strictly lower entropy, since the image $F(S^{\mathbb{Z}})$ is a proper subshift (a closed, shift-invariant subset of $S^{\mathbb{Z}}$).

For the **column subshift** – the subshift of $S^{\mathbb{Z}}$ obtained by intersecting the images $\bigcap_{n\geq 0} F^n(S^{\mathbb{Z}})$ – the entropy can decrease under iteration and converge to a limit, which measures the "effective" long-time entropy of the CA.

Computing the topological entropy of a CA is in general undecidable (Hurd, Kari, and Culik 1992). Even for specific ECA rules, exact values are known only in special cases (e.g., additive rules like Rule 90, where the entropy can be computed via the associated matrix over $\mathbb{F}_2$).

7.12 Two-Dimensional CA: Conway's Game of Life

The **Game of Life** (Conway, 1970) is a 2D CA on $\mathbb{Z}^2$ with $S = \{0, 1\}$ (dead/alive) and the Moore neighborhood (the 8 surrounding cells). The local rule is:

- A **live** cell with 2 or 3 live neighbors **survives**; otherwise it **dies**.
- A **dead** cell with exactly 3 live neighbors becomes **alive**; otherwise it stays dead.

This rule is denoted **B3/S23** in the standard "birth/survival" notation.
Despite the simplicity of this rule, the Game of Life exhibits extraordinary complexity:

- **Still lifes**: stable configurations (e.g., the 2×2 "block", the "beehive").
- **Oscillators**: configurations that cycle through a finite sequence of states (e.g., the "blinker" with period 2, the "pulsar" with period 3).
- **Gliders**: small configurations that translate across the lattice while oscillating. The simplest glider is a 5-cell pattern that moves one cell diagonally every 4 generations.
- **Glider guns**: configurations that periodically emit gliders (discovered by Gosper, 1970).

Theorem 7.7 (Berlekamp, Conway, Guy, 1982; fully proved via later constructions). *The Game of Life is Turing-complete.*

The proof uses glider guns to construct logic gates (AND, OR, NOT) by arranging glider streams to collide in appropriate ways. From logic gates, one builds arbitrary circuits, and hence a universal Turing machine.

The Game of Life is not surjective (Gardens of Eden exist, as shown by early computer searches) and not injective (many configurations can evolve into the all-dead state).

7.13 Connection to Our Framework

Throughout this series, we have studied discrete dynamical systems of the form (f, x_0) where $f : X \to X$ is an iterated map and $x_0 \in X$ is an initial state.

A cellular automaton with a **finite lattice** $\Lambda = \{0, 1, \ldots, n-1\}$ (with periodic boundary conditions) is exactly such a system:

- The state space is $X = S^n$, a finite set of size $|S|^n$.
- The global map $F : S^n \to S^n$ is our iterated map f.
- An initial configuration $c_0 \in S^n$ is our initial state x_0.
- The orbit $c_0, F(c_0), F^2(c_0), \ldots$ is a sequence in a finite set, so it is eventually periodic. The transient length and cycle length are the quantities we have studied.

All the tools from the theory of finite dynamical systems apply: functional graphs, cycle structure, fixed points, Garden of Eden configurations (the non-image elements of F), and so on.

For ECA on a lattice of n cells with periodic boundary, the state space has 2^n configurations. The functional graph of the global map F is a union of ρ-shaped components (tails leading into cycles). The Garden of Eden configurations are the roots of the trees – the configurations with in-degree zero in the functional graph.

Non-autonomous CA. If the local rule itself changes at each time step – say we apply f_0 at time 0, f_1 at time 1, and so on – we obtain a **non-autonomous CA**, which is a sequence of maps $F_0, F_1, F_2, \ldots$ composed in order. This corresponds directly to our non-autonomous DDS framework. If the sequence of rules is periodic with period p, the long-term dynamics are governed by the composite map $F_{p-1} \circ \cdots \circ F_1 \circ F_0$.

7.14 Summary

Cellular automata are a natural and far-reaching class of discrete dynamical systems. Despite their simple definition – a lattice of cells updated synchronously by a local rule – they exhibit the full spectrum of dynamical behavior, from trivial convergence to Turing-complete computation. The Curtis-Hedlund-Lyndon theorem embeds CA theory into symbolic dynamics, giving access to powerful topological and algebraic tools. The Garden of Eden theorem reveals deep connections between local injectivity and global surjectivity. And the finite-lattice restriction recovers exactly the iterated maps on finite sets that form the core of our study.

7.15 References

- Bartholdi, L. (2010). *Gardens of Eden and amenability on cellular automata.* Journal of the European Mathematical Society, 12(1), 241–248.

- Berlekamp, E., Conway, J., and Guy, R. (1982). *Winning Ways for Your Mathematical Plays*, Vol. 2. Academic Press.

- Ceccherini-Silberstein, T., Machi, A., and Scarabotti, F. (1999). Amenable groups and cellular automata. Annales de l'Institut Fourier, 49(2), 673–685.

- Cook, M. (2004). Universality in elementary cellular automata. Complex Systems, 15(1), 1–40.

- Hedlund, G. A. (1969). Endomorphisms and automorphisms of the shift dynamical system. Mathematical Systems Theory, 3(4), 320–375.

- Hurd, L. P., Kari, J., and Culik, K. (1992). The topological entropy of cellular automata is uncomputable. Ergodic Theory and Dynamical Systems, 12(2), 255–265.

- Kurka, P. (1997). Languages, equicontinuity and attractors in cellular automata. Ergodic Theory and Dynamical Systems, 17(2), 417–433.

- Kurka, P. (2003). *Topological and Symbolic Dynamics.* Cours Specialises 11, Societe Mathematique de France.

- Lind, D. and Marcus, B. (2021). *An Introduction to Symbolic Dynamics and Coding*, 2nd ed. Cambridge University Press.

- Moore, E. F. (1962). Machine models of self-reproduction. Proceedings of Symposia in Applied Mathematics, 14, 17–33.

- Myhill, J. (1963). The converse of Moore's Garden-of-Eden theorem. Proceedings of the American Mathematical Society, 14(4), 685–686.

- Shereshevsky, M. A. (1993). Expansiveness, entropy and polynomial growth for groups acting on subshifts by automorphisms. Indagationes Mathematicae, 4(2), 203–210.

- Wolfram, S. (1984). Universality and complexity in cellular automata. Physica D, 10(1–2), 1–35.

- Wolfram, S. (2002). *A New Kind of Science.* Wolfram Media.

7.16 Recommended Reading

For an accessible introduction:

- **Wolfram (2002)**, *A New Kind of Science*, is polarizing but contains extensive empirical data and beautiful figures. Best used as a source of examples rather than proofs.
- **Schiff, J.** (2008). *Cellular Automata: A Discrete View of the World.* Wiley. Undergraduate-level textbook with clear exposition and many exercises.

For the mathematical foundations:

- **Kurka (2003)** is the definitive mathematical treatment, covering the topological dynamics perspective thoroughly.
- **Lind & Marcus (2021)**, Chapter 1, places CA in the broader context of symbolic dynamics and gives clean proofs of the Curtis-Hedlund-Lyndon theorem.

For universality and computation:

- **Cook (2004)** is the original proof of Rule 110's universality. Technical but important.
- **Ollinger, N.** (2012). "Universalities in cellular automata." In *Handbook of Natural Computing*, Springer. Comprehensive survey of universality results across different CA.

For the Game of Life specifically:

- **Berlekamp, Conway, and Guy (1982)**, *Winning Ways*, Chapter 25, remains the classic exposition of Life's basic structures and logical universality.
- **Adamatzky, A.** (ed.) (2010). *Game of Life Cellular Automata.* Springer. Collection of articles on Life's properties and applications.

For connections to physics and reversible CA:

- **Toffoli, T. and Margolus, N.** (1987). *Cellular Automata Machines.* MIT Press. Pioneering work on using CA for physics simulation, with emphasis on reversible rules.

Part II

Complexity and Computation

Chapter 8

Kolmogorov Complexity and Algorithmic Information Theory

8.1 Motivation: Information Content of Individual Objects

Shannon's information theory (1948) measures the information content of a *source*—a random variable or stochastic process. It tells us, for instance, that a fair coin produces one bit of information per toss on average. But it says nothing about a specific, concrete string like

$$x = 01010101010101010101$$

versus

$$y = 10110100011010001101$$

Shannon entropy is a property of *ensembles*, not individual objects. Yet intuitively, x is "simpler" than y: the first has an obvious pattern, the second does not. We need a theory that assigns an information content to *each individual object*.

This is exactly what Kolmogorov complexity provides. Independently developed by Solomonoff (1964), Kolmogorov (1965), and Chaitin (1966), it defines the complexity of a finite object as the length of its shortest effective description. The key insight is that "description" means "program": the complexity of a string is the length of the shortest program that produces it.

8.2 Definition

Fix a universal Turing machine U. For a finite binary string $x \in \{0,1\}^*$, the **Kolmogorov complexity** of x (with respect to U) is

$$K_U(x) = \min\{|p| : U(p) = x\}$$

where the minimum is over all binary programs p such that U on input p halts and outputs x, and $|p|$ denotes the length of p in bits.

If no such p exists, we set $K_U(x) = \infty$, but since U is universal, every computable function is representable, and every finite string is trivially computable (by a program that simply contains it as a literal), so $K_U(x)$ is always finite.

Remark 8.1. The definition depends on the choice of U. The next theorem shows this dependence is benign.

8.3 The Invariance Theorem

Theorem 8.2 (Invariance Theorem, Kolmogorov 1965)**.** *For any two universal Turing machines U_1 and U_2, there exists a constant c (depending on U_1 and U_2 but not on x) such that for all x:*

$$|K_{U_1}(x) - K_{U_2}(x)| \leq c.$$

Proof sketch. Since U_1 is universal, there exists a program s_{21} (a "compiler" or "interpreter") such that for all p, $U_1(s_{21} \cdot p) = U_2(p)$, where $\cdot$ denotes concatenation. Then if p^* is a shortest program for x on U_2, the program $s_{21} \cdot p^*$ produces x on U_1. Therefore

$$K_{U_1}(x) \leq |s_{21}| + K_{U_2}(x).$$

By symmetry, $K_{U_2}(x) \leq |s_{12}| + K_{U_1}(x)$, so taking $c = \max(|s_{12}|, |s_{21}|)$ gives the result.

The invariance theorem justifies writing $K(x)$ without specifying U, with the understanding that all statements hold "up to an additive constant $O(1)$." We adopt this convention henceforth.

8.4 Basic Properties

Proposition 8.3. *For all $x \in \{0,1\}^n$:*

$$K(x) \leq n + c$$

for a constant c independent of x.

Proof. Consider the program "print the following n bits: x." This program has length n plus a fixed overhead c for encoding the instruction and the length n itself.

Proposition 8.4. $K_{prefix}(x) \leq K(x) + O(\log |x|)$. *(More precisely, the prefix-free (self-delimiting) complexity exceeds the plain complexity by at most $O(\log n)$ bits, since the length n can be encoded in $\lceil \log_2 n \rceil + O(\log \log n)$ bits.)*

Theorem 8.5 (Non-computability)**.** *The function $K : \{0,1\}^* \to \mathbb{N}$ is not computable.*

Proof sketch (Berry's paradox). Suppose for contradiction that K is computable. Then consider the following program p: "Enumerate all strings in shortlex order. Output the first string x such that $K(x) > |p| + c_0$." This program has some fixed length $|p| = L$. It outputs a string x with $K(x) > L + c_0$. But p itself is a description of x, so $K(x) \leq L$, a contradiction for sufficiently large c_0.

This is the formalization of Berry's paradox: "the smallest positive integer not definable in fewer than twelve words" is defined in eleven words.

Proposition 8.6 (Upper semi-computability)**.** *$K(x)$ is upper semi-computable: there exists a computable function $\phi(x,t)$ with $\phi(x,t) \geq \phi(x,t+1)$ and $\lim_{t\to\infty} \phi(x,t) = K(x)$. We can approximate $K(x)$ from above by running all programs in parallel and tracking the shortest one found so far that outputs x. We can never know when we have found the true minimum, but any program we find gives a valid upper bound.*

8.5 Incompressible Strings

Definition. A string $x \in \{0,1\}^n$ is **c-incompressible** if $K(x) \geq n - c$. A string is **incompressible** if it is 0-incompressible, i.e., $K(x) \geq |x|$.

Theorem 8.7 (Counting argument)**.** *For every n and every $c \geq 0$:*

1. There exists at least one incompressible string of length n. 2. The fraction of strings of length n that are c-compressible (i.e., $K(x) < n - c$) is at most 2^{-c}.

Interpretation. *Incompressible strings are "random"—they have no exploitable pattern. Most strings are nearly incompressible. This gives an algorithmic definition of randomness for finite objects.*

Proof. There are 2^n binary strings of length n. The total number of programs of length less than $n - c$ is at most

$$\sum_{k=0}^{n-c-1} 2^k = 2^{n-c} - 1 < 2^{n-c}.$$

Each such program produces at most one output. Therefore, at most 2^{n-c} strings of length n have $K(x) < n - c$. The fraction is at most $2^{n-c}/2^n = 2^{-c}$.

In particular, for $c = 0$: at most $2^n - 1$ strings have descriptions shorter than n bits, so at least one string of length n is incompressible. For $c = 10$: at least 99.9% of strings of length n satisfy $K(x) \geq n - 10$. □

8.6 Conditional and Joint Complexity

Definition. The **conditional Kolmogorov complexity** of x given y is

$$K(x \mid y) = \min\{|p| : U(p, y) = x\}$$

where $U(p, y)$ means running U with program p and auxiliary input y.

Definition. The **joint Kolmogorov complexity** of x and y is $K(x, y)$, defined as the shortest program that outputs the pair (x, y) (under some standard pairing function).

Theorem 8.8 (Chain rule for Kolmogorov complexity)**.** *For all strings x, y:*

$$K(x, y) = K(x) + K(y \mid x) + O(\log n)$$

where $n = |x| + |y|$.

The logarithmic term arises from the need to encode the boundary between the descriptions of x and $y \mid x$ within a joint program. This is analogous to Shannon's chain rule $H(X, Y) = H(X) + H(Y \mid X)$, but with an unavoidable $O(\log n)$ "overhead."

Theorem 8.9 (Symmetry of algorithmic information)**.** *For all strings x, y:*

$$K(x \mid y) = K(y \mid x) + O(\log n)$$

where $n = \max(|x|, |y|)$.

Proof sketch. From the chain rule applied in both orders:

$$K(x) + K(y \mid x) + O(\log n) = K(x, y) = K(y) + K(x \mid y) + O(\log n)$$

Rearranging: $K(y \mid x) - K(x \mid y) = K(y) - K(x) + O(\log n)$.

A more refined version (Kolmogorov, Levin) shows $K(x \mid y, K(y)) + K(y) = K(x, y) + O(\log n)$ and similarly with roles swapped, yielding symmetry up to $O(\log n)$.

8.7 Prefix-Free Complexity

The "plain" Kolmogorov complexity $K(x)$ defined above has an inconvenient property: the set of valid programs need not be prefix-free, so the sum $\sum_x 2^{-K(x)}$ can diverge. This prevents a clean connection to probability.

Definition. A **prefix-free Turing machine** is one whose domain (the set of inputs on which it halts) is a prefix-free set: no valid input is a proper prefix of another. The **prefix-free Kolmogorov complexity** is

$$K_{\text{prefix}}(x) = \min\{|p| : U_{\text{prefix}}(p) = x\}$$

where U_{prefix} is a universal prefix-free Turing machine.

Theorem 8.10 (Kraft inequality for prefix-free complexity)**.**

$$\sum_{x \in \{0,1\}^*} 2^{-K_{prefix}(x)} \leq 1.$$

The prefix-free version satisfies $K_{prefix}(x) \geq K(x)$ and $K_{prefix}(x) \leq K(x) + O(\log K(x))$, so the two notions agree up to logarithmic terms. For the remainder of this chapter, we use $K(x)$ to denote prefix-free complexity unless otherwise stated.

Proof. Since the domain of U_{prefix} is prefix-free, the Kraft inequality applies directly:

$$\sum_{p \in \text{dom}(U_{\text{prefix}})} 2^{-|p|} \leq 1.$$

Grouping by output: each x may be produced by multiple programs, but

$$\sum_x 2^{-K_{\text{prefix}}(x)} \leq \sum_x \sum_{p:U(p)=x} 2^{-|p|} = \sum_{p \in \text{dom}(U)} 2^{-|p|} \leq 1.$$

(The first inequality is actually an equality in the lower bound direction since we only take the minimum-length program, but the upper bound suffices.) □

8.8 Algorithmic Probability and Solomonoff's Prior

Definition. The **algorithmic probability** (or universal a priori probability) of x is

$$\mathbf{m}(x) = \sum_{p:U(p)=x} 2^{-|p|}$$

where the sum is over all programs p in the domain of a universal prefix-free Turing machine U that output x.

By the Kraft inequality, $\mathbf{m}$ is a semi-measure: $\sum_x \mathbf{m}(x) \leq 1$. Intuitively, $\mathbf{m}(x)$ is the probability that a random program (generated by fair coin flips for each bit, halting when the machine accepts) produces x.

Theorem 8.11 (Coding theorem, Levin 1974)**.**

$$K(x) = -\log_2 \mathbf{m}(x) + O(1).$$

Connection to Solomonoff's prior. *Solomonoff (1964) proposed $\mathbf{m}$ as a universal prior for prediction: given an observed sequence $x_1 \ldots x_n$, predict the next bit by*

$$P(x_{n+1} = 1 \mid x_1 \ldots x_n) = \frac{\mathbf{m}(x_1 \ldots x_n 1)}{\mathbf{m}(x_1 \ldots x_n)}.$$

This prior is universal in the sense that it dominates every computable measure: for every computable probability distribution μ, there exists $c_\mu > 0$ such that $\mathbf{m}(x) \geq c_\mu \cdot \mu(x)$ for all x. It thus converges (in a precise sense) to the true distribution at a rate depending only on the complexity of the true distribution.

8.9 Martin-Löf Randomness

Algorithmic randomness formalizes the intuition that a random string should pass every conceivable statistical test.

Definition (Martin-Löf, 1966). An **effective statistical test** is a uniformly computably enumerable sequence $\{V_m\}_{m=1}^{\infty}$ of open sets $V_m \subseteq \{0,1\}^\omega$ (in Cantor space) such that $\mu(V_m) \leq 2^{-m}$, where μ is the fair-coin measure.

An infinite sequence $\omega \in \{0,1\}^\omega$ is **Martin-Löf random** if for every effective statistical test, $\omega \notin \bigcap_{m=1}^{\infty} V_m$.

Theorem 8.12 (Schnorr-Levin theorem)**.** *An infinite binary sequence $\omega = \omega_1\omega_2\omega_3\ldots$ is Martin-Löf random if and only if*

$$K_{prefix}(\omega_1 \ldots \omega_n) \geq n - O(1) \quad \textit{for all } n.$$

This gives a clean equivalence: an infinite sequence is random if and only if its initial segments are all (nearly) incompressible.

8.10 Brudno's Theorem: Complexity Meets Ergodic Theory

Kolmogorov complexity connects to dynamical systems through a beautiful theorem due to Brudno (1983).

Theorem 8.13 (Brudno, 1983)**.** *Let (X, T, μ) be an ergodic measure-preserving system on a compact metric space, with a finite generating partition $\mathcal{P}$. Let $x_1x_2x_3\ldots$ be the symbolic itinerary of a point x with respect to $\mathcal{P}$. Then for μ-almost every x:*

$$\lim_{n\to\infty} \frac{K(x_1 \ldots x_n)}{n} = h_\mu(T)$$

where $h_\mu(T)$ is the Kolmogorov-Sinai (metric) entropy of the system.

Interpretation. *The Kolmogorov complexity growth rate of a typical orbit equals the dynamical entropy. This is a profound link between computation theory and ergodic theory: the "algorithmic compressibility" of typical trajectories is determined by the dynamical entropy of the system.*

8.11 Worked Examples

Example: Complexity of a constant string

Let $x = 0^n$ (the string of n zeros). A program to produce this is:

```
print '0' repeated n times
```

The program needs to encode the instruction (constant overhead) and the number n (requiring $\lceil \log_2 n \rceil$ bits). Therefore

$$K(0^n) \leq \log_2 n + O(1).$$

A matching lower bound (up to $O(\log \log n)$) can be shown: any program producing 0^n must somehow specify n, which requires $\Omega(\log n)$ bits. We conclude

$$K(0^n) = \Theta(\log n).$$

Contrast this with $|0^n| = n$. The string 0^n is highly compressible; its complexity grows only logarithmically in its length.

Example: Complexity of the digits of π

Let $x = \pi_n$ denote the first n digits (or bits) of π. There exist short programs that compute π to arbitrary precision (e.g., using the Chudnovsky algorithm). Such a program takes as input only the desired precision n. Therefore

$$K(\pi_n) \leq \log_2 n + O(1).$$

Despite π being irrational and "looking random," it is *algorithmically simple*—the entire infinite expansion is generated by a short, fixed program. The complexity of a finite prefix is dominated by the cost of specifying how many digits to output.

Example: Complexity of an incompressible string

Let r be a specific string of length n produced by a true random number generator (e.g., radioactive decay). By Theorem 8.6, with probability at least $1 - 2^{-c}$, such a string satisfies

$$K(r) \geq n - c.$$

Consider $n = 1000$. With probability $\geq 1 - 1/1024 \approx 99.9\%$, we have $K(r) \geq 990$. No program shorter than 990 bits can produce r. The string is essentially its own shortest description.

To verify the counting argument concretely: among all 2^{1000} strings of length 1000, at most $2^{990} - 1$ have a program shorter than 990 bits. The fraction of "10-compressible" strings is at most $2^{990}/2^{1000} = 2^{-10} < 0.001$.

Example: The chain rule in action

Consider two strings: $x = 0^{500}$ (500 zeros) and $y =$ the binary encoding of x repeated three times, i.e., $y = 0^{1500}$.

We have:

- $K(x) = O(\log 500) = O(1)$ (about 9 bits plus overhead).
- $K(y \mid x) = O(1)$: given x, the program "concatenate x three times" is a short, fixed program.
- $K(x, y) = K(x) + K(y \mid x) + O(\log n) = O(\log n)$.

By contrast, if x and y were independent incompressible strings of length 500 and 1500 respectively:

- $K(x) \approx 500$, $K(y \mid x) \approx 1500$ (knowing x gives no information about y).
- $K(x, y) \approx 500 + 1500 = 2000$.

The chain rule $K(x, y) = K(x) + K(y \mid x) + O(\log n)$ mirrors the Shannon relation $H(X, Y) = H(X) + H(Y \mid X)$ but at the level of individual objects.

8.12 Practical Approximation

Since $K(x)$ is not computable, we cannot calculate it exactly. However, any lossless compression algorithm provides an *upper bound*:

$$K(x) \leq |\text{gzip}(x)| + c_{\text{gzip}}$$

where c_{gzip} is the constant overhead of encoding the gzip decompression algorithm.

Normalized Compression Distance (NCD). Li et al. (2004) proposed a practical similarity metric based on compression:

$$\text{NCD}(x, y) = \frac{C(xy) - \min(C(x), C(y))}{\max(C(x), C(y))}$$

where $C(x)$ is the compressed size of x and $C(xy)$ is the compressed size of the concatenation. This approximates the *normalized information distance*

$$\text{NID}(x, y) = \frac{\max(K(x \mid y), K(y \mid x))}{\max(K(x), K(y))}$$

which is a universal metric (it minorizes every computable metric, up to a vanishing additive term). NCD has been applied successfully to clustering, classification, plagiarism detection, phylogenetics, and music analysis.

8.13 Connection to Discrete Dynamical Systems

Consider a discrete dynamical system (X, f) and a point $x_0 \in X$ with orbit $x_0, f(x_0), f^2(x_0), \ldots$. Under a symbolic encoding, the orbit becomes a string $\sigma = s_0 s_1 s_2 \ldots s_{n-1}$.

The Kolmogorov complexity $K(\sigma)$ measures the intrinsic information content of this orbit segment. Three regimes emerge:

1. **Trivially simple orbits** ($K(\sigma) = O(\log n)$): Fixed points, periodic orbits, and orbits converging to simple attractors. The entire orbit can be specified by the rule, the initial condition, and the length. Example: the orbit $0 \to 0 \to 0 \to \cdots$ under the identity map.

1. **Maximally complex orbits** ($K(\sigma) \approx n$): These are algorithmically random orbits indistinguishable from fair coin flips. They carry maximal information but no discernible structure. By Brudno's theorem, for an ergodic system with entropy h_μ, typical orbits have $K(\sigma) \approx h_\mu \cdot n$.

1. **Intermediate complexity orbits** ($O(\log n) \ll K(\sigma) \ll n$): These are the most interesting from the standpoint of structure and emergence. They are neither trivially predictable nor indistinguishable from noise. They exhibit regularities that can be partially but not fully compressed. In our framework, these are "creative" orbits: they encode genuine structural information.

Brudno's theorem makes the connection precise for ergodic systems. For non-ergodic systems or transient behavior, the relationship between orbit complexity and dynamical properties remains an active area of research.

8.14 Summary of Key Results

Result	Statement
Definition	$K(x) = \min\{$
p	: U(p) = x$\}$
Invariance	
$\mathrm{K}_{U_1}(x) - K_{U_2}(x)$	$\leq c$
Upper bound	$K(x) \leq$
x	+ O(1)
Non-computability	K is not computable
Upper semi-computability	K can be approximated from above
Incompressibility	Most strings satisfy $K(x) \geq$
x	- O(1)
Chain rule	$K(x, y) = K(x) + K(y \mid x) + O(\log n)$
Symmetry	$K(x \mid y) = K(y \mid x) + O(\log n)$
Coding theorem	$K(x) = -\log_2 \mathbf{m}(x) + O(1)$
Schnorr-Levin	ML-random $\iff K_{\text{prefix}}(x_{1:n}) \geq n - O(1)$
Brudno	$K(x_{1:n})/n \to h_\mu$ a.s. for ergodic systems

8.15 References

- Chaitin, G. J. (1966). On the length of programs for computing finite binary sequences. *Journal of the ACM*, 13(4), 547–569.

- Cover, T. M. & Thomas, J. A. (2006). *Elements of Information Theory*, 2nd edition. Wiley. Chapter 14.

- Kolmogorov, A. N. (1965). Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1), 1–7.

- Li, M. & Vitányi, P. (2019). *An Introduction to Kolmogorov Complexity and Its Applications*, 4th edition. Springer.

- Li, M., Chen, X., Li, X., Ma, B., & Vitányi, P. (2004). The similarity metric. *IEEE Transactions on Information Theory*, 50(12), 3250–3264.

- Martin-Löf, P. (1966). The definition of random sequences. *Information and Control*, 9(6), 602–619.

- Brudno, A. A. (1983). Entropy and the complexity of the trajectories of a dynamical system. *Transactions of the Moscow Mathematical Society*, 44, 127–151.

- Solomonoff, R. J. (1964). A formal theory of inductive inference. *Information and Control*, 7(1), 1–22 and 7(2), 224–254.

- Levin, L. A. (1974). Laws of information conservation (nongrowth) and aspects of the foundation of probability theory. *Problems of Information Transmission*, 10(3), 206–210.

- Schnorr, C. P. (1973). Process complexity and effective random tests. *Journal of Computer and System Sciences*, 7(4), 376–388.

8.16 Recommended Reading

For a comprehensive introduction:

- **Li & Vitányi (2019)** is the definitive textbook, covering everything from basic definitions through applications. Chapters 1–3 suffice for the core theory; later chapters cover applications to learning, statistics, and physics.

For a shorter introduction:

- **Cover & Thomas (2006)**, Chapter 14, provides a concise introduction oriented toward information theorists.

For algorithmic randomness:

- **Downey, R. and Hirschfeldt, D.** (2010). *Algorithmic Randomness and Complexity.* Springer. The most comprehensive treatment of the theory of random sequences, covering Martin-Löf randomness and many refinements.

- **Nies, A.** (2009). *Computability and Randomness.* Oxford University Press. Another excellent advanced treatment.

For connections to dynamics:

- **Brudno (1983)** is the original source for the connection between Kolmogorov complexity and dynamical entropy. The paper is short and readable.

For applications:

- **Li et al. (2004)** introduced the normalized compression distance and demonstrated its wide applicability. The IEEE Transactions paper is accessible and includes many examples.

- **Cilibrasi, R. and Vitányi, P.** (2005). "Clustering by compression." *IEEE Transactions on Information Theory*, 51(4), 1523–1545. Extends NCD to clustering algorithms with impressive empirical results.

For philosophical foundations:

- **Chaitin, G.** (2005). *Meta Math! The Quest for Omega.* Vintage. A popular account of the philosophical implications of algorithmic information theory, including connections to Gödel's theorems.

Chapter 9

Logical Depth and Sophistication

9.1 Motivation: The Inadequacy of Kolmogorov Complexity Alone

Kolmogorov complexity $K(x)$ measures the length of the shortest program that produces a string x on a universal Turing machine U. It is a foundational notion: strings with $K(x) \ll |x|$ are compressible and hence "simple," while strings with $K(x) \approx |x|$ are algorithmically random. But there is a striking deficiency. Consider two strings of length n:

1. A passage from Shakespeare, encoded in ASCII.

2. A string produced by fair coin flips.

Both have Kolmogorov complexity close to n – the Shakespeare text because natural language, while compressible, is information-dense; the random string because it is incompressible by definition. Yet these strings are qualitatively different. The Shakespeare passage contains *structure*, *organization*, and *meaning*. It is the product of an extended creative process. The random string is not.

Kolmogorov complexity conflates two very different kinds of objects: those that are complex because they encode organized structure, and those that are complex because they are patternless noise. We need a measure that distinguishes the *interesting* from the *merely random*.

This chapter develops two such measures: **logical depth** (Bennett, 1988), which captures the computational work needed to unfold structure, and **sophistication** (Koppel & Atlan, 1991), which captures the amount of meaningful information in an object. Both refine Kolmogorov complexity by decomposing it into components that separate structure from noise.

9.2 Bennett's Logical Depth

9.2.1 9.2.1 Informal Idea

The key observation is that structured objects are typically produced by *slow* computations. The digits of π, the output of a long cellular automaton evolution, a genome shaped by billions of years of natural selection – all of these have compact descriptions (short programs), but those programs require enormous computational resources to execute. In contrast:

- A string of n zeros, '000...0', has a short program *and* runs quickly.

- A random string has no short program at all.

Neither of these is "deep." The first is trivially simple; the second is trivially random. Deep objects occupy a third category: they have short programs, but those programs take a long time to run. Depth measures the *unavoidable computational work* required to produce an object from a concise description.

9.2.2 9.2.2 Formal Definition

Fix a universal prefix-free Turing machine U. For a program p, let $U(p)$ denote the output of U on input p (if it halts), and let $T(p)$ denote the number of steps U takes before halting.

Definition 9.1 (Logical Depth)**.** Let x be a finite binary string, and let $s \geq 0$ be a non-negative integer called the *significance parameter*. The *logical depth* of x at significance level s is

$$\text{depth}_s(x) = \min\{T(p) : U(p) = x \text{ and } |p| \leq K(x) + s\}$$

where the minimum is taken over all programs p that produce x and whose length exceeds the minimal description length $K(x)$ by at most s bits.

The significance parameter s controls how far from the shortest description we are willing to look. When $s = 0$, we insist on the shortest program; when s is larger, we allow slightly longer programs that may run faster.

Remark 9.2. Logical depth is not computable (it depends on $K(x)$, which is itself uncomputable). It is a theoretical quantity that captures an important conceptual distinction.

9.2.3 9.2.3 The Three Categories

Bennett's framework partitions finite objects into three informal categories:

Category	$K(x)$	$\text{depth}_s(x)$	Example
Simple (trivial)	Small	Small	'000...0'
Random (incompressible)	$\approx$	x	
Small	Random bit string		
Deep (organized)	Small to moderate	Large	Digits of π; evolved genome

The crucial insight: **random strings are shallow**. An incompressible string x of length n has $K(x) \geq n$, so the "program" that produces it is essentially x itself (a print statement containing x as a literal). Such a program runs in $O(n)$ steps – fast. There is no computation to perform; the output is already explicit in the program.

Deep objects, by contrast, hide their output behind a wall of necessary computation. Their short programs encode the output *implicitly*, and unfolding the implicit description takes substantial time.

9.2.4 9.2.4 The Slow Growth Law

The most important structural result about logical depth is that deep objects cannot be quickly manufactured from shallow ones.

Theorem 9.3 (Slow Growth Law, Bennett 1988)**.** *Let f be a computable function and t a time bound. If x is a string with $\text{depth}_s(x) > t$, and y is any string with $\text{depth}_{s'}(y) \leq t'$, then computing x from y requires time at least $\text{depth}_s(x) - t' - O(1)$, up to an additive constant depending on f, s, and s'.*

In informal terms: ***you cannot quickly extract deep objects from shallow ones****. Depth is "conserved" across computable transformations, modulo the depth of the input. A fast computation applied to a shallow input produces a shallow output.*

This has a satisfying physical interpretation: organized complexity (biological organisms, cultural artifacts, scientific knowledge) accumulates slowly. It cannot be conjured from nothing by a fast process. The slow growth law is the algorithmic information theory analogue of the second law of thermodynamics – not that entropy must increase, but that depth must accumulate gradually.

9.3 Worked Examples

Example: The All-Zeros String

Let $x = 0^n$ (the string of n zeros). A program for x is:

```
print("0" * n)
```

This program has length $O(\log n)$ (we need $\log n$ bits to encode the number n), and it runs in $O(n)$ steps. Thus $K(0^n) = O(\log n)$ and $\text{depth}_s(0^n) = O(n)$ for any reasonable s.

But $O(n)$ steps is essentially the minimum time needed to *write down* an output of length n (you must at least print it). So the depth of 0^n is as small as it can be for a string of that length. It is **shallow and simple**.

Example: A Random String

Let $r \in \{0,1\}^n$ be a Kolmogorov-random string, meaning $K(r) \geq n$. The shortest program for r has length at least n and essentially contains r as a literal. This program runs in $O(n)$ time – it just copies r to the output tape.

For any significance level s, $\text{depth}_s(r) = O(n)$. The string is **shallow and random**. It is incompressible, but producing it requires no real computation beyond printing.

Example: The n-th Busy Beaver Number

The Busy Beaver function $\text{BB}(k)$ is the maximum number of 1's that a halting Turing machine with k states can write on a blank tape. Let x be the binary encoding of $\text{BB}(n)$. Then:

- $K(x) = O(\log n)$: the program "compute $\text{BB}(n)$" has length $O(\log n)$.
- $\text{depth}_s(x)$ is enormous: computing $\text{BB}(n)$ requires simulating all n-state Turing machines and determining which ones halt. The running time grows faster than any computable function of n.

This is a paradigmatic deep object. It has a tiny description but requires vast computation to produce.

Example: Digits of a Slowly Converging Series

Consider $x =$ the first n digits of

$$\sum_{k=1}^{\infty} \frac{1}{k^3}.$$

The sum converges, but slowly. The shortest program is roughly "compute $\sum 1/k^3$ to n digits," which has length $O(\log n)$. But to compute n correct digits via the naive series requires summing on the order of $10^{n/3}$ terms (since the tail $\sum_{k>N} 1/k^3 \approx 1/(2N^2)$, we need $N \approx 10^{n/2}$ for n digits). More efficient algorithms exist, but the point is that the computation is substantially longer than the output. This string is **deep**: compact description, slow execution.

Example: Cellular Automaton Output

Let f be a cellular automaton rule (say, Rule 110, which is known to be Turing-complete). Let x be the configuration of the automaton after T steps, starting from a simple initial condition. The shortest program is approximately "run Rule 110 for T steps from initial condition c_0," of length $O(\log T)$. But producing the output requires T steps of simulation. If T is large relative to $|x|$, this output is deep. The depth is a direct measure of the computational history embedded in the evolved state.

9.4 Sophistication

9.4.1 9.4.1 Motivation

Logical depth measures the *time* needed to produce an object. **Sophistication** takes a different approach: it measures the *amount of meaningful information* in an object, as opposed to accidental or random information.

The idea is to decompose the information content of a string x into two parts:

1. **Structure** (regularity, pattern): the "meaningful" part.
2. **Noise** (randomness, accident): the "meaningless" part.

The Kolmogorov complexity $K(x)$ lumps these together. Sophistication attempts to isolate the structural component.

9.4.2 9.4.2 Formal Definition

Definition 9.4 (Sophistication, Koppel & Atlan 1991)**.** Let x be a binary string and $c \geq 0$ a constant. The *sophistication* of x at significance level c is

$$\text{soph}_c(x) = \min\{|p| : p \text{ is a total program, } (\exists d)[U(p,d) = x], \text{ and } |p| + |d| \leq K(x) + c\}$$

where the minimum is taken over all total programs p (programs that halt on every input) such that there exists a "data" string d with $U(p,d) = x$ and the combined length $|p| + |d|$ is within c bits of optimal.

Intuitively, we decompose the shortest description of x into a "model" p (the structure) and "data" d (the noise fed into that model). The sophistication is the length of the smallest model that, when given appropriate random-looking data, produces x.

Example. Consider a string x that consists of English text with a few random typos. The "model" p encodes the rules of English grammar and the vocabulary; the "data" d specifies which particular words and typos appear. The sophistication of x is approximately the size of the English-language model, not the full length of the text.

9.4.3 9.4.3 Properties

Proposition 9.5. *For any string x of length n: - $soph_c(x) \leq K(x) + O(1)$: sophistication is at most the Kolmogorov complexity. - If x is Kolmogorov random (i.e., $K(x) \geq n$), then $soph_c(x) = O(\log n)$ for appropriate c: random strings have low sophistication. Their complexity is entirely noise, not structure. - If $x = 0^n$, then $soph_c(x) = O(\log n)$: trivially structured strings also have low sophistication.*

The first and third points are straightforward. For the second, observe that a random string x of length n can be produced by the program "print the following n-bit string" (of length $O(\log n)$) given the data $d = x$ itself. The model is trivial; all the information is in the data.

Sophistication thus gives low values to both trivially simple and random strings, and high values to strings with genuine structure – precisely the distinction we want.

9.5 Effective Complexity

Gell-Mann and Lloyd (1996) introduced a related notion from the perspective of physics.

Definition 9.6 (Effective Complexity)**.** The *effective complexity* of a string x is the Kolmogorov complexity of the set of "regularities" (or "ensemble") that x belongs to. More precisely, if S is the "most plausible" set of strings containing x (the one that best balances model simplicity against fit), then

$$\mathrm{EC}(x) = K(S)$$

where $K(S)$ is the length of the shortest program that enumerates the members of S.

This is closely related to sophistication. The effective complexity of x is, informally, the complexity of the best "theory" or "model" for x. Random strings have low effective complexity (the best model is "all strings of length n," which has complexity $O(\log n)$). Simple strings also have low effective complexity. Structured strings have high effective complexity.

The main difference from sophistication is philosophical: effective complexity emphasizes the description of an *ensemble* rather than a *program-data decomposition.* In practice, the two notions are closely related and, under suitable formalizations, essentially equivalent (see Antunes & Fortnow, 2009).

9.6 Relationships Between the Notions

Antunes and Fortnow (2009) provided a rigorous treatment of the connections between depth, sophistication, and effective complexity. We summarize the key relationships.

Theorem 9.7 (Antunes & Fortnow 2009)**.** *For appropriate choices of significance parameters:*

1. Sophistication lower-bounds depth: If $\mathrm{soph}_c(x)$ is large, then $\mathrm{depth}_s(x)$ must be at least moderately large. Intuitively, a string with a complex model cannot be produced quickly, because the model itself requires time to "execute."

2. Depth does not upper-bound sophistication: There exist strings of high depth but low sophistication. For example, the string $0^{BB(n)}$ (a string of zeros whose length is the n-th Busy Beaver number) is very deep (computing its length takes enormous time) but has trivial structure (it is just a string of zeros). Its sophistication is $O(\log n)$.

3. Effective complexity and sophistication agree up to logarithmic terms, under appropriate formalizations.

This gives us a clean picture:

*- **Depth** measures the temporal cost of producing an object from a compact description.*
*- **Sophistication** measures the informational cost of the structural component of an object.*
*- **Effective complexity** measures the same thing as sophistication, from a slightly different angle.*

An object can be deep without being sophisticated (Example: $0^{BB(n)}$), and an object can be sophisticated without being extremely deep (Example: a string encoding a complex but efficiently computable mathematical object). The most interesting objects – those we intuitively recognize as exhibiting organized complexity – tend to score high on both measures.

9.7 Connection to Creativity and "Plausible History of Origin"

Bennett explicitly argued that logical depth captures the notion of **plausible history of origin**. An object is deep if the most plausible explanation for its existence involves an extended computational (or physical, or evolutionary) process.

This connects directly to creativity. Consider a creative artifact – a novel, a symphony, a mathematical proof, a well-engineered bridge. Such an artifact:

1. Is **not random**: it has a compact description (it follows rules, has structure, serves a purpose). Thus $K(x) < |x|$.

2. Is **not trivially generated**: no short-cut produces it from its compact description. The author had to think, revise, explore dead ends, and iterate. Thus $\text{depth}_s(x)$ is large.

A creative artifact is precisely one that is deep. It lies in the "interesting" region between trivial simplicity and random noise. The slow growth law then formalizes the intuition that creativity takes time: you cannot produce a deep artifact by a fast, mechanical process applied to shallow inputs.

This perspective also clarifies why plagiarism detectors work: a plagiarized text is *shallow relative to its apparent sophistication.* It has high sophistication (complex structure) but was produced by a fast process (copying), not by the slow process of genuine composition.

9.8 Connection to Discrete Dynamical Systems

In a finite discrete dynamical system (X, f), every orbit $x_0, f(x_0), f^2(x_0), \ldots$ eventually enters a cycle. The number of steps before entering the cycle is the **transient length** $\tau(x_0)$.

The transient length is a finite, computable proxy for logical depth. Consider the state $y = f^t(x_0)$ at the point where the orbit first enters its cycle. To describe y, one can specify f, x_0, and t, then simulate t steps. If f and x_0 have compact descriptions but t is large, then y is deep: its shortest description requires long computation to execute.

Proposition 9.8. *In a finite DDS (X, f) with $|X| = n$, the transient length $\tau(x_0)$ satisfies $0 \leq \tau(x_0) \leq n$. The state $f^{\tau(x_0)}(x_0)$ at the cycle entry point has depth proportional to $\tau(x_0)$ relative to the description of (f, x_0).*

States with long transients are the "deep" states of the system: they encode the accumulated computational history of the iteration. States already on cycles are "shallow" – they can be reached immediately (they are their own description, in a sense).

This gives us a practical, finite analogue of logical depth. In explorations of DDS on small state spaces, the distribution of transient lengths across initial conditions provides a concrete measure of how much "depth" the map f generates. Maps that produce long transients are, in Bennett's sense, more "creative" than those that quickly funnel all states onto short cycles.

9.9 Practical Approximation

Since logical depth is uncomputable, any practical application requires approximations. Several approaches exist:

1. **Transient length in finite systems** (as discussed in Section 9.8): directly measurable, well-defined, and meaningful for DDS.

1. **Compression-decompression time asymmetry**: For a string x, measure the time to compress x (finding a short description) versus the time to decompress it (running the

short description). If decompression is much slower than compression, this suggests depth. In practice, one can use standard compression algorithms (gzip, bzip2, etc.) and measure their running times.

1. **Busy Beaver scaling**: For families of strings parameterized by a complexity parameter n, measure how the generation time scales with n. Faster-than-polynomial scaling suggests depth.

1. **Multi-scale complexity measures**: Compute complexity at multiple scales (e.g., block entropies at different block sizes) and look for structure that persists across scales. This is related to effective complexity and can distinguish structured from random strings.

1. **Lempel-Ziv complexity and its computation time**: The Lempel-Ziv (LZ77/LZ78) factorization of a string provides a measure of compressibility. The *time* needed to find the factorization, relative to the string length, can serve as a rough depth proxy.

9.10 Summary

Kolmogorov complexity tells us how much information a string contains. Logical depth and sophistication tell us *what kind* of information it is.

Measure	What it captures	Simple strings	Random strings	Structured strings
$K(x)$	Total information content	Low	High	Moderate to high
$\text{depth}_s(x)$	Computational time to generate	Low	Low	High
$\text{soph}_c(x)$	Meaningful structural information	Low	Low	High
$\text{EC}(x)$	Complexity of regularities	Low	Low	High

The deep, sophisticated objects – those with moderate Kolmogorov complexity, high depth, and high sophistication – are precisely the ones we recognize as *interesting*: biological organisms, works of art, scientific theories, evolved computational structures. The theoretical framework of this chapter provides the formal underpinning for this intuition.

9.11 References

- Antunes, L. and Fortnow, L. (2009). Sophistication revisited. *Theory of Computing Systems*, 45(1):150–161.

- Bennett, C.H. (1988). Logical depth and physical complexity. In R. Herken (ed.), *The Universal Turing Machine: A Half-Century Survey*, pp. 227–257. Oxford University Press.

- Gell-Mann, M. and Lloyd, S. (1996). Information measures, effective complexity, and total information. *Complexity*, 2(1):44–52.

- Koppel, M. and Atlan, H. (1991). An almost machine-independent theory of program-length complexity, sophistication, and induction. *Information Sciences*, 56(1–3):258–263.

- Li, M. and Vitányi, P.M.B. (2008). *An Introduction to Kolmogorov Complexity and Its Applications*. Third edition. Springer.

- Vitányi, P.M.B. (2006). Meaningful information. *IEEE Transactions on Information Theory*, 52(10):4617–4626.

9.12 Recommended Reading

For the foundational paper:

- **Bennett (1988)** is the original and remains the best exposition of logical depth. It is remarkably accessible and philosophically rich. Read this first.

For the rigorous treatment:

- **Antunes & Fortnow (2009)** clarifies the relationships between depth, sophistication, and effective complexity with full proofs. Essential for understanding the precise mathematical content.

For sophistication and effective complexity:

- **Gell-Mann & Lloyd (1996)** introduces effective complexity from a physics perspective. Short and conceptual rather than technical.

- **Vitányi (2006)** gives a rigorous treatment of "meaningful information" and its relation to sophistication.

For the broader context:

- **Li & Vitányi (2008)**, Chapter 7, covers logical depth and sophistication as part of a comprehensive treatment of algorithmic information theory.

For philosophical implications:

- **Bennett, C.H.** (1995). "Thermodynamics of computation." In *Computational Complexity*, ed. R.A. Meyers, Springer. Connects logical depth to physical notions of irreversibility and the thermodynamics of computation.

- **Dennett, D.** (1995). *Darwin's Dangerous Idea.* Simon & Schuster. Chapter 4 discusses Bennett's concept of depth in the context of evolutionary creativity, making the ideas accessible to a general audience.

For practical applications:

- **Zenil, H. et al.** (2018). "A decomposition method for global evaluation of Shannon entropy and local estimations of algorithmic complexity." *Entropy*, 20(8), 605. Computational approaches to estimating complexity measures including depth-like quantities.

Chapter 10

Computational Mechanics and Epsilon-Machines

10.1 Introduction and Motivation

Consider a stream of symbols $\ldots x_{-2}\, x_{-1}\, x_0\, x_1\, x_2 \ldots$ drawn from a finite alphabet $\mathcal{A}$. This might be the output of a sensor, the orbit of a dynamical system, the text of a language, or a sequence of nucleotides. A fundamental question arises: **what is the intrinsic computational structure of this process?**

Shannon's information theory (1948) provides one answer. The **entropy rate** h_μ quantifies the irreducible unpredictability per symbol. But entropy rate alone is blind to structure. A fair coin produces $h_\mu = 1$ bit/symbol. A process that deterministically alternates between long runs of 0s and long runs of 1s might also have h_μ close to 1 bit/symbol, yet it has obvious patterns that an intelligent observer could detect and exploit for prediction.

What we need is a framework that answers:

1. **What are the patterns** in the process?
2. **How are those patterns organized** into a predictive model?
3. **How much memory** is required for optimal prediction?
4. **How much residual randomness** remains after optimal prediction?

Computational mechanics, developed primarily by James Crutchfield and collaborators beginning in the late 1980s, provides a mathematically rigorous answer to all four questions. The central construction is the **epsilon-machine** (ε-machine): a unique, minimal, optimal predictor for any stationary stochastic process. The theory gives us two fundamental complexity measures — **statistical complexity** C_μ (the memory cost of optimal prediction) and **entropy rate** h_μ (the irreducible randomness) — and reveals that the most "complex" processes live in a specific region of the (h_μ, C_μ) plane, neither fully ordered nor fully random.

This chapter develops the theory from definitions, constructs the epsilon-machine, works through several examples in detail, and connects the framework to the discrete dynamical systems studied in earlier chapters.

10.2 Setup: Stationary Stochastic Processes

Definition 10.1 (Stochastic process)**.** A **discrete-time stochastic process** over a finite alphabet $\mathcal{A}$ is a bi-infinite sequence of random variables $\ldots, X_{-1}, X_0, X_1, X_2, \ldots$ where each X_t takes values in $\mathcal{A}$. We write $\overleftarrow{X}_t = \ldots X_{t-2}\, X_{t-1}$ for the **past** (semi-infinite history up to but not including time t) and $\overrightarrow{X}_t = X_t\, X_{t+1}\, X_{t+2} \ldots$ for the **future** (from time t onward).

Definition 10.2 (Stationarity)**.** The process is **stationary** if the joint distribution of any finite block $(X_{t_1}, \ldots, X_{t_k})$ is invariant under time shifts: $P(X_{t_1} = a_1, \ldots, X_{t_k} = a_k) = P(X_{t_1+\tau} = a_1, \ldots, X_{t_k+\tau} = a_k)$ for all $\tau \in \mathbb{Z}$.

For a stationary process we write $P(\overrightarrow{X} \mid \overleftarrow{X})$ for the conditional distribution of the future given the past, dropping the time subscript. Throughout this chapter, all processes are assumed stationary and ergodic unless otherwise stated.

Notation. We write $\overleftarrow{x}$ for a specific realization (value) of the past, and $\overrightarrow{x}$ for a specific realization of the future. We use $w = x_1 x_2 \ldots x_L$ for finite words (blocks) of length L.

10.3 Causal States

The key insight of computational mechanics is to partition the set of all possible pasts by their **predictive equivalence**: two pasts that induce the same conditional distribution over futures are informationally interchangeable for prediction purposes.

Definition 10.3 (Causal equivalence)**.** Two pasts $\overleftarrow{x}$ and $\overleftarrow{x}'$ are **causally equivalent**, written $\overleftarrow{x} \sim_\varepsilon \overleftarrow{x}'$, if and only if

$$P(\overrightarrow{X} \mid \overleftarrow{X} = \overleftarrow{x}) = P(\overrightarrow{X} \mid \overleftarrow{X} = \overleftarrow{x}').$$

That is, the two pasts produce identical conditional distributions over all possible futures.

This is clearly an equivalence relation (reflexive, symmetric, transitive).

Definition 10.4 (Causal states)**.** The **causal states** of the process are the equivalence classes of $\sim_\varepsilon$:

$$\mathcal{S} = \{\epsilon(\overleftarrow{x}) : \overleftarrow{x} \in \mathcal{A}^{-\mathbb{N}}\}$$

where $\epsilon : \mathcal{A}^{-\mathbb{N}} \to \mathcal{S}$ is the map that assigns each past to its equivalence class. We write $s = \epsilon(\overleftarrow{x})$ for the causal state corresponding to past $\overleftarrow{x}$.

Remark 10.5. The causal state set $\mathcal{S}$ can be finite, countably infinite, or uncountably infinite. When $\mathcal{S}$ is finite, we say the process has **finite statistical complexity** and the theory is especially clean. All examples in this chapter have finite $\mathcal{S}$.

Proposition 10.6 (Causal states are sufficient statistics)**.** *The causal state $\epsilon(\overleftarrow{x})$ is a sufficient statistic of the past $\overleftarrow{x}$ for predicting the future $\overrightarrow{X}$. That is,*

$$P(\overrightarrow{X} \mid \overleftarrow{X} = \overleftarrow{x}) = P(\overrightarrow{X} \mid S = \epsilon(\overleftarrow{x})).$$

Proof. This follows immediately from the definition: the causal state is defined so that all pasts in the same equivalence class yield the same conditional future distribution.

Proposition 10.7 (Causal states are minimal sufficient statistics)**.** *If $\eta : \mathcal{A}^{-\mathbb{N}} \to \mathcal{Q}$ is any other function of the past that is a sufficient statistic for predicting the future, then η is a refinement of ϵ: for all $\overleftarrow{x}, \overleftarrow{x}'$, if $\eta(\overleftarrow{x}) = \eta(\overleftarrow{x}')$ then $\epsilon(\overleftarrow{x}) = \epsilon(\overleftarrow{x}')$.*

Equivalently, $|\mathcal{S}| \leq |\mathcal{Q}|$ (when both are finite), and there is a function $g : \mathcal{Q} \to \mathcal{S}$ such that $\epsilon = g \circ \eta$.

Proof. Suppose $\eta(\overleftarrow{x}) = \eta(\overleftarrow{x}')$. Since η is a sufficient statistic, $P(\overrightarrow{X} \mid \overleftarrow{X} = \overleftarrow{x}) = P(\overrightarrow{X} \mid \eta = \eta(\overleftarrow{x})) = P(\overrightarrow{X} \mid \eta = \eta(\overleftarrow{x}')) = P(\overrightarrow{X} \mid \overleftarrow{X} = \overleftarrow{x}')$. Therefore $\overleftarrow{x} \sim_\varepsilon \overleftarrow{x}'$, so $\epsilon(\overleftarrow{x}) = \epsilon(\overleftarrow{x}')$.

*This is a crucial result: the causal states form the **coarsest** partition of pasts that retains all predictive information. Any other sufficient statistic either coincides with the causal states or is unnecessarily fine-grained.*

10.4 The Epsilon-Machine

Definition 10.8 (Epsilon-machine)**.** The **epsilon-machine** (ε-machine) of a stationary process is the tuple $(\mathcal{S}, \mathcal{A}, \{T^{(a)}\}_{a\in\mathcal{A}})$ where:

- $\mathcal{S}$ is the set of causal states.
- $\mathcal{A}$ is the process alphabet.
- $T^{(a)}_{s,s'}$ is the **labeled transition probability**: the probability that, when the process is in causal state s, the next emitted symbol is a and the next causal state is s':

$$T^{(a)}_{s,s'} = P(X_t = a,\, S_{t+1} = s' \mid S_t = s).$$

The transition probabilities satisfy $\sum_{a\in\mathcal{A}} \sum_{s'\in\mathcal{S}} T^{(a)}_{s,s'} = 1$ for each s.

Theorem 10.9 (Unifilarity)**.** *The epsilon-machine is **unifilar**: for each causal state $s \in \mathcal{S}$ and each symbol $a \in \mathcal{A}$, there is at most one successor state s' such that $T^{(a)}_{s,s'} > 0$.*

In other words, if we know the current causal state and observe the next symbol, the next causal state is determined with certainty. This is a strong structural property that most HMMs do not possess.

Proof sketch. If the process is in causal state $s = \epsilon(\overleftarrow{x})$ and emits symbol a, then the new past is $\overleftarrow{x}\,a$. The new causal state is $\epsilon(\overleftarrow{x}\,a)$, which depends only on s and a, not on the particular representative $\overleftarrow{x}$ of s. (This can be verified: if $\epsilon(\overleftarrow{x}) = \epsilon(\overleftarrow{x}')$, then $P(\overrightarrow{X} \mid \overleftarrow{X} = \overleftarrow{x}) = P(\overrightarrow{X} \mid \overleftarrow{X} = \overleftarrow{x}')$, and conditioning both sides on the first future symbol being a yields $P(\overrightarrow{X}' \mid \overleftarrow{X}' = \overleftarrow{x}\,a) = P(\overrightarrow{X}' \mid \overleftarrow{X}' = \overleftarrow{x}'a)$, so $\epsilon(\overleftarrow{x}\,a) = \epsilon(\overleftarrow{x}'a)$.) Thus the successor state is a function of (s, a), which is exactly unifilarity.

We can therefore write the successor state as a deterministic function:

$$\delta : \mathcal{S} \times \mathcal{A} \to \mathcal{S}, \qquad \delta(s, a) = \textit{the unique } s' \textit{ with } T^{(a)}_{s,s'} > 0$$

(defined only when $P(X_t = a \mid S_t = s) > 0$).

Theorem 10.10 (Minimality and uniqueness)**.** *The epsilon-machine is the **unique** minimal unifilar HMM generating the process. Any other unifilar HMM that generates the same process has at least as many states, and any unifilar HMM with the same number of states is isomorphic to the epsilon-machine.*

This is the fundamental representation theorem of computational mechanics. The proof can be found in Shalizi & Crutchfield (2001), Theorem 1.

Remark 10.11. The epsilon-machine is a **hidden** Markov model: an external observer sees only the emitted symbols $a \in \mathcal{A}$, not the internal causal states $s \in \mathcal{S}$. However, because of unifilarity, if the observer knows (or can infer) the initial causal state, then all subsequent states can be tracked deterministically from the observed symbol sequence.

10.5 Statistical Complexity and Entropy Rate

The epsilon-machine naturally defines two complexity measures.

Definition 10.12 (Stationary distribution over causal states)**.** Since the process is stationary, the Markov chain over causal states (with transition matrix $T_{s,s'} = \sum_a T^{(a)}_{s,s'}$) has a unique stationary distribution π. We write $\pi(s) = P(S_t = s)$.

Definition 10.13 (Statistical complexity)**.** The **statistical complexity** is the Shannon entropy of the stationary distribution over causal states:

$$C_\mu = H[\mathcal{S}] = -\sum_{s\in\mathcal{S}} \pi(s)\log_2 \pi(s).$$

C_μ measures the **minimum amount of information about the past** that must be stored to optimally predict the future. It is measured in bits. Because the causal states are the minimal sufficient statistics, C_μ is the smallest possible memory for an optimal predictor.

Definition 10.14 (Entropy rate)**.** The **entropy rate** of the process is

$$h_\mu = \lim_{L\to\infty} H[X_L \mid X_1,\ldots,X_{L-1}] = \lim_{L\to\infty} \frac{1}{L} H[X_1,\ldots,X_L].$$

Because the epsilon-machine is unifilar, h_μ can be computed directly from the epsilon-machine:

$$h_\mu = H[X_t \mid S_t] = -\sum_{s\in\mathcal{S}} \pi(s) \sum_{a\in\mathcal{A}} P(a \mid s)\log_2 P(a \mid s)$$

where $P(a \mid s) = \sum_{s'} T^{(a)}_{s,s'}$ is the probability of emitting symbol a from state s.

This is a key computational advantage of the epsilon-machine: because knowing the causal state and the emitted symbol determines the next state (unifilarity), conditioning on the causal state captures all the predictive structure, and h_μ reduces to a single-step conditional entropy.

10.6 Excess Entropy (Predictive Information)

Definition 10.15 (Excess entropy)**.** The **excess entropy** (also called **predictive information** or **effective measure complexity**) is the mutual information between the semi-infinite past and the semi-infinite future:

$$E = I[\overleftarrow{X};\overrightarrow{X}].$$

E measures the total amount of apparent memory or structure in the process: how much information the past carries about the future (and vice versa).

Proposition 10.16. *The excess entropy can be expressed as:*

$$E = \sum_{k=1}^{\infty} \left[H[X_k \mid X_1,\ldots,X_{k-1}] - h_\mu\right].$$

Proof. Define $h_k = H[X_k \mid X_1,\ldots,X_{k-1}]$. This is the conditional entropy of the k-th symbol given all previous symbols. As $k\to\infty$, $h_k \to h_\mu$. The block entropy $H_L = H[X_1,\ldots,X_L] = \sum_{k=1}^{L} h_k$. The mutual information between past and future can be shown (Crutchfield & Feldman 2003) to equal $E = \lim_{L\to\infty}(H_L - L\cdot h_\mu) = \sum_{k=1}^{\infty}(h_k - h_\mu)$. Each term $h_k - h_\mu \geq 0$ since conditioning on more past can only reduce (or maintain) entropy.

Proposition 10.17 (Bounds)**.** *The excess entropy satisfies:*

$$0 \leq E \leq C_\mu.$$

The lower bound $E = 0$ holds for IID processes (no temporal structure). The upper bound $E = C_\mu$ holds when the causal states can be determined with certainty from a finite past (i.e., when the epsilon-machine's states are "synchronizing").

10.7 The Complexity-Entropy Plane

A powerful way to visualize and classify processes is to plot them in the (h_μ, C_μ) **plane**: the horizontal axis is the entropy rate (irreducible randomness) and the vertical axis is the statistical complexity (memory cost of optimal prediction).

The key regions are:

- **Bottom-left corner** ($h_\mu \approx 0, C_\mu \approx 0$): Simple periodic processes. Fully predictable, almost no memory needed (e.g., a constant sequence "000...").
- **Bottom-right corner** ($h_\mu \approx \log|\mathcal{A}|, C_\mu \approx 0$): IID random processes. Maximally unpredictable, but no structure to remember (e.g., fair coin).
- **Upper middle region** (intermediate h_μ, high C_μ): **Complex processes**. These are neither trivially ordered nor trivially random. They have significant internal structure requiring substantial memory to track, yet remain partially unpredictable. This is the "edge of chaos" region.

The bottom edge of the plane ($C_\mu = 0$) is occupied only by IID processes (the only processes requiring zero memory). Not all points in the plane are achievable: there are upper bounds on C_μ as a function of h_μ (see Feldman & Crutchfield 2003 for detailed analysis of the allowed region).

The complexity-entropy diagram provides a principled answer to the informal notion that "complexity lies between order and chaos." Processes with the richest computational structure are neither the most predictable nor the most random.

10.8 Worked Example 1: The Fair Coin (IID Bernoulli-1/2)

Process. Each X_t is independently drawn from $\{0, 1\}$ with $P(X_t = 0) = P(X_t = 1) = 1/2$.

Causal states. For any two pasts $\overleftarrow{x}$ and $\overleftarrow{x}'$, the future distribution is the same: each future symbol is independently Bernoulli(1/2), regardless of the past. Therefore all pasts are causally equivalent, and there is exactly **one causal state**: $\mathcal{S} = \{s_0\}$.

Epsilon-machine. The machine has a single state s_0. From s_0, symbol 0 is emitted with probability 1/2 and the machine returns to s_0; symbol 1 is emitted with probability 1/2 and the machine returns to s_0.

```
Epsilon-machine diagram:

         0 | 1/2
    +----------+
    |          |
    V          |
  (s_0) ------+
    |          ^
    |          |
    +----------+
         1 | 1/2
```

Statistical complexity. $C_\mu = H[\mathcal{S}] = -1 \cdot \log_2 1 = 0$ bits.

Entropy rate. $h_\mu = H[X_t \mid S_t = s_0] = -(1/2)\log_2(1/2) - (1/2)\log_2(1/2) = 1$ bit/symbol.

Excess entropy. $E = 0$ (past and future are independent).

Position in the complexity-entropy plane. $(h_\mu, C_\mu) = (1, 0)$: the bottom-right corner. Maximally random, zero structural complexity.

10.9 Worked Example 2: The Period-2 Process ("010101...")

Process. The sequence is deterministic and periodic: $\ldots 0\,1\,0\,1\,0\,1 \ldots$ (We can model this as a stochastic process concentrated on a single bi-infinite sequence, or more properly, on the two sequences $\ldots 0101 \ldots$ and $\ldots 1010 \ldots$ with equal probability, depending on the phase.)

Causal states. Consider the process where the observer does not know the phase. After observing a 0, the observer knows the next symbol is 1. After observing a 1, the observer knows the next symbol is 0. The past's predictive content is fully captured by the last symbol observed. Thus there are **two causal states**:

- s_0: "the most recent symbol was 0" (equivalently, the next symbol is 1).
- s_1: "the most recent symbol was 1" (equivalently, the next symbol is 0).

Epsilon-machine.

```
Epsilon-machine diagram:

  (s_0) ---- 1 | p=1 ----> (s_1)
    ^                         |
    |                         |
    +---- 0 | p=1 ------------+
```

From s_0, the machine deterministically emits 1 and transitions to s_1. From s_1, it deterministically emits 0 and transitions to s_0.

Stationary distribution. By symmetry, $\pi(s_0) = \pi(s_1) = 1/2$.

Statistical complexity. $C_\mu = -(1/2)\log_2(1/2) - (1/2)\log_2(1/2) = 1$ bit.

Entropy rate. From each state, the emitted symbol is deterministic: $h_\mu = H[X_t \mid S_t] = (1/2) \cdot 0 + (1/2) \cdot 0 = 0$ bits/symbol.

Excess entropy. $E = C_\mu = 1$ bit (the past perfectly determines the future through the causal state; the machine is synchronizing).

Position in the complexity-entropy plane. $(h_\mu, C_\mu) = (0, 1)$: zero randomness, $C_\mu = 1$ bit needed to track the phase.

10.10 Worked Example 3: The Golden Mean Process

Process. The **Golden Mean process** (also called the "no consecutive 1s" process) is the stationary stochastic process on $\mathcal{A} = \{0, 1\}$ that is the maximum-entropy (Markov) process supported on the Golden Mean shift: the subshift forbidding the word 11. After a 0, the next symbol can be 0 or 1; after a 1, the next symbol must be 0.

More precisely, we define a Markov chain with states corresponding to the last emitted symbol:

- After emitting 0: emit 0 with probability p and 1 with probability $1 - p$.
- After emitting 1: emit 0 with probability 1.

The parameter p is determined by the requirement that the chain be stationary. Let π_0, π_1 be the stationary probabilities of being in state "last symbol was 0" and "last symbol was 1" respectively. The balance equations are:

$$\pi_0 = p \cdot \pi_0 + 1 \cdot \pi_1, \qquad \pi_1 = (1 - p) \cdot \pi_0.$$

With the normalization $\pi_0 + \pi_1 = 1$, substituting the second equation into the first:

$$\pi_0 = p \cdot \pi_0 + (1-p) \cdot \pi_0 = \pi_0.$$

This is always satisfied, so p is a free parameter. Different values of p give different members of the family. The **maximum-entropy** case corresponds to $p = 1/\varphi$ where $\varphi = (1+\sqrt{5})/2$ is the golden ratio. But for concreteness, let us work with the **uniform Golden Mean process** where each allowed symbol is chosen with equal probability given the constraint. This gives:

- After a 0: emit 0 or 1 each with probability 1/2.
- After a 1: emit 0 with probability 1.

Causal states. What does the past tell us about the future? All that matters is whether the last symbol was 0 or 1:

- If the last symbol was 0: the next symbol is 0 or 1 with equal probability.
- If the last symbol was 1: the next symbol is 0 with certainty.

Two pasts ending in 0 have the same conditional future distribution, and two pasts ending in 1 have the same conditional future distribution. Thus there are **two causal states**:

- s_A: "last symbol was 0" (or: at the start).
- s_B: "last symbol was 1."

Epsilon-machine.

```
Epsilon-machine diagram:

          0 | 1/2
      +----------+
      |          |
      V          |
    (s_A) ------+
      |
      |  1 | 1/2
      V
    (s_B)
      |
      |  0 | 1
      V
    (s_A)
```

More explicitly:

- From s_A: emit 0 with probability 1/2, go to s_A; emit 1 with probability 1/2, go to s_B.
- From s_B: emit 0 with probability 1, go to s_A.

Stationary distribution. Let $\pi_A = \pi(s_A)$ and $\pi_B = \pi(s_B)$.

$$\pi_A = \frac{1}{2}\pi_A + 1 \cdot \pi_B, \qquad \pi_B = \frac{1}{2}\pi_A.$$

From the second equation, $\pi_B = \pi_A/2$. With $\pi_A + \pi_B = 1$: $\pi_A + \pi_A/2 = 1$, so $\pi_A = 2/3$ and $\pi_B = 1/3$.

Statistical complexity.

$$C_\mu = -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} = \frac{2}{3}\log_2\frac{3}{2} + \frac{1}{3}\log_2 3 = \log_2 3 - \frac{2}{3} \approx 0.918 \text{ bits.}$$

Entropy rate.

$$h_\mu = \pi_A \cdot H_A + \pi_B \cdot H_B$$

where $H_A = H[\text{Bernoulli}(1/2)] = 1$ bit and $H_B = H[\text{Bernoulli}(1)] = 0$ bits. Therefore:

$$h_\mu = \frac{2}{3}\cdot 1 + \frac{1}{3}\cdot 0 = \frac{2}{3} \approx 0.667 \text{ bits/symbol.}$$

Position in the complexity-entropy plane. $(h_\mu, C_\mu) \approx (0.667, 0.918)$: intermediate randomness, moderate complexity.

10.11 Worked Example 4: The Even Process

Process. The **Even process** is a well-known example in computational mechanics. The process generates binary strings where, between consecutive 1s, there must be an **even** number of 0s. That is, the blocks of 0s separating 1s have lengths $0, 2, 4, 6, \ldots$.

One way to define it: the process is generated by a hidden Markov model with two states $\{A, B\}$:

- **State** A: emit 1 with probability p, stay in A; emit 0 with probability $1-p$, go to B.
- **State** B: emit 0 with probability 1, go to A.

(The emitted 0 from state B is the "second" 0 in a pair, ensuring evenness.)

For concreteness, let $p = 1/2$, so from state A: each of $\{0, 1\}$ with probability $1/2$.

Claim: This HMM is already the epsilon-machine. We need to verify that the two states A and B are causally distinct (have different predictive futures) and that the machine is unifilar.

Unifilarity check. From A: symbol 0 leads deterministically to B, symbol 1 leads deterministically to A. From B: symbol 0 leads deterministically to A (symbol 1 is not emitted). Each (state, symbol) pair determines the next state uniquely. The machine is unifilar.

Causal distinctness. From state A, the next symbol can be 0 or 1 (each with probability $1/2$). From state B, the next symbol is 0 with certainty. These are different conditional distributions, so A and B are causally distinct.

Therefore the epsilon-machine has **two causal states**: $\mathcal{S} = \{s_A, s_B\}$.

Epsilon-machine diagram.

```
      1 | 1/2
  +----------+
  |          |
  V          |
(s_A) ------+
  |
  |  0 | 1/2
  V
(s_B)
  |
  |  0 | 1
  V
(s_A)
```

- From s_A: emit 1 with probability 1/2, go to s_A; emit 0 with probability 1/2, go to s_B.
- From s_B: emit 0 with probability 1, go to s_A.

Remark 10.18. This has exactly the same transition structure as the Golden Mean process in Example 3, but with the labels 0 and 1 swapped on the self-loop at s_A. The numerical calculations below are therefore identical.

Stationary distribution. $\pi_A = 2/3$, $\pi_B = 1/3$ (same balance equations as the Golden Mean example).

Statistical complexity.

$$C_\mu = -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} \approx 0.918 \text{ bits.}$$

Entropy rate.

$$h_\mu = \frac{2}{3}\cdot H[\mathrm{Ber}(1/2)] + \frac{1}{3}\cdot H[\mathrm{Ber}(1)] = \frac{2}{3}\cdot 1 + \frac{1}{3}\cdot 0 = \frac{2}{3} \approx 0.667 \text{ bits/symbol.}$$

Why the Even process is interesting. Despite being generated by a two-state HMM, the Even process is **not** itself a Markov chain of any finite order. Knowing any finite number of past symbols does not suffice to determine the causal state. To see this, note that after observing the past $\ldots 0\,0$, the causal state could be s_A (if the run of 0s has even length) or s_B (if odd length). Determining the state requires counting the parity of the 0-run length, which can be arbitrarily long. The Even process is thus a prototypical example of a **hidden Markov process** that is not a finite-order Markov chain — it has finite C_μ but requires arbitrarily long memory to synchronize.

10.12 Summary of Examples

Process	$\lvert\mathcal{S}\rvert$	C_μ (bits)	h_μ (bits/sym)	E (bits)	Region
Fair coin (IID)	1	0	1.000	0	Bottom-right
Period-2	2	1.000	0	1.000	Left edge
Golden Mean ($p = 1/2$)	2	0.918	0.667	—	Middle
Even process ($p = 1/2$)	2	0.918	0.667	—	Middle

The table illustrates the range of behaviors: from zero-complexity maximal randomness (fair coin) to zero-randomness moderate complexity (periodic), to processes with both nontrivial structure and nontrivial randomness (Golden Mean, Even process).

10.13 The CSSR Algorithm

In practice, we do not have access to the true process distribution — only a finite sample of observed data. The **Causal State Splitting Reconstruction** (CSSR) algorithm of Shalizi and Klinkner (2004) provides a principled method to infer the epsilon-machine from data.

Algorithm outline (CSSR).

Input: A finite symbol sequence $x_1, x_2, \ldots, x_N$ over alphabet $\mathcal{A}$, and a maximum history length $L_{\max}$.

Output: An estimated epsilon-machine $(\hat{\mathcal{S}}, \mathcal{A}, \{\hat{T}^{(a)}\})$.

1. **Initialize.** Start with histories of length $\ell = 1$. For each word $w \in \mathcal{A}^\ell$ that appears in the data, estimate the conditional distribution $\hat{P}(X_{t+1} \mid X_{t-\ell+1}^t = w)$ by counting occurrences.

1. **Test and merge.** Two histories w and w' are provisionally placed in the same causal state if a statistical test (e.g., a χ^2 test or Kolmogorov-Smirnov test) fails to reject the null hypothesis $P(\cdot \mid w) = P(\cdot \mid w')$ at some significance level α.

1. **Increase history length.** Increment $\ell \to \ell + 1$. For each existing causal state, check whether histories of the longer length within the same state still have the same conditional next-symbol distribution. If not, **split** the state: separate histories whose distributions are statistically distinguishable.

1. **Iterate** steps 2–3 until $\ell = L_{\max}$ or until the state set stabilizes.

1. **Determinize.** Ensure the resulting machine is unifilar by checking that each (state, symbol) pair leads to a unique successor state. If not, apply additional splits.

1. **Estimate transitions.** From the final state assignments, estimate the transition probabilities $\hat{T}^{(a)}_{s,s'}$ by counting transitions in the data.

Consistency. Shalizi and Crutchfield (2001) proved that, under mild conditions, CSSR is **asymptotically consistent**: as $N \to \infty$ (with $L_{\max}$ growing appropriately), the inferred epsilon-machine converges to the true one.

Practical considerations.

- The choice of significance level α and maximum length $L_{\max}$ are tuning parameters. Typical choices are $\alpha = 0.001$ and $L_{\max}$ determined by the amount of available data ($L_{\max} \sim \log N$ is a rough guideline).

- CSSR can overfit on small data, producing too many states. Cross-validation or information-theoretic model selection (e.g., BIC) can help.

- More recent algorithms (e.g., Bayesian structural inference) address some of CSSR's limitations.

10.14 Connection to Dynamical Systems

For the discrete dynamical systems studied in earlier chapters of this book, computational mechanics provides a natural framework for characterizing the "computational complexity" of orbits.

Setting. Let (X, f) be a deterministic discrete dynamical system (DDS) with state space X and transition map $f : X \to X$. Suppose we observe the system through a **measurement function** $\phi : X \to \mathcal{A}$ that maps each state to a symbol in a finite alphabet $\mathcal{A}$. The observed sequence is

$$a_0, a_1, a_2, \ldots \quad \text{where} \quad a_t = \phi(f^t(x_0)).$$

If we introduce stochasticity — either through a distribution over initial conditions x_0, through noisy dynamics, or through a coarse measurement function ϕ — the observed sequence becomes a stochastic process, and we can construct its epsilon-machine.

What the epsilon-machine captures. The epsilon-machine of the observed process reveals the **computational structure** of the dynamics as seen through the observation. Specifically:

- C_μ measures **how much memory** the dynamics effectively uses: how much information from the past orbit is relevant for predicting the future orbit.

- h_μ measures the **intrinsic unpredictability** of the orbit (as observed through ϕ). For a deterministic system with a deterministic measurement, $h_\mu = 0$; the process becomes interesting when ϕ is many-to-one (symbolic dynamics) or when noise is present.

- High C_μ with intermediate h_μ indicates dynamics that are neither trivially periodic nor trivially random — they are performing nontrivial "computation" in the sense of maintaining and transforming stored information.

Example: Logistic map at the edge of chaos. The logistic map $f(x) = rx(1-x)$ on $[0,1]$, observed through a binary partition $\phi(x) = 0$ if $x < 1/2$ and $\phi(x) = 1$ if $x \geq 1/2$, produces symbolic sequences whose epsilon-machine complexity varies dramatically with the parameter r:

- For r in the periodic regime: the epsilon-machine is a simple cycle, C_μ is small, $h_\mu = 0$.

- For r in the fully chaotic regime: the epsilon-machine may be trivial (for the full shift) or have moderate complexity.

- At the **onset of chaos** ($r = r_\infty \approx 3.5699\ldots$, the Feigenbaum point): the epsilon-machine has infinitely many states, $C_\mu = \infty$, and $h_\mu = 0$. This is a signature of maximal computational complexity at the edge of chaos.

Connection to meta-rules and rule-generated systems. When the DDS is itself generated by a rule (e.g., a cellular automaton, a tag system, or one of the meta-rule systems studied in earlier chapters), the epsilon-machine of the observed orbit provides a quantitative fingerprint of the rule's computational behavior. Rules that generate high C_μ with intermediate h_μ are the "interesting" ones — they are performing nontrivial information processing, neither trivially repetitive nor trivially destructive of information.

10.15 Further Results and Connections

Crypticity. The **crypticity** χ of a process is the expected number of observed symbols needed to synchronize to the correct causal state, starting from complete ignorance. Formally, $\chi = C_\mu - E$ measures the "hidden" complexity not revealed by the excess entropy. The Even process has $\chi > 0$ (it takes arbitrarily long to determine the parity of the 0-run), making it cryptic. The Golden Mean process has $\chi = 0$ (one symbol suffices to determine the state).

Causal irreversibility. Constructing the epsilon-machine of the time-reversed process generally yields a different machine (the **reverse epsilon-machine**) with a different statistical complexity C_μ^+. The quantity $C_\mu - C_\mu^+$ measures the **causal asymmetry** between past-to-future and future-to-past prediction.

Transducers and input-output systems. The epsilon-machine framework extends to **epsilon-transducers**: when we have an input process and an output process, the epsilon-transducer captures the minimal model of the input-output relation. This connects to the theory of finite-state transducers and to questions about the computational capacity of physical systems.

Quantum extensions. Recent work by Gu, Wiesner, and others has shown that quantum epsilon-machines can achieve strictly lower complexity than classical ones ($C_q < C_\mu$), demonstrating a quantum advantage for prediction. The difference $C_\mu - C_q$ has been proposed as a measure of the "quantumness" of a process's structure.

10.16 Summary

Computational mechanics provides a principled, unique decomposition of any stationary process into:

1. **Causal states** $\mathcal{S}$: the minimal sufficient statistics of the past for predicting the future.
2. **The epsilon-machine** $(\mathcal{S}, \mathcal{A}, \{T^{(a)}\})$: the unique minimal unifilar HMM generating the process.
3. **Statistical complexity** $C_\mu = H[\mathcal{S}]$: the memory cost of optimal prediction.
4. **Entropy rate** h_μ: the irreducible randomness per symbol.
5. **Excess entropy** $E = I[\overleftarrow{X}; \overrightarrow{X}]$: the total shared information between past and future.

The key theorems are:

- Causal states are minimal sufficient statistics (Proposition 10.6).
- The epsilon-machine is unifilar (Theorem 10.8).
- The epsilon-machine is the unique minimal unifilar HMM (Theorem 10.9).
- $0 \leq E \leq C_\mu$ (Proposition 10.15).

The complexity-entropy plane (h_μ, C_μ) reveals that "complexity" in the colloquial sense — rich, structured, interesting behavior — corresponds to the upper-middle region of the plane, between pure order ($h_\mu = 0$) and pure randomness ($h_\mu = \log |\mathcal{A}|$).

10.17 References

1. **Crutchfield, J. P. and Young, K.** (1989). "Inferring statistical complexity." *Physical Review Letters*, 63(2):105–108. [The founding paper of computational mechanics.]

1. **Shalizi, C. R. and Crutchfield, J. P.** (2001). "Computational mechanics: Pattern and prediction, structure and simplicity." *Journal of Statistical Physics*, 104(3/4):817–879. [The comprehensive mathematical development of the theory, including proofs of the main theorems.]

1. **Crutchfield, J. P.** (2012). "Between order and chaos." *Nature Physics*, 8:17–24. [An accessible overview of computational mechanics and the complexity-entropy plane.]

1. **Feldman, D. P. and Crutchfield, J. P.** (2003). "Structural information in two-dimensional patterns: Entropy convergence and excess entropy." *Physical Review E*, 67:051104. [Detailed analysis of excess entropy, the complexity-entropy plane, and the allowed region.]

1. **Shalizi, C. R. and Klinkner, K. L.** (2004). "Blind construction of optimal nonlinear recursive predictors for discrete sequences." In *Uncertainty in Artificial Intelligence: Proceedings of the Twentieth Conference (UAI 2004)*, pp. 504–511. [The CSSR algorithm for epsilon-machine inference from data.]

1. **Crutchfield, J. P. and Feldman, D. P.** (2003). "Regularities unseen, randomness observed: Levels of entropy convergence." *Chaos*, 13(1):25–54. [Systematic treatment of entropy convergence and its connection to structure.]

1. **Upper, D. R.** (1997). *Theory and algorithms for hidden Markov models and generalized hidden Markov models.* Ph.D. thesis, University of California, Berkeley. [Early detailed treatment of epsilon-machines and their relationship to HMMs.]

1. **Travers, N. F. and Crutchfield, J. P.** (2014). "Equivalence of history and generator epsilon-machines." *arXiv:1111.4500.* [Rigorous proof that the forward-time and history-based constructions yield the same object.]

10.18 Recommended Reading

For getting started:

- **Crutchfield (2012)**, "Between order and chaos" in *Nature Physics*, is the best short introduction. It is accessible to a general scientific audience and conveys the key ideas in 8 pages.

- **Shalizi & Crutchfield (2001)** is the comprehensive technical reference. Sections 1–3 are accessible; later sections require more mathematical maturity. Essential for anyone wanting to use the theory rigorously.

For practical inference:

- **Shalizi & Klinkner (2004)** describes the CSSR algorithm with implementation details. The companion software (written in Python) is available online.

- **Strelioff, C. C. and Crutchfield, J. P.** (2014). "Bayesian structural inference for hidden processes." *Physical Review E*, 89:042119. A Bayesian alternative to CSSR that handles small data better.

For connections to physics:

- **Crutchfield, J. P.** (1994). "The calculi of emergence: Computation, dynamics and induction." *Physica D*, 75:11–54. Places computational mechanics in the broader context of complex systems science.

For quantum extensions:

- **Gu, M. et al.** (2012). "Quantum mechanics can reduce the complexity of classical models." *Nature Communications*, 3:762. Introduces quantum epsilon-machines and proves the quantum advantage.

For software implementations:

- The CMPy package (Computational Mechanics in Python) provides implementations of CSSR and related algorithms: https://github.com/csc-ubc-okanagan/cmpy

Part III

Self-Reference and Reflection

Chapter 11

Kleene's Recursion Theorem and Fixed Points of Program Transformations

11.1 Introduction

Kleene's recursion theorem is one of the most surprising results in computability theory. It says, roughly, that for any computable way of transforming programs, there exists a program that is a *fixed point* of that transformation — not in the sense that the program text is unchanged, but in the sense that the transformed program computes the same function as the original.

This result has far-reaching consequences. It explains why quines (self-printing programs) must exist in any sufficiently powerful programming language, why self-replicating programs (viruses) are inevitable, why Rice's theorem holds, and why certain self-referential strategies are available in algorithmic learning theory. In the context of discrete dynamical systems with evolving rules — the $(f, x) \mapsto (\varphi(f, x), f(x))$ framework of earlier chapters — it guarantees the existence of "self-consistent" rules that no meta-rule can escape.

This chapter develops the necessary background (Gödel numbering, the s-m-n theorem), states and proves the recursion theorem, and works through its principal applications.

11.2 Setup: Acceptable Numberings and the s-m-n Theorem

11.2.1 11.2.1 Gödel Numbering

Fix a standard model of computation (Turing machines, register machines, lambda calculus — the choice does not matter). Every program in this model can be encoded as a natural number. Such an encoding is called a *Gödel numbering* or *acceptable numbering* of programs.

Definition 11.1 (Partial Computable Function)**.** For each $e \in \mathbb{N}$, we write φ_e for the partial function $\mathbb{N} \rightharpoonup \mathbb{N}$ computed by the program with index (Gödel number) e. The family $\{\varphi_e\}_{e\in\mathbb{N}}$ is called an *acceptable numbering* of the partial computable functions.

Some conventions:

- $\varphi_e(x)\downarrow$ means the computation of program e on input x halts (is defined).
- $\varphi_e(x)\uparrow$ means it diverges (is undefined).
- $\varphi_e = \varphi_{e'}$ means the two programs compute the same partial function: for all x, $\varphi_e(x)\downarrow = \varphi_{e'}(x)\downarrow$ or both diverge.
- $W_e = \mathrm{dom}(\varphi_e) = \{x : \varphi_e(x)\downarrow\}$ is the domain of program e.

Remark 11.2. Many different indices compute the same function. The map $e \mapsto \varphi_e$ is surjective onto the partial computable functions but far from injective: every partial computable function has infinitely many indices.

11.2.2 11.2.2 The s-m-n Theorem (Parameter Theorem)

The s-m-n theorem (also called the parameter theorem or parametrization theorem) is a basic structural property of acceptable numberings. It says that "hard-coding" an input into a program can itself be done computably.

Theorem 11.3 (s-m-n Theorem; Kleene 1943)**.** *There exists a total computable function $s : \mathbb{N}^2 \to \mathbb{N}$ such that for all $e, x, y \in \mathbb{N}$:*

$$\varphi_{s(e,x)}(y) = \varphi_e(x, y).$$

More generally, for any $m, n \geq 1$, there exists a total computable injection s_n^m such that

$$\varphi_{s_n^m(e,x_1,\ldots,x_m)}(y_1, \ldots, y_n) = \varphi_e(x_1, \ldots, x_m, y_1, \ldots, y_n).$$

Interpretation. Suppose program e takes two inputs (x, y). Then $s(e, x)$ is the index of a new program that has x "baked in" and only needs the second input y. The function s is a compiler that performs partial evaluation, and it is itself total and computable.

Example 11.4. Suppose $\varphi_e(x, y) = x + y$. Then $\varphi_{s(e,3)}(y) = 3 + y$. The index $s(e, 3)$ is the Gödel number of a program that adds 3 to its input.

11.3 Kleene's Second Recursion Theorem

11.3.1 11.3.1 Statement

Theorem 11.5 (Kleene's Second Recursion Theorem; Kleene 1938, 1952)**.** *Let $f : \mathbb{N} \to \mathbb{N}$ be any total computable function. Then there exists an index $e \in \mathbb{N}$ such that*

$$\varphi_e = \varphi_{f(e)}.$$

In words: for any computable transformation of programs, there is a program that is a fixed point of that transformation in the extensional sense — the program and its transform compute the same partial function.

11.3.2 11.3.2 Proof

The proof is short but uses a clever diagonal trick combined with the s-m-n theorem.

Proof. Define a computable function d as follows. Given input x, the function d does:

1. Compute $s(x, x)$ — this is the index of the program φ_x with its first argument specialized to x.
2. Apply f to the result: compute $f(s(x, x))$.
3. Return the index $f(s(x, x))$.

So $d(x) = f(s(x, x))$. Since s and f are total computable, d is total computable. Let d_0 be an index for the function d, meaning $\varphi_{d_0}(x) = d(x) = f(s(x, x))$ for all x.

But actually, we need d to *simulate* the program $f(s(x, x))$ rather than merely return its index. More precisely, define d so that:

$$\varphi_{d_0}(x, y) = \varphi_{f(s(x,x))}(y).$$

That is, d_0 is a program of two arguments: given (x, y), it computes $s(x, x)$, applies f to get an index $f(s(x, x))$, and then runs that program on y.

Now set:

$$e = s(d_0, d_0).$$

We verify this is a fixed point. For any y:

$$\varphi_e(y) = \varphi_{s(d_0,d_0)}(y) = \varphi_{d_0}(d_0, y) = \varphi_{f(s(d_0,d_0))}(y) = \varphi_{f(e)}(y).$$

The first equality is the definition of e. The second is the s-m-n theorem. The third is the definition of d_0. The fourth substitutes $e = s(d_0, d_0)$ back.

Since this holds for all y, we have $\varphi_e = \varphi_{f(e)}$.

11.3.3 11.3.3 Dissecting the Proof

The proof has the structure of a diagonalization that "undoes itself." Compare it with the proof of the undecidability of the halting problem, which constructs a program that does the *opposite* of what a purported decider predicts. Here, instead of contradicting, we construct a program that *agrees* with its own transform.

The key insight is the self-application $s(x, x)$. The program indexed by $s(x, x)$ behaves like "φ_x applied to itself" — reminiscent of the λ-calculus combinator $\omega = \lambda x.x\,x$. The fixed-point construction builds a form of self-reference without any explicit self-reference in the programming language.

11.4 Rogers's Fixed-Point Theorem

The following equivalent formulation, due to Rogers (1967), rephrases the result slightly.

Theorem 11.6 (Rogers's Fixed-Point Theorem)**.** *Let $f : \mathbb{N} \to \mathbb{N}$ be any total computable function. Then there exists $n \in \mathbb{N}$ with $\varphi_n = \varphi_{f(n)}$.*

This is identical in content to Theorem 11.4. Rogers's contribution was to place the result in a broader framework of numbering theory and to show that the existence of fixed points characterizes acceptable numberings: an effective numbering of the partial computable functions is acceptable if and only if it satisfies the recursion theorem (Rogers 1958).

Theorem 11.7 (Myhill–Shepherdson; Rogers 1958)**.** *A numbering $\{\psi_e\}_{e\in\mathbb{N}}$ of the partial computable functions has the fixed-point property (i.e., every total computable f has an index n with $\psi_n = \psi_{f(n)}$) if and only if it is an acceptable numbering.*

11.5 What the Theorem Does NOT Say

A common misunderstanding deserves emphasis.

The theorem does not claim $e = f(e)$. It claims $\varphi_e = \varphi_{f(e)}$. The indices e and $f(e)$ are in general different natural numbers — they are different programs. What the theorem guarantees is that these two different programs compute the same function.

- **Intensional fixed point:** $e = f(e)$. The program index itself is unchanged.
- **Extensional fixed point:** $\varphi_e = \varphi_{f(e)}$. The computed function is unchanged.

Kleene's theorem provides extensional fixed points. Intensional fixed points need not exist. For instance, $f(e) = e + 1$ has no intensional fixed point (no e satisfies $e = e + 1$), but it does have an extensional one: there exists e with $\varphi_e = \varphi_{e+1}$.

11.6 Worked Examples

11.6.1 11.6.1 Example 1: A Concrete Fixed Point

Let $f(e) = 2e+7$. This is a total computable function. The recursion theorem guarantees there exists e with $\varphi_e = \varphi_{2e+7}$.

We cannot easily exhibit a specific numerical value of e without committing to a particular Gödel numbering, but the theorem assures us the number exists. Intuitively: among the infinitely many indices for every partial computable function, at least one index e is "aligned" so that $2e + 7$ is another index for the same function.

11.6.2 11.6.2 Example 2: Quine Construction

A *quine* is a program that, when run with no input, outputs its own source code (its own Gödel number, in our formalism).

Claim. In any acceptable numbering, there exists an index e such that $\varphi_e(x) = e$ for all x (i.e., φ_e is the constant function with value e).

Proof using the recursion theorem. Define a total computable function f as follows: given an index i, let $f(i)$ be the index of the program that, on any input x, outputs i. That is:

$$\varphi_{f(i)}(x) = i \quad \text{for all } x.$$

The function f is total computable: we can effectively construct, from any number i, a program that ignores its input and prints i.

By the recursion theorem, there exists e with $\varphi_e = \varphi_{f(e)}$. But $\varphi_{f(e)}$ is the constant function that always outputs e. Therefore $\varphi_e(x) = e$ for all x.

The program e outputs its own index on every input. In particular, on input 0 (or with no input), it prints e — its own source code. This is a quine.

Explicit construction (without invoking the full theorem). Following the proof of Theorem 11.4 directly:

1. Define $\varphi_{d_0}(x, y) = \varphi_{f(s(x,x))}(y) = s(x, x)$. That is, d_0 is a program that, given (x, y), computes $s(x, x)$ and returns it. (Here f maps i to a program printing i, so $\varphi_{f(s(x,x))}(y) = s(x, x)$.)

2. Set $e = s(d_0, d_0)$.

3. Then $\varphi_e(y) = \varphi_{s(d_0,d_0)}(y) = \varphi_{d_0}(d_0, y) = s(d_0, d_0) = e$.

This mirrors the structure of real quines in programming languages. In Python, for instance:

```
s = 's = %r\nprint(s %% s)'
print(s % s)
```

The string ‘s‘ plays the role of d_0, and the substitution ‘s

11.6.3 11.6.3 Example 3: A Fixed Point of "Double the Program"

Consider a computable function f that takes a program index e and produces a new program $f(e)$ that runs φ_e twice: $\varphi_{f(e)}(x) = \varphi_e(\varphi_e(x))$ (composition of φ_e with itself, where the result is undefined if either application diverges).

The recursion theorem gives us e^* with $\varphi_{e^*} = \varphi_{f(e^*)}$. This means:

$$\varphi_{e^*}(x) = \varphi_{e^*}(\varphi_{e^*}(x)) \quad \text{for all } x.$$

What functions satisfy $g(x) = g(g(x))$ for all x? Any function satisfying $g(g(x)) = g(x)$ works — that is, any *idempotent* function. Examples include:

- The identity: $g(x) = x$.
- Any constant function: $g(x) = c$.
- Any projection onto a fixed set: $g(x) = x$ if $x \in S$, $g(x) = c \in S$ otherwise, provided $g(c) = c$.
- The everywhere-undefined function: $g(x)\uparrow$ for all x.

The theorem guarantees that at least one index e^* for such an idempotent function exists as a fixed point of f.

11.6.4 11.6.4 Example 4: Self-Referential Programs

The recursion theorem can be used to construct programs that "know their own index." Suppose we want a program e that, on input x, outputs the pair (e, x) — it reports its own index together with its input.

Define $f(i)$ to be the index of the program: "on input x, output (i, x)." Then $\varphi_{f(i)}(x) = (i, x)$, and f is total computable. The recursion theorem gives e with $\varphi_e = \varphi_{f(e)}$, so $\varphi_e(x) = (e, x)$.

This is the standard way to build self-referential programs in computability theory: you write the program *as if* you had access to your own index as a parameter, and then invoke the recursion theorem to "close the loop."

11.7 Applications

11.7.1 11.7.1 Application 1: Self-Replicating Programs (Viruses)

A *self-replicating program* (or virus, in the informal sense) is a program that produces a copy of itself as output, possibly interleaved with other behavior.

More precisely, let $\text{write}(i)$ be a computable function that returns the index of a program that writes i to some specified location. Define $f(i) = \text{write}(i)$. Then f is total computable, and the recursion theorem gives e with $\varphi_e = \varphi_{f(e)} = \varphi_{\text{write}(e)}$. The program e writes its own index — it copies itself.

The existence of self-replicating programs is thus a mathematical inevitability in any sufficiently expressive computational model. This observation has been attributed to von Neumann (in the context of self-reproducing automata) and was made rigorous by the recursion theorem. Adleman (1988) explicitly connected the recursion theorem to computer viruses.

11.7.2 11.7.2 Application 2: Rice's Theorem

Theorem 11.8 (Rice 1953)**.** *Let $\mathcal{A}$ be a set of partial computable functions that is non-trivial (i.e., $\mathcal{A} \neq \emptyset$ and $\mathcal{A}$ does not contain all partial computable functions). Then the index set*

$$A = \{e \in \mathbb{N} : \varphi_e \in \mathcal{A}\}$$

is not computable (not decidable).

Proof using the recursion theorem. *Suppose for contradiction that A is decidable. Since $\mathcal{A}$ is non-trivial, choose indices a and b such that $\varphi_a \in \mathcal{A}$ and $\varphi_b \notin \mathcal{A}$.*

Define the total computable function:

$$f(e) = \begin{cases} b \& \textit{if } e \in A, \\ a \& \textit{if } e \notin A. \end{cases}$$

This is computable because we assumed A is decidable.

By the recursion theorem, there exists e^* with $\varphi_{e^*} = \varphi_{f(e^*)}$.

- *If $e^* \in A$, then $f(e^*) = b$, so $\varphi_{e^*} = \varphi_b \notin \mathcal{A}$, contradicting $e^* \in A$.*
- *If $e^* \notin A$, then $f(e^*) = a$, so $\varphi_{e^*} = \varphi_a \in \mathcal{A}$, contradicting $e^* \notin A$.*

Either case is a contradiction, so A is not decidable.

This is one of the cleanest proofs of Rice's theorem. The recursion theorem provides the fixed point that the diagonalization needs.

11.7.3 11.7.3 Application 3: Fixed Points in Learning Theory

In *algorithmic learning theory* (also called inductive inference), a learner M receives an enumeration of the values of a total function f and must, in the limit, output an index e such that $\varphi_e = f$. The recursion theorem enables the construction of *self-referential learners* — learners that can refer to their own future output.

Case (1994) showed that self-referential learners (those that exploit the recursion theorem to access their own index) can identify strictly more function classes than non-self-referential ones. The key idea: a learner that "knows its own program" can reason about what it would do in hypothetical situations, gaining an advantage that provably cannot be replicated without self-reference.

Theorem 11.9 (Case 1994)**.** *There exist classes of computable functions that are identifiable in the limit by a self-referential learner but not by any non-self-referential learner.*

This shows that the recursion theorem is not merely a curiosity — it provides genuine computational power in the context of learning.

11.8 The Recursion Theorem for Dynamical Systems

11.8.1 11.8.1 Meta-Rules and Fixed Points

Recall the framework from earlier chapters: a discrete dynamical system with evolving rules is a pair (f, x) that evolves by

$$(f, x) \longmapsto (\varphi(f, x),\ f(x)),$$

where φ is a *meta-rule* that updates the function component f based on the current state.

If we focus on the function component alone, the meta-rule defines a transformation $T : f \mapsto \varphi(f, x)$ (parametrized by the state x, but let us first consider the state-independent case $T : f \mapsto \varphi(f)$).

Theorem 11.10 (Fixed Points of Meta-Rules)**.** *Let φ be any computable meta-rule, viewed as a total computable function on program indices: $\varphi : \mathbb{N} \to \mathbb{N}$ maps the index of f to the index of the "updated" function. Then there exists an index e^* such that*

$$\varphi_{e^*} = \varphi_{\varphi(e^*)}.$$

That is, $f^* = \varphi_{e^*}$ is an extensional fixed point of the meta-rule: applying φ to f^* yields a program that computes the same function as f^*.

Proof. This is an immediate application of Kleene's recursion theorem (Theorem 11.4) with $f = \varphi$.

□

11.8.2 11.8.2 Consequences for the (f, x) Framework

In the full framework $(f, x) \mapsto (\varphi(f, x), f(x))$, the meta-rule depends on the current state. If we fix a state x_0 and define $T_{x_0}(f) = \varphi(f, x_0)$, then for each x_0 there is a fixed point $f^*_{x_0}$. But the more interesting observation is global:

Observation 11.11. *For any computable meta-rule φ, there exist "self-consistent" functions f^* such that $\varphi(f^*, x) = f^*$ (extensionally) for every state x, provided φ does not depend on x when restricted to such f^*.*

In the general state-dependent case, self-consistency becomes: the function f^* is such that the meta-rule, evaluated at any state the system actually visits along the orbit of f^*, returns (an index of) f^*. Formally, if $(f^*, x_0) \mapsto (f^*, f^*(x_0)) \mapsto (f^*, f^*(f^*(x_0))) \mapsto \cdots$, then f^* is a true fixed point in the sense that the function component never changes.

These fixed points are unavoidable: no matter how cleverly φ is designed, it cannot escape having extensional fixed points. This is a fundamental structural constraint on any computable system of evolving rules.

Example 11.12. Consider the meta-rule "given program e, produce the program that runs φ_e and then adds 1 to the result." Formally, $\varphi_{\text{meta}}(e)$ is the index of the program $x \mapsto \varphi_e(x) + 1$.

One might think this meta-rule has no fixed point — after all, it always increments the output. But the recursion theorem says otherwise. There exists e^* with $\varphi_{e^*} = \varphi_{\text{meta}(e^*)}$, meaning $\varphi_{e^*}(x) = \varphi_{e^*}(x) + 1$ for all x in the domain. The only way this can hold is if $\varphi_{e^*}(x)$ is *undefined* for all x — the fixed point is the everywhere-undefined (nowhere-halting) function. The meta-rule does have a fixed point; it is just the trivial one $\varphi_{e^*} = \perp$.

This example illustrates an important subtlety: the recursion theorem guarantees existence of fixed points but says nothing about their computability or non-triviality. The fixed point may be the everywhere-undefined function.

11.8.3 11.8.3 From Extensional to Intensional Fixed Points in Finite Systems

In the computability-theoretic setting (countably infinite function space), only extensional fixed points are guaranteed. But in the finite discrete dynamical systems studied in earlier chapters, the function space S^S is finite. Every function $T : S^S \to S^S$ on a finite set must have a periodic orbit, and in particular:

Proposition 11.13. *If $|S|$ is finite, then any meta-rule $T : S^S \to S^S$ has an intensional fixed point or a finite cycle. In particular, there exists $k \geq 1$ and $f^* \in S^S$ such that $T^k(f^*) = f^*$.*

When T has an actual fixed point ($k = 1$), we get $T(f^*) = f^*$: the function is unchanged even as a syntactic object (since in a finite setting, the function is the object — there is no distinction between a function and its index).

This is stronger than what Kleene's theorem provides. Finiteness buys us intensional fixed points for free.

11.9 Limitations and Caveats

1. **Extensional only.** In the computable setting, the fixed point satisfies $\varphi_e = \varphi_{f(e)}$ but generally $e \neq f(e)$. The program *text* changes; only the *behavior* is preserved.

1. **No control over which fixed point.** The proof gives a specific e, but there may be many fixed points of f, and the theorem does not let us choose one with desirable properties (e.g., a total function, an efficient program).

1. **The fixed point may be trivial.** As Example 11.11 showed, the fixed point may be the everywhere-undefined function. The theorem does not guarantee interesting or useful fixed points.

1. **Requires totality of f.** The standard recursion theorem assumes f is total computable. There are extensions to partial functions (the recursion theorem with parameters), but the basic statement requires totality.

1. **Non-constructive flavor.** While the proof is constructive (it gives an explicit $e = s(d_0, d_0)$), computing e requires knowing the index d_0, which depends on the Gödel numbering. In practice, the theorem is used as an existence result.

11.10 Summary

Concept	Statement
s-m-n theorem	$\varphi_{s(e,x)}(y) = \varphi_e(x, y)$; parameter specialization is computable
Kleene's recursion theorem	For total computable f, $\exists\, e$: $\varphi_e = \varphi_{f(e)}$
Fixed point type	Extensional ($\varphi_e = \varphi_{f(e)}$), not intensional ($e = f(e)$)
Quines	Exist in every acceptable numbering; follow from the recursion theorem
Rice's theorem	No non-trivial extensional property of programs is decidable
Finite DDS	Meta-rules on finite S^S always have intensional fixed points or cycles

The recursion theorem tells us something deep about the nature of computation: self-reference is inescapable. Any computable system that transforms programs must have programs that are invariant (up to extensional equivalence) under that transformation. For the study of discrete dynamical systems with evolving rules, this means that every meta-rule φ has unavoidable fixed points — functions that φ cannot change.

11.11 References

- Adleman, L. M. (1988). An abstract theory of computer viruses. *Advances in Cryptology — CRYPTO '88*, LNCS 403, pp. 354–374.

- Case, J. (1994). Infinitary self-reference in learning theory. *Journal of Experimental and Theoretical Artificial Intelligence*, 6(1), 3–16.

- Cutland, N. J. (1980). *Computability: An Introduction to Recursive Function Theory.* Cambridge University Press.

- Kleene, S. C. (1938). On notation for ordinal numbers. *Journal of Symbolic Logic*, 3(4), 150–155.

- Kleene, S. C. (1943). Recursive predicates and quantifiers. *Transactions of the AMS*, 53(1), 41–73.
- Kleene, S. C. (1952). *Introduction to Metamathematics*. North-Holland.
- Kiselyov, O. (2002). Kleene's second recursion theorem: A functional pearl. Unpublished manuscript.
- Rice, H. G. (1953). Classes of recursively enumerable sets and their decision problems. *Transactions of the AMS*, 74(2), 358–366.
- Rogers, H., Jr. (1958). Gödel numberings of partial recursive functions. *Journal of Symbolic Logic*, 23(3), 331–341.
- Rogers, H., Jr. (1967). *Theory of Recursive Functions and Effective Computability*. McGraw-Hill. Reprinted by MIT Press, 1987.
- Soare, R. I. (1987). *Recursively Enumerable Sets and Degrees*. Springer-Verlag.

11.12 Recommended Reading

For learning computability theory:

- **Rogers (1967)** remains the gold standard textbook for classical recursion theory. Chapter 11 covers the recursion theorem with full proofs and applications. Dense but rewarding.
- **Cutland (1980)** is a gentler introduction suitable for undergraduates. Chapter 11 covers the recursion theorem clearly.
- **Sipser, M.** (2012). *Introduction to the Theory of Computation*, 3rd ed. Cengage. Chapter 6 covers computability basics; less depth on the recursion theorem but more accessible.

For the recursion theorem specifically:

- **Kleene (1952)**, *Introduction to Metamathematics*, is the original source. Historical but still readable.
- **Moschovakis, Y.** (2010). "Kleene's amazing second recursion theorem." *Bulletin of Symbolic Logic*, 16(2), 189–239. An excellent modern survey with historical context and advanced applications.

For applications to computer science:

- **Adleman (1988)** applies the recursion theorem to computer viruses. Short, elegant, and somewhat unsettling.
- **Jones, N. D.** (1997). *Computability and Complexity: From a Programming Perspective*. MIT Press. Connects recursion-theoretic ideas to programming language theory.

For quines specifically:

- **Thompson, K.** (1984). "Reflections on Trusting Trust." *Communications of the ACM*, 27(8), 761–763. The famous Turing Award lecture on self-replicating programs and their security implications.

Chapter 12

Self-Modifying Computation and Reflective Towers

12.1 Introduction

Every discrete dynamical system (X, f) studied so far has a fixed transition rule: the function f does not change as the system evolves. Even in the (f, x) framework where f is updated by a meta-rule φ, the meta-rule itself is fixed. But what if the meta-rule could also change? And the meta-meta-rule? This line of questioning leads to *reflective towers*: infinite stacks of interpreters, each governing the one below, each potentially subject to modification from below.

The idea has independent roots in automata theory (von Neumann's self-reproducing automata), programming language semantics (Smith's procedural reflection), and theoretical biology (Kampis's self-modifying systems). This chapter traces the key ideas, gives precise definitions where possible, and connects the tower construction back to the (f, x) dynamical systems framework.

12.2 Von Neumann's Self-Reproducing Automata

In unpublished lectures from the late 1940s (edited posthumously by Burks, 1966), John von Neumann posed the question: *Can a machine build a copy of itself?* At first glance, self-reproduction seems paradoxical. A machine that constructs an arbitrary target from a blueprint must be at least as complex as any target it can build. But then the machine itself, being the most complex thing it can build, would require a blueprint at least as complex as itself — an apparent infinite regress.

Von Neumann resolved this by separating the roles of the description.

Definition 12.1 (Universal Constructor)**.** A *universal constructor* is a machine U that, given a description tape $d(M)$ encoding any machine M, performs two operations:

1. **Interpretation.** U reads $d(M)$ as instructions and constructs a physical copy of M. 2. **Copying.** U duplicates the tape $d(M)$ and attaches the copy to the newly built M.

The result is a new machine M equipped with its own description $d(M)$. When $M = U$, the universal constructor reproduces itself: U reads $d(U)$, builds a copy of U, copies $d(U)$, and attaches it. No infinite regress arises because the description $d(U)$ plays two distinct roles — it is *interpreted* (as construction instructions) and *copied* (as passive data).

Remark 12.2. Von Neumann's construction anticipates the mechanism of biological reproduction discovered a decade later. DNA serves the dual role: it is *interpreted* by ribosomes (transcription and translation) to produce proteins, and it is *copied* by DNA polymerase during cell division. The description-interpretation-copying architecture is identical.

Formally, von Neumann embedded the universal constructor in a two-dimensional cellular automaton with 29 states per cell. While the engineering details are elaborate, the logical structure is simple and general. It depends on two properties:

- **Universality.** The constructor U can build any machine describable by a tape. - **Program-data duality.** The tape $d(M)$ functions simultaneously as a program (to be interpreted) and as data (to be copied verbatim).

12.3 Program-Data Duality

Von Neumann's insight generalizes far beyond cellular automata. In any stored-program computer, programs and data occupy the same memory. A program is a bitstring; so is its input. Nothing prevents a program from reading, modifying, or generating other programs — including its own code.

Definition 12.3 (Program-Data Duality)**.** A computational system exhibits *program-data duality* if its programs are represented in the same format as its data, and programs can manipulate other programs (or themselves) as data.

This is the foundation of self-modifying computation. Some concrete manifestations:

- **Lisp.** Programs are S-expressions; data are S-expressions. The function 'eval' takes a data structure and executes it as a program. The function 'quote' takes a program fragment and treats it as data. This symmetry is not accidental — McCarthy designed Lisp to explore exactly this duality (McCarthy, 1960).

- **Gödel numbering.** Every formula φ in a formal system can be encoded as a natural number $\lceil\varphi\rceil$. The system can then reason about its own formulas by reasoning about their codes. This is the mechanism behind Gödel's incompleteness theorems: the sentence "this sentence is unprovable" is constructed by exploiting program-data duality at the level of formal arithmetic.

- **Quines.** A *quine* is a program that outputs its own source code. Its existence in any sufficiently expressive language follows from Kleene's recursion theorem (a fixed-point theorem for computable functions). A quine is the computational analogue of von Neumann's self-reproducing automaton, stripped to its essence.

Theorem 12.4 (Kleene's Recursion Theorem)**.** *For any total computable function $f : \mathbb{N} \to \mathbb{N}$, there exists an index e such that $\varphi_e = \varphi_{f(e)}$, where φ_e denotes the partial computable function with index e.*

In other words, for any computable transformation f of programs, there is a program e that behaves identically to its own transformation $f(e)$. This is the formal basis for self-reference in computation, and it directly implies the existence of quines (take f to be the function that, given a program index, produces an index for a program that prints that index's source code).

12.4 Procedural Reflection: Smith's 3-Lisp

Program-data duality gives a program access to *other* programs as data. *Procedural reflection* goes further: a program gains access to the data structures of *its own interpreter* while that interpreter is running it.

The foundational work is Brian Cantwell Smith's doctoral dissertation (Smith, 1982) and the associated paper (Smith, 1984). Smith designed **3-Lisp**, a dialect of Lisp with a precise reflective architecture.

12.4.1 12.4.1 The Reflective Tower

In 3-Lisp, every program is executed by an interpreter. That interpreter is itself a 3-Lisp program, which is executed by another interpreter, and so on. This produces a conceptually infinite *reflective tower*:

$$\begin{aligned} \text{Level } 0: &\quad \text{the user's program } P_0 \\ \text{Level } 1: &\quad \text{the interpreter } I_1 \text{ running } P_0 \\ \text{Level } 2: &\quad \text{the interpreter } I_2 \text{ running } I_1 \\ &\vdots \\ \text{Level } k: &\quad \text{the interpreter } I_k \text{ running } I_{k-1} \\ &\vdots \end{aligned}$$

Each interpreter I_k is a *meta-circular* interpreter: it is written in the same language (3-Lisp) that it interprets. The tower is homogeneous — every level has the same structure.

Definition 12.5 (Reflective Tower)**.** A *reflective tower* is a countable sequence $(I_k)_{k\geq 1}$ of interpreters such that:

1. Each I_k is written in the same language $\mathcal{L}$. 2. Each I_k interprets the program at level $k-1$. 3. Programs at level $k-1$ can, via designated reflective operations, access and modify the state of I_k.

12.4.2 12.4.2 Reification and Reflection

Two operations connect adjacent levels of the tower.

Definition 12.6 (Reification)**.** *Reification* takes an implicit part of the interpreter's state — such as the current continuation, the environment, or the pending expression — and makes it available as an explicit, first-class data structure at the level below.

Definition 12.7 (Reflection)**.** *Reflection* (also called *absorption*) takes a data structure at level k and installs it as part of the running state of the interpreter at level $k+1$. It is the inverse of reification: data is "absorbed" back into the interpreter's implicit machinery.

In 3-Lisp, the mechanism for ascending the tower is the **reflective lambda**, written $\lambda\uparrow$. When a reflective lambda is invoked at level k, its body executes at level $k+1$, with access to the continuation and environment of level k's interpreter. Concretely:

Example 12.8 (Reflective operation in 3-Lisp)**.** Suppose a user program at level 0 defines:

```
(define current-continuation
  (lambda-reflect (env cont)
    ;; This body runs at level 1 (inside the interpreter).
    ;; 'env' is the reified environment of the call site.
    ;; 'cont' is the reified continuation of the call site.
    (cont cont)))  ;; Return the continuation as a value.
```

When '(current-continuation)' is called at level 0, control shifts to level 1. The interpreter's current environment and continuation are *reified* as the arguments 'env' and 'cont'. The body '(cont cont)' then passes the continuation *as data* back down to level 0. The continuation — normally an implicit, invisible part of the interpreter — has been made into a first-class value that the user program can inspect, store, or invoke.

This is analogous to 'call/cc' (call-with-current-continuation) in Scheme, but more general: in a reflective tower, the program can access *any* aspect of the interpreter's state, not just the continuation.

12.4.3 12.4.3 Formal Sketch

Let $\mathcal{S}_k$ denote the state of the interpreter at level k. Typically, $\mathcal{S}_k$ consists of a triple $(\text{expr}_k, \text{env}_k, \text{cont}_k)$: the expression being evaluated, the environment binding variables to values, and the continuation representing "what to do next."

The transition at level k is governed by the interpreter at level $k+1$:

$$\mathcal{S}_k^{(t+1)} = I_{k+1}(\mathcal{S}_k^{(t)})$$

A reflective operation at level k modifies $\mathcal{S}_{k+1}$ — the state of the interpreter one level up. Let R be a reflective operation. Then:

$$R : \mathcal{S}_{k+1} \to \mathcal{S}'_{k+1}$$

After R executes, the interpreter at level $k+1$ runs with modified state $\mathcal{S}'_{k+1}$. Since I_{k+1} governs how level k executes, modifying $\mathcal{S}_{k+1}$ changes the rules by which level k operates. This is genuine self-modification: the dynamics itself changes, not just the data.

12.5 The Implementation Problem: Lazy Tower Construction

An immediate objection arises: one cannot actually run an infinite tower of interpreters. Each level requires computational resources, and there are infinitely many levels.

Des Rivières and Smith (1984) resolved this with **lazy instantiation**.

Definition 12.9 (Lazy Reflective Tower)**.** A *lazy reflective tower* materializes interpreter levels on demand. At any given moment, only finitely many levels are active. When a reflective operation at level k requires access to level $k+1$, and level $k+1$ has not yet been instantiated, the system creates it at that point.

The key observation: in practice, most computations never perform reflective operations. A program that does no reflection executes entirely at level 0, and the interpreters at levels 1, 2, 3, ... remain dormant. A program that reflects once activates level 1 but no higher. Only pathological programs (e.g., ones that recursively reflect upward) would require many levels.

Proposition 12.10. *If a computation performs at most k nested reflective operations, then at most $k+1$ levels of the tower are instantiated during execution.*

This is sufficient for all practical purposes. The infinite tower is a conceptual device — it provides clean semantics — but the implementation is always finite.

12.6 Denotational Semantics: Wand and Friedman

The reflective tower, defined operationally as an infinite stack of running interpreters, raises a semantic question: can we give a *non-circular* mathematical meaning to the tower? The interpreters are defined in terms of each other — what anchors the semantics?

Wand and Friedman (1986, 1988) answered this by showing that the tower's semantics is a **fixed point** of an operator on interpretation functions.

Definition 12.11 (Interpretation Operator)**.** Let $\mathcal{D}$ be a suitable semantic domain. Define the operator $\Phi : (\mathcal{D} \to \mathcal{D}) \to (\mathcal{D} \to \mathcal{D})$ as follows: $\Phi(I)$ is the interpretation function that results from running one level of the meta-circular interpreter, using I as the semantics of the level above.

Theorem 12.12 (Wand & Friedman, 1986)**.** *Under appropriate conditions on the semantic domain, Φ has a unique fixed point $I^* = \Phi(I^*)$. The semantics of the reflective tower is this fixed point.*

The intuition: the fixed-point condition $I^* = \Phi(I^*)$ says that interpreting a program using I^* as the meta-level semantics gives back I^* itself. The tower is self-consistent because every level has the same behavior, and the semantics of "one more level of interpretation" is the same as the semantics of the whole tower.

This is analogous to the denotational semantics of recursion: a recursive function $f(x) = \ldots f \ldots$ is defined as the least fixed point of an operator. Wand and Friedman apply the same technique to the tower.

Remark 12.13. The fixed-point characterization demystifies the apparent circularity. The tower is *not* circular in a vicious sense — it is a well-defined mathematical object, namely the fixed point of Φ. The "infinite tower" language is a convenient description of this fixed point, not a literal requirement to run infinitely many processes.

12.7 Collapsing Towers: Amin and Rompf

Running even a few levels of the reflective tower incurs interpretive overhead: each level of interpretation slows execution. Amin and Rompf (2018) showed how to eliminate this overhead entirely using **partial evaluation** (also known as *staging*).

Definition 12.14 (Partial Evaluation / Staging)**.** Given a program $P(s, d)$ with a *static* input s (known at compile time) and a *dynamic* input d (known only at run time), a *partial evaluator* produces a specialized program $P_s(d) = P(s, d)$ that has "pre-computed" everything that depends only on s.

The key idea of Amin and Rompf: the meta-circular interpreter at each level of the tower is a program whose *static* input is the program it interprets. By partially evaluating the interpreter with respect to the program, one eliminates the interpretive overhead of that level. Applying this to every level of the tower *collapses* the entire stack into a single compiled program.

Definition 12.15 (Stage Polymorphism)**.** A *stage-polymorphic* evaluator is a single piece of code that can operate in two modes:

1. **Interpretation mode.** Given a program and an input, it directly executes the program on the input. 2. **Code generation mode.** Given a program, it produces compiled code that, when run on an input, produces the same result.

The two modes share the same source code; only a staging parameter differs.

Theorem 12.16 (Amin & Rompf, 2018, informal)**.** *An N-level reflective tower, where each level is a meta-circular interpreter, can be collapsed by iterated partial evaluation into a single pass that runs with no interpretive overhead. The resulting code is equivalent to a compiler that has been derived from the tower of interpreters.*

Example 12.17 (Two-level collapse)**.** Consider a two-level tower: a user program P interpreted by a meta-circular evaluator eval, which is itself interpreted by another copy of eval.

- *Without staging:* Run $\text{eval}(\text{eval}, \langle P, \text{input}\rangle)$. Two layers of dispatch, two layers of environment lookup, two layers of overhead. - *With staging:* Partially evaluate eval with respect to P, producing compiled code P_c. Then partially evaluate eval with respect to eval (using P_c as the program), producing a single compiled pass. The two levels collapse into one.

This connects to the **Futamura projections** (Futamura, 1971):

- **First projection.** Partially evaluating an interpreter with respect to a program yields a compiled program: $\text{PE}(\text{eval}, P) = P_c$. - **Second projection.** Partially evaluating a partial evaluator with respect to an interpreter yields a compiler: $\text{PE}(\text{PE}, \text{eval}) = \text{compiler}$. - **Third projection.** Partially evaluating a partial evaluator with respect to itself yields a compiler generator: $\text{PE}(\text{PE}, \text{PE}) = \text{cogen}$.

Amin and Rompf generalize this to arbitrary tower depth.

12.8 State-Update versus Function-Update: Kampis

Most systems described as "self-modifying" do not actually modify their own dynamics. George Kampis (1991) introduced a precise distinction.

Definition 12.18 (State-Update System)**.** A system (X, f) is a *state-update system* if the transition function f is fixed and only the state $x \in X$ changes over time:

$$x_{t+1} = f(x_t)$$

Definition 12.19 (Function-Update System)**.** A system is a *function-update system* if the transition function itself changes over time:

$$x_{t+1} = f_t(x_t), \quad f_{t+1} = \varphi(f_t, x_t)$$

where φ is the meta-rule governing how f evolves.

Definition 12.20 (Genuine Self-Modification, Kampis)**.** A system exhibits *genuine self-modification* if the meta-rule φ itself is subject to change — i.e., the "reading frame" or interpretive context shifts. This requires not just function-update but *function-update of the function-update rule.*

Kampis's central argument: most systems that appear to "self-modify" are actually state-update systems operating within a fixed, higher-level dynamics. A cellular automaton with a complex rule is state-update (the rule is fixed). A genetic algorithm that evolves programs is function-update if we view the programs as transition functions, but the genetic algorithm *itself* (selection, crossover, mutation) is fixed — so at the meta-level, it is again state-update.

Example 12.21 (Pseudo vs. genuine self-modification)**.** *(a) Pseudo self-modification.* Consider a computer virus that modifies its own code to evade detection. The virus code is the "function" and it changes (function-update). But the modification strategy (the polymorphic engine) is a fixed algorithm. At the meta-level, the system is state-update: the polymorphic engine is a fixed φ.

(b) Genuine self-modification. Consider a system where the modification strategy itself evolves based on experience. The meta-rule φ at time t is φ_t, and there is a meta-meta-rule ψ such that $\varphi_{t+1} = \psi(\varphi_t, f_t, x_t)$. If ψ is also subject to change, we have the beginning of a reflective tower.

Kampis argues that genuine self-modification requires a change in the system's *reading frame*: the way the system interprets its own description must itself be part of the dynamics. This connects directly to the reflective tower, where a reflective operation changes the interpreter (the reading frame) rather than just the program (the data being read).

12.9 Connection to the (f, x) Framework

We now make the connection to the discrete dynamical systems framework developed in earlier chapters precise.

12.9.1 12.9.1 The Tower as a Stack of (f_i, x_i) Pairs

Recall the basic (f, x) framework: the state is a pair $(f, x) \in F \times X$, and the transition is

$$(f, x) \longmapsto (\varphi(f, x),\ f(x))$$

where $\varphi : F \times X \to F$ is the meta-rule.

The reflective tower generalizes this to a *stack* of such pairs. Define:

- **Level 0.** State: (f_0, x_0). Here f_0 is the "user program" and x_0 is the data. The transition at level 0 is $x_0 \mapsto f_0(x_0)$.
- **Level 1.** State: (f_1, x_1) where $x_1 = (f_0, x_0)$ is the state of level 0, and f_1 is the interpreter that governs how level 0 evolves. The transition at level 1 is $x_1 \mapsto f_1(x_1)$, which computes one step of level 0.
- **Level k.** State: (f_k, x_k) where $x_k = (f_{k-1}, x_{k-1})$ and f_k is the interpreter at level k.

Schematically:

$$\begin{aligned} \text{Level 0:} \quad & \&(f_0, x_0) \\ \text{Level 1:} \quad & \&(f_1, (f_0, x_0)) \\ \text{Level 2:} \quad & \&(f_2, (f_1, (f_0, x_0))) \\ & \vdots \end{aligned}$$

Each f_k plays the role of both a transition function (it governs the dynamics at level $k-1$) and data (it is part of the state seen by level $k+1$).

12.9.2 12.9.2 Reflective Operations as Meta-Rule Applications

In the (f, x) framework, a reflective operation at level k is an application of the meta-rule φ_k that modifies f_k. Specifically:

- **Ordinary computation** at level k: $x_k \mapsto f_k(x_k)$ while f_k remains unchanged. This is state-update.
- **Reflective operation** at level k: $f_k \mapsto \varphi_k(f_k, x_k)$. This is function-update. The dynamics at level $k-1$ changes because f_k — which governs level $k-1$ — has been modified.

Reification corresponds to the passage from (f_k, x_k) to explicit data: the function f_k and state x_k become visible as components of x_{k+1}. Reflection corresponds to modifying f_k by writing new data into the function component of level k's state.

12.9.3 12.9.3 The Tower Arises from Self-Applicable Meta-Rules

The full tower structure arises when the meta-rule φ is itself an element of the function space and is subject to modification. Define the hierarchy:

$$\begin{aligned} & \varphi_0 : F_0 \times X \to F_0\& \quad \&(\text{meta-rule: updates } f_0) \\ & \varphi_1 : F_1 \times (F_0 \times X) \to F_1\& \quad \&(\text{meta-meta-rule: updates } \varphi_0) \\ & \varphi_2 : F_2 \times (F_1 \times F_0 \times X) \to F_2\& \quad \&(\text{meta-meta-meta-rule}) \\ & \quad \vdots \end{aligned}$$

where F_k is the space of possible functions at level k. In the reflective tower, the homogeneity condition (all levels use the same meta-circular interpreter) means $\varphi_k = \varphi$ for all k, and the tower's semantics is the fixed point described in Section 12.6.

12.9.4 12.9.4 Worked Example

Consider a minimal tower over $X = \{0, 1\}$ with two levels.

- **Level 0.** $f_0 : \{0, 1\} \to \{0, 1\}$, $x_0 \in \{0, 1\}$. There are $2^2 = 4$ possible functions f_0, so the state space at level 0 is $F_0 \times X = \{0, 1\}^{\{0,1\}} \times \{0, 1\}$, with 8 states.

- **Level 1.** $f_1 : F_0 \times X \to F_0 \times X$ is the "interpreter" that maps one state of level 0 to the next. Under ordinary (non-reflective) computation, f_1 acts as $f_1(f_0, x_0) = (f_0, f_0(x_0))$ — it applies f_0 to x_0 and leaves f_0 unchanged.

A reflective operation at level 0 would modify f_1, changing how level 0's dynamics is computed. For instance, a reflective operation might install a new interpreter f_1' that applies f_0 twice per step:

$$f_1'(f_0, x_0) = (f_0, f_0(f_0(x_0)))$$

After this reflective operation, level 0 effectively runs at "double speed." The function f_1 has changed — this is function-update at level 1, triggered by a reflective operation originating at level 0.

In the (f, x) notation, we have a meta-rule φ_1 at level 1 such that:

$$\varphi_1(f_1, (f_0, x_0)) = \begin{cases} f_1 \& \text{if no reflective operation occurs,} \\ f_1' \& \text{if level 0 triggers a reflective operation.} \end{cases}$$

This is precisely the structure described abstractly in Section 12.4.3.

12.10 Self-Modifying Code in Practice

The theoretical framework of reflective towers manifests in several practical settings where a system modifies its own computational behavior.

12.10.1 12.10.1 JIT Compilation

A just-in-time (JIT) compiler is a two-level reflective tower. At level 0, a program P runs on data x. At level 1, the JIT compiler monitors P's execution and, when it identifies "hot" code paths, replaces interpreted code with compiled machine code. This is a reflective operation: the interpreter f_1 modifies itself (replacing interpretation with direct execution for certain functions). In (f, x) terms:

$$(f_1, (f_0, x_0)) \longmapsto (f_1', (f_0', x_0'))$$

where f_1' is the modified interpreter and f_0' is the compiled version of the hot path.

12.10.2 12.10.2 Genetic Programming

In genetic programming (GP), a population of programs $(f_0^{(1)}, f_0^{(2)}, \ldots, f_0^{(N)})$ evolves under selection and variation operators. The GP system is the meta-rule φ: it evaluates each $f_0^{(i)}$ on training data, selects the fittest, and produces modified programs via crossover and mutation. The programs $f_0^{(i)}$ change (function-update), but φ is fixed (state-update at the meta-level).

Meta-GP — where the selection and variation operators themselves evolve — would constitute genuine self-modification in Kampis's sense. This remains largely an open research direction.

12.10.3 12.10.3 Neural Architecture Search

Neural architecture search (NAS) searches over the space of neural network architectures (functions f_θ parameterized by both weights θ and architecture α). The search algorithm acts as a meta-rule:

$$(\alpha_{t+1}, \theta_{t+1}) = \varphi(\alpha_t, \theta_t, \text{data})$$

Standard NAS keeps φ fixed. More recent approaches learn the search strategy itself (learning to learn, or meta-learning), moving toward function-update of the meta-rule. In the (f, x) framework, this is a two-level tower where the architecture-search algorithm is the interpreter at level 1, and modifications to the search strategy are reflective operations.

12.11 Summary

The main ideas of this chapter:

1. **Von Neumann's universal constructor** resolves the self-reproduction paradox via program-data duality: a description serves as both instructions (interpreted) and data (copied).

1. **Program-data duality** is fundamental to self-modifying computation. In stored-program computers, programs *are* data. Kleene's recursion theorem guarantees the existence of self-referential programs (quines).

1. **Procedural reflection** (Smith, 1984) gives a program access to its own interpreter's state. The reflective tower is an infinite stack of interpreters, each governing the level below.

1. **Reification** makes the interpreter's implicit state explicit; **reflection** installs data back as interpreter state. These operations connect adjacent levels of the tower.

1. **Lazy instantiation** (des Rivières & Smith, 1984) resolves the implementation problem: tower levels are created only when needed.

1. **Denotational semantics** (Wand & Friedman, 1986): the tower's meaning is a fixed point $I^* = \Phi(I^*)$ of an operator on interpretation functions, resolving the apparent circularity.

1. **Collapsing towers** (Amin & Rompf, 2018): partial evaluation eliminates interpretive overhead, collapsing N levels into a single compiled pass via the Futamura projections.

1. **Kampis's distinction**: state-update (fixed dynamics, changing state) versus function-update (changing dynamics). Genuine self-modification requires the latter, and in its fullest form, requires the meta-rule itself to change.

1. **Connection to** (f, x): the reflective tower is a stack of (f_k, x_k) pairs where f_k interprets level $k-1$. A reflective operation is a meta-rule application $f_k \mapsto \varphi_k(f_k, x_k)$. The tower arises when φ_k is itself subject to modification. The tower's semantics as a fixed point mirrors the fixed-point semantics of recursive computation.

12.12 References

- Amin, N. and Rompf, T. (2018). "Collapsing Towers of Interpreters." *Proceedings of the ACM on Programming Languages*, 2(POPL), Article 52. Demonstrates how staged partial evaluation collapses a reflective tower into a single compiled pass, via stage polymorphism.

- des Rivières, J. and Smith, B. C. (1984). "The Implementation of Procedurally Reflective Languages." *Conference Record of the 1984 ACM Symposium on Lisp and Functional Programming*, pp. 331–347. Introduces lazy instantiation of the reflective tower: meta-levels are created on demand.

- Futamura, Y. (1971). "Partial Evaluation of Computation Process — An Approach to a Compiler-Compiler." *Systems, Computers, Controls*, 2(5), pp. 45–50. The three Futamura projections relating partial evaluation, compilation, and compiler generation.

- Kampis, G. (1991). *Self-Modifying Systems in Biology and Cognitive Science: A New Framework for Dynamics, Information, and Complexity.* Pergamon Press. Distinguishes state-update from function-update systems and argues that genuine self-modification requires a change in the reading frame.

- McCarthy, J. (1960). "Recursive Functions of Symbolic Expressions and Their Computation by Machine, Part I." *Communications of the ACM*, 3(4), pp. 184–195. The original paper on Lisp, establishing program-data duality via S-expressions.

- Smith, B. C. (1982). *Procedural Reflection in Programming Languages.* Ph.D. dissertation, Massachusetts Institute of Technology. The foundational work on reflective towers and procedural reflection.

- Smith, B. C. (1984). "Reflection and Semantics in Lisp." *Conference Record of the Eleventh Annual ACM Symposium on Principles of Programming Languages*, pp. 23–35. Presents 3-Lisp and the theory of procedural reflection in a more concise form.

- von Neumann, J. (1966). *Theory of Self-Reproducing Automata.* Edited and completed by A. W. Burks. University of Illinois Press. Posthumous publication of von Neumann's lectures on self-reproducing cellular automata and the universal constructor.

- Wand, M. and Friedman, D. P. (1986). "The Mystery of the Tower Revealed: A Non-Reflective Description of the Reflective Tower." *Conference Record of the 1986 ACM Conference on Lisp and Functional Programming*, pp. 298–307. Gives denotational (fixed-point) semantics to the reflective tower, showing that the tower's meaning can be characterized without circularity.

- Wand, M. and Friedman, D. P. (1988). "The Mystery of the Tower Revealed: A Non-Reflective Description of the Reflective Tower." In *Meta-Level Architectures and Reflection*, edited by P. Maes and D. Nardi, pp. 111–134. Elsevier. Extended version of the 1986 paper.

12.13 Recommended Reading

For the foundations:

- **Smith (1984)** is the essential source for procedural reflection. Dense but rewarding; introduces the key concepts of reification, reflection, and the tower architecture.

- **Wand & Friedman (1988)** provides the semantic foundation, showing that the tower has a well-defined fixed-point meaning. More accessible than Smith's original work.

For von Neumann's ideas:

- **von Neumann (1966)**, edited by Burks, is the primary source. Part I covers the logical theory of automata; Part II covers the self-reproducing universal constructor.

- **Sipper, M.** (1998). *Fifty Years of Research on Self-Replication: An Overview.* Artificial Life, 4(3), 237–257. A good survey of developments since von Neumann.

For partial evaluation and tower collapsing:

- **Jones, N., Gomard, C., and Sestoft, P.** (1993). *Partial Evaluation and Automatic Program Generation.* Prentice Hall. The standard reference on partial evaluation, including the Futamura projections.

- **Amin & Rompf (2018)** is accessible and demonstrates the practical power of collapsing towers. The paper includes working Scala code.

For self-modifying systems:

- **Kampis (1991)** is philosophically rich and scientifically rigorous. Challenging but essential for understanding the difference between superficial and genuine self-modification.

For connections to programming language theory:

- **Friedman, D. and Wand, M.** (2008). *Essentials of Programming Languages*, 3rd ed. MIT Press. A textbook approach to interpreters and their towers, accessible to advanced undergraduates.

Chapter 13

AIXI and Gödel Machines

13.1 Introduction

The preceding chapters developed discrete dynamical systems on finite state spaces and explored Kolmogorov complexity as a measure of description length. We now turn to two landmark theoretical constructions that connect these ideas to the problem of *optimal sequential decision-making*: Hutter's AIXI and Schmidhuber's Gödel Machine.

Both constructions grapple with the same fundamental question: what does it mean for an agent to act optimally in an unknown environment? AIXI answers this by maintaining a Bayesian mixture over all computable environments, weighted by a universal prior. The Gödel Machine answers it by searching for provably beneficial self-modifications to its own code. Both are incomputable in their full generality, yet both illuminate the structure of the problem in ways that computable approximations cannot.

The thread connecting them to our framework is direct. In the notation of Chapter 1, a discrete dynamical system is a pair (X, f) where $f : X \to X$. When we allow the map itself to change—$(f, x) \mapsto (\phi(f, x),\ f(x))$—we enter the territory of self-modifying systems. AIXI reasons *about* an unknown f without modifying itself. Gödel Machines modify their own f subject to a proof constraint. The general $(f, x) \mapsto (\phi(f, x), f(x))$ framework studied in this text imposes no such constraint, allowing us to study the full landscape of self-modification dynamics.

We begin with the foundation on which AIXI rests: Solomonoff's theory of inductive inference.

13.2 Solomonoff Induction

13.2.1 13.2.1 The Prediction Problem

Consider the following problem. You observe a binary sequence $x_1, x_2, \ldots, x_n$ and wish to predict x_{n+1}. You do not know the process that generated the sequence. It might be a deterministic rule, a biased coin, a pseudorandom generator, or any other mechanism.

Classical statistics requires you to choose a model class (e.g., "the data are i.i.d. Bernoulli") before you can do inference. If the true source lies outside your model class, your predictions may be systematically wrong. Solomonoff induction eliminates this dependence on a pre-chosen model class by considering *all* computable hypotheses simultaneously.

13.2.2 13.2.2 The Universal Prior

Fix a prefix-free universal Turing machine U. (Prefix-free means that the set of programs on which U halts forms a prefix-free set: no halting program is a proper prefix of another. This is needed to make the prior a proper probability measure.)

Definition 13.1 (Solomonoff's Universal Prior)**.** For a finite binary string $x \in \{0,1\}^*$, the *universal a priori probability* (or *Solomonoff prior*) is

$$M(x) = \sum_{\substack{p \in \{0,1\}^* \\ U(p)=x^*}} 2^{-|p|}$$

where the sum is over all programs p such that U with input p outputs a string starting with x (written x^* to denote that x is a prefix of the output), and $|p|$ is the length of p in bits.

Equivalently, $M(x)$ is the probability that the universal machine U, fed with a sequence of fair coin flips as input, produces an output beginning with x.

Example 13.2. Consider three binary strings of length 20: - $a = 00000000000000000000$ (all zeros) - $b = 01010101010101010101$ (alternating) - $c = 10110100011010001101$ (apparently random)

Under the Solomonoff prior, $M(a) > M(b) > M(c)$ (up to constant factors). The string a can be generated by very short programs ("print 0 twenty times"), so many short programs contribute to the sum. The string b requires a slightly longer program. The string c, if truly incompressible, can essentially only be produced by a program that contains it as a literal, so the shortest contributing program has length ≈ 20 bits, giving $M(c) \approx 2^{-20}$.

13.2.3 13.2.3 The Dominance Property

The crucial property of M is that it is *universal* among computable probability measures, in the following precise sense.

Theorem 13.3 (Dominance, Solomonoff 1964, Zvonkin and Levin 1970)**.** *For any computable probability measure μ on $\{0,1\}^*$, there exists a constant $c_\mu > 0$ (depending on μ but not on x) such that for all $x \in \{0,1\}^*$:*

$$M(x) \geq c_\mu \cdot \mu(x).$$

Proof sketch. Since μ is computable, there exists a program p_μ that, given random bits as input, samples from μ. This program has some fixed length $|p_\mu|$. The contribution of programs that begin with p_μ to the sum defining $M(x)$ is at least $2^{-|p_\mu|} \cdot \mu(x)$. Setting $c_\mu = 2^{-|p_\mu|}$ gives the result.

This theorem says that M assigns probability at least as large (up to a multiplicative constant) as any computable measure. It "dominates" the entire class of computable distributions. The constant $c_\mu = 2^{-|p_\mu|}$ is small for complex hypotheses and large for simple ones, which is precisely Occam's razor implemented in a mathematically rigorous way.

13.2.4 13.2.4 Convergence of Solomonoff Prediction

Given observations $x_1, \ldots, x_n$ generated by some computable measure μ, the Solomonoff predictor assigns

$$M(x_{n+1} \mid x_1 \ldots x_n) = \frac{M(x_1 \ldots x_n x_{n+1})}{M(x_1 \ldots x_n)}.$$

Theorem 13.4 (Solomonoff's Convergence Theorem, Solomonoff 1978)**.** *If the data are generated by a computable measure μ, then the expected total KL divergence between μ's predictions and M's predictions is bounded:*

$$\sum_{n=1}^{\infty} \mathbb{E}_\mu \left[D_{\mathrm{KL}}(\mu(\cdot \mid x_{<n}) \,\|\, M(\cdot \mid x_{<n})) \right] \leq \ln \frac{1}{c_\mu} = |p_\mu| \cdot \ln 2.$$

In particular, $D_{\mathrm{KL}}(\mu(\cdot \mid x_{<n}) \| M(\cdot \mid x_{<n})) \to 0$ *with* μ*-probability 1.*

This is a remarkable result: the universal predictor learns any computable environment, and the total prediction error over the agent's entire lifetime is bounded by the complexity of the true environment. Simple environments are learned quickly; complex ones take longer, but all are learned eventually.

Example 13.5. Suppose the true environment produces the sequence $0, 1, 0, 1, 0, 1, \ldots$ deterministically. After observing just a few alternations, the Solomonoff predictor assigns overwhelming probability to programs that generate the alternating pattern (these are short and thus heavily weighted). Programs generating $0, 1, 0, 1, 0, 1, 0, 0, \ldots$ (which agree on the first n bits but then diverge) are also present in the mixture, but their collective weight shrinks as more confirming data arrives. The predictor rapidly converges to the correct rule.

13.2.5 13.2.5 Incomputability

Proposition 13.6. $M(x)$ *is not computable.*

Proof. Computing $M(x)$ requires summing $2^{-|p|}$ over all programs p that produce output beginning with x. Determining whether $U(p)$ halts and produces output starting with x is at least as hard as the halting problem.

However, $M(x)$ is *enumerable from above* (upper semi-computable): we can run all programs in parallel, and as we discover more programs that output strings starting with x, our running estimate of $M(x)$ can only increase. We can approximate $M(x)$ from above to arbitrary precision, but we can never know when we have converged.

This incomputability is not a minor technical inconvenience—it is the price of universality. Any computable predictor must fail on some computable environments (this follows from diagonalization), whereas M succeeds on all of them, at the cost of being incomputable.

□

13.3 AIXI

13.3.1 13.3.1 The Agent-Environment Framework

We now extend from passive prediction to *active* sequential decision-making. Consider an agent interacting with an environment over discrete time steps $t = 1, 2, 3, \ldots, m$ (where m may be finite or a horizon parameter):

1. At time t, the agent selects an action $a_t \in \mathcal{A}$. 2. The environment responds with an observation $o_t \in \mathcal{O}$ and a reward $r_t \in \mathcal{R} \subset [0, 1]$. 3. Both agent and environment may depend on the entire interaction history.

The environment is modeled as a computable function (program) q that maps action histories to observation-reward pairs:

$$q : (a_1 o_1 r_1 \ldots a_{t-1} o_{t-1} r_{t-1} a_t) \mapsto (o_t, r_t).$$

The agent's goal is to maximize cumulative reward $\sum_{t=1}^{m} r_t$.

13.3.2 13.3.2 The AIXI Agent

Definition 13.7 (AIXI, Hutter 2000)**.** The AIXI agent selects actions by maximizing expected future reward, where the expectation is taken over all computable environments weighted by the universal prior:

$$a_t^* = \arg\max_{a_t} \sum_{o_t, r_t} \ldots \max_{a_m} \sum_{o_m, r_m} \left[\sum_{k=t}^{m} r_k \right] \sum_{\substack{q \text{ consistent} \\ \text{with history}}} 2^{-|q|}$$

where "consistent with history" means $q(a_1 o_1 r_1 \ldots a_{k-1} o_{k-1} r_{k-1} a_k) = (o_k, r_k)$ for all $k < t$. Let us unpack this formula. Reading from left to right:

- $\arg\max_{a_t}$: choose the best current action.
- $\sum_{o_t, r_t}$: sum over all possible immediate responses.
- $\ldots \max_{a_m} \sum_{o_m, r_m}$: for all future time steps, maximize over actions and sum over environment responses. The alternation of max and $\sum$ reflects that future actions are chosen optimally while future observations are uncertain.
- $\sum_{k=t}^{m} r_k$: the total future reward.
- $\sum_q 2^{-|q|}$: the posterior weight on environment q, which is proportional to $2^{-|q|}$ restricted to programs consistent with what has been observed.

AIXI is simultaneously performing **Bayesian learning** (maintaining the posterior over environments via the Solomonoff prior) and **optimal planning** (choosing the reward-maximizing action given this posterior).

13.3.3 13.3.3 A Concrete Scenario

Example 13.8 (Cheese-in-a-maze)**.** An agent is placed in a maze. At each step, it chooses a direction (north, south, east, west). The environment returns the agent's new position (observation) and a reward of 1 if the agent reaches a cheese and 0 otherwise. The maze layout is fixed but unknown to the agent.

AIXI's posterior initially assigns weight to all computable maze-generating programs. Simple mazes (small, few walls) correspond to short programs and thus receive higher prior weight. As the agent explores and observes walls and open spaces, most maze programs become inconsistent with the data and are eliminated from the posterior. After sufficient exploration, the posterior concentrates on programs generating mazes similar to the true one. The agent then exploits this knowledge to navigate efficiently to the cheese.

The key insight is that AIXI's exploration is *automatic*—it follows from the Bayesian framework. Actions that yield informative observations help distinguish between environments, which helps maximize *future* reward. AIXI does not need a separate exploration strategy bolted on; exploration emerges from the expected reward maximization over the full horizon.

13.3.4 13.3.4 Optimality

Theorem 13.9 (Pareto Optimality, Hutter 2002)**.** *AIXI is Pareto-optimal over the class of all computable environments: there is no other policy π that achieves higher expected reward than AIXI in some computable environment without achieving lower expected reward in another.*

This is the strongest optimality statement one can make without assuming which environment the agent is in. It says AIXI cannot be uniformly improved upon.

Theorem 13.10 (Self-optimizing, Hutter 2002)**.** *In any computable environment μ, the per-step expected reward of AIXI converges to the optimal value achievable by a policy that knows μ.*

In other words, AIXI eventually performs as well as an agent that knows the true environment. The "eventually" may take arbitrarily long for complex environments, but convergence is guaranteed.

13.3.5 13.3.5 Limitations

AIXI is a mathematical ideal, not a practical algorithm. Its limitations are severe and illuminating:

1. **Incomputability.** AIXI requires computing the Solomonoff prior and solving an infinite-depth expectimax search. Both are incomputable.

1. **No self-reference.** AIXI models the environment as a program q that is separate from the agent. It does not model itself as part of the environment. If the environment can inspect the agent's code or predict the agent's actions (as in game-theoretic settings), AIXI's model is incomplete. This is the **embedded agency** problem: real agents are *within* their environments, not separate from them.

1. **The grain of truth assumption.** AIXI assumes the true environment is computable. If the environment involves genuinely uncomputable elements (e.g., depends on a halting oracle), the Solomonoff prior assigns it zero weight.

1. **Lack of self-modification.** AIXI's policy is defined by a fixed formula. It does not modify its own source code or architecture. It treats the problem as "fixed agent, unknown environment."

13.3.6 13.3.6 AIXItl: A Computable Approximation

Definition 13.11 (AIXItl, Hutter 2002)**.** Fix resource bounds: a maximum program length l and a maximum computation time t per step. The **AIXItl** agent is identical to AIXI except that the sum over programs is restricted to those of length $\leq l$ that run in time $\leq t$.

AIXItl is computable (since we only consider finitely many programs, each run for bounded time). It is not optimal in the same sense as AIXI, but it satisfies a different guarantee:

Theorem 13.12. *AIXItl is as good as any other agent of computational resources (t, l). For any computable policy π of description length $\leq l$ running in time $\leq t$, AIXItl achieves expected reward at least as high (up to terms that vanish with the horizon).*

This makes AIXItl the best possible use of bounded computational resources, in a well-defined sense. The practical difficulty is that the constants are astronomical: even modest values of l yield an intractably large set of programs to consider.

13.4 Gödel Machines

13.4.1 13.4.1 Motivation

AIXI is a fixed agent reasoning about an unknown environment. But what about an agent that can improve *itself*? A programmer who writes better versions of their own code? This is the problem of **self-modification**, and it leads directly to the Gödel Machine.

The key observation motivating the Gödel Machine is simple: if an agent can prove that a modification to its own code would increase expected future reward, then it should perform that modification. The challenge is making this idea rigorous.

13.4.2 13.4.2 Structure

Definition 13.13 (Gödel Machine, Schmidhuber 2003)**.** A Gödel Machine is a computational system consisting of:

1. **A program** f, stored in memory and initially containing a proof searcher s. The program f includes instructions for interacting with the environment and for searching for self-improvements.

2. **An axiom system** $\mathcal{A}$, formalized in a language powerful enough to describe: - The machine's own code and state. - The environment's properties (to whatever extent they are known). - Arithmetic and the theory of computation.

3. **A utility function** u, specifying the agent's objective (e.g., expected cumulative reward).

The machine operates as follows at each step:

- Execute the current instruction of f (which may involve interacting with the environment, performing computations, or searching for proofs).
- The proof searcher s systematically enumerates all possible proofs in $\mathcal{A}$.
- If s finds a proof that replacing the current program f with some f' would yield strictly higher expected utility than continuing with f (including continuing to search for better modifications), then **perform the rewrite**: replace f with f'.

13.4.3 13.4.3 The Self-Referential Proof Search

The proof searcher is the heart of the machine. It enumerates candidate theorems of the form:

> "Switching from the current policy f to policy f' at the current time, given the current state, yields higher expected utility than continuing with f (which includes continuing the proof search)."

Formally, let state(t) be the complete state of the machine at time t, and let $u(f, \text{state}(t))$ be the expected utility of running policy f from state state(t) onward. The proof searcher looks for a theorem of the form:

$$\mathcal{A} \vdash u(f', \text{state}(t)) > u(f, \text{state}(t))$$

where f' is some candidate replacement program. If such a proof is found, the machine switches to f'.

The remarkable feature is that f' is unrestricted: it can be *any* program. In particular, f' can contain a different proof searcher, a different utility function interpretation, a different environment model—anything. The only constraint is that the improvement must be *proved*, not merely conjectured.

13.4.4 13.4.4 Key Properties

Property 1: Global optimality guarantee. If a beneficial rewrite exists and is provable within $\mathcal{A}$, the proof searcher will eventually find it (since it enumerates all proofs). The machine will not get stuck in a local optimum if the global optimum is reachable via a provable improvement.

Property 2: Soundness of self-modification. Every self-modification is backed by a formal proof. The machine never makes a change that it cannot justify. This prevents the pathological self-modifications that plague naive self-modifying systems (e.g., a program that "optimizes" itself by deleting all safety checks).

Property 3: Self-referential stability. The machine can rewrite the proof searcher itself. If there is a provably faster search strategy, it will be found and adopted. This makes the Gödel Machine optimally self-reflective in a precise sense: the proof searcher has no bias toward preserving itself.

Property 4: Gödel's incompleteness applies. By Gödel's first incompleteness theorem, any consistent axiom system $\mathcal{A}$ that is powerful enough to encode arithmetic contains true but

unprovable statements. This means there may exist beneficial self-rewrites that the machine can never prove are beneficial. The Gödel Machine is not omniscient—it is limited to *provably* beneficial modifications.

Example 13.14 (Proof-driven self-improvement)**.** Suppose a Gödel Machine starts with a naive sorting algorithm (bubble sort, $O(n^2)$) as a subroutine used in its environment interactions. The axiom system includes the theory of arithmetic and facts about algorithm analysis. The proof searcher eventually constructs a proof that replacing bubble sort with merge sort ($O(n \log n)$) will decrease computation time for all inputs of size $\geq n_0$, thereby increasing expected utility (since faster computation means more time for other useful activities within a fixed time budget). Upon finding this proof, the machine rewrites its sorting subroutine.

Note that the machine does not merely test that merge sort is faster empirically. It *proves* it, within the formal system, using the axioms of arithmetic and the definition of the algorithms. This proof may take enormous time to find, but once found, the improvement is guaranteed to be correct.

13.4.5 13.4.5 Comparison with AIXI

Property	AIXI	Gödel Machine
Self-modification	No	Yes
Handles unknown environments	Yes (Bayesian)	Partially (via axioms)
Optimality guarantee	Pareto-optimal over computable envs	Optimal among provably good pol
Computability	Incomputable	Incomputable (in general)
Limitation principle	Halting problem	Gödel's incompleteness

The two approaches address complementary aspects of the optimal agent problem. AIXI solves the learning problem (how to act in an unknown environment) but ignores self-modification. The Gödel Machine solves the self-modification problem (how to improve one's own code) but requires an axiom system that adequately describes the environment.

13.5 OOPS: Optimal Ordered Problem Solver

13.5.1 13.5.1 Motivation

Schmidhuber (2004) introduced the **Optimal Ordered Problem Solver (OOPS)** as a more concrete (though still idealized) approach to incremental self-improvement. The key idea is to reuse solutions to previously solved problems when solving new ones.

13.5.2 13.5.2 Setup

OOPS faces a sequence of problems $P_1, P_2, P_3, \ldots$ and maintains a growing *code store* C. For each problem P_k:

1. OOPS performs **Levin search** (universal search, Levin 1973): enumerate all programs p by increasing length, running each for time proportional to $2^{-|p|}$ times the total available time.

2. Crucially, programs in the search can *call* previously found solutions as subroutines. If P_3 is similar to P_1, the solution to P_1 (already in the code store) can be invoked as a building block.

3. When a solution is found, it is added to C.

13.5.3 13.5.3 Bias-Optimality

Definition 13.15 (Bias-Optimality)**.** A search method is *bias-optimal* if no other method with the same set of initial assumptions (prior over programs) can solve the given problem more than a constant factor faster.

Theorem 13.16 (Schmidhuber 2004)**.** *OOPS is bias-optimal for its given problem sequence.*

This means OOPS is provably the fastest way (up to a constant) to solve a sequence of problems given Levin's universal prior, with the added benefit of reusing prior solutions.

13.5.4 13.5.4 OOPS as a Dynamical System

OOPS naturally generates a trajectory of increasingly capable solvers:

$$C_0, \quad C_1 = C_0 \cup \{s_1\}, \quad C_2 = C_1 \cup \{s_2\}, \quad \ldots$$

where s_k is the solution to problem P_k. Each new solution potentially makes future searches faster (by providing callable subroutines). This is a dynamical system on the space of code stores, where the transition rule is "solve the next problem using all available code, then add the solution."

In our notation: $f_k = C_k$ is the agent's "program" at stage k, and $x_k = P_k$ is the current problem. The update is $f_{k+1} = \phi(f_k, x_k) = f_k \cup \{\text{solve}(x_k, f_k)\}$. This is a concrete instance of the self-modifying dynamical system $(f, x) \mapsto (\phi(f, x), g(x))$ where the map itself grows monotonically.

13.6 Synthesis: Self-Modifying Dynamical Systems

We now step back and connect these constructions to the framework developed in previous chapters.

13.6.1 13.6.1 Three Approaches to $(f, x) \mapsto (f', x')$

Consider the general update rule where both the state and the map can change:

$$(f_t, x_t) \mapsto (f_{t+1}, x_{t+1}) = (\phi(f_t, x_t),\ f_t(x_t)).$$

The three constructions studied in this chapter correspond to three different choices of the meta-rule ϕ:

AIXI: $\phi(f, x) = f$ (no self-modification). The map f is the fixed AIXI policy. The state x encodes the interaction history. AIXI updates its *beliefs* about the environment, but this is a change in x (data), not in f (policy). The agent-environment system evolves as $(f, x) \mapsto (f, f(x))$—a standard iterated map with a fixed function.

Gödel Machine: $\phi(f, x) = f'$ where f' is the result of a provably-beneficial rewrite if one is found, or f otherwise. The meta-rule ϕ is constrained:

$$\phi(f, x) = \begin{cases} f' \& \text{if } \mathcal{A} \vdash u(f', x) > u(f, x) \text{ is found at this step} \\ f \& \text{otherwise.} \end{cases}$$

This is a self-modifying dynamical system with a *conservative* transition rule: changes to f are infrequent and always beneficial (relative to the axiom system).

OOPS: $\phi(f, x) = f \cup \{\text{new solution}\}$. The meta-rule is *monotone*: the code store only grows. This guarantees that capabilities never decrease, at the cost of ever-increasing memory usage.

13.6.2 13.6.2 The Unconstrained Case

Our general framework imposes no constraints on ϕ. The meta-rule could be:

- **Arbitrary:** ϕ is any computable function. This allows studying phenomena like **model collapse** (where f degenerates) or **creative transients** (long sequences of distinct f values before settling).
- **Stochastic:** ϕ selects f' randomly from some distribution depending on (f, x).
- **Adversarial:** ϕ is chosen to make things as bad as possible (useful for studying robustness).

By not requiring provable improvement, we gain the ability to study the *dynamics* of self-modification itself—the trajectories, attractors, transients, and bifurcations that arise when programs rewrite programs.

Example 13.17 (Degenerate self-modification)**.** Let $\mathcal{F} = \{f_0, f_1, f_2\}$ be three programs and let ϕ be defined by: $\phi(f_0, x) = f_1$, $\phi(f_1, x) = f_2$, $\phi(f_2, x) = f_2$ for all x. Then regardless of the initial state, the meta-dynamics converge to the fixed point f_2 in at most two steps. If f_2 is a degenerate program (e.g., one that always outputs the same action), this represents model collapse: a self-modifying system that destroys its own capabilities. A Gödel Machine would never reach f_2 (it could not prove that switching to a degenerate program improves utility), but the unconstrained dynamics allow it.

13.6.3 13.6.3 What These Constructions Share

Despite their differences, AIXI, Gödel Machines, and OOPS share several deep structural features:

1. **Universality.** Each construction ranges over *all* programs (or all proofs, or all solutions), not a pre-specified model class. This universality is what gives the theoretical guarantees their power—and what makes the constructions incomputable.

1. **Occam weighting.** Programs are weighted by $2^{-|p|}$, favoring short descriptions. This is the algorithmic formalization of Occam's razor and is the common thread linking Kolmogorov complexity (Chapter 8), the Solomonoff prior, and AIXI.

1. **Incomputability barriers.** All three hit fundamental limits: the halting problem for Solomonoff/AIXI, Gödel's incompleteness for the Gödel Machine. These are not bugs but features—they delineate the boundary of what is achievable by any formal system.

13.7 Summary

Construction	Year	Core idea	Optimality	
Solomonoff prior	1964	Universal mixture over programs	Dominates all computable measures	
AIXI	2000	Bayesian RL with Solomonoff prior	Pareto-optimal	In
AIXItl	2002	AIXI with bounded resources	Optimal for given resources	
Gödel Machine	2003	Provably beneficial self-rewrites	Optimal among provable policies	
OOPS	2004	Incremental universal search with reuse	Bias-optimal	

The progression from Solomonoff induction through AIXI to Gödel Machines represents a series of increasingly ambitious attempts to formalize the notion of an optimal intelligent agent. Solomonoff solves prediction. AIXI extends this to action. The Gödel Machine extends further to self-modification. Each step broadens the scope while preserving formal guarantees.

For the study of discrete dynamical systems on function spaces, these constructions provide the conceptual endpoints: AIXI is the best possible fixed map, and the Gödel Machine is the best possible self-modifying map (within the limits of provability). The space between them—self-modification without proof constraints—is where the interesting dynamics live.

13.8 References

- Hutter, M. (2005). *Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability.* Springer.
- Hutter, M. (2000). A theory of universal artificial intelligence based on algorithmic complexity. arXiv:cs/0004001.
- Hutter, M. (2007). Universal algorithmic intelligence: A mathematical top-down approach. In *Artificial General Intelligence*, 227–290.
- Legg, S. (2008). *Machine Super Intelligence.* PhD thesis, University of Lugano.
- Levin, L. A. (1973). Universal sequential search problems. *Problems of Information Transmission*, 9(3):265–266.
- Schmidhuber, J. (2003). Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. arXiv:cs/0309048.
- Schmidhuber, J. (2004). Optimal ordered problem solver. *Machine Learning*, 54(3):211–254.
- Schmidhuber, J. (2009). Ultimate cognition à la Gödel. *Cognitive Computation*, 1(2):177–193.
- Solomonoff, R. J. (1964). A formal theory of inductive inference. *Information and Control*, 7(1):1–22 and 7(2):224–254.
- Solomonoff, R. J. (1978). Complexity-based induction systems: Comparisons and convergence theorems. *IEEE Transactions on Information Theory*, 24(4):422–432.
- Zvonkin, A. K. and Levin, L. A. (1970). The complexity of finite objects and the development of the concepts of information and randomness by means of the theory of algorithms. *Russian Mathematical Surveys*, 25(6):83–124.

13.9 Recommended Reading

For Solomonoff induction and AIXI:

- **Hutter (2005)** is the definitive monograph. The first half covers Solomonoff induction; the second half covers AIXI. Technically demanding but comprehensive.
- **Legg (2008)**, *Machine Super Intelligence*, provides a more accessible introduction to the main ideas, including a formal definition of intelligence based on AIXI.

- **Rathmanner, S. and Hutter, M.** (2011). "A philosophical treatise of universal induction." *Entropy*, 13(6):1076–1136. Covers the philosophical foundations and addresses common objections.

For Gödel Machines:

- **Schmidhuber (2003)** is the original paper and remains the best introduction. Available on arXiv as cs/0309048.

- **Schmidhuber (2009)**, "Ultimate cognition à la Gödel," provides a shorter and more accessible overview.

For OOPS and incremental self-improvement:

- **Schmidhuber (2004)**, "Optimal ordered problem solver," in *Machine Learning*. The original paper, with full technical details.

For broader context on AGI theory:

- **Goertzel, B. and Pennachin, C.** (eds.) (2007). *Artificial General Intelligence*. Springer. Collection of papers from the AGI community, including Hutter's overview of AIXI.

For embedded agency and limitations of AIXI:

- **Demski, A. and Garrabrant, S.** (2019). "Embedded Agency." MIRI Technical Report. Analyzes the assumptions underlying AIXI and identifies problems that arise when the agent is embedded within its environment.

For practical approximations:

- **Veness, J. et al.** (2011). "A Monte-Carlo AIXI Approximation." *Journal of Artificial Intelligence Research*, 40:95–142. The most successful practical approximation to AIXI, with experiments on Atari games.

Part IV

Language Models and Novelty

Chapter 14

LLMs as Dynamical Systems

14.1 Introduction

Large language models (LLMs) are, at bottom, functions: they accept a sequence of tokens and produce a probability distribution over the next token. When equipped with a decoding strategy—greedy, beam search, nucleus sampling—they become maps on the space of token sequences. When their outputs are fed back as inputs, they become discrete dynamical systems.

This chapter develops this perspective systematically. We begin by formalizing what an LLM computes as a map on symbolic sequences, connecting to the sliding block codes and subshifts of Chapters 2–3. We then study the dynamics of iterated LLM generation—feeding outputs back as inputs—and show how recent empirical findings (convergence to periodic orbits, model collapse, self-improvement) fit naturally into the framework of discrete dynamical systems developed throughout this book.

The key conceptual contribution is the **(f, x) decomposition**: the observation that a prompt simultaneously encodes a function specification f and an input x, and that iteration can mutate both. This framework unifies phenomena as diverse as model collapse, self-refinement, and multi-agent debate under a single dynamical systems umbrella.

14.2 The LLM as a Map

14.2.1 14.2.1 The Conditional Distribution

Let Σ be a finite token vocabulary with $|\Sigma| = V$ (typically $V \approx 32{,}000$ to $128{,}000$ for modern LLMs). An autoregressive language model defines, for each finite sequence $w = (x_1, x_2, \ldots, x_n) \in \Sigma^*$, a conditional probability distribution over the next token:

$$P_\theta(x_{n+1} \mid x_1, x_2, \ldots, x_n),$$

where θ denotes the model parameters. The full joint distribution over sequences factorizes autoregressively:

$$P_\theta(x_1, x_2, \ldots, x_N) = \prod_{n=1}^{N} P_\theta(x_n \mid x_1, \ldots, x_{n-1}).$$

This is a standard probability chain rule; the content of the model lies in the parametric form of P_θ.

14.2.2 14.2.2 Context Windows and Finite Memory

In practice, transformer-based LLMs have a finite **context window** of length k (typically $k = 2{,}048$ to $128{,}000$ tokens). The conditional distribution depends only on the most recent k

tokens:

$$P_\theta(x_{n+1} \mid x_1, \ldots, x_n) = P_\theta(x_{n+1} \mid x_{n-k+1}, \ldots, x_n)$$

for all $n \geq k$.

Definition 14.1 (LLM local rule)**.** An LLM with context window k and vocabulary Σ defines a function

$$p : \Sigma^k \to \Delta(\Sigma),$$

where $\Delta(\Sigma)$ is the probability simplex over Σ. The function p maps a k-tuple of tokens to a probability distribution over the next token.

This is the stochastic analogue of the block maps from Chapter 3. Specifically, if we fix a deterministic decoding strategy, we obtain a deterministic local rule.

14.2.3 14.2.3 Deterministic Decoding as a Sliding Block Code

Definition 14.2 (Greedy decoding map)**.** Given an LLM local rule $p : \Sigma^k \to \Delta(\Sigma)$, the **greedy decoding map** is the function $f : \Sigma^k \to \Sigma$ defined by

$$f(x_{n-k+1}, \ldots, x_n) = \arg\max_{s \in \Sigma} \; p(s \mid x_{n-k+1}, \ldots, x_n).$$

(Ties are broken by a fixed ordering on Σ.)

Proposition 14.3. *An LLM with context window k and greedy decoding induces a one-sided sliding block code $\Phi : \Sigma^{\mathbb{N}} \to \Sigma^{\mathbb{N}}$ with memory $k-1$ and anticipation 0.*

Proof. The output at position n depends on positions $n-k+1, \ldots, n$ of the input. This is exactly a sliding block code with memory $m = k-1$ and anticipation $a = 0$, in the sense of Definition 3.2.2. The one-sidedness reflects the causal (autoregressive) structure: the LLM cannot look ahead.

Remark 14.4. This is an unusual sliding block code in two respects. First, the local rule f is not given by a simple formula but by a neural network with billions of parameters. Second, the code is one-sided: it acts on $\Sigma^{\mathbb{N}}$ rather than $\Sigma^{\mathbb{Z}}$, reflecting the causal nature of language generation.

□

14.2.4 14.2.4 The Generated Subshift

Definition 14.5 (Generated language)**.** Given an LLM with greedy decoding map $f : \Sigma^k \to \Sigma$ and a set of seed prompts $\mathcal{P} \subseteq \Sigma^k$, the **generated language** is

$$\mathcal{L}(f, \mathcal{P}) = \{w \in \Sigma^* : w \text{ is a contiguous block of some sequence generated by iterating } f \text{ from some seed in } \mathcal{P}\}.$$

The closure of all bi-infinite sequences whose every finite subword lies in $\mathcal{L}(f, \mathcal{P})$ forms a subshift $X_f \subseteq \Sigma^{\mathbb{Z}}$.

Example 14.6 (Tiny vocabulary)**.** Let $\Sigma = \{a, b\}$, context window $k = 2$, and suppose the greedy decoding map is:

(x_1, x_2)	$f(x_1, x_2)$
(a, a)	b
(a, b)	a
(b, a)	b
(b, b)	a

Starting from the seed aa: we generate b, giving aab. The window slides to ab, generating a: $aaba$. Then $ba \to b$: $aabab$. Then $ab \to a$: $aababa$. The pattern $\ldots ababababab \ldots$ emerges: a period-2 orbit of the sliding window.

This is a subshift of finite type (an edge shift on the de Bruijn graph of order 2), and the attractor is the single periodic orbit $\overline{ab}$.

14.2.5 14.2.5 Stochastic Generation as a Measure

With temperature $\tau > 0$, the LLM generates tokens by sampling from the softmax distribution:

$$P_\tau(x_{n+1} = s \mid x_{n-k+1}, \ldots, x_n) = \frac{\exp(\ell_s/\tau)}{\sum_{s' \in \Sigma} \exp(\ell_{s'}/\tau)},$$

where ℓ_s are the logits. As $\tau \to 0$, this concentrates on the greedy argmax. As $\tau \to \infty$, it approaches the uniform distribution.

The stochastic process defined by sampling with a fixed temperature defines a **shift-invariant probability measure** μ_τ on $\Sigma^{\mathbb{N}}$, supported on a subshift that depends on τ. In the language of ergodic theory, $(\Sigma^{\mathbb{N}}, \sigma, \mu_\tau)$ is a measure-preserving system (after suitable stationarization), and the entropy rate of the process is

$$h(\mu_\tau) = -\lim_{n \to \infty} \frac{1}{n} \sum_{w \in \Sigma^n} \mu_\tau([w]) \log \mu_\tau([w]),$$

where $[w]$ is the cylinder set of sequences beginning with w. This is the Shannon entropy rate of the LLM's output process.

14.3 Iterated LLM Generation

14.3.1 14.3.1 The Iteration Scheme

We now study the core dynamical system: feeding an LLM's output back as its input.

Definition 14.7 (LLM iteration)**.** Let $\mathrm{LLM} : \Sigma^* \to \Sigma^*$ denote the map that takes a prompt string to the LLM's full response (a finite string produced by generating tokens until a stop condition). Define the **iteration**:

$$x_0 \in \Sigma^*, \qquad x_{n+1} = \mathrm{LLM}(x_n), \qquad n = 0, 1, 2, \ldots$$

This is a discrete dynamical system (S, F) where $S = \Sigma^*$ (finite strings over the token vocabulary) and $F = \mathrm{LLM}$.

Remark 14.8. There is a subtlety: Σ^* is countably infinite, not finite. The theory of Chapter 1 (iterated maps on finite sets) does not directly apply—orbits need not be eventually periodic. However, if we restrict attention to strings of bounded length (as enforced by a maximum output length), the state space becomes finite and all orbits are eventually periodic.

14.3.2 14.3.2 Convergence to Periodic Attractors

Theorem 14.9 (Wang et al., 2025)**.** *In experiments with successive paraphrasing—where an LLM is asked to rephrase its own output—the dynamics converge to* ***2-cycles*** *(period-2 attractors). That is, there exist strings $a, b \in \Sigma^*$ with $\mathrm{LLM}(a) = b$ and $\mathrm{LLM}(b) = a$, and orbits from generic initial conditions converge to one of these 2-cycles.*

Discussion. This is Class 2 behavior in the Wolfram classification (Chapter 7): the system converges to periodic attractors from generic initial conditions. The 2-cycles arise because the LLM has systematic biases—preferred phrasings, syntactic patterns, and lexical choices—that act as attractors under iteration. A paraphrase of a paraphrase tends to oscillate between two "canonical forms" that are paraphrases of each other.

Example 14.10. Consider successive paraphrasing of "The cat sat on the mat." A typical trajectory might look like:

- x_0: "The cat sat on the mat." - x_1: "A feline was seated upon the mat." - x_2: "A cat was sitting on the mat." - x_3: "A feline was seated on the mat." - x_4: "A cat was sitting on the mat."

From x_2 onward, the system oscillates between two phrasings: a 2-cycle.

14.3.3 14.3.3 Geometric Framework

Tacheny (2025) provides a formal geometric framework for LLM iteration, identifying two regimes:

1. **Contractive regime.** The iteration map is (locally) contractive in an appropriate metric on string space. Orbits converge to fixed points or short cycles. This occurs with "neutral" rewriting tasks like paraphrasing or summarization.

1. **Exploratory regime.** The iteration map is expansive: small perturbations grow. Orbits diverge and exhibit sensitive dependence on initial conditions. This occurs with "adversarial" tasks like "summarize the following text, then negate the main claim."

Proposition 14.11 (Informal)**.** *The choice of prompt—specifically, the instruction component—controls which regime the iteration inhabits. Prompts that preserve meaning (paraphrase, reformat) tend to produce contractive dynamics. Prompts that transform meaning (negate, extrapolate) tend to produce expansive dynamics.*

This is a direct analogue of the role of the local rule in cellular automata (Chapter 7): different rules produce different Wolfram classes.

14.3.4 14.3.4 The Telephone Game

Perez et al. (2024) study iterated LLM generation as a "cultural telephone game": a message is passed from LLM to LLM (or from the same LLM to itself), and small biases accumulate across iterations. The key finding is that outputs converge toward **cultural attractors**—stereotypical, high-frequency patterns in the training data.

This is the dynamical systems analogue of the **basin of attraction** (Chapter 1, Definition 1.8): the set of initial conditions that converge to a given attractor. The cultural attractors of an LLM are the periodic orbits or fixed points of the iteration map, and their basins of attraction encompass the space of "reasonable" inputs.

14.4 The (f, x) Decomposition

14.4.1 14.4.1 Prompts Encode Functions and Inputs

The key structural observation underlying all of iterated LLM generation is:

Observation 14.12. *A prompt $w \in \Sigma^*$ simultaneously encodes two components: - A **function specification** f: what the LLM should do (the instruction). - An **input** x: what the LLM should do it to (the data).*

The LLM's output is (approximately) $f(x)$.

Example 14.13. Consider the prompt:

> "Given x=3, add one and output in the same format: Given x=[result], add one and output in the same format."

This encodes: - f: the function "add one to x and format the result as a new prompt with the same instruction." - $x = 3$: the current input.

The expected output is:

> "Given x=4, add one and output in the same format: Given x=[result], add one and output in the same format."

This output encodes the same function f but with updated input $x' = 4 = f(3)$.

14.4.2 14.4.2 The (f, x) Dynamical System

Definition 14.14 (Function-input decomposition)**.** Let $\mathcal{F}$ denote the space of "function specifications" (the instruction component of a prompt) and $\mathcal{X}$ the space of "inputs" (the data component). An LLM iteration defines a map

$$\Psi : \mathcal{F} \times \mathcal{X} \to \mathcal{F} \times \mathcal{X}$$

given by

$$\Psi(f, x) = (\varphi(f, x),\ f(x)),$$

where:

- $f(x)$ is the result of applying the function f to the input x (the "computation" component).
- $\varphi(f, x)$ is the **meta-rule**: the function specification that appears in the output (the "instruction mutation" component).

Case 1: Perfect instruction preservation ($\varphi(f, x) = f$ for all x). The dynamics reduce to ordinary iteration of f:

$$(f, x) \to (f, f(x)) \to (f, f^2(x)) \to \cdots$$

This is the setting of Chapter 1: a fixed map iterated on a state space.

Case 2: Instruction mutation ($\varphi \neq \mathrm{id}$). Both the function and the input evolve:

$$(f_0, x_0) \to (f_1, x_1) \to (f_2, x_2) \to \cdots$$

where $f_{n+1} = \varphi(f_n, x_n)$ and $x_{n+1} = f_n(x_n)$. This is a **self-modifying dynamical system**: the rule itself changes at each step.

Remark 14.15. Self-modifying dynamics are well-studied in the context of **iterated function systems** (Chapter 6), where the map applied at each step is chosen from a finite set. The (f, x) framework generalizes this by allowing the map to mutate continuously.

14.4.3 14.4.3 Worked Example: Iterated Increment

Let $\mathcal{X} = \mathbb{Z}$, $f_0(x) = x + 1$, and suppose the LLM preserves instructions perfectly: $\varphi(f, x) = f$.

Then:

- $(f_0, 3) \to (f_0, 4) \to (f_0, 5) \to \cdots$

The orbit is $\{3, 4, 5, \ldots\}$—a divergent trajectory with no periodic orbit (since $\mathcal{X}$ is infinite). If $\mathcal{X}$ is taken modulo N (i.e., $\mathcal{X} = \mathbb{Z}/N\mathbb{Z}$), the orbit is periodic with period N.

14.4.4 14.4.4 Worked Example: Instruction Drift

Now suppose the LLM is imperfect: it occasionally mutates the instruction. Let $f_0(x) = x + 1$ but with probability ϵ per iteration the instruction drifts to $f'(x) = x + 2$, and once there, it stays.

The trajectory might look like:

- $(f_0, 3) \to (f_0, 4) \to (f', 5) \to (f', 7) \to (f', 9) \to \cdots$

The "increment by 1" rule mutated to "increment by 2" at step 2, and all subsequent dynamics follow f'. This illustrates how instruction mutation changes the qualitative behavior of the system.

14.5 Model Collapse

14.5.1 14.5.1 The Phenomenon

Model collapse occurs when a generative model is trained on data produced by itself (or by a previous generation of itself). The distribution of outputs progressively loses diversity and converges to a low-variance, low-entropy approximation of the original.

Theorem 14.16 (Shumailov et al., 2024)**.** *Consider a sequence of models $M_0, M_1, M_2, \ldots$ where M_0 is trained on real data, and M_{n+1} is trained on data generated by M_n. Then: 1.* ***Tail erosion.*** *The tails of the learned distribution shrink at each generation. Rare events become rarer and eventually disappear. 2.* ***Variance collapse.*** *For Gaussian models with training set size N, the variance at generation n satisfies*

$$\sigma_n^2 \approx \sigma_0^2 \cdot \frac{1}{1 + n/N}.$$

The variance decreases as $O(1/n)$.

3. ***Convergence to a point mass.*** *In the limit $n \to \infty$, the model's distribution collapses to a point mass (or near-point-mass) at the mode.*

14.5.2 14.5.2 Information-Theoretic Explanation

Proposition 14.17 (Data Processing Inequality)**.** *Let X be the true data distribution, M_n the model at generation n, and Y_n the data generated by M_n. By the data processing inequality,*

$$I(X; Y_{n+1}) \leq I(X; Y_n),$$

where $I(\cdot;\cdot)$ denotes mutual information. Each generation of self-training is a lossy channel: information about the original distribution can only decrease.

Proof. The chain $X \to Y_n \to M_{n+1} \to Y_{n+1}$ is a Markov chain. The data processing inequality states that $I(X; Y_{n+1}) \leq I(X; Y_n)$ for any Markov chain $X \to Y \to Z$.

Corollary 14.18. *Self-training cannot, in expectation, increase the fidelity of a model to the true data distribution. Model collapse is thermodynamically inevitable in the absence of fresh data.*

□

14.5.3 14.5.3 Self-Consuming Loops

Alemohammad et al. (2023) identify three types of **self-consuming generative loops**:

1. **Fully synthetic loop.** All training data for M_{n+1} comes from M_n. Collapse is fastest.
2. **Synthetic augmentation loop.** Training data is a mixture of real and synthetic data. Collapse is slower but still occurs unless the fraction of real data exceeds a threshold.
3. **Fresh data loop.** Each generation receives fresh real data. Collapse is avoided.

They term the pathological behavior **Model Autophagy Disorder (MAD)**: the model consumes itself and degenerates.

14.5.4 14.5.4 Model Collapse in the (f, x) Framework

In the (f, x) decomposition, model collapse corresponds to a **destructive meta-rule** φ:

$$\varphi(f, x) = f_{\text{degraded}},$$

where f_{degraded} is a lower-fidelity version of f. At each generation:

$$(f_0, x_0) \to (f_1, f_0(x_0)) \to (f_2, f_1(f_0(x_0))) \to \cdots$$

with $f_0 \succ f_1 \succ f_2 \succ \cdots$ in some quality ordering. The sequence converges to (f_∞, x_∞) where f_∞ is maximally degenerate (e.g., it maps all inputs to the mode) and x_∞ is a fixed point of f_∞.

Example 14.19. Consider a Gaussian model. Let f_n denote "sample from $\mathcal{N}(\mu, \sigma_n^2)$." Each self-training step shrinks σ_n:

$$\sigma_{n+1}^2 = \sigma_n^2 \cdot \frac{N}{N+1} < \sigma_n^2.$$

The fixed point is $\sigma_\infty^2 = 0$: the collapsed model is a delta function at μ.

14.6 Self-Improvement and Its Limits

14.6.1 14.6.1 Self-Refine

Madaan et al. (2023) propose **Self-Refine**: a three-step iterative loop.

1. **Generate.** The LLM produces an initial response $y_0 = \text{LLM}(x)$.
2. **Critique.** The LLM evaluates its own response: $c_0 = \text{LLM}(\text{"Critique: "} \| y_0)$.
3. **Refine.** The LLM improves the response using the critique: $y_1 = \text{LLM}(\text{"Refine: "} \| y_0 \| c_0)$.

Steps 2–3 repeat: $y_0 \to y_1 \to y_2 \to \cdots$.

Observation 14.20. *Empirically, the quality of y_n improves for the first few iterations and then **saturates**: $y_n \approx y_{n+1}$ for n sufficiently large. This is convergence to an **approximate fixed point** of the refine map.*

In the (f, x) framework: f = "refine the response" and $x_n = y_n$. The dynamics are:

$$x_{n+1} = f(x_n),$$

*and the system converges to a fixed point x^* with $f(x^*) \approx x^*$. The convergence is consistent with the refine map being **contractive** in some implicit quality metric.*

14.6.2 14.6.2 Self-Rewarding

Yuan et al. (2024) propose a more ambitious scheme: the LLM serves simultaneously as **policy** (generator) and **reward model** (evaluator). At each iteration:

1. The LLM generates responses.
2. The same LLM scores them (acting as reward model).
3. The LLM is fine-tuned on high-scoring responses (reinforcement learning from self-feedback).

This is a genuine (f, x) system where **both components improve**:

$$\Psi(f_n, x_n) = (f_{n+1}, x_{n+1})$$

with f_{n+1} being a better policy/reward model than f_n, and x_{n+1} being a higher-quality response than x_n. Empirically, self-rewarding models improve over multiple iterations, with each generation outperforming the last on benchmarks.

The contrast with model collapse (Section 14.5) is instructive: here φ is *constructive* (improving f) rather than destructive (degrading f). The difference lies in the feedback mechanism: self-rewarding includes an explicit quality signal, while self-consuming loops do not.

14.6.3 14.6.3 Limits of Self-Correction

Theorem 14.21 (Huang et al., 2023, informal)**.** *LLMs cannot reliably self-correct their reasoning without external feedback. Specifically: for tasks requiring multi-step logical reasoning, the "critique-and-revise" map often* ***worsens*** *correct answers—the refinement map has wrong fixed points.*

Discussion. The issue is that the LLM's internal model of "correctness" is imperfect. When the same flawed model is used for both generation and evaluation, systematic errors reinforce themselves. In dynamical systems terms: the refinement map has ***spurious fixed points*** *(wrong answers that the model believes are correct), and the basin of attraction of these spurious fixed points can be large.*

This is analogous to the fixed-point structure of Chapter 1: a map can have both "good" fixed points (correct answers) and "bad" fixed points (wrong answers that are stable under iteration). Without external perturbation, the system cannot escape a bad basin.

14.6.4 14.6.4 Constitutional AI

Bai et al. (2022) introduce **Constitutional AI (CAI)**: a set of explicit principles ("the constitution") that constrain the LLM's behavior during self-improvement. In the (f, x) framework, the constitution acts as a **constraint on the meta-rule** φ:

$$\varphi(f, x) \in \mathcal{C},$$

where $\mathcal{C} \subset \mathcal{F}$ is the set of function specifications consistent with the constitution. This prevents φ from drifting toward degenerate or harmful behaviors—it restricts the dynamics to a controlled subspace.

14.7 Computational Power of Iterated LLMs

14.7.1 14.7.1 A Single Forward Pass: TC^0

Theorem 14.22 (Merrill and Sabharwal, 2024)**.** *A single forward pass of a transformer with $O(1)$ layers, $O(\mathrm{poly}(n))$ embedding dimension, and $O(1)$ precision per parameter lies in the*

circuit complexity class TC^0*: the class of problems solvable by constant-depth, polynomial-size threshold circuits.*

TC^0 *is powerful—it contains addition, multiplication, sorting, and matrix operations—but it does not contain all of* P*. In particular, problems requiring* $\omega(1)$*-depth circuits (such as evaluating Boolean formulas of unbounded depth) cannot be solved in a single forward pass.*

14.7.2 14.7.2 Chain of Thought: Turing Completeness

Theorem 14.23 (Merrill and Sabharwal, 2024)**.** *A transformer augmented with* $O(\mathrm{poly}(n))$ *chain-of-thought (CoT) steps—where intermediate tokens are generated and fed back—can simulate any polynomial-time Turing machine. That is, transformers with CoT are* ***Turing complete*** *(up to the polynomial bound on the number of steps).*

Proof sketch. Each CoT step is a TC^0 *computation. A polynomial number of such steps can simulate a polynomial-time Turing machine step by step, using the CoT tokens as the tape. The transformer encodes the transition function, and each forward pass executes one (or a bounded number of) steps.*

14.7.3 14.7.3 Prompt-Based Turing Completeness

Theorem 14.24 (Qiu et al., 2024)**.** *For any computable function* $g : \{0,1\}^* \to \{0,1\}^*$*, there exists a prompt* w *such that a fixed transformer, given* w *concatenated with input* x*, computes* $g(x)$ *using chain-of-thought. That is, the space of prompts parametrizes all computable functions.*

This is a remarkable result: ***prompting alone suffices for Turing completeness.*** *The prompt plays the role of a program, and the transformer plays the role of a universal machine.*

14.7.4 14.7.4 Implications for the (f, x) Framework

Combining Theorems 14.20 and 14.21 with the (f, x) decomposition:

Corollary 14.25. *The dynamical system* $\Psi : (f, x) \mapsto (\varphi(f, x), f(x))$*, instantiated with a sufficiently powerful LLM and chain-of-thought, can in principle realize any computable dynamical system. In particular, for any computable map* $g : \mathcal{X} \to \mathcal{X}$*, there exists a prompt encoding* $f = g$ *such that the iteration* $x_{n+1} = f(x_n)$ *computes the orbit of* g*.*

This means the theory of LLM iteration inherits the full richness (and undecidability) of general computation. Questions like "does this iterated LLM converge?" are, in the general case, undecidable (by reduction to the halting problem).

14.8 Multi-Agent LLM Systems

14.8.1 14.8.1 The Product Dynamical System

When multiple LLM agents interact, the state space becomes a product. Let agents A and B maintain internal states (conversation histories) $s_A, s_B \in \Sigma^*$, and let $m \in \Sigma^*$ be the current message being exchanged.

Definition 14.26 (Two-agent LLM system)**.** A **coupled LLM dynamical system** with agents A, B is a map

$$\Psi : \Sigma^* \times \Sigma^* \times \Sigma^* \to \Sigma^* \times \Sigma^* \times \Sigma^*$$

defined by alternating updates:

$$\Psi(s_A, s_B, m) = \begin{cases} (s_A \| m,\ s_B,\ \mathrm{LLM}_A(s_A \| m)) \& \text{if it is } A\text{'s turn}, \\ (s_A,\ s_B \| m,\ \mathrm{LLM}_B(s_B \| m)) \& \text{if it is } B\text{'s turn}, \end{cases}$$

where $\|$ denotes concatenation.

14.8.2 14.8.2 Role-Playing Agents: CAMEL

Li et al. (2023) instantiate this framework with **CAMEL**: two LLM agents assigned complementary roles (e.g., "AI user" and "AI assistant") that collaborate through structured dialogue. Each agent's prompt includes a role specification and a task description.

In the (f, x) framework, this is:

$$\Psi(f_A, f_B, x) = (f_A, f_B, f_B(f_A(x))),$$

where f_A is the "user" role (generates requests) and f_B is the "assistant" role (generates responses). The two roles are fixed, and only x (the conversation content) evolves. This is a composition: the effective single-step map is $f_B \circ f_A$.

14.8.3 14.8.3 Emergent Collective Dynamics

Park et al. (2023) simulate **25 generative agents** in a virtual town, each with:

- A memory stream (list of past observations and reflections).
- A retrieval mechanism (attention over memory).
- A planning module (daily schedule generation).
- A reflection module (periodic synthesis of memories into higher-level insights).

The resulting multi-agent system exhibits emergent social behaviors: agents form relationships, spread information, coordinate activities, and throw a party that was not explicitly programmed.

In dynamical systems terms, the state is a vector $(s_1, s_2, \ldots, s_{25}) \in (\Sigma^*)^{25}$, and the update map couples all agents through a shared environment. The emergent social structures—stable relationships, group activities, information cascades—are **collective attractors** of this high-dimensional coupled dynamical system. They are analogous to the synchronized oscillations and pattern formation seen in coupled map lattices (a continuous-space analogue of the coupled cellular automata of Chapter 7).

14.9 Open Questions

The dynamical systems perspective on LLMs raises several fundamental questions, many of which connect back to themes developed throughout this book.

Question 14.27 (Attractor control)**.** *Can we design meta-rules φ that avoid both model collapse (convergence to degenerate fixed points) and chaos (no convergence)? What is the analogue of "edge of chaos" for LLM iteration—the boundary between contractive dynamics (Section 14.3.3, regime 1) and expansive dynamics (regime 2)?*

In the cellular automata setting (Chapter 7), the edge of chaos corresponds to Wolfram Class 4 rules, which produce complex, long-lived transients and support computation. The analogous question for LLM iteration is whether there exist prompts that induce Class 4 behavior: complex, structured, non-repeating output that nonetheless has internal organization.

Question 14.28 (Logical depth of LLM outputs)**.** *Is there a formal connection between the **logical depth** (Chapter 9) of an LLM's output and the "creativity" or "interestingness" of the generation process? Recall that logical depth measures the computational effort required to produce a string from its shortest description. An LLM that generates logically deep text is, in*

some sense, performing substantial computation—not producing trivial repetition (shallow) or random noise (incompressible but also shallow in Bennett's framework).

Question 14.29 (Prompt engineering as dynamical systems design)**.** *Prompt engineering is, in the (f, x) framework, the design of the initial function specification f_0 and the meta-rule φ. Can the attractor structure of the resulting dynamical system be predicted from properties of the prompt? Is there a "bifurcation theory" for prompt parameters—small changes in the prompt that cause qualitative changes in the long-term dynamics?*

Question 14.30 (Fresh data as external forcing)**.** *Alemohammad et al. (2023) show that fresh data prevents model collapse. In dynamical systems terms, fresh data is an* ***external forcing*** *or* ***perturbation*** *that prevents the system from settling into a degenerate attractor. What is the minimum rate of fresh data injection needed to maintain a given level of distributional diversity? This is analogous to the problem of* ***noise-sustained patterns*** *in nonlinear dynamics.*

Question 14.31 (Multi-agent phase transitions)**.** *As the number of LLM agents and the topology of their interactions change, do the collective dynamics exhibit phase transitions? Is there a critical connectivity at which coherent collective behavior emerges, analogous to percolation thresholds or the onset of synchronization in coupled oscillator networks?*

14.10 Summary

Concept	Dynamical Systems Formalization
LLM with greedy decoding	One-sided sliding block code, memory $k-1$
LLM with temperature $\tau > 0$	Shift-invariant measure μ_τ on $\Sigma^{\mathbb{N}}$
Iterated generation	DDS (Σ^*, LLM)
Successive paraphrasing	Convergence to 2-cycles (Class 2)
Prompt = (instruction, data)	(f, x) decomposition
Perfect instruction following	Ordinary iteration: $x_{n+1} = f(x_n)$
Instruction drift	Self-modifying system: $(f_{n+1}, x_{n+1}) = (\varphi(f_n, x_n), f_n(x_n))$
Model collapse	Destructive φ; convergence to degenerate fixed point
Self-improvement	Constructive φ; convergence to improved fixed point
Constitutional AI	Constraint $\varphi(f, x) \in \mathcal{C}$
Single forward pass	TC^0 computation
With chain of thought	Turing-complete computation
Multi-agent system	Coupled DDS on product state space

The perspective developed in this chapter reveals that many apparently disparate phenomena in LLM research—model collapse, self-improvement, multi-agent emergence, prompt sensitivity—are manifestations of a single underlying structure: the dynamics of iterated maps on the space of strings. The tools of Chapters 1–9—functional graphs, symbolic dynamics, sliding block codes, entropy, logical depth—apply directly and provide both conceptual clarity and quantitative predictions.

14.11 References

- Alemohammad, S., Casco-Rodriguez, J., Luzi, L., Humayun, A. I., Babaei, H., LeJeune, D., Siahkoohi, A., and Baraniuk, R. G. (2023). Self-consuming generative models go MAD. *arXiv preprint* arXiv:2307.01850.

- Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., et al. (2022). Constitutional AI: Harmlessness from AI feedback. *arXiv preprint* arXiv:2212.08073.

- Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. (2023). Large language models cannot self-correct reasoning yet. *arXiv preprint* arXiv:2310.01798.

- Li, G., Hammoud, H. A. A. K., Itani, H., Khizbullin, D., and Ghanem, B. (2023). CAMEL: Communicative agents for "mind" exploration of large language model society. *arXiv preprint* arXiv:2303.17760.

- Lind, D. and Marcus, B. (2021). *An Introduction to Symbolic Dynamics and Coding.* 2nd edition. Cambridge University Press.

- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., et al. (2023). Self-Refine: Iterative refinement with self-feedback. *arXiv preprint* arXiv:2303.17651.

- Merrill, W. and Sabharwal, A. (2024). The expressive power of transformers with chain of thought. *arXiv preprint* arXiv:2310.07923.

- Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. *arXiv preprint* arXiv:2304.03442.

- Perez, C. R., Morvant, E., and Gauthier, E. (2024). Cultural attractors in LLM telephone games. *Proceedings of the Annual Meeting of the Cognitive Science Society.*

- Qiu, L., Jiang, N., and Ma, T. (2024). Prompting is Turing-complete. *arXiv preprint* arXiv:2411.01992.

- Shumailov, I., Shumaylov, Z., Zhao, Y., Gal, Y., Papernot, N., and Anderson, R. (2024). The curse of recursion: Training on generated data makes models forget. *arXiv preprint* arXiv:2305.17493.

- Tacheny, N. (2025). A geometric framework for iterative LLM generation. *arXiv preprint* arXiv:2512.10350.

- Wang, S., Dong, Y., and Cotterell, R. (2025). Successive paraphrasing converges to fixed cycles. *arXiv preprint* arXiv:2502.15208.

- Yuan, W., Pang, R. Y., Cho, K., Sukhbaatar, S., Xu, J., and Weston, J. (2024). Self-rewarding language models. *arXiv preprint* arXiv:2401.10020.

14.12 Recommended Reading

For the dynamical systems perspective on LLMs:

- **Wang et al. (2025)** is essential reading: the first rigorous demonstration of convergence to periodic orbits in LLM iteration. Short and well-written.

- **Shumailov et al. (2024)** is the definitive treatment of model collapse. The theoretical analysis is clean and the experimental evidence compelling.

For computational expressiveness of transformers:

- **Merrill & Sabharwal (2024)** establishes the TC^0 bound for single forward passes and Turing completeness with chain-of-thought. Accessible and important.

- **Qiu et al. (2024)** proves Turing completeness via prompting alone. Short paper with a beautiful result.

For multi-agent LLM systems:

- **Park et al. (2023)** on Generative Agents is a tour de force: 25 agents exhibiting emergent social behavior. The supplementary materials detail the architecture.
- **Li et al. (2023)** on CAMEL provides a simpler two-agent framework useful for understanding coupled dynamics.

For self-improvement and its limits:

- **Madaan et al. (2023)** on Self-Refine shows what works. **Huang et al. (2023)** shows what doesn't. Reading both together gives a balanced picture.

For background on symbolic dynamics and sliding block codes:

- **Lind & Marcus (2021)** is the standard reference. Chapters 1–3 and 6 are directly relevant to understanding LLMs as block maps.

For thinking about AI systems dynamically:

- This textbook's earlier chapters, especially Chapters 1 (iterated maps), 7 (cellular automata), and 9 (logical depth), provide the conceptual foundations for analyzing LLM iteration.

Chapter 15

Measuring Novelty Within a Sequence

15.1 Introduction

The preceding chapters have focused on dynamics *across time*: how states evolve under iteration, how orbits converge to attractors, how entropy rates characterize the information produced per step. But there is another dimension of novelty that has received less formal attention: **novelty within a single output**.

When an LLM generates a response, that response is itself a sequence of N tokens. Before we ask "how does the n-th output differ from the $(n-1)$-th output?" we can ask: "how diverse is the n-th output internally?" A sequence that repeats "the the the the" $N/2$ times is grammatically valid but informationally impoverished. A sequence with high internal diversity explores more of the vocabulary and exhibits richer structure.

This chapter develops the theory of **intra-sequence novelty**: metrics that quantify diversity, complexity, and information content within a single finite sequence. We connect these measures to the complexity theory of Chapters 8–10 and to the LLM dynamics of Chapter 14, showing how intra-sequence novelty provides a complementary perspective on the quality-diversity trade-off in generative systems.

15.2 The Setting

15.2.1 15.2.1 Finite Sequences

Let Σ be a finite alphabet with $|\Sigma| = V$. A **finite sequence** (or **string**) of length N is an element $w = (w_1, w_2, \ldots, w_N) \in \Sigma^N$. We write $|w| = N$ for the length.

In the LLM context, Σ is the token vocabulary (typically $V \approx 32{,}000$ to $128{,}000$), and w is a single generated response.

Definition 15.1 (Empirical distribution)**.** Given $w \in \Sigma^N$, the **empirical distribution** (or **type**) of w is the probability distribution $\hat{p}_w$ on Σ defined by

$$\hat{p}_w(s) = \frac{|\{i : w_i = s\}|}{N}$$

for each $s \in \Sigma$. This is the relative frequency of symbol s in w.

15.2.2 15.2.2 Two Notions of Novelty

We distinguish two related but distinct concepts:

1. **Lexical diversity**: How many distinct symbols appear, and how evenly are they distributed? This is captured by type-token ratios and entropy measures.

1. **Structural complexity**: How much internal structure does the sequence have? A sequence of N distinct symbols has high lexical diversity but may have trivial structure (e.g., the alphabet in order). Compression-based measures capture structural complexity.

Both contribute to what we intuitively call "novelty" or "interestingness" of a sequence.

15.3 Type-Token Measures

15.3.1 15.3.1 Type-Token Ratio

The simplest diversity measure counts distinct symbols.

Definition 15.2 (Type-Token Ratio)**.** The **type-token ratio** (TTR) of $w \in \Sigma^N$ is

$$\mathrm{TTR}(w) = \frac{|\{w_i : 1 \leq i \leq N\}|}{N} = \frac{\text{number of distinct symbols}}{\text{total symbols}}.$$

Example 15.3. Let $\Sigma = \{a, b, c, d\}$ and consider: - $w_1 = abcdabcd$ (length 8, 4 distinct): $\mathrm{TTR}(w_1) = 4/8 = 0.5$ - $w_2 = aaaaaaaa$ (length 8, 1 distinct): $\mathrm{TTR}(w_2) = 1/8 = 0.125$ - $w_3 = abcdefgh$ over $\Sigma' = \{a, \ldots, h\}$ (length 8, 8 distinct): $\mathrm{TTR}(w_3) = 1.0$

15.3.2 15.3.2 The Length Dependence Problem

Proposition 15.4 (Heaps' Law)**.** *For natural language text, the number of distinct words $V(N)$ in a corpus of N words follows approximately*

$$V(N) \approx K \cdot N^{\beta},$$

where $K > 0$ and $0 < \beta < 1$ (typically $\beta \approx 0.4$–0.6).
Consequence. The TTR decreases with sequence length:

$$\mathrm{TTR}(N) = \frac{V(N)}{N} \approx K \cdot N^{\beta-1} \to 0 \text{ as } N \to \infty.$$

This means TTR is not comparable across sequences of different lengths: a 1000-token sequence will almost always have lower TTR than a 100-token sequence, regardless of their relative diversity.

15.3.3 15.3.3 Length-Normalized Variants

To address the length dependence, several normalizations have been proposed:

Definition 15.5 (Normalized TTR variants)**.** 1. **Root TTR**: $\mathrm{RTTR}(w) = \dfrac{|\text{types}|}{\sqrt{N}}$

2. **Log TTR**: $\mathrm{LogTTR}(w) = \dfrac{\log |\text{types}|}{\log N}$

3. **Moving-Average TTR (MATTR)**: Fix a window size k. Compute TTR for each window of k consecutive tokens. MATTR is the average:

$$\mathrm{MATTR}_k(w) = \frac{1}{N-k+1} \sum_{i=1}^{N-k+1} \mathrm{TTR}(w_i, \ldots, w_{i+k-1}).$$

Proposition 15.6. *Under Heaps' Law with exponent β: - $\mathrm{RTTR}(N) \approx K \cdot N^{\beta-0.5}$, which still decreases if $\beta < 0.5$. - $\mathrm{LogTTR}(N) \approx \beta + \frac{\log K}{\log N} \to \beta$ as $N \to \infty$.*

Thus LogTTR converges to a constant (the Heaps exponent), making it more suitable for cross-length comparison.

15.3.4 15.3.4 Hapax Legomena

Definition 15.7 (Hapax ratio)**.** The **hapax legomena** of w are the symbols that appear exactly once. The **hapax ratio** is

$$\text{Hapax}(w) = \frac{|\{s \in \Sigma : |\{i : w_i = s\}| = 1\}|}{N}.$$

A high hapax ratio indicates the sequence is exploring vocabulary rather than reusing a small set of symbols.

15.4 N-gram Diversity Measures

15.4.1 15.4.1 Distinct-n

Token-level measures ignore sequential structure. N-gram measures capture local patterns.

Definition 15.8 (N-grams and Distinct-n)**.** For $w \in \Sigma^N$ and $n \geq 1$, the n**-grams** of w are the tuples

$$\mathcal{N}_n(w) = \{(w_i, w_{i+1}, \ldots, w_{i+n-1}) : 1 \leq i \leq N - n + 1\}.$$

The **Distinct-n** score is

$$\text{Distinct-n}(w) = \frac{|\{\text{unique } n\text{-grams}\}|}{|\mathcal{N}_n(w)|} = \frac{|\text{set}(\mathcal{N}_n(w))|}{N - n + 1}.$$

Example 15.9. Let $w = abcabcabc$ (length 9). - $n = 1$: 9 unigrams, 3 unique $\{a, b, c\}$. Distinct-1 $= 3/9 = 0.333$. - $n = 2$: 8 bigrams $(ab, bc, ca, ab, bc, ca, ab, bc)$, 3 unique. Distinct-2 $= 3/8 = 0.375$. - $n = 3$: 7 trigrams, 3 unique (abc, bca, cab). Distinct-3 $= 3/7 = 0.429$.

Remark 15.10. Distinct-n increases with n when the sequence has short repeated patterns but no long repeated patterns. For a sequence with no repeated n-grams (maximal diversity), Distinct-n $= 1$.

15.4.2 15.4.2 Repetition Rate

Definition 15.11 (Rep-n)**.** The **repetition rate at order** n is the complement of Distinct-n:

$$\text{Rep-n}(w) = 1 - \text{Distinct-n}(w) = \frac{\text{number of repeated } n\text{-grams}}{N - n + 1}.$$

Theorem 15.12 (Welleck et al., 2020)**.** *In human-written text (e.g., Wikitext-103), sentence-level repetition is rare: Rep-4* $\approx 0.5\%$*. In LLM-generated text with greedy or beam-search decoding, Rep-4 can exceed* 50%*, indicating severe degeneration.*

This dramatic gap motivates Distinct-n as a diagnostic for text quality.

15.4.3 15.4.3 Self-BLEU

Definition 15.13 (Self-BLEU)**.** Given a corpus of M generated sequences $\{w^{(1)}, \ldots, w^{(M)}\}$, the **Self-BLEU** score is the average BLEU score of each sequence against the others:

$$\text{Self-BLEU} = \frac{1}{M} \sum_{i=1}^{M} \text{BLEU}(w^{(i)}, \{w^{(j)} : j \neq i\}).$$

High Self-BLEU indicates the generated sequences are similar to each other (low corpus-level diversity).

Adaptation to single sequences. For a single long sequence w, partition it into M segments of length L: $w = w^{(1)} \| w^{(2)} \| \cdots \| w^{(M)}$. Compute Self-BLEU across segments. High intra-sequence Self-BLEU indicates the sequence repeats itself.

15.5 Entropy-Based Measures

15.5.1 15.5.1 Empirical Entropy

Definition 15.14 (Unigram entropy)**.** The **(empirical) unigram entropy** of $w \in \Sigma^N$ is the Shannon entropy of its empirical distribution:

$$H_1(w) = -\sum_{s \in \Sigma} \hat{p}_w(s) \log_2 \hat{p}_w(s),$$

where $0 \log 0 := 0$.

Proposition 15.15 (Entropy bounds)**.** *1. $H_1(w) = 0$ if and only if w consists of a single repeated symbol. 2. $H_1(w) \leq \log_2 |\mathrm{supp}(\hat{p}_w)| \leq \log_2 \min(N, V)$. 3. Equality in (2) holds if and only if $\hat{p}_w$ is uniform over its support.*

Proof. Standard properties of Shannon entropy.

Example 15.16. For $w = abcabcabc$ (9 tokens, each of a, b, c appears 3 times):

$$H_1(w) = -3 \cdot \frac{3}{9} \log_2 \frac{3}{9} = -3 \cdot \frac{1}{3} \log_2 \frac{1}{3} = \log_2 3 \approx 1.585 \text{ bits}.$$

This is the maximum possible for 3 symbols (uniform distribution).

□

15.5.2 15.5.2 Higher-Order Empirical Entropy

Definition 15.17 (Empirical n-gram entropy)**.** Define the empirical distribution over n-grams:

$$\hat{p}_w^{(n)}(g) = \frac{|\{i : (w_i, \ldots, w_{i+n-1}) = g\}|}{N - n + 1}$$

for $g \in \Sigma^n$. The **empirical n-gram entropy** is

$$H_n(w) = -\sum_{g \in \Sigma^n} \hat{p}_w^{(n)}(g) \log_2 \hat{p}_w^{(n)}(g).$$

Proposition 15.18 (Entropy rate estimate)**.** *For large N, the **empirical entropy rate** is approximated by*

$$h(w) \approx H_n(w) - H_{n-1}(w),$$

the conditional entropy of the n-th symbol given the previous $n - 1$. This estimates the per-symbol information content.

15.5.3 15.5.3 Perplexity

Definition 15.19 (Empirical perplexity)**.** The **perplexity** of the empirical unigram distribution is

$$\mathrm{PPL}_1(w) = 2^{H_1(w)}.$$

More generally, $\mathrm{PPL}_n(w) = 2^{H_n(w)/(n)}$ is the per-symbol perplexity at order n.

Interpretation. Perplexity is the effective vocabulary size: a sequence with perplexity k is "as diverse as" a sequence drawn uniformly from k symbols.

Example 15.20. For the uniform distribution over 3 symbols: $H_1 = \log_2 3$, so $\mathrm{PPL}_1 = 3$. For a degenerate sequence of one repeated symbol: $H_1 = 0$, so $\mathrm{PPL}_1 = 1$.

15.6 Compression-Based Measures

15.6.1 15.6.1 Connection to Kolmogorov Complexity

Chapter 8 introduced Kolmogorov complexity $K(w)$: the length of the shortest program that outputs w. While $K(w)$ is uncomputable, practical compression algorithms provide upper bounds.

Proposition 15.21. *For any compressor C (gzip, LZ77, etc.):*

$$K(w) \leq |C(w)| + O(1),$$

where $|C(w)|$ is the length of the compressed representation and the $O(1)$ term accounts for the decompressor.

15.6.2 15.6.2 Compression Ratio

Definition 15.22 (Compression ratio)**.** The **compression ratio** of w under compressor C is

$$\rho_C(w) = \frac{|C(w)|}{|w|}.$$

- $\rho_C(w) \approx 1$: The sequence is incompressible (high complexity, possibly random).
- $\rho_C(w) \ll 1$: The sequence is highly compressible (low complexity, repetitive).

Proposition 15.23. *For a sequence of N i.i.d. symbols from distribution p:*

$$\mathbb{E}[\rho_C(w)] \to H(p)/\log_2 |\Sigma| \text{ as } N \to \infty,$$

where $H(p)$ is the entropy of p. Optimal compression achieves the entropy rate.

15.6.3 15.6.3 Lempel-Ziv Complexity

Definition 15.24 (LZ complexity)**.** The **Lempel-Ziv complexity** $c_{LZ}(w)$ is the number of distinct substrings in the LZ76 parsing of w: the sequence is parsed left-to-right, and at each step, the longest prefix that has appeared before is identified; the new phrase is that prefix plus one new symbol.

Example 15.25. Let $w = abcabcabc$. - Parse: $a \cdot b \cdot c \cdot ab \cdot ca \cdot bc$ (6 phrases). - $c_{LZ}(w) = 6$.

For comparison, $w' = aaaaaaaaa$ parses as $a \cdot aa \cdot aaa \cdot aaa$ (or similar), with $c_{LZ}(w') \approx O(\sqrt{N})$.

Theorem 15.26 (Lempel-Ziv theorem)**.** *For a stationary ergodic source with entropy rate h:*

$$\frac{c_{LZ}(w)\log N}{N} \to h \text{ almost surely as } N \to \infty.$$

Thus normalized LZ complexity converges to the entropy rate.

15.6.4 15.6.4 Normalized Compression Distance

Definition 15.27 (NCD)**.** For two sequences w_1, w_2 and compressor C, the **normalized compression distance** is

$$\mathrm{NCD}(w_1, w_2) = \frac{|C(w_1 \| w_2)| - \min(|C(w_1)|, |C(w_2)|)}{\max(|C(w_1)|, |C(w_2)|)}.$$

This measures similarity: NCD ≈ 0 if w_1, w_2 share much structure; NCD ≈ 1 if they are unrelated.

Application. To measure self-similarity within a sequence w, split it into halves $w = w_1 \| w_2$ and compute $\mathrm{NCD}(w_1, w_2)$. Low NCD indicates the two halves are similar (internal repetition).

15.7 The Quality-Diversity Frontier

15.7.1 15.7.1 The Trade-off

High intra-sequence novelty is not intrinsically desirable. Consider two extremes:

1. **Random sequence**: Draw each w_i uniformly from Σ. This achieves TTR ≈ 1 (for $N \ll V$), $H_1 \approx \log_2 V$, and $\rho_C \approx 1$. But the sequence is meaningless noise.

1. **Deterministic repetition**: $w = ssss\ldots$ for some $s \in \Sigma$. This achieves TTR $= 1/N \to 0$, $H_1 = 0$, $\rho_C \to 0$. The sequence is predictable and boring.

Neither extreme is desirable for language generation. The goal is to achieve high diversity **subject to coherence constraints**.

15.7.2 15.7.2 Pareto Optimality

Definition 15.28 (Quality-diversity Pareto frontier)**.** Let $Q(w)$ be a quality measure (e.g., fluency, coherence, task performance) and $D(w)$ a diversity measure (e.g., Distinct-2, entropy). A sequence w^* is **Pareto optimal** if there is no w with $Q(w) \geq Q(w^*)$ and $D(w) \geq D(w^*)$ with at least one strict inequality.

The set of Pareto optimal sequences forms a **frontier** in (Q, D) space. Different decoding strategies (greedy, beam search, nucleus sampling) trace different points on or below this frontier.

Theorem 15.29 (Holtzman et al., 2020, informal)**.** *Nucleus (top-p) sampling achieves a better quality-diversity trade-off than beam search or pure sampling. Specifically: - Beam search maximizes Q but achieves low D (degeneration). - Pure sampling achieves high D but low Q (incoherence). - Nucleus sampling with appropriate p approximates the Pareto frontier.*

15.7.3 15.7.3 Entropy Bounds in Human Text

Observation 15.30 (Holtzman et al., 2020)**.** *Human-written text maintains entropy within a "Goldilocks zone": neither too high (incoherent) nor too low (repetitive). Empirically, the per-token entropy of human text lies in a narrow band across domains and styles.*

LLM-generated text that violates these bounds—either by entropy collapse (repetition) or entropy explosion (incoherence)—is perceived as low quality.

15.8 Connection to Logical Depth

15.8.1 15.8.1 Review of Logical Depth

Chapter 9 introduced Bennett's **logical depth**: the computational time required to produce a string from its shortest description. Intuitively:

- **Shallow strings**: Either simple (short description, fast to compute) or random (long description, fast to "compute" by just listing).
- **Deep strings**: Require significant computation to produce, even from the shortest description. They are "neither simple nor random."

15.8.2 15.8.2 Depth as Structured Novelty

Proposition 15.31 (Informal)**.** *A sequence with high intra-sequence novelty (high entropy, low compression) can be either: 1.* ***Random****: High Kolmogorov complexity, low logical depth. 2.* ***Structured-complex****: Moderate Kolmogorov complexity, high logical depth.*

True "creativity" or "interestingness" corresponds to case (2): the sequence is neither trivially compressible nor trivially generated.

Example 15.32. Consider: - w_1 = a random bit string of length 1000. High entropy, high K, but depth ≈ 0 (just output the bits). - w_2 = the first 1000 digits of π. Moderate entropy, low K (short program: "compute π"), but high depth (computing π takes time). - w_3 = "0000...0" (1000 zeros). Low entropy, low K, low depth.

Logical depth distinguishes w_1 (random, shallow) from w_2 (structured, deep).

15.8.3 15.8.3 Implications for LLM Outputs

A high-quality LLM output should have:

- Moderate-to-high entropy (not degenerate).
- Moderate Kolmogorov complexity (not incompressible noise).
- High logical depth (structured, meaningful).

Current intra-sequence novelty metrics (TTR, Distinct-n, compression ratio) capture the first two but not the third. Developing practical proxies for logical depth in generated text remains an open problem.

15.9 Connection to Computational Mechanics

15.9.1 15.9.1 Epsilon-Machines and Statistical Complexity

Chapter 10 introduced the **epsilon-machine**: the minimal deterministic automaton that generates a stationary process with the same statistical structure as the observed data. The **statistical complexity** C_μ is the entropy of the epsilon-machine's state distribution.

15.9.2 15.9.2 Statistical Complexity of a Finite Sequence

For a finite sequence w, we can estimate statistical complexity by:

1. Inferring an epsilon-machine from w (e.g., via the CSSR algorithm).
2. Computing the entropy of the inferred causal states.

Definition 15.33 (Empirical statistical complexity)**.** Let $\hat{\epsilon}(w)$ be an epsilon-machine inferred from w, with causal state distribution $\hat{\pi}$. The **empirical statistical complexity** is

$$\hat{C}_\mu(w) = H(\hat{\pi}).$$

15.9.3 15.9.3 Excess Entropy

Definition 15.34 (Excess entropy)**.** The **excess entropy** (or **predictive information**) of a process is

$$E = I(W_{\text{past}}; W_{\text{future}}) = \lim_{L\to\infty} [H(W_1^L) - L \cdot h],$$

where h is the entropy rate. For a finite sequence, this is estimated by:

$$\hat{E}(w) = H_L(w) - L \cdot \hat{h}(w),$$

where H_L is the block entropy and $\hat{h}$ is the estimated entropy rate.

Interpretation. Excess entropy measures the total amount of structure in the process—how much knowing the past helps predict the future, beyond the per-symbol entropy rate. High excess entropy indicates long-range correlations and complex structure.

15.9.4 15.9.4 Complexity vs. Entropy

Proposition 15.35. *For finite sequences: - **Random sequences**: High entropy rate h, low statistical complexity C_μ, low excess entropy E. - **Periodic sequences**: Low h, low C_μ, low E. - **Complex structured sequences**: Moderate h, high C_μ, high E.*

This echoes the logical depth distinction: complex sequences are neither too ordered nor too random.

15.10 Practical Metrics for LLM Evaluation

15.10.1 15.10.1 Standard Metrics

Based on the theoretical framework, we recommend the following metrics for evaluating intra-sequence novelty in LLM outputs:

Metric	Formula	Measures	Range
Distinct-1	unique unigrams / total	Vocabulary diversity	[0, 1]
Distinct-2	unique bigrams / total	Local pattern diversity	[0, 1]
Distinct-4	unique 4-grams / total	Phrase-level diversity	[0, 1]
Rep-4	1 - Distinct-4	Repetition rate	[0, 1]
H_1	Unigram entropy	Information content	[0, log V]
ρ_{gzip}	Compression ratio	Structural complexity	(0, 1+]

15.10.2 15.10.2 Implementation

```
import math
import gzip
from collections import Counter
from typing import List

def distinct_n(tokens: List[str], n: int) -> float:
    """Compute Distinct-n score."""
```

```
    if len(tokens) < n:
        return 0.0
    ngrams = [tuple(tokens[i:i+n]) for i in range(len(tokens) - n + 1)]
    return len(set(ngrams)) / len(ngrams)

def unigram_entropy(tokens: List[str]) -> float:
    """Compute empirical unigram entropy in bits."""
    counts = Counter(tokens)
    total = len(tokens)
    return -sum((c/total) * math.log2(c/total) for c in counts.values())

def compression_ratio(text: str) -> float:
    """Compute gzip compression ratio."""
    original = text.encode('utf-8')
    compressed = gzip.compress(original)
    return len(compressed) / len(original)

def lz_complexity(tokens: List[str]) -> int:
    """Compute Lempel-Ziv complexity (number of phrases)."""
    seen = set()
    complexity = 0
    current = []
    for token in tokens:
        current.append(token)
        current_tuple = tuple(current)
        if current_tuple not in seen:
            seen.add(current_tuple)
            complexity += 1
            current = []
    if current:  # Handle remaining tokens
        complexity += 1
    return complexity

def novelty_report(tokens: List[str]) -> dict:
    """Compute comprehensive novelty metrics."""
    text = ' '.join(tokens)
    return {
        'length': len(tokens),
        'vocab_size': len(set(tokens)),
        'ttr': len(set(tokens)) / len(tokens),
        'distinct_1': distinct_n(tokens, 1),
        'distinct_2': distinct_n(tokens, 2),
        'distinct_4': distinct_n(tokens, 4),
        'rep_4': 1 - distinct_n(tokens, 4),
        'entropy_h1': unigram_entropy(tokens),
        'perplexity': 2 \textbf{ unigram_entropy(tokens),
        'compression_ratio': compression_ratio(text),
        'lz_complexity': lz_complexity(tokens),
    }
```

15.10.3 15.10.3 Interpreting the Metrics

Guidelines for LLM-generated text:

1. **Distinct-2** < 0.5: Likely degenerate (excessive repetition).
2. **Distinct-2** > 0.9: Possibly incoherent or enumeration-like.
3. **Rep-4** $> 10\%$: Significant phrase repetition; investigate.
4. **Compression ratio** < 0.3: Highly repetitive structure.
5. **Compression ratio** > 0.8: Little structure; possibly random or very diverse.

Human text typically achieves Distinct-2 $\approx$ 0.8–0.95 and compression ratio $\approx$ 0.3–0.5.

15.11 Connection to Model Collapse

15.11.1 15.11.1 Intra-Sequence Novelty During Collapse

Chapter 14 (Section 14.5) described **model collapse**: when models are trained on their own outputs, diversity progressively decreases. This manifests in intra-sequence metrics:

Proposition 15.36 (Novelty decay under collapse)**.** *Let M_n be the model at generation n in a self-consuming loop. As n increases: 1.* Distinct-k$(M_n) \to 0$ *for $k \geq 2$. 2. $H_1(M_n) \to 0$. 3. $\rho_C(M_n) \to 0$.*

The outputs converge to repetitive, low-entropy sequences.

15.11.2 15.11.2 Diversity as Collapse Diagnostic

Monitoring intra-sequence novelty provides early warning of model collapse:

- Track Distinct-2 and H_1 across training generations.
- Significant drops indicate onset of collapse.
- Intervention (fresh data injection) can be triggered when metrics fall below thresholds.

15.12 Novelty-Increasing Dynamics in the (f, x) Framework

The preceding sections characterized novelty as a static property of sequences. But in the (f, x) framework of Chapter 14, we can ask a dynamical question: **what kinds of systems increase novelty over time, rather than decrease it?**

15.12.1 15.12.1 The Default: Novelty Decay

Most iterated LLM systems exhibit **novelty decay**. Recall from Chapter 14 that model collapse, successive paraphrasing, and self-consuming loops all converge to low-entropy attractors. The meta-rule φ is typically *contractive* in novelty space:

$$D(x_{n+1}) \leq D(x_n),$$

where D is any diversity measure. This is the second law of thermodynamics for generative systems: without external input, diversity decreases.

Proposition 15.37 (Novelty contraction under paraphrasing)**.** *Let f be a "paraphrase" function (meaning-preserving rewrite). Then for typical LLM implementations: 1. The vocabulary of $f(x)$ is a subset of common words, regardless of x's vocabulary. 2. Rare words in x are replaced by common synonyms in $f(x)$. 3.* $\mathrm{TTR}(f^n(x)) \to \mathrm{TTR}^*$ *as $n \to \infty$, where* TTR^* *is the TTR of the attractor.*

The attractor vocabulary is determined by the LLM's frequency biases, not by the input.

15.12.2 15.12.2 Novelty-Expanding Functions

What functions f satisfy $D(f(x)) > D(x)$? We identify several classes:

Definition 15.38 (Divergent prompt)**.** A prompt specification f is **divergent** if, for typical inputs x:

$$\mathbb{E}[D(f(x))] > D(x).$$

Example 15.39 (Divergent prompts)**.** 1. **Elaboration**: "Expand on the following with additional details and examples." Each application adds new content, increasing vocabulary and n-gram diversity.

2. **Multi-perspective**: "Rewrite this from three different perspectives: a scientist, an artist, and a child." The output contains three sub-sequences with different vocabularies.

3. **Brainstorming**: "List 10 different ways to approach this problem." Enumeration naturally produces high Distinct-n.

4. **Adversarial rewrite**: "Rewrite this using completely different words while preserving the meaning." Explicitly pressures vocabulary change.

5. **Translation round-trip with divergence**: "Translate to French, then to Japanese, then back to English, keeping any interesting variations." Each translation introduces new phrasings.

15.12.3 15.12.3 The Exploratory Regime Revisited

Chapter 14 (Section 14.3.3) identified two regimes:

- **Contractive**: Meaning-preserving operations (paraphrase, summarize). Novelty decreases.
- **Exploratory**: Meaning-transforming operations (negate, extrapolate, fictionalize). Novelty can increase.

We now formalize the exploratory regime in terms of novelty dynamics.

Definition 15.40 (Novelty Lyapunov exponent)**.** For a (f, x) system, define the **novelty Lyapunov exponent**:

$$\lambda_D = \lim_{n\to\infty} \frac{1}{n} \log \frac{D(x_n)}{D(x_0)},$$

when the limit exists.

- $\lambda_D < 0$: Novelty contracts exponentially (typical for paraphrasing).
- $\lambda_D = 0$: Novelty is conserved on average (edge of chaos).
- $\lambda_D > 0$: Novelty expands exponentially (exploratory/divergent regime).

Conjecture 15.41. *For most LLM-based (f, x) systems, $\lambda_D < 0$. Systems with $\lambda_D > 0$ are unstable and eventually produce incoherent outputs (the quality constraint is violated before novelty grows unboundedly).*

15.12.4 15.12.4 Strategies for Sustained Novelty

How can we design (f, x) systems that maintain high novelty without collapsing or diverging to incoherence?

Strategy 1: External Injection.

Periodically inject fresh content from outside the system:

$$x_{n+1} = f(x_n) \oplus \xi_n,$$

where ξ_n is drawn from an external source (new data, user input, retrieved documents). This is analogous to the "fresh data" intervention in model collapse (Alemohammad et al., 2023).

Strategy 2: Diversity-Regularized Objectives.

Modify the generation process to penalize low novelty:

$$\tilde{f}(x) = \arg\max_y \left[\log P(y \mid x) + \alpha \cdot D(y)\right],$$

where $\alpha > 0$ controls the diversity pressure. This is the **Maximum Mutual Information (MMI)** objective of Li et al. (2016).

Strategy 3: Rejection Sampling.

Generate multiple candidates and select for diversity:

$$x_{n+1} = \arg \max_{y \in \{f_1(x_n), \ldots, f_k(x_n)\}} D(y).$$

This can be combined with quality filtering: reject candidates below a coherence threshold, then select the most diverse among survivors.

Strategy 4: Adversarial Multi-Agent Dynamics.

Use multiple agents with opposing objectives:

$$x_{n+1} = g(f_A(x_n), f_B(x_n)),$$

where f_A maximizes agreement with x_n and f_B maximizes disagreement. The aggregation function g (e.g., debate, synthesis) produces outputs more diverse than either agent alone.

Example 15.42 (Debate dynamics)**.** Two agents debate a proposition: - Agent A argues for the proposition. - Agent B argues against. - The combined transcript has higher vocabulary diversity than either monologue.

This is the diversity mechanism underlying Constitutional AI debates (Bai et al., 2022) and multi-agent reasoning (Du et al., 2023).

Strategy 5: Novelty Search.

Borrowed from evolutionary computation (Lehman & Stanley, 2011): rather than optimizing for a fixed objective, optimize for **novelty relative to the archive of past outputs**:

$$x_{n+1} = \arg\max_y \min_{z \in \mathcal{A}_n} d(y, z),$$

where $\mathcal{A}_n = \{x_0, x_1, \ldots, x_n\}$ is the archive and d is a distance metric. This explicitly rewards exploration of new regions of output space.

15.12.5 15.12.5 Patterns of Novelty-Increasing Systems

What do novelty-increasing trajectories look like? We identify several characteristic patterns:

Pattern A: Branching/Enumeration. The system produces lists, alternatives, or variations. Each item is distinct, yielding high Distinct-n.

Example: "List 20 uses for a paperclip" → high diversity, but potentially low depth.

Pattern B: Compositional Expansion. New elements are generated by combining existing elements in novel ways.

Example: Metaphor generation combines concepts from different domains, increasing semantic diversity.

Pattern C: Hierarchical Elaboration. The system recursively expands on sub-parts, adding detail at each level.

Example: Outline → Draft → Expanded draft. Each level has higher token count and (typically) higher absolute vocabulary, though TTR may decrease.

Pattern D: Stochastic Exploration. High-temperature sampling explores the tail of the distribution.

Example: Temperature $\tau > 1$ generation produces more diverse but less coherent outputs.

Pattern E: Contradiction/Negation Chains. Each step negates or contradicts the previous, forcing vocabulary change.

Example: "The cat sat" → "The cat didn't sit" → "No cat was present" → "Dogs filled the room" → ...

15.13 Relative Novelty

15.13.1 15.13.1 The Reference Problem

All novelty metrics so far have been **absolute**: they measure properties of a sequence in isolation. But our intuitive notion of novelty is **relative**: something is novel *with respect to* what we've seen before.

A sequence that is internally diverse but identical to a million existing sequences is not novel. A sequence that repeats one word but has never been generated before is, in some sense, novel.

Definition 15.43 (Relative novelty)**.** Let w be a sequence and $\mathcal{R}$ a **reference set** (corpus, distribution, or collection of prior outputs). The **relative novelty** of w with respect to $\mathcal{R}$ measures how different w is from the elements of $\mathcal{R}$.

15.13.2 15.13.2 Reference Sets

The choice of reference set $\mathcal{R}$ determines what kind of novelty we measure:

Reference Set	Novelty Interpretation
Training corpus	Novelty w.r.t. what the model has seen
Prior outputs of this model	Novelty w.r.t. the model's own history
Outputs of other models	Novelty w.r.t. the "consensus"
Human-written text	Novelty w.r.t. human baseline
The current conversation	Novelty w.r.t. local context

15.13.3 15.13.3 Distance-Based Relative Novelty

Definition 15.44 (Nearest-neighbor novelty)**.** Given w and reference set $\mathcal{R}$, define:

$$\mathrm{Nov}_{\mathcal{R}}(w) = \min_{r \in \mathcal{R}} d(w, r),$$

where d is a distance metric (edit distance, embedding distance, etc.).

High $\mathrm{Nov}_{\mathcal{R}}(w)$ means w is far from everything in $\mathcal{R}$—it is novel.

Definition 15.45 (Average-distance novelty)**.**

$$\mathrm{Nov}_{\mathcal{R}}^{\mathrm{avg}}(w) = \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} d(w, r).$$

This is less sensitive to outliers in $\mathcal{R}$.

15.13.4 15.13.4 Information-Theoretic Relative Novelty

Definition 15.46 (Surprisal)**.** Given a probabilistic model P of the reference distribution, the surprisal** of w is:

$$S_P(w) = -\log P(w).$$

High surprisal means w is unlikely under P—it is novel relative to the distribution.

Remark 15.47. This is the standard cross-entropy loss. A model trained on $\mathcal{R}$ assigns low probability to sequences unlike $\mathcal{R}$.

Definition 15.48 (KL-divergence novelty)**.** For a sequence w with empirical distribution $\hat{p}_w$ and a reference distribution q (e.g., the unigram distribution of $\mathcal{R}$):

$$D_{KL}(\hat{p}_w \| q) = \sum_{s \in \Sigma} \hat{p}_w(s) \log \frac{\hat{p}_w(s)}{q(s)}.$$

High KL-divergence means w's token distribution differs from the reference.

Proposition 15.49. *If w uses words that are rare in $\mathcal{R}$, then $D_{KL}(\hat{p}_w \| q)$ is large (assuming $q(s) > 0$ for all s). Conversely, if w uses only common words in their typical proportions, $D_{KL} \approx 0$.*

15.13.5 15.13.5 Kernel-Based Novelty

Definition 15.50 (Maximum Mean Discrepancy)**.** Let $\phi : \Sigma^* \to \mathcal{H}$ be a feature map into a reproducing kernel Hilbert space. The **MMD** between sequence w and reference set $\mathcal{R}$ is:

$$\mathrm{MMD}(w, \mathcal{R}) = \left\| \phi(w) - \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} \phi(r) \right\|_{\mathcal{H}}.$$

This measures how far w is from the centroid of $\mathcal{R}$ in feature space.

Definition 15.51 (Kernel Entropy Novelty)**.** Kirchherr et al. (2024) propose **Kernel-based Entropic Novelty (KEN)**: identify modes in the feature space that are present in w but underrepresented in $\mathcal{R}$. The number and weight of such "novel modes" quantifies novelty.

15.13.6 15.13.6 N-gram Overlap Measures

Definition 15.52 (Novel n-gram ratio)**.** The fraction of n-grams in w that do not appear in $\mathcal{R}$:

$$\mathrm{NovelNgram}_n(w, \mathcal{R}) = \frac{|\mathcal{N}_n(w) \setminus \bigcup_{r \in \mathcal{R}} \mathcal{N}_n(r)|}{|\mathcal{N}_n(w)|}.$$

Example 15.53. If w contains the trigram "quantum frog dancing" and no document in $\mathcal{R}$ contains this trigram, it contributes to the novel n-gram count.

This is related to the **BLEU score** (but inverted): BLEU measures overlap; NovelNgram measures non-overlap.

15.13.7 15.13.7 Self-Relative Novelty: Novelty Over Time

A particularly important reference set is the system's own history.

Definition 15.54 (Cumulative novelty)**.** In an iterated system (f, x) with trajectory $x_0, x_1, x_2, \ldots$, define:

$$\mathrm{CumNov}(x_n) = \mathrm{Nov}_{\{x_0,\ldots,x_{n-1}\}}(x_n).$$

This measures how different the n-th output is from all previous outputs.

Proposition 15.55. *For a system converging to a periodic attractor of period p:*

$$\mathrm{CumNov}(x_n) \to 0 \text{ as } n \to \infty.$$

Once the system enters the attractor, each output has been seen before (within the last p steps), so cumulative novelty vanishes.

Corollary 15.56. *Sustained high cumulative novelty implies the system has not converged to a periodic attractor. This is a diagnostic for "creative" or "exploratory" dynamics.*

15.13.8 15.13.8 The Novelty-Coherence-Reference Triangle

Combining absolute novelty, relative novelty, and coherence yields a three-dimensional characterization:

High Absolute	High Relative	High Coherence	Interpretation
✓	✓	✓	**Ideal creative output**: diverse, original, sensible
✓	✓	×	Random/incoherent novelty
✓	×	✓	Diverse but clichéd (common patterns)
×	✓	✓	Repetitive but original (rare word repeated)
✓	×	×	Chaotic regurgitation
×	✓	×	Novel nonsense
×	×	✓	Coherent cliché (safe, boring)
×	×	×	Degenerate collapse

The goal of generative systems is to occupy the (✓, ✓, ✓) corner: high diversity within the sequence, high differentiation from existing content, and high coherence.

15.13.9 15.13.9 Formalizing "Interestingness"

We can now attempt a formal definition of what makes a sequence "interesting":

Definition 15.57 (Interestingness score)**.** Define:

$$I(w; \mathcal{R}) = D(w) \cdot \mathrm{Nov}_{\mathcal{R}}(w) \cdot Q(w),$$

where:

- $D(w)$ is an absolute diversity measure (e.g., Distinct-2),
- $\mathrm{Nov}_{\mathcal{R}}(w)$ is relative novelty,
- $Q(w)$ is a quality/coherence measure.

The product structure ensures all three factors must be high for the sequence to be deemed interesting.

Remark 15.58. This multiplicative form is one of many possibilities. Weighted sums, geometric means, or Pareto rankings are alternatives. The right formulation depends on the application.

15.14 Additional Mathematical Frameworks

Several mathematical theories beyond information theory and dynamical systems offer relevant perspectives on novelty.

15.14.1 15.14.1 Optimal Transport and Wasserstein Distance

The **Wasserstein distance** (or earth mover's distance) between probability distributions P and Q is:

$$W_p(P, Q) = \left(\inf_{\gamma \in \Gamma(P,Q)} \int d(x, y)^p \, d\gamma(x, y) \right)^{1/p},$$

where $\Gamma(P, Q)$ is the set of couplings with marginals P and Q.

Application to novelty. Let P be the empirical distribution of a generated sequence and Q the reference distribution. Unlike KL-divergence, Wasserstein distance:

- Is symmetric.
- Respects the geometry of the underlying space (nearby tokens contribute less distance).
- Is defined even when P and Q have non-overlapping support.

Definition 15.59 (Wasserstein novelty)**.** For sequence w with empirical distribution $\hat{p}_w$ and reference distribution q:

$$\mathrm{Nov}_W(w, q) = W_1(\hat{p}_w, q).$$

This measures how much "work" is needed to transform the token distribution of w into the reference.

15.14.2 15.14.2 Rate-Distortion Theory

Rate-distortion theory characterizes the fundamental trade-off between compression rate R and distortion D. The **rate-distortion function** $R(D)$ gives the minimum bits needed to represent a source with average distortion at most D.

Application to novelty. Consider the quality-diversity trade-off as a rate-distortion problem:

- **Rate**: The "complexity" or "effort" required to generate the output.
- **Distortion**: The deviation from a target quality level.

The Pareto frontier of Section 15.7 is analogous to the rate-distortion curve.

Conjecture 15.60. *The quality-diversity Pareto frontier for LLM generation has a convex shape analogous to the rate-distortion function, with diversity playing the role of rate and quality loss playing the role of distortion.*

15.14.3 15.14.3 Determinantal Point Processes

A **Determinantal Point Process (DPP)** is a probability distribution over subsets of a ground set, where the probability of selecting a subset S is proportional to $\det(L_S)$ for a positive semidefinite kernel matrix L.

DPPs naturally produce **diverse** subsets: items that are similar (high L_{ij}) are unlikely to be selected together.

Application to novelty. For text generation:

1. Represent candidate continuations as vectors (embeddings).
2. Define the kernel $L_{ij} = q_i q_j \cdot \phi(x_i)^\top \phi(x_j)$, where q_i is quality and $\phi(x_i)$ is the embedding.
3. Sample from the DPP to obtain diverse, high-quality outputs.

Proposition 15.61 (DPP diversity guarantee)**.** *Samples from a DPP with kernel L satisfy:*

$$\mathbb{E}[|S|] = \operatorname{tr}((I + L)^{-1} L),$$

and the expected pairwise distance between selected items is bounded below by a function of the eigenvalues of L.

15.14.4 15.14.4 Information Geometry

Information geometry treats the space of probability distributions as a Riemannian manifold with the **Fisher information metric**:

$$g_{ij}(\theta) = \mathbb{E}\left[\frac{\partial \log p(x;\theta)}{\partial \theta_i} \frac{\partial \log p(x;\theta)}{\partial \theta_j}\right].$$

Application to novelty. The Fisher-Rao distance between distributions is:

$$d_{FR}(P, Q) = \arccos\left(\int \sqrt{p(x)q(x)}\, dx\right).$$

This provides a geometrically principled measure of how different two distributions are, accounting for the curvature of probability space.

Definition 15.62 (Fisher-Rao novelty)**.** For a sequence w and reference distribution q:

$$\mathrm{Nov}_{FR}(w, q) = d_{FR}(\hat{p}_w, q).$$

This measures the geodesic distance on the probability simplex.

15.14.5 15.14.5 Algorithmic Statistics and Typicality

Algorithmic statistics (Gács, Tromp, Vitányi) studies the "stochastic complexity" of individual objects: the shortest description of an object as a sample from a computable distribution.

Definition 15.63 (Randomness deficiency)**.** The **randomness deficiency** of x with respect to model P is:

$$d(x|P) = \log |P| - K(x|P),$$

where $|P|$ is the "size" of the model's support and $K(x|P)$ is the complexity of x given P. Low deficiency means x is "typical" of P; high deficiency means x is "atypical."

Application to novelty. A sequence with high randomness deficiency relative to the training distribution is "surprising"—it doesn't fit the typical pattern. This formalizes novelty as atypicality.

15.14.6 15.14.6 Intrinsic Motivation and Curiosity

In reinforcement learning, **intrinsic motivation** provides rewards for novel states:

$$r_{\text{intrinsic}}(s) = f(\text{novelty of } s).$$

Common formulations include:

s' - '

- **State visitation counts**: $r = 1/\sqrt{N(s)}$ (novelty as rarity).

Application to LLMs. These ideas can inform novelty-seeking generation:

$$x_{n+1} = \arg\max_y \left[\log P(y|x_n) + \beta \cdot r_{\text{intrinsic}}(y)\right].$$

The parameter β controls the exploration-exploitation trade-off.

15.14.7 15.14.7 Topological Data Analysis

Persistent homology tracks topological features (connected components, loops, voids) across scales. The **persistence diagram** records when features appear and disappear.

Application to novelty. Embed sequences into a metric space (via sentence embeddings). Compute persistent homology of:

1. A single sequence's token embeddings (intra-sequence topology).
2. A collection of sequences (corpus-level topology).

Conjecture 15.64. *Novel sequences have persistence diagrams that differ significantly from the "typical" diagram of the reference corpus. Features that persist at unusual scales indicate structural novelty.*

15.14.8 15.14.8 Renormalization and Multi-Scale Analysis

The **renormalization group** from statistical physics provides tools for analyzing systems at multiple scales. Coarse-graining a system reveals which features persist across scales.

Application to novelty. Consider a hierarchy of representations:

- Token level: $w_1, w_2, \ldots, w_N$
- Phrase level: Chunks of 5–10 tokens
- Sentence level: Semantic units
- Paragraph level: Thematic units

Novelty at each scale can be measured independently. A sequence might be token-diverse but phrase-repetitive, or vice versa.

Definition 15.65 (Multi-scale novelty profile)**.** Define:

$$\text{Nov}^{(k)}(w) = D(\text{level-}k \text{ representation of } w)$$

for $k = 1, 2, \ldots, K$. The tuple $(\text{Nov}^{(1)}, \ldots, \text{Nov}^{(K)})$ is the **multi-scale novelty profile**.

Different applications may weight different scales: poetry might prioritize token-level novelty; scientific writing might prioritize concept-level novelty.

15.15 Cognitive Foundations of Novelty

The metrics developed in this chapter are not arbitrary mathematical constructs. They correspond to cognitive mechanisms that evolved for novelty detection in biological agents. This section integrates cognitive science findings, showing that our formal framework captures genuine aspects of how minds process novelty.

15.15.1 15.15.1 Predictive Processing and Surprisal

The **predictive processing** framework (Rao & Ballard, 1999; Friston, 2010) posits that the brain is fundamentally a prediction machine. Perception, cognition, and action are all understood as minimizing **prediction error**—the discrepancy between expected and observed inputs.

Definition 15.66 (Prediction error)**.** Let P be the brain's generative model and x an observed stimulus. The prediction error is:

$$\epsilon(x) = -\log P(x) = S_P(x),$$

the surprisal of x under P.

This is precisely Definition 15.50. The cognitive science literature thus provides independent motivation for surprisal as a novelty measure: **novelty is prediction error**.

Proposition 15.67 (Novelty as free energy)**.** *In the free energy principle (Friston, 2010), agents minimize variational free energy F, which upper-bounds surprisal:*

$$F \geq S_P(x) = -\log P(x).$$

Novel stimuli have high $S_P(x)$, creating pressure to either: 1. Update the model P to accommodate x (learning), or 2. Act to avoid x (avoidance of surprising states).

This explains why novelty is both attractive (learning opportunity) and aversive (prediction failure)—the dual nature captured by curiosity research (Section 15.15.4).

15.15.2 15.15.2 Hippocampal Novelty Detection

The hippocampus implements a **comparator function** that detects mismatches between predicted and actual inputs (Lisman & Grace, 2005).

Empirical finding 15.86. Novel stimuli elicit increased hippocampal activation and trigger dopaminergic responses in the VTA (ventral tegmental area). The signal strength correlates with the degree of novelty—i.e., with surprisal under the current memory model.

Formalization. Let $\mathcal{M}$ be the set of stored memory traces. The hippocampal novelty signal for stimulus x is:

$$N_{\text{hipp}}(x) \propto \min_{m \in \mathcal{M}} d(x, m),$$

where d is a similarity metric in representational space. This is nearest-neighbor novelty (Definition 15.48).

Implication. The reference set $\mathcal{R}$ in our relative novelty framework corresponds to the contents of memory. The choice of $\mathcal{R}$ (training data, prior outputs, etc.) is the computational analogue of asking "what does the system remember?"

15.15.3 15.15.3 Habituation and Cumulative Novelty

Habituation is the decrease in response to repeated stimuli. It is one of the simplest forms of learning, observed across all species with nervous systems.

Definition 15.68 (Habituation curve)**.** Let r_n be the response magnitude to the n-th presentation of stimulus x. The habituation curve is:

$$r_n = r_0 \cdot e^{-\lambda n},$$

where $\lambda > 0$ is the habituation rate.

Connection to cumulative novelty. Habituation implies that the novelty of x decreases with repeated exposure:

$$\mathrm{Nov}_{\{x,x,\ldots,x\}}(x) \to 0.$$

This is captured by cumulative novelty (Definition 15.57): as the reference set accumulates copies of x, the nearest-neighbor distance goes to zero.

Dishabituation. A novel stimulus can restore responsiveness to habituated stimuli. In our framework, this corresponds to:

$$\mathrm{CumNov}(x_n) > 0 \implies \text{system has not fully habituated.}$$

15.15.4 15.15.4 Curiosity and the Goldilocks Zone

Berlyne (1960) distinguished two types of curiosity:

- **Diversive curiosity**: Seeking stimulation to escape boredom (too low novelty).
- **Specific curiosity**: Seeking information to resolve uncertainty (moderate novelty).

Empirical finding 15.88 (Inverted-U relationship). Preference for stimuli follows an inverted-U curve as a function of novelty/complexity: stimuli that are too simple are boring; stimuli that are too complex are aversive; stimuli of intermediate complexity are most engaging (Berlyne, 1970).

Formalization. Let $D(x)$ be an absolute novelty measure and $Q(x)$ be perceived quality/engagement. The Berlyne curve is:

$$Q(x) = f(D(x)) \quad \text{where } f \text{ is concave with interior maximum.}$$

A simple parametric form:

$$Q(x) = D(x)^{\alpha} \cdot (D_{\max} - D(x))^{\beta},$$

which peaks at $D^* = \frac{\alpha}{\alpha+\beta} D_{\max}$.

Connection to quality-diversity frontier. The inverted-U explains why the Pareto frontier (Section 15.7) is the right framing: maximum diversity is not optimal; there is a sweet spot. The "edge of chaos" is cognitively validated.

15.15.5 15.15.5 Schema Violation and Bisociation

Schema theory (Bartlett, 1932; Rumelhart, 1980) posits that knowledge is organized into structured frames or schemas. Novelty arises from **schema violation**: inputs that don't fit existing frames.

Definition 15.69 (Schema-relative novelty)**.** Let $\mathcal{S} = \{S_1, \ldots, S_k\}$ be a set of schemas (structured templates). Define:

$$\mathrm{Nov}_{\mathcal{S}}(x) = \min_{S \in \mathcal{S}} \mathrm{dist}(x, S),$$

where $\mathrm{dist}(x, S)$ measures how poorly x fits schema S.

Koestler (1964) defined **bisociation** as the creative act of connecting two previously unconnected schemas. A bisociative sequence w activates schemas S_1 and S_2 that have low prior association:

$$\mathrm{Bisoc}(w) \propto \mathrm{P}(S_1 \in w) \cdot \mathrm{P}(S_2 \in w) \cdot (1 - \mathrm{Assoc}(S_1, S_2)).$$

Implication for LLMs. Truly creative outputs combine concepts from distant domains. This suggests a novelty metric based on co-activation of dissimilar embedding clusters.

15.15.6 15.15.6 Conceptual Spaces

Gärdenfors (2000) proposed that concepts are represented as regions in **conceptual spaces**—geometric structures where similarity corresponds to distance.

Definition 15.70 (Conceptual space novelty)**.** Let $\mathcal{C}$ be a conceptual space with metric d. For a sequence w mapped to a trajectory γ_w in $\mathcal{C}$:

$$\mathrm{Nov}_{\mathcal{C}}(w) = \int_{\gamma_w} \rho(c)^{-1}\, dc,$$

where $\rho(c)$ is the density of prior experience at point c. This weights novelty by how unexplored each region is.

Connection to embedding-based metrics. Modern sentence embeddings (BERT, etc.) define a conceptual space. The semantic diversity metrics of Section 15.6 are implementations of conceptual space novelty.

15.15.7 15.15.7 Working Memory and Chunk Diversity

Miller (1956) established that working memory capacity is approximately 7 ± 2 items—but "items" can be **chunks** of arbitrary complexity.

Definition 15.71 (Chunk entropy)**.** Parse sequence w into chunks $c_1, \ldots, c_m$. The chunk entropy is:

$$H_{\text{chunk}}(w) = H(\text{empirical distribution over chunk types}).$$

Proposition 15.72. *For sequences within working memory span, perceived novelty scales with chunk entropy, not raw token entropy. A sequence of 7 novel chunks is more striking than 7 repetitions of a familiar chunk, even if both have similar token-level entropy.*

Implication. *Multi-scale novelty (Section 15.14.8) is cognitively grounded: the brain computes novelty at the chunk level, not just the symbol level.*

15.15.8 15.15.8 Semantic Priming and Spreading Activation

In semantic memory, concepts are linked in a network. **Spreading activation** (Collins & Loftus, 1975) propagates from activated nodes to related nodes.

Definition 15.73 (Activation novelty)**.** Let A_t be the activation pattern at time t and A_{t-1} the pattern after the previous token. Define:

$$\mathrm{Nov}_{\mathrm{act}}(w_t|w_{<t}) = \|A_t - A_{t-1}\|,$$

the magnitude of activation change.

Tokens that activate distant regions of semantic memory (low overlap with current activation) have high activation novelty. This corresponds to conditional entropy (Definition 15.16): how unpredictable is the next semantic region given the current one?

15.15.9 15.15.9 The Orienting Response

The **orienting response** (Sokolov, 1963) is an involuntary shift of attention toward novel stimuli. It includes physiological markers (pupil dilation, skin conductance, heart rate change).

Empirical finding 15.94. Orienting response magnitude correlates with:

1. Stimulus intensity
2. Unexpectedness (surprisal)
3. Significance (relevance to goals)
4. Change from previous stimuli (local novelty)

Connection to metrics. The orienting response validates that novelty is not purely abstract—it has measurable physiological correlates. An LLM-generated sequence that would elicit strong orienting responses (measurable via eye-tracking or GSR) is one with high operational novelty.

15.15.10 15.15.10 Toward a Unified Cognitive-Computational Theory

The cognitive findings converge on a unified picture:

Cognitive Mechanism	Computational Formalization
Prediction error	Surprisal $S_P(x) = -\log P(x)$
Hippocampal comparison	Nearest-neighbor novelty $\min_r d(w, r)$
Habituation	Cumulative novelty decay
Curiosity (Goldilocks)	Inverted-U preference function
Schema violation	Distance to nearest schema
Bisociation	Co-activation of dissimilar clusters
Conceptual space	Trajectory in embedding space
Chunking	Multi-scale entropy
Spreading activation	Activation change magnitude

Theorem 15.74 (Informal)**.** *The novelty metrics of Sections 15.3–15.6 correspond to cognitive mechanisms that evolved for adaptive behavior in uncertain environments. A sequence w is "cognitively novel" if it: 1. Has high surprisal under the predictive model (high S_P). 2. Is distant from stored memories (high $\mathrm{Nov}_{\mathcal{R}}$). 3. Falls in the Goldilocks zone of the Berlyne curve (moderate D). 4. Violates or connects schemas in unexpected ways. 5. Causes large activation changes in semantic memory.*

Design principle. *To build generative systems that produce human-interesting outputs, optimize for these cognitively-grounded novelty metrics rather than purely information-theoretic ones. The brain is the ultimate novelty detector; our metrics should approximate its computations.*

15.16 Open Questions

Question 15.75 (Practical depth estimation). *Can we develop efficient estimators for logical depth or depth-like quantities that distinguish "structured complex" from "random" sequences in LLM outputs?*

Question 15.76 (Novelty vs. coherence). *Is there a formal relationship between intra-sequence novelty and inter-sequence coherence (e.g., in a conversation)? Can high intra-sequence diversity compensate for low inter-sequence diversity, or vice versa?*

Question 15.77 (Optimal decoding). *Given a quality function Q and a diversity function D, what decoding strategy achieves the Pareto frontier? Can this be characterized analytically for simple models?*

Question 15.78 (Multi-scale novelty). *Novelty can be measured at multiple scales: token, phrase, sentence, paragraph, document. How do these scales interact? Can a document have high paragraph-level diversity but low sentence-level diversity?*

Question 15.79 (Semantic vs. lexical). *Our metrics are largely lexical. How can we measure* ***semantic novelty****—diversity in meaning rather than surface form? Embedding-based metrics (e.g., variance of sentence embeddings) are one approach, but their relationship to information-theoretic quantities is unclear.*

Question 15.80 (Lyapunov exponents for novelty). *Can the novelty Lyapunov exponent λ_D (Definition 15.44) be estimated empirically for real LLM systems? What is its typical value for different prompt types?*

Question 15.81 (Optimal reference set). *For relative novelty measures, how should the reference set $\mathcal{R}$ be chosen? Is there an "optimal" reference that balances sensitivity to genuine novelty against robustness to noise?*

Question 15.82 (DPP decoding). *Can Determinantal Point Processes be used for beam selection in LLM decoding to guarantee diverse outputs? What is the computational cost, and does it improve over simpler diversity heuristics?*

Question 15.83 (Information-geometric novelty). *Does the Fisher-Rao distance provide better discrimination of novelty than KL-divergence in practice? When is the geometric structure of the probability simplex important?*

15.17 Summary

Concept	Metric	Captures
Vocabulary exploration	TTR, Hapax ratio	Number of distinct symbols
Local pattern diversity	Distinct-n	Uniqueness of n-grams
Repetition	Rep-n, Self-BLEU	Degree of self-copying
Information content	Entropy H_1, H_n	Bits per symbol
Structural complexity	Compression ratio, LZ complexity	Non-random patterns
Long-range structure	Statistical complexity, Excess entropy	Memory in the process
"Interestingness"	Logical depth (theoretical)	Computation required to generate
Relative novelty	$\mathrm{Nov}_{\mathcal{R}}$, KL-divergence	Difference from reference
Novelty dynamics	Lyapunov exponent λ_D	Growth/decay rate of diversity
Geometric novelty	Wasserstein, Fisher-Rao distance	Geometry-respecting divergence
Diverse sampling	DPP kernel methods	Guaranteed diversity in selection
Typicality	Randomness deficiency	Atypicality relative to model
Multi-scale structure	Renormalization, TDA	Features across scales

Intra-sequence novelty complements the dynamical systems perspective of earlier chapters. Where Chapters 1–14 ask "how does the system evolve over time?", this chapter asks "how rich is each individual state?" Together, they provide a comprehensive framework for analyzing the outputs of generative systems.

The key insight is that novelty alone is insufficient: random sequences have maximum novelty but zero value. The goal is **structured novelty**—high diversity within the constraints of coherence, grammar, and meaning. This is the "creative zone" between order and chaos, the edge where interesting computation happens.

We identified two key extensions to the basic framework:

1. **Novelty-increasing dynamics**: Most LLM systems exhibit novelty decay (model collapse, attractor convergence). But divergent prompts, adversarial multi-agent systems, and novelty search can sustain or increase diversity. The novelty Lyapunov exponent λ_D characterizes whether a system is contractive ($\lambda_D < 0$), conservative ($\lambda_D = 0$), or expansive ($\lambda_D > 0$).

1. **Relative novelty**: Absolute diversity measures miss the question "novel compared to what?" Relative novelty with respect to a reference set $\mathcal{R}$ captures originality. The choice of $\mathcal{R}$—training data, prior outputs, human baselines—determines what kind of novelty we measure. The ideal generative output occupies the (high absolute, high relative, high coherence) corner of the novelty-coherence-reference triangle.

15.18 Recommended Reading

Foundational:

- Shannon, C. E. (1948). "A Mathematical Theory of Communication" — the origin of information-theoretic measures of diversity.
- Lempel, A. and Ziv, J. (1976). "On the Complexity of Finite Sequences" — introduces the LZ complexity measure.
- Bennett, C. H. (1988). "Logical Depth and Physical Complexity" — distinguishes structured complexity from randomness.

Text Generation Diversity:

- Li et al. (2016). "A Diversity-Promoting Objective Function for Neural Conversation Models" — introduces Distinct-n and MMI decoding.
- Holtzman et al. (2020). "The Curious Case of Neural Text Degeneration" — essential reading on the degeneration problem and nucleus sampling.
- Welleck et al. (2020). "Neural Text Generation With Unlikelihood Training" — unlikelihood objective to reduce repetition.

Computational Mechanics:

- Crutchfield, J. P. and Young, K. (1989). "Inferring Statistical Complexity" — epsilon-machines and statistical complexity.
- Shalizi, C. R. and Crutchfield, J. P. (2001). "Computational Mechanics: Pattern and Prediction, Structure and Simplicity" — comprehensive introduction to computational mechanics.

Cognitive Science of Novelty:

- Berlyne, D. E. (1960). *Conflict, Arousal, and Curiosity* — classic work on curiosity and the inverted-U relationship.
- Friston, K. (2010). "The Free-Energy Principle: A Unified Brain Theory?" — predictive processing framework.
- Schmidhuber, J. (2010). "Formal Theory of Creativity, Fun, and Intrinsic Motivation" — computational theory of curiosity.

Modern Diversity Metrics:

- Pillutla et al. (2021). "MAUVE: Measuring the Gap Between Neural Text and Human Text" — distribution comparison for generation evaluation.
- Tevet & Berant (2021). "Evaluating the Evaluation of Diversity in Natural Language Generation" — critical analysis of diversity metrics.
- Kirchherr et al. (2024). "Kernel-based Entropic Novelty" — advanced kernel-based novelty measures.

Mathematical Foundations:

- Cover, T. M. and Thomas, J. A. (2006). *Elements of Information Theory* — comprehensive reference for entropy and information measures.
- Peyré, G. and Cuturi, M. (2019). "Computational Optimal Transport" — Wasserstein distances and optimal transport.
- Kulesza, A. and Taskar, B. (2012). "Determinantal Point Processes for Machine Learning" — DPPs for diverse sampling.

15.19 References

- Alemohammad, S., et al. (2023). Self-consuming generative models go MAD. *arXiv preprint* arXiv:2307.01850.
- Amari, S. and Nagaoka, H. (2000). *Methods of Information Geometry.* American Mathematical Society.
- Bai, Y., et al. (2022). Constitutional AI: Harmlessness from AI feedback. *arXiv preprint* arXiv:2212.08073.
- Bartlett, F. C. (1932). *Remembering: A Study in Experimental and Social Psychology.* Cambridge University Press.
- Bennett, C. H. (1988). Logical depth and physical complexity. In *The Universal Turing Machine: A Half-Century Survey*, pp. 227–257. Oxford University Press.
- Berlyne, D. E. (1960). *Conflict, Arousal, and Curiosity.* McGraw-Hill.
- Berlyne, D. E. (1970). Novelty, complexity, and hedonic value. *Perception & Psychophysics*, 8(5):279–286.
- Collins, A. M. and Loftus, E. F. (1975). A spreading-activation theory of semantic processing. *Psychological Review*, 82(6):407–428.

- Crutchfield, J. P. and Young, K. (1989). Inferring statistical complexity. *Physical Review Letters*, 63(2):105–108.
- Du, Y., et al. (2023). Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint* arXiv:2305.14325.
- Friston, K. (2010). The free-energy principle: A unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138.
- Gács, P., Tromp, J. T., and Vitányi, P. M. B. (2001). Algorithmic statistics. *IEEE Transactions on Information Theory*, 47(6):2443–2463.
- Gärdenfors, P. (2000). *Conceptual Spaces: The Geometry of Thought*. MIT Press.
- Heaps, H. S. (1978). *Information Retrieval: Computational and Theoretical Aspects*. Academic Press.
- Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. (2020). The curious case of neural text degeneration. In *ICLR 2020*.
- Koestler, A. (1964). *The Act of Creation*. Hutchinson.
- Kulesza, A. and Taskar, B. (2012). Determinantal point processes for machine learning. *Foundations and Trends in Machine Learning*, 5(2–3):123–286.
- Lehman, J. and Stanley, K. O. (2011). Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2):189–223.
- Li, J., Galley, M., Brockett, C., Gao, J., and Dolan, B. (2016). A diversity-promoting objective function for neural conversation models. In *NAACL-HLT 2016*.
- Lempel, A. and Ziv, J. (1976). On the complexity of finite sequences. *IEEE Transactions on Information Theory*, 22(1):75–81.
- Lisman, J. E. and Grace, A. A. (2005). The hippocampal-VTA loop: Controlling the entry of information into long-term memory. *Neuron*, 46(5):703–713.
- Miller, G. A. (1956). The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63(2):81–97.
- Pathak, D., Agrawal, P., Efros, A. A., and Darrell, T. (2017). Curiosity-driven exploration by self-supervised prediction. In *ICML 2017*.
- Peyré, G. and Cuturi, M. (2019). Computational optimal transport. *Foundations and Trends in Machine Learning*, 11(5–6):355–607.
- Rao, R. P. N. and Ballard, D. H. (1999). Predictive coding in the visual cortex: A functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1):79–87.
- Rumelhart, D. E. (1980). Schemata: The building blocks of cognition. In *Theoretical Issues in Reading Comprehension*, pp. 33–58. Lawrence Erlbaum.
- Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423.

- Shumailov, I., et al. (2024). The curse of recursion: Training on generated data makes models forget. *arXiv preprint* arXiv:2305.17493.

- Sokolov, E. N. (1963). *Perception and the Conditioned Reflex.* Pergamon Press.

- Villani, C. (2009). *Optimal Transport: Old and New.* Springer.

- Welleck, S., Kulikov, I., Roller, S., Dinan, E., Cho, K., and Weston, J. (2020). Neural text generation with unlikelihood training. In *ICLR 2020.*

- Ziv, J. and Lempel, A. (1977). A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3):337–343.

15.20 Exercises

Exercise 15.1. Compute Distinct-1, Distinct-2, and H_1 for the following sequences over $\Sigma = \{a, b, c\}$: (a) $w_1 = abcabcabcabc$ (b) $w_2 = aaaaaaaaaaaa$ (c) $w_3 = abcbcacabacb$

Exercise 15.2. Prove that for any sequence w of length N over alphabet Σ:

$$\text{Distinct-1}(w) = \text{TTR}(w).$$

Exercise 15.3. Show that if w is a periodic sequence with period p (i.e., $w_i = w_{i+p}$ for all i), then for $n \leq p$:

$$\text{Distinct-n}(w) \leq \frac{p}{N - n + 1}.$$

Exercise 15.4. The **Rényi entropy** of order α is defined as:

$$H_\alpha(w) = \frac{1}{1-\alpha} \log_2 \sum_{s \in \Sigma} \hat{p}_w(s)^\alpha.$$

Show that $\lim_{\alpha \to 1} H_\alpha(w) = H_1(w)$ (Shannon entropy).

Exercise 15.5. Implement the ‘novelty$_{r}eport‘functionandcomputemetricsfor : (a)Aparagraphfromaneu$

Exercise 15.6. (Research) Find an LLM-generated text that has high Distinct-2 but low human-judged quality. What other metrics might detect this failure mode?

Chapter 16

Evolution, Discovery, and Open-Ended Systems

16.1 Introduction

The preceding chapters developed a framework for understanding LLMs as dynamical systems, with the (f, x) decomposition revealing how prompts encode both functions and data, and how iteration can mutate both. Chapter 15 introduced metrics for measuring novelty within outputs—the diversity, complexity, and originality of what these systems produce.

This chapter extends the framework in two directions that turn out to be deeply connected:

1. **AI-as-scientist**: Systems that autonomously generate hypotheses, design experiments, and make discoveries—treating the scientific method itself as a dynamical system.

1. **Evolutionary computation**: Algorithms that evolve populations of solutions, where LLMs now serve as intelligent mutation operators—treating evolution as a search through function space.

Both domains share a fundamental structure: an iterative process that generates novelty under selection pressure, with the goal of producing not just *good* solutions but *diverse* solutions that explore the space of possibilities. The (f, x) framework applies directly, and the novelty metrics of Chapter 15 become fitness functions.

The synthesis reveals a common challenge: **open-endedness**. How can we design systems that produce unbounded novelty without collapsing into repetitive patterns or diverging into incoherence? This is the "edge of chaos" problem writ large—the search for dynamics that sustain creative exploration indefinitely.

16.2 Scientific Discovery as a Dynamical System

16.2.1 16.2.1 The Scientific Method as Iteration

The scientific method is, at its core, an iterated process:

1. **Observe** phenomena

2. **Hypothesize** explanations

3. **Predict** consequences

4. **Test** predictions experimentally

5. **Revise** hypotheses based on results
6. **Repeat**

Definition 16.1 (Scientific discovery system)**.** A **scientific discovery system** is a tuple (H, E, f, g) where:

- H is the space of hypotheses (theories, models, explanations).
- E is the space of evidence (experimental results, observations).
- $f : H \times E \to H$ is the **revision function**: how hypotheses update given evidence.
- $g : H \to E$ is the **prediction function**: what evidence a hypothesis predicts.

The dynamics are:

$$h_{n+1} = f(h_n, e_n), \qquad e_n = g(h_n) + \xi_n,$$

where ξ_n is experimental noise or the "surprise" of nature.

16.2.2 16.2.2 The (f, x) Decomposition of Science

Mapping to the framework of Chapter 14:

Scientific Concept	(f, x) Component
Current hypothesis	State x
Scientific method / reasoning	Function f
Methodology evolution	Meta-rule φ
Experimental results	External input ξ
Paradigm shift	Change in f itself

Observation 16.2. *Normal science (Kuhn, 1962) corresponds to fixed f: hypotheses evolve but the method stays constant.* ***Paradigm shifts*** *correspond to changes in f itself—a new way of doing science.*

In the (f, x) framework, a paradigm shift is:

$$(f_{old}, h) \to (f_{new}, h'),$$

where f_{new} may be incommensurable with f_{old} (different standards of evidence, different ontologies).

16.2.3 16.2.3 Attractors in Hypothesis Space

Definition 16.3 (Scientific attractor)**.** A hypothesis h^* is an **attractor** of the scientific discovery system if, for hypotheses h in some neighborhood, repeated application of revision converges to h^*:

$$f(f(\cdots f(h, e_1), e_2), \ldots, e_n) \to h^*.$$

Example 16.4. The heliocentric model is an attractor: sufficiently careful observation and hypothesis revision, starting from various geocentric models, eventually converges to heliocentrism. The Ptolemaic system was a **metastable state**—stable under small perturbations but not a global attractor.

Proposition 16.5. *Under the Bayesian interpretation, the attractor of rational belief revision is the hypothesis with maximum posterior probability. If the true model is in H and evidence is unbiased, the attractor is the truth.*

16.2.4 16.2.4 Novelty in Scientific Discovery

Where does novelty enter science?

1. **Hypothesis generation**: Proposing new $h \in H$ not previously considered.
2. **Experimental design**: Choosing which e to seek (not just passively receiving).
3. **Conceptual innovation**: Expanding H itself to include previously inconceivable hypotheses.
4. **Methodological innovation**: Changing f to a more powerful revision procedure.

The metrics of Chapter 15 apply:

- **Distinct-n on hypotheses**: How many qualitatively different hypotheses has the system explored?
- **Relative novelty**: Is the current hypothesis far from all previously considered ones?
- **Cumulative novelty**: Is the trajectory through hypothesis space exploring new regions?

Definition 16.6 (Scientific creativity)**.** A scientific discovery system exhibits **creativity** if: 1. It generates hypotheses with high relative novelty (unlike prior hypotheses). 2. Those hypotheses have high coherence (consistent with evidence). 3. The novelty is sustained (cumulative novelty doesn't decay to zero).

This is the (✓, ✓, ✓) corner of the novelty-coherence-reference triangle (Section 15.13.8) applied to hypothesis space.

16.2.5 16.2.5 Boden's Taxonomy of Creativity

Boden (2004) distinguishes three types of creativity, which map directly onto the (f, x) framework:

Definition 16.7 (Boden's creativity types)**.** 1. **Combinatorial creativity**: Novel combinations of familiar ideas. In (f, x) terms: new states x generated by fixed f operating on known elements. Example: a new metaphor combining two existing concepts.

2. **Exploratory creativity**: Systematic exploration of a conceptual space defined by implicit rules. In (f, x) terms: traversing the state space $\mathcal{X}$ under fixed dynamics f, discovering regions not previously visited. Example: a jazz improvisation within the rules of chord progressions.

3. **Transformational creativity**: Changing the rules that define the conceptual space itself. In (f, x) terms: changing f, expanding or restructuring $\mathcal{X}$. Example: the invention of non-Euclidean geometry.

Proposition 16.8. *Transformational creativity corresponds to paradigm shifts (Section 16.2.2). It requires meta-level operations: the system must represent and modify its own rules. In the (f, x) framework, this is the action of the meta-rule φ on f.*

Observation 16.9. *Most computational creativity systems achieve only combinatorial or exploratory creativity. Transformational creativity—genuine expansion of the conceptual space—remains rare. This is the "paradigm shift problem" for AI.*

16.3 Evolutionary Computation as a Dynamical System

16.3.1 16.3.1 Genetic Algorithms

A **genetic algorithm** (GA) evolves a population of candidate solutions:

1. **Initialize** population $P_0 = \{x_1, \ldots, x_N\}$.
2. **Evaluate** fitness $f(x_i)$ for each individual.
3. **Select** parents based on fitness.
4. **Recombine** (crossover) parents to produce offspring.
5. **Mutate** offspring with some probability.
6. **Replace** population: $P_{t+1} = \text{selection}(P_t \cup \text{offspring})$.
7. **Repeat** from step 2.

Definition 16.10 (GA as DDS)**.** A genetic algorithm defines a discrete dynamical system on the space of populations:

$$\mathcal{P} = \{\text{finite subsets of } \mathcal{X}\},$$

with transition function:

$$F : \mathcal{P} \to \mathcal{P}, \qquad P_{t+1} = F(P_t).$$

This is a higher-order DDS: the "state" is itself a set of solutions.

16.3.2 16.3.2 Fitness Landscapes

Definition 16.11 (Fitness landscape)**.** For search space $\mathcal{X}$ with fitness function $f : \mathcal{X} \to \mathbb{R}$ and neighborhood relation $N : \mathcal{X} \to 2^{\mathcal{X}}$, the **fitness landscape** is the graph (X, E, f) where edge $(x, y) \in E$ iff $y \in N(x)$.

Proposition 16.12 (Landscape structure determines dynamics)**.** *- **Smooth, unimodal landscape**: GA converges to global optimum. Single attractor. - **Rugged, multimodal landscape**: GA may get stuck in local optima. Multiple attractors. - **Deceptive landscape**: Gradient points away from global optimum. Attractor is not optimal. - **Neutral landscape**: Large plateaus of equal fitness. Random drift dominates.*

16.3.3 16.3.3 Premature Convergence as Collapse

Definition 16.13 (Premature convergence)**.** A GA exhibits **premature convergence** when the population loses diversity and concentrates on a suboptimal solution before finding the global optimum.

This is the evolutionary analogue of **model collapse** (Chapter 14, Section 14.5): self-reinforcing dynamics reduce diversity. In GA terms:

$$\text{Distinct-1}(P_t) \to 0 \text{ as } t \to \infty,$$

even though the population hasn't found the global optimum.

Proposition 16.14. *Premature convergence corresponds to the GA being captured by a **spurious attractor**—a local optimum that is stable under the selection-mutation dynamics but not globally optimal.*

16.3.4 16.3.4 Diversity Maintenance

To prevent premature convergence, GAs employ diversity maintenance:

1. **Fitness sharing**: Reduce fitness of individuals similar to others.
2. **Crowding**: Offspring replace similar parents.
3. **Island models**: Maintain separate subpopulations with occasional migration.
4. **Novelty pressure**: Explicitly reward individuals different from the population.

These correspond to **diversity-regularized objectives** (Section 15.12.4, Strategy 2):

$$\tilde{f}(x) = f(x) + \alpha \cdot \mathrm{Nov}_P(x),$$

where $\mathrm{Nov}_P(x)$ measures novelty of x relative to population P.

16.4 Novelty Search and Quality-Diversity

16.4.1 16.4.1 Novelty Search

Lehman and Stanley (2011) proposed a radical idea: **abandon fitness entirely** and search purely for novelty.

Definition 16.15 (Novelty search)**.** Let $\mathcal{A}_t$ be the **archive** of previously discovered behaviors. The novelty of individual x is:

$$\mathrm{Nov}(x) = \frac{1}{k} \sum_{i=1}^{k} d(x, i\text{-th nearest neighbor in } \mathcal{A}_t),$$

the average distance to the k nearest archived behaviors.
Selection is based on novelty, not fitness:

$$P_{t+1} = \mathrm{select}(P_t, \mathrm{Nov}).$$

Theorem 16.16 (Lehman & Stanley, 2011, informal)**.** *On deceptive fitness landscapes, novelty search often outperforms fitness-based search. By ignoring the misleading fitness gradient, novelty search explores broadly and stumbles upon the global optimum.*

16.4.2 16.4.2 The Paradox of Objectives

Observation 16.17 (The objective paradox)**.** *For complex problems, explicitly optimizing for the objective can prevent finding solutions, while searching for novelty—ignoring the objective—can succeed.*

This connects to the exploration-exploitation trade-off. Pure exploitation (greedy fitness optimization) converges prematurely. Pure exploration (random search) is inefficient. Novelty search is a structured form of exploration that can outperform both.

16.4.3 16.4.3 Quality-Diversity Algorithms

Quality-Diversity (QD) algorithms seek the best of both worlds: collections of solutions that are both **high-quality** and **diverse**.

Definition 16.18 (MAP-Elites)**.** **MAP-Elites** (Mouret & Clune, 2015) maintains a grid of cells, each representing a region of **behavior space**. For each cell, the algorithm stores the highest-fitness individual whose behavior falls in that region.

Algorithm: 1. **Initialize**: Randomly generate solutions; place each in the appropriate cell. 2. **Select**: Randomly choose an occupied cell. 3. **Mutate**: Produce offspring from the cell's elite. 4. **Evaluate**: Compute fitness and behavior descriptor of offspring. 5. **Update**: If offspring's cell is empty or offspring has higher fitness, replace. 6. **Repeat**.

Proposition 16.19. *MAP-Elites simultaneously optimizes for: - **Quality**: Each cell contains the best solution for that behavior. - **Diversity**: Different cells represent different behaviors.*

*The output is a **repertoire**—a diverse collection of high-quality solutions.*

16.4.4 16.4.4 Connection to Chapter 15 Metrics

Quality-Diversity algorithms operationalize the novelty-coherence-reference triangle:

QD Concept	Chapter 15 Concept
Fitness	Quality $Q(w)$
Behavior distance	Relative novelty $\mathrm{Nov}_{\mathcal{R}}(w)$
Archive coverage	Absolute diversity $D(w)$
MAP-Elites grid	Multi-scale novelty profile

The QD objective can be written:

$$\max_{P} \sum_{c \in \text{cells}} Q(\text{elite}_c) \cdot \mathbb{1}[\text{cell } c \text{ occupied}],$$

which rewards both quality (high Q) and coverage (many cells occupied).

16.5 LLM-Augmented Evolution

16.5.1 16.5.1 Evolution Through Large Models (ELM)

Traditional GAs use random mutation. **Evolution through Large Models (ELM)** (Lehman et al., 2022) replaces random mutation with LLM-guided mutation.

Definition 16.20 (LLM mutation operator)**.** Given a program x (or any structured solution), the LLM mutation is:

$$\text{mutate}_{\text{LLM}}(x) = \text{LLM}(\text{"Improve or vary this solution: "} \| x).$$

The LLM uses its training knowledge to propose *intelligent* modifications rather than random changes.

Proposition 16.21. *LLM mutation has several advantages over random mutation: 1. **Syntax preservation**: The LLM generates syntactically valid programs. 2. **Semantic awareness**: Modifications are more likely to be meaningful. 3. **Knowledge transfer**: The LLM applies patterns from its training data. 4. **Adaptive step size**: The LLM can make small tweaks or large restructurings.*

16.5.2 16.5.2 FunSearch: LLM + Evolution for Mathematical Discovery

FunSearch (Romera-Paredes et al., 2023) applies ELM to mathematical discovery. The key insight: search for **programs that generate solutions** rather than solutions directly.

Algorithm (FunSearch).

1. **Skeleton**: User provides a program skeleton with a critical function to evolve.
2. **Initialize**: Start with a simple implementation.
3. **Prompt**: Sample high-performing programs from the archive; prompt the LLM to improve.
4. **Evaluate**: Run the generated program; score based on the solutions it produces.
5. **Update**: Add high-scoring programs to the archive.
6. **Iterate** with island-based parallelism.

Theorem 16.22 (FunSearch results, Nature 2024)**.** *FunSearch discovered: - New constructions for the **cap set problem** (extremal combinatorics), improving the asymptotic lower bound for the first time in 20 years. - New heuristics for **online bin packing** that outperform hand-designed baselines.*

Remark 16.23. FunSearch searches in **function space**, not solution space. The programs it discovers are interpretable and generalizable—a scientist can read them and understand the strategy.

16.5.3 16.5.3 AlphaEvolve: General-Purpose Algorithm Discovery

AlphaEvolve (DeepMind, 2025) extends FunSearch to general-purpose algorithm discovery.

Key features:

- **Ensemble of LLMs**: Uses both Gemini 2.0 Flash (high throughput) and Gemini 2.0 Pro (high quality).
- **Multi-objective optimization**: Can optimize for multiple metrics (accuracy, efficiency, etc.).
- **Whole-codebase evolution**: Can modify entire programs, not just single functions.

Theorem 16.24 (AlphaEvolve results, 2025)**.** *AlphaEvolve achieved: - First improvement in 56 years to Strassen's algorithm for 4×4 complex matrix multiplication. - 0.7- 23–32- Improvements on 20*

16.5.4 16.5.4 The (f, x) View of LLM-Evolution

In the (f, x) framework:

Evolution Concept	(f, x) Component
Current population	State $x = P$
Selection + LLM mutation	Function $f = \text{select} \circ \text{mutate}_{\text{LLM}}$
Prompt engineering	Specification of f
Archive/elites	Memory component of x
Fitness function	Quality metric Q
Behavior descriptor	Novelty metric D

Proposition 16.25. *LLM-augmented evolution is a (f, x) system where the mutation operator f is itself a large neural network with rich prior knowledge. The "instruction" component of f (the prompt) can be evolved alongside the population, enabling* ***meta-learning*** *of the evolutionary process.*

16.6 Autonomous AI Scientists

16.6.1 16.6.1 The AI Scientist

The AI Scientist (Lu et al., 2024) automates the full scientific workflow:

1. **Idea generation**: LLM proposes research questions and hypotheses.
2. **Literature review**: Automated search and synthesis of related work.
3. **Experiment design**: LLM writes code to test hypotheses.
4. **Execution**: Automated running of experiments.
5. **Analysis**: LLM interprets results and generates visualizations.
6. **Writing**: LLM produces a complete scientific manuscript.
7. **Review**: Automated peer review evaluates the paper.

Definition 16.26 (AI Scientist as (f, x) system). The AI Scientist is a (f, x) system where:

- $x =$ (current manuscript, experimental results, literature context).
- $f =$ the LLM-driven research pipeline (idea $\to$ experiment $\to$ write $\to$ review).
- The output $f(x)$ is a revised manuscript incorporating new experiments.

Observation 16.26. *The AI Scientist exhibits both* ***novelty generation*** *(new ideas, experiments) and* ***quality control*** *(automated review). This implements the quality-diversity trade-off at the level of scientific papers.*

16.6.2 16.6.2 AI Scientist v2 and Agentic Tree Search

AI Scientist v2 (2025) replaces template-based generation with **agentic tree search**:

Definition 16.27 (Experiment tree)**.** The **experiment tree** is a tree where:

- Each node is an experimental state (code, results, manuscript draft).
- Each edge is an action (modify code, run experiment, revise text).
- The tree expands via LLM-proposed actions.
- An **experiment manager agent** guides search.

This is **Monte Carlo Tree Search for science**—balancing exploration of new experimental directions with exploitation of promising results.

Theorem 16.28 (AI Scientist v2 results, 2025)**.** *AI Scientist v2 produced the first entirely AI-generated peer-review-accepted workshop paper, demonstrating that LLM-driven systems can produce novel, validated scientific contributions.*

16.6.3 16.6.3 Open-Ended Scientific Discovery

Definition 16.29 (Open-ended scientific discovery)**.** A scientific discovery system is **open-ended** if: 1. It generates novel hypotheses indefinitely (no convergence to a fixed set). 2. The novelty is adaptive (hypotheses explain more phenomena over time). 3. Complexity grows unboundedly (explanations become more sophisticated).

Conjecture 16.30. *Current AI scientist systems are not truly open-ended. They can produce novel papers within the distribution of their training data, but cannot: - Invent fundamentally new mathematical frameworks. - Recognize when the entire paradigm needs revision. - Generate experimental setups requiring physical intuition beyond their training.*

True open-ended scientific discovery may require: - Embodiment (physical experimentation, not just simulation). - Social embedding (interaction with human scientists). - Meta-level reflection (questioning the scientific method itself).

16.7 The Adjacent Possible and Open-Endedness

16.7.1 16.7.1 The Adjacent Possible

Definition 16.31 (Adjacent possible, Kauffman 1996)**.** The **adjacent possible** is the set of states reachable in one step from the current state. As the system explores and expands, new states become adjacent that were not previously accessible.

Example 16.32. In technology evolution: - Before the wheel, the cart is not in the adjacent possible. - Once the wheel exists, the cart becomes adjacent. - Once the cart exists, wheeled vehicles, gears, and pulleys become adjacent.

Each innovation expands the adjacent possible, enabling further innovation.

16.7.2 16.7.2 Formalizing Open-Endedness

Definition 16.33 (Open-ended evolution)**.** An evolutionary system is **open-ended** if it: 1. **Never converges**: $\lim_{t\to\infty} \mathrm{CumNov}(P_t) \neq 0$. 2. **Increases complexity**: Some measure of complexity grows without bound. 3. **Generates major transitions**: Qualitatively new types of entities emerge.

Observation 16.34. *Biological evolution on Earth appears open-ended: 3.5 billion years have produced increasing complexity (from prokaryotes to eukaryotes to multicellular life to intelligence) without convergence.*

16.7.3 16.7.3 Obstacles to Open-Endedness

Why do computational systems fail to be open-ended?

Obstacle 1: Finite state space. If $|\mathcal{X}| < \infty$, the system must eventually repeat. Novelty is bounded.

Obstacle 2: Fixed fitness landscape. If the fitness function is static, the system converges to optima. No pressure for continued exploration.

Obstacle 3: Representational limits. If solutions have fixed complexity (e.g., fixed-length genomes), major transitions are impossible.

Obstacle 4: Attractor collapse. As in model collapse (Chapter 14), iterative systems tend toward low-diversity equilibria.

16.7.4 16.7.4 Conditions for Open-Endedness

Based on biological analogies and computational experiments:

Condition 1: Expanding state space. The set of possible solutions must grow with the system. In biology: new genes, new cell types, new body plans.

Condition 2: Coevolution. Entities evolve in response to each other, creating an ever-shifting fitness landscape. Red Queen dynamics: "running just to stay in place."

Condition 3: Niche construction. Agents modify their environment, creating new selection pressures. The adjacent possible expands as entities create new niches.

Condition 4: Open-ended evaluation. The fitness function itself evolves or is learned, preventing convergence to fixed optima.

Conjecture 16.36 (Open-endedness requires (f, x) co-evolution). A system is open-ended only if both the function f (the rules of the game) and the state x (the solutions) co-evolve. Fixed f leads to convergence; only $\varphi \neq \mathrm{id}$ (changing rules) can sustain open-ended novelty.

16.8 Synthesis: The DDS Framework for Discovery and Evolution

16.8.1 16.8.1 Unifying Table

System	State x	Function f	Meta-rule φ
Scientific discovery	Hypothesis	Revision	Paradigm shift
Genetic algorithm	Population	Selection + mutation	Hyperparameter adapt
Novelty search	Population + archive	Selection by novelty	Archive update ru
MAP-Elites	Grid of elites	Mutation + cell update	Behavior space defini
FunSearch	Program archive	LLM mutation + scoring	Prompt evolution
AI Scientist	(paper, results, context)	Research pipeline	Methodology learni
Open-ended evolution	(population, environment)	Selection + niche construction	Major transitions

16.8.2 16.8.2 The Quality-Diversity Lyapunov Exponent

Extending Section 15.12.3:

Definition 16.35 (QD Lyapunov exponent)**.** For a quality-diversity system with quality Q_t and diversity D_t at generation t, define:

$$\lambda_{QD} = \lim_{t\to\infty} \frac{1}{t} \log \frac{Q_t \cdot D_t}{Q_0 \cdot D_0}.$$

- $\lambda_{QD} < 0$: System collapses (quality or diversity decreasing).
- $\lambda_{QD} = 0$: Equilibrium (quality-diversity maintained).
- $\lambda_{QD} > 0$: Expansion (both improving)—the open-ended regime.

16.8.3 16.8.3 The Role of LLMs

LLMs transform the landscape of evolutionary and discovery systems by:

1. **Informed search**: LLMs encode vast prior knowledge, enabling intelligent exploration rather than blind mutation.

1. **Linguistic interface**: Problems can be specified in natural language; solutions can be programs or text.

1. **Transfer learning**: Strategies that work in one domain (from training data) transfer to new problems.

1. **Scalable creativity**: LLMs can generate many diverse candidates cheaply.

Proposition 16.36. *LLMs shift the exploration-exploitation balance toward informed exploration. The "adjacent possible" is larger because the LLM can propose solutions that are not merely one mutation away but are semantically related.*

However, LLMs also introduce risks:

1. ***Mode collapse****: LLMs have their own biases; LLM-driven evolution may converge to LLM-preferred solutions.*

2. ***Training distribution limits****: LLMs cannot easily generate solutions outside their training distribution.*

3. ***Hallucination****: LLMs may propose plausible-sounding but invalid solutions.*

The combination of LLMs with explicit evaluation (fitness functions, proof checkers, experiments) addresses these risks by grounding creativity in verification.

16.8.4 16.8.4 Implementation Examples

The following code snippets illustrate key components of the systems discussed in this chapter.

Genetic Algorithm with Novelty Pressure:

```
import random
from typing import List, Callable, TypeVar

T = TypeVar('T')

def genetic_algorithm_with_novelty(
    population: List[T],
    fitness: Callable[[T], float],
    distance: Callable[[T, T], float],
    mutate: Callable[[T], T],
    crossover: Callable[[T, T], T],
    generations: int = 100,
    novelty_weight: float = 0.3,
    k_nearest: int = 5,
) -> List[T]:
    """GA with novelty-weighted fitness (Section 16.3.4)."""
    archive = list(population)  # Historical solutions

    for gen in range(generations):
        # Compute novelty for each individual
        def novelty(x: T) -> float:
            distances = sorted([distance(x, a) for a in archive])
            return sum(distances[:k_nearest]) / k_nearest

        # Combined fitness: quality + novelty
        def combined_fitness(x: T) -> float:
            return (1 - novelty_weight) \textit{ fitness(x) + novelty_weight } novelty(x)
```

```
        # Selection (tournament)
        def select() -> T:
            contestants = random.sample(population, 3)
            return max(contestants, key=combined_fitness)

        # Generate offspring
        offspring = []
        for _ in range(len(population)):
            p1, p2 = select(), select()
            child = crossover(p1, p2)
            child = mutate(child)
            offspring.append(child)

        # Update population and archive
        population = offspring
        archive.extend(offspring)

    return population
```

Novelty Search (Section 16.4.1):

```
def novelty_search(
    initial_population: List[T],
    behavior: Callable[[T], List[float]],  # Map solution to behavior descriptor
    distance: Callable[[List[float], List[float]], float],
    mutate: Callable[[T], T],
    generations: int = 100,
    archive_threshold: float = 0.5,
) -> tuple[List[T], List[T]]:
    """Pure novelty search: selection based only on behavioral novelty."""
    population = list(initial_population)
    archive = []  # Archive of novel behaviors

    for gen in range(generations):
        # Compute behavior descriptors
        behaviors = [behavior(x) for x in population]

        # Compute novelty scores (average distance to k-nearest in archive + population)
        def novelty_score(b: List[float]) -> float:
            all_behaviors = behaviors + [behavior(a) for a in archive]
            if not all_behaviors:
                return float('inf')
            distances = sorted([distance(b, ob) for ob in all_behaviors])
            return sum(distances[:15]) / min(15, len(distances))

        novelties = [novelty_score(b) for b in behaviors]

        # Add novel individuals to archive
        for x, b, nov in zip(population, behaviors, novelties):
            if nov > archive_threshold:
                archive.append(x)
```

```
        # Selection by novelty (tournament)
        def select() -> T:
            indices = random.sample(range(len(population)), 3)
            best_idx = max(indices, key=lambda i: novelties[i])
            return population[best_idx]

        # Generate next generation
        population = [mutate(select()) for _ in range(len(population))]

    return population, archive
```

MAP-Elites (Section 16.4.3):

```
from typing import Dict, Tuple, Optional
import numpy as np

def map_elites(
    initial_solutions: List[T],
    fitness: Callable[[T], float],
    behavior_descriptor: Callable[[T], Tuple[int, ...]],  # Maps to grid cell
    mutate: Callable[[T], T],
    grid_dims: Tuple[int, ...],
    iterations: int = 10000,
) -> Dict[Tuple[int, ...], Tuple[T, float]]:
    """MAP-Elites: maintain a grid of behavioral niches, each with its elite."""
    # Grid: maps cell coordinates to (solution, fitness)
    grid: Dict[Tuple[int, ...], Tuple[T, float]] = {}

    # Initialize with random solutions
    for x in initial_solutions:
        cell = behavior_descriptor(x)
        fit = fitness(x)
        if cell not in grid or fit > grid[cell][1]:
            grid[cell] = (x, fit)

    for iteration in range(iterations):
        if not grid:
            break

        # Select random occupied cell
        cell = random.choice(list(grid.keys()))
        elite, _ = grid[cell]

        # Mutate to produce offspring
        offspring = mutate(elite)
        offspring_cell = behavior_descriptor(offspring)
        offspring_fit = fitness(offspring)

        # Update grid if offspring is better for its cell
        if offspring_cell not in grid or offspring_fit > grid[offspring_cell][1]:
            grid[offspring_cell] = (offspring, offspring_fit)
```

```
    return grid

def qd_metrics(grid: Dict, grid_dims: Tuple[int, ...]) -> dict:
    """Compute Quality-Diversity metrics for a MAP-Elites grid."""
    total_cells = np.prod(grid_dims)
    occupied = len(grid)

    fitnesses = [fit for _, fit in grid.values()]

    return {
        'coverage': occupied / total_cells,
        'qd_score': sum(fitnesses),  # Sum of all elite fitnesses
        'max_fitness': max(fitnesses) if fitnesses else 0,
        'mean_fitness': np.mean(fitnesses) if fitnesses else 0,
    }
```

LLM Mutation Operator (Section 16.5.1):

```
def llm_mutation_operator(
    solution: str,
    llm_generate: Callable[[str], str],
    mutation_type: str = 'improve',
) -> str:
    """Use an LLM as an intelligent mutation operator for programs/text."""

    prompts = {
        'improve': f"""Improve this solution while keeping its core approach:

{solution}

Return only the improved solution, no explanation.""",

        'vary': f"""Create a variation of this solution with a different approach:

{solution}

Return only the new solution, no explanation.""",

        'simplify': f"""Simplify this solution while maintaining functionality:

{solution}

Return only the simplified solution, no explanation.""",

        'combine': f"""This is a template. Create a novel variation inspired by it:

{solution}

Return only the new solution, no explanation.""",
    }
```

```
    prompt = prompts.get(mutation_type, prompts['improve'])
    return llm_generate(prompt)


def funsearch_iteration(
    archive: List[Tuple[str, float]],  # (program, score) pairs
    llm_generate: Callable[[str], str],
    evaluate: Callable[[str], float],
    num_samples: int = 2,
) -> List[Tuple[str, float]]:
    """One iteration of FunSearch (Section 16.5.2)."""

    # Sample high-scoring programs from archive
    if len(archive) < num_samples:
        samples = archive
    else:
        # Weighted sampling by score
        weights = [score for _, score in archive]
        min_w = min(weights)
        weights = [w - min_w + 1 for w in weights]  # Ensure positive
        samples = random.choices(archive, weights=weights, k=num_samples)

    # Create prompt with sampled programs
    programs_text = "\n\n".join([
        f"# Score: {score:.3f}\n{prog}"
        for prog, score in samples
    ])

    prompt = f"""Here are some high-scoring programs:

{programs_text}

Write a new, improved program that could score even higher.
Return only the program code."""

    # Generate new program
    new_program = llm_generate(prompt)

    try:
        score = evaluate(new_program)
        return archive + [(new_program, score)]
    except Exception:
        return archive  # Skip invalid programs
```

Measuring Scientific Discovery Dynamics:

```
def discovery_system_metrics(
    hypotheses: List[str],
    embedder: Callable[[str], List[float]],
    evidence_scores: List[float],
) -> dict:
    """Metrics for a scientific discovery trajectory (Section 16.2)."""
```

```
import numpy as np

if len(hypotheses) < 2:
    return {'trajectory_length': len(hypotheses)}

# Embed all hypotheses
embeddings = [embedder(h) for h in hypotheses]
embeddings = np.array(embeddings)

# Cumulative novelty: distance from each hypothesis to all previous
cumulative_novelties = []
for i in range(1, len(embeddings)):
    distances = np.linalg.norm(embeddings[:i] - embeddings[i], axis=1)
    cumulative_novelties.append(np.min(distances))

# Evidence improvement over time
evidence_improvement = [
    evidence_scores[i] - evidence_scores[i-1]
    for i in range(1, len(evidence_scores))
]

return {
    'trajectory_length': len(hypotheses),
    'total_novelty': sum(cumulative_novelties),
    'mean_cumulative_novelty': np.mean(cumulative_novelties),
    'novelty_trend': np.polyfit(range(len(cumulative_novelties)),
                                cumulative_novelties, 1)[0],  # Slope
    'evidence_improvement': sum(evidence_improvement),
    'final_evidence': evidence_scores[-1] if evidence_scores else 0,
}
```

16.9 Open Questions

Question 16.37 (Open-ended AI). *Can we build systems that exhibit biological-like open-ended evolution? What are the minimal conditions? Current systems (MAP-Elites, FunSearch, AI Scientist) seem to plateau rather than generate unbounded novelty.*

Question 16.38 (Paradigm shifts). *Can AI systems generate genuine paradigm shifts—not just new solutions within existing frameworks but entirely new frameworks? This seems to require meta-level reasoning about the problem formulation itself.*

Question 16.39 (Coevolution with humans). *What is the role of human scientists in AI-augmented discovery? The most successful systems (FunSearch, AlphaEvolve) involve human problem specification and result interpretation. Is human-AI coevolution the path to open-endedness?*

Question 16.40 (Evaluation bottleneck). *Autonomous scientific discovery is limited by the ability to evaluate hypotheses. Physical experiments are slow and expensive. Can we develop better "world models" that enable rapid hypothesis testing?*

Question 16.41 (Diversity collapse in LLM-evolution). *Do LLM-driven evolutionary systems suffer from diversity collapse analogous to model collapse? If LLM outputs are fed back as inputs (prompts containing LLM-generated code), does diversity decay?*

Question 16.42 (Measuring scientific novelty)**.** *How should we measure the novelty of scientific discoveries? Citation-based metrics capture impact but not originality. Information-theoretic measures (Section 15.13) could be applied to scientific papers, but the right formalization is unclear.*

Question 16.43 (The creativity spectrum)**.** *Is there a hierarchy of creative capabilities? 1. Generating variations within a known space. 2. Discovering new solutions to known problems. 3. Discovering new problems worth solving. 4. Inventing new frameworks that redefine what counts as a solution.*

Where do current AI systems fall? Can LLMs reach level 4?

16.10 Summary

Concept	Dynamical Systems View	Key Insight
Scientific method	Iteration on hypothesis space	Paradigm shifts = changes in f
Genetic algorithms	DDS on population space	Premature convergence = spurious attractor
Novelty search	Selection by archive distance	Ignoring objectives can help
Quality-Diversity	Multi-objective: fitness + coverage	MAP-Elites as grid of local optima
LLM mutation	Intelligent, knowledge-based variation	Prior knowledge expands adjacent possible
FunSearch	Program space evolution with LLM	Search for programs, not solutions
AlphaEvolve	General-purpose algorithm discovery	LLM ensemble + multi-objective
AI Scientist	Full research pipeline automation	Science as (f, x) iteration
Open-endedness	Unbounded novelty generation	Requires (f, x) co-evolution

The synthesis reveals that **discovery and evolution are two faces of the same coin**: both involve iterative search through vast spaces, balancing exploration (novelty) with exploitation (quality), and seeking dynamics that sustain creativity without collapse.

LLMs have transformed both domains by providing intelligent mutation operators and automated evaluation. But the deepest challenge—achieving true open-endedness—remains unsolved. Biological evolution achieved it; can we engineer it?

The framework developed in this textbook—discrete dynamical systems, symbolic dynamics, computational mechanics, the (f, x) decomposition, novelty metrics—provides the conceptual tools to analyze these systems. Whether those tools suffice to *design* open-ended systems is the frontier.

16.11 Recommended Reading

Philosophy and History of Science:

- Kuhn, T. S. (1962). *The Structure of Scientific Revolutions* — paradigm shifts and normal science.
- Popper, K. (1959). *The Logic of Scientific Discovery* — falsificationism and the scientific method.
- Kauffman, S. A. (1996). *At Home in the Universe* — the adjacent possible and self-organization.

Evolutionary Computation Foundations:

- Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems* — foundational text on genetic algorithms.

- Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning* — comprehensive GA textbook.
- Eiben, A. E. and Smith, J. E. (2015). *Introduction to Evolutionary Computing* — modern introduction to the field.

Novelty Search and Quality-Diversity:

- Lehman, J. and Stanley, K. O. (2011). "Abandoning Objectives: Evolution Through the Search for Novelty Alone" — the foundational novelty search paper.
- Mouret, J.-B. and Clune, J. (2015). "Illuminating Search Spaces by Mapping Elites" — introduces MAP-Elites.
- Pugh, J. K., Soros, L. B., and Stanley, K. O. (2016). "Quality Diversity: A New Frontier for Evolutionary Computation" — comprehensive survey.

LLM-Augmented Evolution:

- Lehman, J., et al. (2022). "Evolution Through Large Models" — introduces ELM concept.
- Romera-Paredes, B., et al. (2024). "Mathematical Discoveries from Program Search with Large Language Models" — FunSearch (Nature).
- Chen, A., Dohan, D., and So, D. R. (2024). "EvoPrompt: Language Models for Code-Level Neural Architecture Search" — LLMs for NAS.

AI for Scientific Discovery:

- Lu, C., et al. (2024). "The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery" — full scientific pipeline automation.
- Wang, H., et al. (2023). "Scientific Discovery in the Age of Artificial Intelligence" — Nature survey of AI in science.
- Jumper, J., et al. (2021). "Highly Accurate Protein Structure Prediction with AlphaFold" — landmark AI discovery (Nature).

Open-Endedness:

- Stanley, K. O., Lehman, J., and Soros, L. (2017). "Open-Endedness: The Last Grand Challenge You've Never Heard Of" — accessible introduction.
- Soros, L. B. and Stanley, K. O. (2014). "Identifying Necessary Conditions for Open-Ended Evolution" — formal conditions.
- Bedau, M. A. (2003). "Artificial Life: Organization, Adaptation, and Complexity from the Bottom Up" — ALife perspective.

Computational Creativity:

- Boden, M. A. (2004). *The Creative Mind: Myths and Mechanisms* — cognitive science of creativity.
- Colton, S. and Wiggins, G. A. (2012). "Computational Creativity: The Final Frontier?" — survey of computational creativity.
- Schmidhuber, J. (2010). "Formal Theory of Creativity, Fun, and Intrinsic Motivation" — information-theoretic creativity.

16.12 References

- Clune, J. (2019). AI-GAs: AI-generating algorithms, an alternate paradigm for producing general artificial intelligence. *arXiv preprint* arXiv:1905.10985.

- DeepMind (2025). AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint* arXiv:2506.13131.

- Kauffman, S. A. (1996). *At Home in the Universe: The Search for the Laws of Self-Organization and Complexity.* Oxford University Press.

- Kuhn, T. S. (1962). *The Structure of Scientific Revolutions.* University of Chicago Press.

- Lehman, J. and Stanley, K. O. (2011). Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2):189–223.

- Lehman, J., Gordon, J., Jain, S., Ndousse, K., Yeh, C., and Stanley, K. O. (2022). Evolution through large models. *arXiv preprint* arXiv:2206.08896.

- Lu, C., Lu, C., Lange, R. T., Foerster, J., Clune, J., and Ha, D. (2024). The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint* arXiv:2408.06292.

- Mouret, J.-B. and Clune, J. (2015). Illuminating search spaces by mapping elites. *arXiv preprint* arXiv:1504.04909.

- Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M. P., Dupont, E., Ruiz, F. J. R., Ellenberg, J. S., Wang, P., Fawzi, O., Kohli, P., and Fawzi, A. (2024). Mathematical discoveries from program search with large language models. *Nature*, 625:468–475.

- Sakana AI (2025). The AI Scientist-v2: Workshop-level automated scientific discovery via agentic tree search.

- Soros, L. B. and Stanley, K. O. (2014). Identifying necessary conditions for open-ended evolution through the artificial life world of Chromaria. *Proceedings of the Artificial Life Conference.*

- Stanley, K. O., Lehman, J., and Soros, L. (2017). Open-endedness: The last grand challenge you've never heard of. *O'Reilly Radar.*

16.13 Exercises

Exercise 16.1. Model the discovery of Newtonian mechanics as a (f, x) dynamical system. What was the state space H? What evidence E drove the revision? When did f change (paradigm shift)?

Exercise 16.2. Implement a simple genetic algorithm for symbolic regression (finding a formula that fits data). Track Distinct-1 on the population over generations. Does premature convergence occur? At what generation?

Exercise 16.3. Implement novelty search for a simple maze navigation task. Compare with fitness-based search. In which environments does novelty search outperform?

Exercise 16.4. Design a prompt for an LLM that acts as a mutation operator for Python functions. Test it: does it produce syntactically valid variations? Are the variations meaningfully different?

Exercise 16.5. Propose a metric for "paradigm-shift novelty" in scientific papers. How would you distinguish papers that work within an existing paradigm from papers that establish a new one?

Exercise 16.6. (Research) Read the FunSearch paper. What is the role of the "islands" in the evolutionary algorithm? How does this relate to the diversity maintenance techniques of Section 16.3.4?

Exercise 16.7. (Research) Can you design an (f, x) system where both f and x co-evolve? Sketch an algorithm where the mutation operator itself is subject to selection.